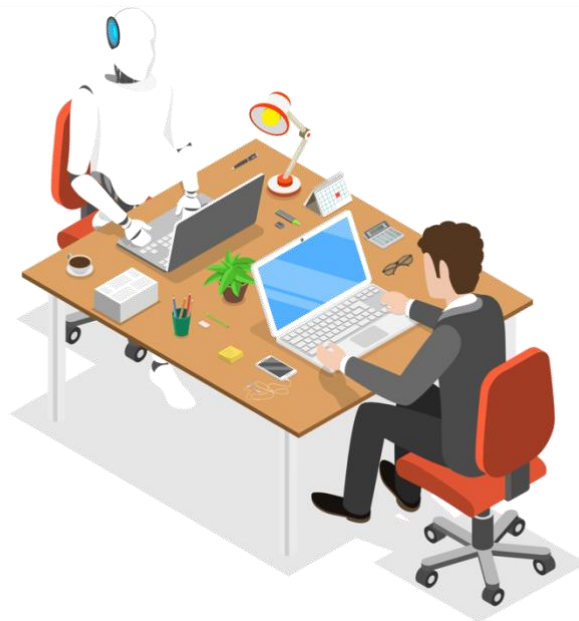




**Hochschule
Augsburg** University of
Applied Sciences

Fakultät für Informatik
M.Sc. Business Information
Systems

Thesis – Einsatzmöglichkeiten und Nutzen von Künstlicher Intelligenz zur Automatisierung von BPMN-basierten Prozessen



Masterarbeit im Studiengang
“Business Information Systems”

(WS2021/22)

Angefertigt & Eingereicht von:

Marco Schäck

marco.schaeck1@hs-augsburg.de

Matrikelnummer: 2081852

Abgabedatum: 20.10.2022

Abstract

Die Künstliche Intelligenz (KI) wird als eine Schlüsseltechnologie der Zukunft gesehen. Vor allem in Prozessen, bzw. zur Automatisierung von diesen wird sie bereits heute häufig eingesetzt. Dabei ist es absehbar, dass ihr Einsatz in diesem Bereich in Zukunft noch stark steigen wird. Der Grund dafür sind die enormen Potentiale, die durch ihren Einsatz gehoben werden können. Andererseits zeigt sich jedoch auch, dass beim Einsatz einer KI, in einem Prozess, konkrete Herausforderungen auftreten können. Zudem muss beachtet werden, dass Prozesse schon heute meist sehr komplex sind. Denn an ihrer Ausführung können diverse Akteure, Systeme und Technologien teilhaben. Um eine KI in eine solche Umgebung zu integrieren, werden geeignete Lösungen benötigt. Eine von diesen könnte die BPMN, in Kombination mit dem Einsatz einer Process Engine sein.

Die folgende Thesis analysiert den Einsatz von Künstlicher Intelligenz im Hinblick auf diese beiden Lösungen. Sie geht darauf ein, wie der Einsatz einer KI aus einer fachlichen Sicht abgebildet und aus einer technischen Sicht automatisiert werden kann. Im Fazit soll hierdurch die Fragestellung beantwortet werden, welchen konkreten Nutzen die Kombination von KI, BPMN und einer Process Engine, zum Zweck der Prozessautomatisierung erzeugen kann.

Daher werden zuallererst die relevanten Grundlagen zu Prozessen und der Künstlichen Intelligenz vermittelt. Folgend darauf werden diese vereinigt. Dabei wird ergründet, für welche operativen Anwendungsfälle eine KI in Prozessen eingesetzt werden kann. Ferner wird darauf eingegangen, welche Herausforderungen beim Einsatz der Technologie in diesen Szenarien auftreten können. Danach wird auf die Theorie zur BPMN und der DMN eingegangen.

Darauf aufbauend werden jegliche der Grundlagen kombiniert. Dabei zeigt sich, dass der Einsatz einer KI in einem Prozess als eine Aktivität interpretiert – und somit durch die BPMN abgebildet werden kann. Diese kann als ein Service-, ein Nachrichten-Sendender-, oder ein Geschäftsregel Task repräsentiert werden. Weiterhin wird erörtert, dass es die Möglichkeit gibt, einen eigenen Aufgabentyp zu erstellen. Dieser könnte als KI-Task bezeichnet werden. Ferner werden Entwürfe erstellt, welche beschreiben, wie die Tasks in einer Process Engine automatisiert werden können. Abschließend wird thematisiert, wie die Notation und eine Engine, bei der Bewältigung der oben erörterten Herausforderungen helfen können. Dazu werden ebenfalls spezifische Lösungsansätze entworfen. Auf Basis der Entwürfe, sowie Lösungsansätze werden sodann Prototypen entwickelt, um die theoretischen Schlussfolgerungen zu testen.

Insgesamt kommt die Arbeit zum Schluss, dass die Kombination der BPMN und der KI zum Zweck der Prozessautomatisierung sehr sinnvoll ist. So lässt sich ableiten, dass durch den Einsatz einer KI in Prozessen ein hoher Nutzen erzeugt werden kann. Weiterhin zeigt sich, dass die Verwendung der BPMN und einer geeigneten Process Engine, um den Einsatz einer KI zu automatisieren viele Vorteile bringen kann. So kann der Einsatz jener KI auf verschiedene Weisen abgebildet, und auf unterschiedliche Arten in die Ausführungslogik des Prozesses technisch integriert werden. Zudem stellt sich heraus, dass der Einsatz der beiden Lösungen dabei helfen kann, die oben genannten Herausforderungen zu lösen. Darüber hinaus ermöglicht die Engine auch die Automatisierung von weiteren Schritten in einem Prozess. Alles in allem kann daher ein wesentlich höherer Mehrwert erzielt werden, als wenn die KI-Lösung alleinstehend eingesetzt wird.

Jedoch zeigt sich durch die Thesis auch, dass die Automatisierung eines Prozesses durch die KI, die BPMN und eine Engine einen hohen Aufwand erzeugt. Daher wird gefolgert, dass sie nicht für alle Prozesse eines Unternehmens geeignet ist – sondern nur für solche, in welchen ein positiver Return on Investment oder ein gleichwertiger Wettbewerbsvorteil erzielt werden kann.

Inhaltsverzeichnis

Inhaltsverzeichnis.....	I
Abbildungsverzeichnis.....	V
Tabellenverzeichnis.....	VI
Anhangsverzeichnis.....	VII
Abkürzungsverzeichnis	VII
1: Einleitung	1
1.1: Motivation	1
1.2: Problemstellung.....	1
1.3: Zielsetzung.....	2
1.4: Aufbau der Arbeit	2
2: Grundlagen – zu Prozessen	3
2.1: Prozess – als Gegenstand der Automatisierung.....	3
2.1.1: Grundkonstrukt eines Prozesses	3
2.1.2: Entscheidung – als Aktivität in einem Prozess	4
2.2: Prozessautomatisierung – als Hauptthematik.....	5
2.3: Business Process Management.....	6
2.3.1: Prozesslebenszyklus	7
2.3.2: Camunda BPM-Kreislauf.....	7
2.3.3: Prozessmodell	9
2.3.4: Grundbegriffe zu Modellierungsnotationen	9
3: Grundlagen – zur künstlichen Intelligenz.....	10
3.1: Künstliche Intelligenz – als Oberbegriff	10
3.2: Machine Learning – als Teilgebiet der KI	10
3.2.1: Daten im Kontext des Machine Learnings.....	11
3.2.2: Lernstrategien des Machine Learnings.....	11
3.2.3: Supervised Learning.....	12
3.2.4: Training von Supervised Learning Modellen.....	12
3.2.5: Entscheidungsbaum	13
3.2.5.1: Aufbau eines Entscheidungsbaums	14
3.2.5.2: Funktionsweise eines Entscheidungsbaums.....	14
3.2.5.3: Training eines Entscheidungsbaums.....	15
3.2.6: Validierung von Supervised Learning Modellen.....	16
3.2.7: Deep Learning.....	17
3.2.7.1: Grundidee von künstlichen neuronalen Netzen	17

3.2.7.2: Aufbau eines künstlichen neuronalen Netzes	17
3.2.7.3: Der Weg vom neuronalen Netz zum Deep Learning	18
3.3: PMML als Standard	19
4: Fusion – von Prozessen und KI	20
4.1: Möglichkeiten zum Einsatz von KI in Unternehmen	20
4.2: Einsatz von KI zur Prozessautomatisierung	21
4.3: KI als Make-Or-Buy Entscheidung	22
4.4: Herausforderungen beim Einsatz von KI	23
4.4.1: Transparenz & Akzeptanz	23
4.4.2: Integration	23
4.4.3: Qualität der Ergebnisse	24
4.4.4: Erklärbarkeit	24
5: Grundlagen – zur BPMN und DMN	26
5.1: Business Process Model and Notation	26
5.1.1: BPMN als Modellierungsnotation	26
5.1.2: BPMN als Notation mit Fokus auf Automatisierung	27
5.1.2.1: Automatisierung durch Process Engines	28
5.1.2.2: Technische Konfiguration der BPMN-Modelle	28
5.1.3: Die Notation im Detail	29
5.1.3.1: Aktivitäten	29
5.1.3.2: Aufgaben	30
5.1.3.3: Ereignisse	31
5.1.3.4: Gateways	33
5.1.3.5: Swimlanes	34
5.1.3.6: Artefakte	35
5.2: Process Engine	36
5.2.1: Funktionsweise einer Process Engine	37
5.2.2: Exemplarische Process Engines	38
5.3: Decision Model and Notation	38
5.3.1: Decision Requirement Diagram	39
5.3.1.1: Input Data	39
5.3.1.2: Knowledge Source	40
5.3.1.3: Business Knowledge Model	40
5.3.1.4: Decision	40
5.3.2: Entscheidungslogik	40
5.3.2.1: Entscheidungstabelle	41
5.3.3: DMN als Ergänzung zur BPMN	43

5.4: DMN-Engine	44
5.4.1: Funktionsweise einer DMN-Engine.....	44
5.4.2: Überblick über DMN-Engines	45
6: Entwurf – Einsatz von KI in BPMN-basierten Prozessen.....	46
6.1: Rollen einer KI im Prozesskontext	46
6.1.1: Künstliche Intelligenz als Orchestrator	46
6.1.2: Künstliche Intelligenz als Prozessakteur	47
6.2: Modellierung eines KI-Einsatzes in der BPMN	47
6.2.1: Typen zur Modellierung eines KI-Einsatzes	48
6.2.1.1: Modellierung als Service Task	48
6.2.1.2: Modellierung als Nachrichten-Sender Task	48
6.2.1.3: Modellierung als Geschäftsregel Task	49
6.2.1.4: Modellierung als KI-Task.....	49
6.3: Automatisierung eines KI-Einsatzes in der BPMN.....	49
6.3.1: Automatisierung mittels Service Task	50
6.3.2: Automatisierung mittels Nachrichten-Sender Task	51
6.3.3: Automatisierung mittels Geschäftsregel Task	51
6.3.3.1: Variante 1 – PMML-Modell in Business Knowledge Model	51
6.3.3.2: Variante 2 – Entscheidungstabelle	52
6.4: Analyse der Herausforderungen.....	53
6.4.1: Transparenz & Akzeptanz.....	53
6.4.1.1: Einsatz von Swimlanes	54
6.4.1.2: Einsatz von Artefakten.....	55
6.4.2: Integration	56
6.4.2.1: Einsatz in Folgeschritten.....	56
6.4.2.2: Einsatz zur Prozesssteuerung.....	57
6.4.3: Qualität der Ergebnisse	58
6.4.3.1: Gezielte Prüfung der Ergebnisse	58
6.4.3.2: Zufällige Prüfung der Ergebnisse	60
6.4.4: Erklärbarkeit	61
6.4.4.1: Orchestration transparenter Modelle.....	61
6.4.4.2: Orchestration verschiedener Modelle durch Ensemble Learning.....	62
6.4.4.3: Umwandlung in DMN-Entscheidungstabelle	63
7: Entwurf – Prototypischer Einsatz von KI in BPMN-Prozessen.....	64
7.1: Technisches Setting zur Automatisierung der Prozesse	64
7.2: Prototyp 1 – Automatisierung mittels Service Task	64
7.3: Prototyp 2 – Automatisierung mittels Geschäftsregel Task.....	67

7.3.1: Variante 1 – Einsatz eines PMML-Modells.....	70
7.3.2: Variante 2 – Einsatz einer Entscheidungstabelle.....	70
8: Evaluation – Bewertung der Prototypen.....	71
8.1: Prototyp 1 – Automatisierung mittels Service Task.....	71
8.2: Prototyp 2 – Automatisierung mittels Geschäftsregel Task.....	73
8.2.1: Variante 1 – Spezielle Erkenntnisse zum Einsatz des PMML-Modells	75
8.2.2: Variante 2 – Spezielle Erkenntnisse zum Einsatz der Entscheidungstabelle	75
9: Evaluation – Nutzen der Kombination von KI und BPMN	77
9.1: Allgemeine Kombination.....	77
9.1.1: Nutzen des Einsatzes einer KI.....	77
9.1.2: Möglichkeiten zum KI-Einsatz.....	78
9.1.2.1: Service Task.....	78
9.1.2.2: Nachrichten-Sendender Task	78
9.1.2.3: Geschäftsregel Task	79
9.1.2.4: KI-Task.....	79
9.1.2.5: Fazit zu untersuchten Typen	80
9.1.3: Daten als Grundvoraussetzung.....	80
9.1.4: Aufwand und Komplexität der Implementierung	81
9.2: Kombination zur Lösung der KI-spezifischen Herausforderungen.....	82
9.2.1: Transparenz & Akzeptanz.....	82
9.2.2: Integration	83
9.2.3: Qualität der Ergebnisse	84
9.2.4: Erklärbarkeit	86
9.3: Gesamtfazit zur Kombination	87
9.3.1: Nutzen der BPMN und einer Process Engine	87
9.3.2: Nutzen der DMN und einer DMN-Engine	89
9.3.3: Eignung eines Prozesses.....	90
10: Fazit	91
10.1: Abschlussbetrachtung.....	91
10.2: Ausblick	92
Literaturverzeichnis.....	93

Abbildungsverzeichnis

Abbildung 1: Prozess als Abfolge von Aktivitäten	3
Abbildung 2: Grundkonstrukt einer Entscheidung.....	4
Abbildung 3: Ebenen des BPM und Phasen im zugehörigen Prozesslebenszyklus	6
Abbildung 4: Camunda BPM-Kreis als Vorgehensmodell zum BPM	8
Abbildung 5: Zusammenhang zwischen KI, Machine Learning und Deep Learning	10
Abbildung 6: Beispiele für lösbare Problemstellungen im Supervised Learning.....	12
Abbildung 7: Schritte zur Erstellung eines Supervised Learning Modells	13
Abbildung 8: Aufteilung von Beispieldaten in Trainings- und Testdaten.....	13
Abbildung 9: Exemplarischer Entscheidungsbaum zur Kategorisierung von Bankkunden	14
Abbildung 10: Funktionsweise eines Entscheidungsbaums	15
Abbildung 11: Training eines Entscheidungsbaums durch Extraktion von Zusammenhängen ...	15
Abbildung 12: Schritte zur Erstellung eines exemplarischen Entscheidungsbaumes.....	16
Abbildung 13: Überprüfung des Entscheidungsbaumes mittels Testdaten.....	16
Abbildung 14: Berechnung der Modellqualität mittels einer Konfusionsmatrix.....	16
Abbildung 15: Grundlegender Aufbau eines künstlichen neuronalen Netzes	18
Abbildung 16: Basisstruktur eines PMML-Modells	19
Abbildung 17: Ausschnitt der relevanten Notationselemente der BPMN	26
Abbildung 18: BPMN-basierter Prozess zur Stellenausschreibung	27
Abbildung 19: Automatisierung eines Prozesses durch Konfiguration der Elemente	29
Abbildung 20: Unterscheidung von Aktivitäten in atomar und nicht-atomar.....	29
Abbildung 21: Unterschiedliche Typen von BPMN-Aufgaben.....	31
Abbildung 22: BPMN – Arten von modellierbaren Ereignissen	32
Abbildung 23: Unterscheidung von Ereignissen in eingetretene und ausgelöste Ereignisse.....	33
Abbildung 24: Zweck und Arten von Gateways in der BPMN.....	33
Abbildung 25: Beispiele für die Verwendung von exklusiven und parallelen Gateways.....	34
Abbildung 26: Nutzung von Swimlanes zur Strukturierung eines Vertriebsprozesses	35
Abbildung 27: Daten und Anmerkungen als beispielhafte Artefakte der BPMN	36
Abbildung 28: Exemplarische Architektur eines Business Process Management Systems	36
Abbildung 29: Funktionsweise einer Process Engine	37
Abbildung 30: Notationselemente und Abhängigkeiten in einem abstrakten DMN-Modell	39
Abbildung 31: Grundlegender Aufbau einer DMN-basierten Entscheidungstabelle.....	41
Abbildung 32: Aufbau und Logik einer Entscheidungsregel	42
Abbildung 33: Gesamtkontext zwischen BPMN- und DMN-Modellen	43
Abbildung 34: Grundkonzept und Funktionsweise einer Business Rule Engine	44

Abbildung 35: Künstliche Intelligenz als Orchestrator bzw. Intermediär im Prozess	46
Abbildung 36: Künstliche Intelligenz als aktiv am Prozess beteiligter Akteur.....	47
Abbildung 37: Möglichkeiten zur Typisierung einer Aufgabe in der eine KI eingesetzt wird	48
Abbildung 38: Modellierung eines KI-Einsatzes als Nachrichten-Sendender Task.....	49
Abbildung 39: Technische Implementierung eines Service Tasks	50
Abbildung 40: Technische Implementierung eines Nachrichten-Sendenden Tasks	51
Abbildung 41: Technische Implementierung zum Einsatz eines PMML-Modells.....	52
Abbildung 42: Transformation eines Entscheidungsbaums in eine Entscheidungstabelle	52
Abbildung 43: Technische Implementierung einer DMN-Entscheidungstabelle.....	53
Abbildung 44: Abbildung einer KI-Lösung als interner Prozessakteur	54
Abbildung 45: Abbildung einer KI-Lösung als externer Prozessakteur	55
Abbildung 46: Einsatz von Artefakten zur Kontextualisierung eines KI-Einsatzes	56
Abbildung 47: Verwendung des Ergebnisses aus einem KI-Einsatz in einer Folgeaktivität	57
Abbildung 48: Verwendung des Ergebnisses aus einem KI-Einsatz zur Prozesssteuerung.....	58
Abbildung 49: Allgemeine Möglichkeiten zur Kontrolle des Ergebnisses einer KI	59
Abbildung 50: Fallabhängige Kontrolle des Ergebnisses einer KI	60
Abbildung 51: Zufällige Prüfung der Ergebnisse aus dem Einsatz einer KI	61
Abbildung 52: Orchestration eines transparenten ML-Modells	62
Abbildung 53: Erklärbarkeit der Vorhersagen einer KI durch Ensemble Learning	63
Abbildung 54: Log-Daten zur Ausführung einer DMN-Entscheidungstabelle	63
Abbildung 55: Ausgangslage – Prozess zur Bearbeitung von Eingangsrechnungen.....	64
Abbildung 56: Grundidee zur Extraktion von Daten aus einer Rechnung	65
Abbildung 57: Prototyp – Automatisierter Prozess zur Bearbeitung von Eingangsrechnungen..	66
Abbildung 58: Ausgangslage – Prozess zur Bearbeitung von Kreditanfragen	67
Abbildung 59: Klassifikator zur Bewertung der Kreditwürdigkeit von Bankkunden	68
Abbildung 60: Prototyp – Automatisierter Prozess zur Bearbeitung von Kreditanfragen.....	69
Abbildung 61: Varianten zur Technischen Implementierung des Klassifikators.....	70
Abbildung 62: Process Engine als Datenquelle für ML-Modell Training	81
Abbildung 63: Sinnhaftigkeit der Einbindung von PMML-Modellen in DMN-Modelle.....	90

Tabellenverzeichnis

Tabelle 1: Logische Vergleiche mittels FEEL-Operatoren	42
--	----

Anhangsverzeichnis

- Anhang 1: BPMN Poster – alle Notationselemente der BPMN
- Anhang 2: Nutzung der Invoice OCR API des Unternehmens verifyfi
- Anhang 3: Automatisierte Bearbeitung von Eingangsrechnungen
- Anhang 4: Daten zur Erstellung eines ML-Modells zur Klassifikation von Kreditanfragen
- Anhang 5: Erstellung eines ML-Entscheidungsbaums mit KNIME
- Anhang 6: Exemplarisches PMML-Modell
- Anhang 7: Automatisierte Bearbeitung von Kreditanfragen mittels einem PMML-Modell
- Anhang 8: Transformation eines PMML-Modells in eine DMN-Entscheidungstabelle
- Anhang 9: Automatisierte Bearbeitung von Kreditanfragen mittels einer Entscheidungstabelle
- Anhang 10: Entscheidungstabelle zur Klassifikation der Kreditwürdigkeit von Kreditkunden
- Anhang 11: Korrektheit der Umwandlung des Baumes in eine Entscheidungstabelle

Abkürzungsverzeichnis

Abb.	Abbildung
API	Application Programming Interface (Programmierschnittstelle)
BPM	Business Process Management
BKM	Business Knowledge Model
BPMN	Business Process Model and Notation
bzw.	beziehungsweise
ERP	Enterprise Resource Planning
FEEL	Friendly Enough Expression Language
DRD	Decision Requirement Diagramm
KI	Künstliche Intelligenz
KNN	Künstliches Neuronales Netz
ML	Machine Learning
SL	Supervised Learning
SDK	Software Development Kit
u.a.	unter anderem
z.B.	zum Beispiel
sog.	sogenannte

1: Einleitung

1.1: Motivation

Die Künstliche Intelligenz (KI) ist gegenwärtig einer der präsentesten Trends im Themenkomplex der IT. Sie befasst sich damit, Maschinen eine Form von Intelligenz zu verleihen. Statt also für jede Problemstellung explizit programmiert zu werden, sollen Maschinen durch diese Intelligenz selbstständig lernen, sich Wissen aneignen und somit komplexe Probleme lösen können, welche bisher entweder gar nicht lösbar waren oder nur durch Menschen gelöst werden konnten.

Im Rahmen der Digitalisierung nimmt die Künstliche Intelligenz einen großen Stellenwert ein. Für Unternehmen ermöglicht sie unter anderem die Schaffung neuer Produkte, Dienstleistungen, oder sogar ganzer Geschäftsmodelle. So ist die KI z.B. die Grundlage für bereits am Markt verfügbare Produkte, wie die Übersetzer „DeepL“ und „Google Translate“. Sie wird aber auch als Enabler für noch in Entwicklung befindliche Themen, wie das Autonome Fahren gesehen.

Doch auch abseits dieser, hauptsächlich in der Öffentlichkeit diskutierten, Anwendungsszenarien ist die KI für Unternehmen von hoher Relevanz. So sehen viele Unternehmen in der KI – laut einer Studie des IDG Research Service aus dem Jahr 2021 – vor allem ein Mittel zur Optimierung und zur Automatisierung ihrer Prozesse (siehe Reder 2021, S. 19).

Als ein solches wird die Technologie bereits heute für diverseste Zwecke eingesetzt – wie z.B. zur Automatisierung von einzelnen Schritten in einem Prozess, oder zur Entscheidungsunterstützung von Mitarbeitern. Es ist jedoch absehbar, dass es bei diesem Status Quo nicht bleiben wird. Es wird erwartet, dass die Künstliche Intelligenz in der Zukunft eine noch wesentlich größere Rolle spielen wird. Sie wird als eine Schlüsseltechnologie angesehen, welche für nahezu jeden Bereich eines Unternehmens relevant ist (vgl. Bitkom 2017, S. 8).

1.2: Problemstellung

Insgesamt kann also abgeleitet werden, dass der Einsatz einer KI im Kontext zur Automatisierung von Prozessen viele Potentiale heben und somit einen hohen Nutzen erzeugen kann. Dabei muss jedoch beachtet werden, dass durch den Einsatz der KI auch einige konkrete Herausforderungen entstehen können. Zu diesen zählen, dass der Einsatz transparent und komplett in den jeweiligen Prozess integriert sein muss. Weiterhin ist es eine Herausforderung, dass die Ergebnisse einer KI fehlerhaft sein können, und dass die Gründe für die Entscheidungen einer KI für Menschen häufig nicht nachvollziehbar sind. Außerdem muss beachtet werden, dass viele Prozesse bereits heute hochkomplex sind. Denn an ihrer Ausführung sind oft diverse Akteure, Systeme und Technologien beteiligt. Zu diesen zählen bspw. Menschen, Web-Schnittstellen und Legacy-Systeme. Daher sind die Systemarchitekturen in Prozessen für gewöhnlich sehr heterogen. Dies macht diese Prozesse schwer zu automatisieren.

Um eine KI-Lösung in eine solche Architektur zu integrieren, und um die Herausforderungen, die mit dem Einsatz dieser verbunden sind zu bewältigen, werden daher konkrete Lösungen benötigt. Eine von diesen könnte die „Business Process Model and Notation“ (BPMN) sein. Sie ist zu deuten als eine Notation zur Modellierung von Prozessen. Durch ihren Einsatz kann ein Prozess, aus einer fachlichen Sicht, graphisch abgebildet werden. Daneben ist die BPMN aber auch ein technisch ausführbarer Standard. Er bietet die Möglichkeit, einen Prozess, auf Basis eines Prozessmodells, durch den Einsatz einer sogenannten Process Engine zu automatisieren, bzw. zu steuern.

1.3: Zielsetzung

Schlussfolgernd aus der Problemstellung kann die Hypothese aufgestellt werden, dass es sinnvoll sein kann, die BPMN und die KI zu kombinieren. So könnte die BPMN z.B. eingesetzt werden, um den Einsatz einer KI aus fachlicher Sicht abzubilden, oder um diesen mittels einer Process Engine zu automatisieren. Im Zuge dieser Thesis soll diese Hypothese im Detail untersucht werden.

Daher soll zunächst ein grundlegendes Verständnis für Prozesse und die Thematik der Künstlichen Intelligenz aufgebaut werden. Basierend auf diesem sollen die Themengebiete zusammengeführt werden. Dabei soll vor allem darauf eingegangen werden, für welche Zwecke eine KI in Prozessen eingesetzt werden kann und welche konkreten Herausforderungen dabei entstehen.

Folgend darauf soll die BPMN einbezogen werden. Im Hinblick auf diese soll untersucht werden, welche spezifischen Möglichkeiten es gibt, um den Einsatz von KI-Lösungen aus fachlicher Sicht zu modellieren und aus technischer Sicht zu automatisieren. Zudem soll erörtert werden, ob und wie die BPMN bei der Lösung der oben genannten Herausforderungen helfen kann. Abschließend sollen die theoretischen Überlegungen durch die Erstellung von Prototypen validiert werden.

Im Fazit soll dadurch die Fragestellung beantwortet werden können, welchen konkreten Nutzen ein Unternehmen durch den Einsatz einer Künstlichen Intelligenz, in Kombination mit der BPMN und einer Process Engine, zu Zwecken der Prozessautomatisierung erzielen kann.

1.4: Aufbau der Arbeit

Das wissenschaftliche Vorgehen, welches bei der Anfertigung der Thesis verfolgt wird, orientiert sich an der Methode der sog. gestaltungsorientierten Wirtschaftsinformatik (vgl. Oesterle et al., 2010). Gemäß dieser wird sie in drei aufeinanderfolgende Phasen unterteilt:

Phase 1 - Analyse

In der Analysephase wird die Problemstellung untersucht. Daher werden in dieser, bzw. in Kapitel 2 und 3, zunächst die Grundlagen zu Prozessen, sowie zur Thematik der Künstlichen Intelligenz vermittelt. Folgend darauf wird in Kapitel 4 diskutiert, inwiefern die KI im Kontext zu Prozessen eingesetzt werden kann – wobei der Fokus auf operative Anwendungsfälle gelegt wird. Darüber hinaus wird untersucht, welche Herausforderungen, beim Einsatz einer KI in Prozessen auftreten können. Die Phase endet mit Kapitel 5, dass eine Einführung in die BPMN und die DMN gibt.

Phase 2 – Entwurf

Die Entwurfsphase umfasst die Kapitel 6 und 7. In dieser wird das Wissen aus der Analysephase angewendet, um spezifische Entwürfe zur Lösung der Fragestellungen zu konzipieren. Daher wird in Kapitel 6 darauf eingegangen, welche Rollen eine KI in einem Prozess annehmen kann, wie ihr Einsatz mittels der BPMN abgebildet und automatisiert werden kann. Zudem wird erörtert, ob und wie die BPMN bei der Lösung der oben genannten Herausforderungen helfen kann. Auf Basis dieser theoretischen Überlegungen werden in Kapitel 7, durch den Einsatz der Camunda Process Engine, exemplarische Prototypen erstellt, mittels welchen diese validiert werden sollen.

Phase 3 - Evaluation

Die Evaluationsphase beinhaltet die restlichen Kapitel. In diesen werden die jeweiligen Entwürfe und ihr Nutzen bewertet. Faktisch wird durch die Phase die Fragestellung beantwortet, welchen Nutzen ein Unternehmen durch die Kombination von KI, BPMN und einer Engine erzielen kann.

2: Grundlagen – zu Prozessen

Im Rahmen dieser Thesis soll untersucht werden, wie die BPMN und die Künstliche Intelligenz zur Automatisierung von Prozessen nutzenbringend kombiniert werden können. Um dies zu erörtern, müssen jedoch zuerst die Grundlagen zu den relevanten Themen behandelt werden. Das folgende Kapitel macht hiermit einen Anfang. Es befasst sich daher mit den Grundlagen zu Prozessen, der Prozessautomatisierung und dem Prozessmanagement. Zudem leitet es kurz in die Thematik der Prozessmodellierung ein. Alles in allem bildet es somit einen einleitenden Rahmen für die Arbeit. Dieser ist vor allem relevant für das Verständnis der BPMN, und um in den späteren Kapiteln zu analysieren, wie die Künstliche Intelligenz in Prozessen eingesetzt werden kann.

2.1: Prozess – als Gegenstand der Automatisierung

Prozesse sind das wohl wichtigste Instrument eines jeden Unternehmens. Nahezu jeder Vorgang in einem Unternehmen – von der Durchführung einer Bestellung, über den Vertrieb von Produkten, bis hin zur Buchhaltung – ist als ein solcher zu verstehen. Folglich gibt es viele Definitionen sowie Perspektiven auf das Konstrukt eines Prozesses.

Werden die Zusammenhänge aus diesen verknüpft, so kann ein Prozess grundsätzlich verstanden werden als ein konkreter Geschäftsvorgang in einem Unternehmen. Ein solcher wird oft auch als Geschäftsprozess bezeichnet. Er zeichnet sich dadurch aus, dass er regelmäßig ausgeführt wird, an der Strategie eines Unternehmens orientiert ist und die Erreichung spezifischer Ziele dieses Unternehmens unterstützt (vgl. Gadatsch 2020, S. 71). Zudem kann angemerkt werden, dass ein Prozess, aus einer abstrakten Sicht, stets aus einem ähnlichen Grundkonstrukt besteht. Dieses ist in Abbildung 1 dargestellt und wird in der Folge im Detail erörtert.

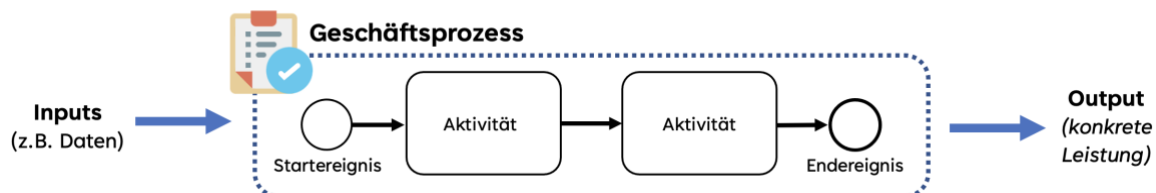


Abbildung 1: Prozess als Abfolge von Aktivitäten

2.1.1: Grundkonstrukt eines Prozesses

Abstrakt betrachtet besteht jeder Prozess aus einer endlichen Abfolge von Arbeitsschritten. Diese werden gemeinhin als Aktivitäten bezeichnet, sind logisch miteinander verbunden und können entweder von Menschen oder Maschinen ausgeführt werden (vgl. Freund und Rücker 2017, S. 1). Dabei ist jedoch zu beachten, dass – unabhängig davon wer sie ausführt – die Ausführung einer Aktivität, und somit auch jene eines Prozesses, immer eine bestimmte Zeit benötigt.

Insgesamt kann ein Prozess daher als inhaltlich abgeschlossen und zeitlich begrenzt verstanden werden (vgl. Becker und Schütte 2004, S. 32). Dies bedeutet er besitzt klar definierte Start- sowie Endpunkte. Diese werden repräsentiert durch sogenannte Start- und Endereignisse.

Ausgeführt wird ein Prozess, wenn eines der Startereignisse ausgelöst wird. Eine Ausführung wird dabei als eine Prozessinstanz bezeichnet. Der Auslöser einer solchen ist für gewöhnlich ein sog.

Prozesskunde. Der Begriff des Kunden darf hier allerdings nicht missinterpretiert werden. Er kann aus der Sicht einer Organisation sowohl von interner als auch externer Art sein.

Ziel jeder Prozessausführung ist es, konkrete Ergebnisse zu erzielen, bzw. Leistungen zu erstellen. Diese werden im Allgemeinen als der **Output** eines Prozesses bezeichnet. Dieser kann bspw. ein Produkt, eine Dienstleistung, oder eine Information sein. Der Kunde hat einen Bedarf, respektive ein Interesse am Output, denn dieser erzeugt für ihn einen Mehrwert (vgl. Scheer 2020, S. 71).

Um einen Output zu generieren, konsumieren Prozesse bestimmte **Inputs**. Diese sind zu verstehen als Faktoren, wie etwa Material zur Fertigung eines Produktes, oder Informationen, bzw. Daten zur Erstellung einer Dienstleistung. Durch die Ausführung der einzelnen Aktivitäten des Prozesses wird auf Basis der Inputs der erforderliche Output erzeugt. Dabei ist zu beachten, dass nicht nur der Prozess als Ganzes, sondern auch die einzelnen Aktivitäten jeweils In- und Outputs besitzen. Diese werden in der Kette der Aktivitäten verarbeitet, bis der finale Output erbracht ist.

2.1.2: Entscheidung – als Aktivität in einem Prozess

Neben dem Begriff des Prozesses spielt auch jener einer Entscheidung in dieser Thesis eine Rolle. Das Konzept einer solchen ist grundsätzlich jedem – ob bewusst oder unbewusst – bekannt. Denn jede Person und jedes Unternehmen, treffen täglich eine Vielzahl von Entscheidungen. Dabei ist einleitend anzumerken, dass sich hinter dem Begriff der Entscheidung ein breites Themengebiet verbirgt. Für gewöhnlich werden Entscheidungen daher eigenständig betrachtet. In dieser Thesis werden sie allerdings, aus Gründen der Vereinfachung, nur im Kontext zu betrieblichen Prozessen betrachtet – und als eine spezielle Form einer Aktivität in einem solchen Prozess verstanden.

Im Grundsatz sind Entscheidungen daher zu beschreiben als eine Auswahl einer Option aus einer Menge von mehreren alternativen Optionen (vgl. Gillenkirch 2018). Sie werden, mit der Absicht spezifische Ziele zu erreichen, analog zu Aktivitäten, entweder von Menschen oder von Maschinen getroffen (vgl. Debevoise und Taylor 2016, S. 9). Dabei hat jede Option in einer Entscheidung konkrete Konsequenzen für einen Prozess.

Exemplarisch kann dies an einem Verkaufsprozess erörtert werden. Eine Entscheidung, welche in einem solchen oft getroffen wird, ist jene ob für einen Kunden eine Bonitätsprüfung durchgeführt werden muss. Die Optionen für diese Entscheidung sind ein „Ja“ und ein „Nein. Ihre Konsequenz ist entweder die Ausführung oder die Unterlassung der Prüfung – sowie resultierend daraus, ein Abbruch des Verkaufs, bei mangelnder Bonität. Insgesamt wird dabei das Ziel verfolgt das Risiko eines Zahlungsausfalles und den Aufwand durch unnötige Prüfungen zu verringern.



Abbildung 2: Grundkonstrukt einer Entscheidung

Eine Entscheidung, wie die eben beschriebene, zeichnet sich dabei dadurch aus, dass ihr Ergebnis situationsabhängig ist. Sie basiert auf konkreten Informationen, welche zu dieser verfügbar sind. Im Falle der Bonitätsprüfung sind dies z.B. Daten zum Warenkorb des Verkaufs und zum Kunden

selbst. Sie wird für gewöhnlich getroffen, indem diese Daten durch eine sog. Entscheidungslogik verarbeitet werden (vgl. OMG 2021, S. 13). Diese wird abgebildet durch eine spezielle Form von Wissen – wie etwa Entscheidungsregeln. Solche Regeln geben an, unter welchen Umständen eine konkrete Option gewählt werden soll. Dadurch ist der Vorgang einer Entscheidungsfindung nicht willkürlich. Er ist logisch begründet und daher für ein Unternehmen jederzeit nachvollziehbar.

2.2: Prozessautomatisierung – als Hauptthematik

Der Begriff der Prozessautomatisierung kann auf verschiedene Weisen ausgelegt werden. Er kann zum Beispiel verstanden werden, als ein Oberbegriff für ein Themengebiet, welches sich – wie es sich aus dem Namen ableiten lässt – mit der Automatisierung von Prozessen, durch den Einsatz von Informationstechnologie beschäftigt. Er kann aber auch definiert werden, als ein spezifischer und systematischer Vorgang, durch welchen diese Automatisierung realisiert wird.

Unabhängig davon, auf welche Art der Begriff gedeutet wird – er sollte nie zu wörtlich genommen werden. Er impliziert nicht, dass alle Schritte in einem Prozess komplett automatisiert ausgeführt werden (vgl. Freund und Rücker 2017, S. 6). Dies hängt primär mit dem Aspekt zusammen, dass selten alle Schritte in einem Prozess automatisierbar sind. Denn zur Automatisierung eignen sich insbesondere Aufgaben, welche repetitiv und monoton sind. Oftmals sind die Aufgaben in einem Prozess allerdings komplex und es wird eine Form von Intelligenz benötigt, um sie auszuführen. Es kann aber auch sein, dass zur Ausführung der Aufgabe verschiedene Systeme genutzt werden müssen. Beide Aspekte machen einige Tätigkeiten daher nur schwer zu automatisieren.

Der Fokus der Prozessautomatisierung liegt daher primär darauf, die Ausführung von Prozessen, durch den Einsatz von IT zu unterstützen. Dadurch soll ein Prozess, sowohl in einer qualitativen als auch einer quantitativen Perspektive optimiert werden. Ein qualitatives Ziel ist dabei z.B. die Erhöhung der Mitarbeiter- und Kundenzufriedenheit. Ein quantitatives Ziel hingegen kann die Reduzierung von Prozesskosten, oder einer Durchlaufzeit sein.

Um diese Ziele zu erreichen, stellt die Automatisierung von konkreten Prozessschritten nur eine von, in der Regel, mehreren Teilaufgaben dar. So wird im Zuge einer Prozessautomatisierung ein Prozess üblicherweise vollständig digital abgebildet. Die Steuerung dieses Prozesses soll durch eine Software übernommen werden. Zudem werden Aktivitäten, die nicht automatisiert werden können, und durch menschliche Akteure ausgeführt werden müssen, bestmöglich durch Software unterstützt. Darüber hinaus sollen Informationen, die im Rahmen der Schritte benötigt werden, in Form von Daten, zwischen den beteiligten Systemen, bzw. Akteuren ausgetauscht werden.

Im Hinblick auf den Begriff der Automatisierung kann also differenziert werden in eine Voll- oder Teilautomatisierung von Prozessen (vgl. Voigt 2018). Ein Prozess gilt dabei als vollautomatisiert, wenn er völlig ohne menschlichen Eingriff abläuft. Eine Teilautomatisierung hingegen liegt vor, sofern nur einzelne Schritte und die Steuerung eines Prozesses automatisiert sind.

Techniken zur Prozessautomatisierung

Im Gesamten kann die Prozessautomatisierung als eine konkrete Form der Softwareentwicklung verstanden werden (vgl. Freund und Rücker 2017, S. 6). Dementsprechend kann sie über diverse Weisen praktiziert werden. Bekannte Techniken zur Durchführung einer Prozessautomatisierung sind dabei unter anderem:

- Individualprogrammierung von Prozessabläufen, in eigenentwickelten Systemen
- Customizing von Prozessen, in Standardsystemen, wie einem ERP-System

- Abbildung von Prozessen, auf Basis von Prozessmodellen, in sog. Process Engines

Der Fokus dieser Thesis liegt, im Hinblick auf diese Techniken zur Prozessautomatisierung, auf der letztgenannten Variante – den Process Engines. Deren Grundidee wird in Kapitel 5 untersucht.

2.3: Business Process Management

Die bisherigen Grundlagen zusammenfassend kann erörtert werden, dass Prozesse eine immense Bedeutung für ein Unternehmen haben. Durch diese werden, in aller Regel, dessen Strategie und Geschäftsmodell operationalisiert (vgl. Amberg et al. 2011, S. 60). Folglich sind Prozesse kritisch zur Erreichung jeglicher strategischer Ziele. Soll also ein konkreter Prozess automatisiert werden, ist es demzufolge nur logisch, dass dabei ein strukturiertes Vorgehen verfolgt werden sollte. Einen Rahmen für ein solches Vorgehen bietet das sogenannte Business Process Management (BPM).

Dieses ist zu deuten als ein Managementkonzept zur Planung, Steuerung und Kontrolle jeglicher Prozesse in einer Organisation (vgl. Becker und Kahn 2012, S. 8). Im Deutschen wird es teils auch als Geschäftsprozessmanagement, oder kurz, nur als Prozessmanagement bezeichnet.

Ziel des BPM ist es, sicherzustellen, dass die Prozesse einer Organisation konstant optimiert, und an der Strategie, sowie den Kunden orientiert sind. Dabei werden sie infolge der zunehmenden Digitalisierung, sowohl aus einer fachlichen als auch aus einer technischen Sicht betrachtet. Der Fokus des Konzepts liegt dabei nicht auf einzelnen Prozessschritten, sondern auf den Prozessen als Ganzes und deren Beziehungen untereinander (vgl. Dumas et al. 2018, S. 1).

Um diese Ziele zu erreichen, folgt das BPM strukturierten Konzepten. Ein solches ist zum Beispiel das durch Gadatsch definierte, in Abbildung 3 visualisierte, Konzept eines integrierten Prozess- und Workflowmanagements (vgl. Gadatsch 2020, S. 19-22). Laut diesem agiert das BPM primär auf einer fachlich-konzeptionellen und einer operativen Ebene. Außerdem weist es Schnittstellen zu weiteren Ebenen auf. Durch Interaktionen zwischen diesen stellt es sicher, dass die Prozesse bestmöglich durch IT unterstützt werden, effektiv in die Unternehmensstruktur eingebettet und an der Strategie, sowie den Kunden ausgerichtet sind (vgl. Amberg et al. 2011, S. 60).

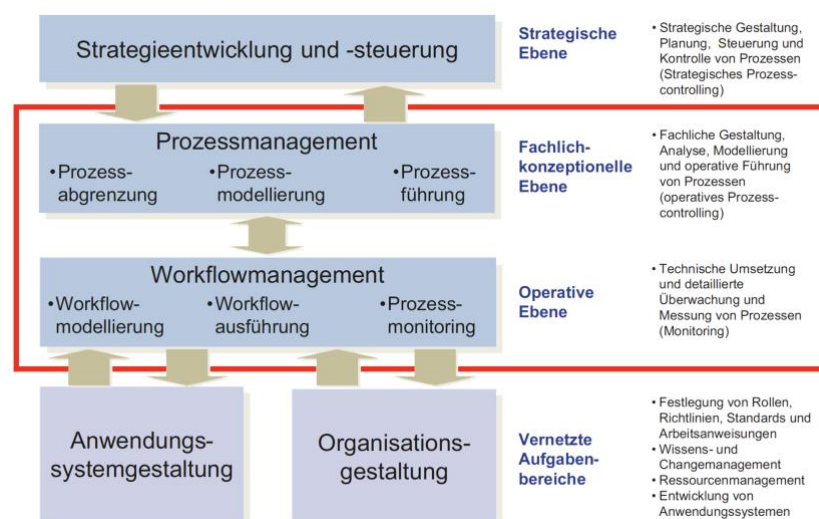


Abbildung 3: Ebenen des BPM und Phasen im zugehörigen Prozesslebenszyklus ¹

¹ Quelle: Gadatsch 2020, S. 20

Nach Freund und Rücker wird das BPM in der Praxis primär für drei Szenarien verwendet: Für die Dokumentation, die Verbesserung und die Einführung von Prozessen (vgl. Freund und Rücker 2017, S. 2). Dabei ist zu erwähnen, dass im Rahmen der beiden letzten Szenarien, stets ein Fokus darauf liegt, dass ein Prozess bestmöglich durch den Einsatz von IT unterstützt wird. Demzufolge nimmt die Prozessautomatisierung also eine wichtige Rolle im BPM ein.

Je nach Szenario umfasst das BPM dabei verschiedenste Teilaufgaben. Zu diesen zählen, gemäß unterschiedlicher Autoren, Aufgaben wie etwa die Analyse, Modellierung, Optimierung, operative Steuerung, oder das Monitoring von Prozessen (vgl. Dumas et al. 2018, S. 6; Weske 2012, S. 5). Sie werden durch systematische Vorgehensmodelle strukturiert. Ein Beispiel für ein solches sind sogenannte Prozesslebenszyklusmodelle. Auf diese wird im Folgenden eingegangen.

2.3.1: Prozesslebenszyklus

Ein Prozesslebenszyklusmodell ist ein Modell, welches jegliche Phasen abbildet, die ein konkreter Prozess während seiner Lebenszeit in einem Unternehmen durchläuft. Diese Zeit beginnt mit der Einführung dieses Prozesses, und endet, wenn er durch einen anderen Prozess abgelöst, oder aus sonstigen Gründen eingestellt wird.

Wie es im letzten Kapitel beschrieben ist, definiert und strukturiert ein solches Modell die Arbeit, die im Zuge eines BPM-Vorhabens ausgeführt werden muss. Folglich stellt es auch ein Vorgehen dar, dass zur Prozessautomatisierung verwendet werden kann.

In der Literatur und Praxis gibt es einige Prozesslebenszyklusmodelle. Diese unterscheiden sich vorwiegend in ihrer Komplexität und Ausrichtung. Eines dieser Modelle, auf welches in der Folge exemplarisch eingegangen wird, ist der sogenannte **Camunda BPM-Kreislauf**.

2.3.2: Camunda BPM-Kreislauf

Der Camunda BPM-Kreislauf ist ein, durch Freund und Rücker formuliertes, in Abb. 4 illustriertes, iteratives Vorgehensmodell zum BPM. Es verfolgt das Ziel, Prozesse kontinuierlich zu verbessern. Das Modell kann auf zwei Weisen ausgelöst werden: Entweder, indem ein existierender Prozess verbessert, respektive dokumentiert, oder ein neuer Prozess eingeführt werden soll (vgl. Freund und Rücker 2017, S. 4). Die einzelnen Phasen des Modells sind dabei – orientiert an der Definition der Autoren – wie folgt zu beschreiben:

Prozesserhebung & -dokumentation

Durch eine Prozesserhebung wird der aktuelle Stand eines bestehenden Prozesses aufgenommen. Eine Erhebung kann auf mehrere Weisen erfolgen – z.B. über Formate, wie Workshops, Interviews, oder Beobachtungen. Deren Resultat ist eine detaillierte Beschreibung des Prozesses. Diese wird im Folgeschritt, der Prozessdokumentation, in ein sogenanntes Ist-Prozessmodell überführt.

Prozessanalyse

Die Prozessanalyse untersucht einen Ist-Prozess eingehend. Der Fokus liegt dabei primär auf der Suche nach Schwachstellen, der Ausrichtung am Kunden und weiteren Optimierungspotentialen. Das Resultat der Phase ist ein Artefakt, in dem alle Erkenntnisse zum Prozess gesammelt sind.

Prozesskonzeption

In der Prozesskonzeption wird ein optimierter Prozess entworfen, welcher am Kunden und an der Strategie einer Organisation orientiert ist. Er wird als Soll-Prozess bezeichnet, und in einem sog. Soll-Prozessmodell dokumentiert. Beim Entwurf von diesem liegt in aller Regel ein Fokus darauf, dass der Prozess optimal durch IT unterstützt wird. Demzufolge wird also in dieser Phase die Prozessautomatisierung geplant. Dabei gibt es genau zwei Auslöser für diese. Sie wird entweder durchgeführt, um einen neuen Prozess von Grund auf zu entwerfen, oder um einen bestehenden Prozess, auf Basis der Ergebnisse aus einer Prozessanalyse, zu verbessern.

Prozessumsetzung

Im Zuge der Prozessumsetzung wird der eben entworfene Soll-Prozess operativ eingeführt. Dazu werden in aller Regel sowohl organisatorische als auch technische Maßnahmen durchgeführt. Zu den Organisatorischen zählen dabei jegliche Maßnahmen zur Kommunikation des Prozesses. Die Technischen dagegen umfassen vorwiegend die technische Abbildung und Automatisierung des Prozesses. Alles in allem wird also in dieser Phase die Prozessautomatisierung, die in der vorigen Phase geplant wird, praktisch realisiert. Das Resultat der Einführung ist sodann ein neuer Ist-Prozess (vgl. Freund und Rücker 2017, S. 4). Durch diesen neuen Ist-Prozess wird gegebenenfalls ein bisher bestehender Ist-Prozess abgelöst.

Prozesscontrolling

Im Rahmen des Prozesscontrollings wird der Betrieb des Ist-Prozesses kontinuierlich überwacht. Dabei werden Daten zu den einzelnen Instanzen des Prozesses gesammelt und ausgewertet. Ziel des Controllings ist es, den Prozess zu steuern und Probleme an diesem zu beheben, respektive zu identifizieren. Werden Probleme gefunden, für deren Lösung eine Änderung am Prozessmodell notwendig ist, so kann die Phase eine neue Iteration im Kreislauf auslösen.

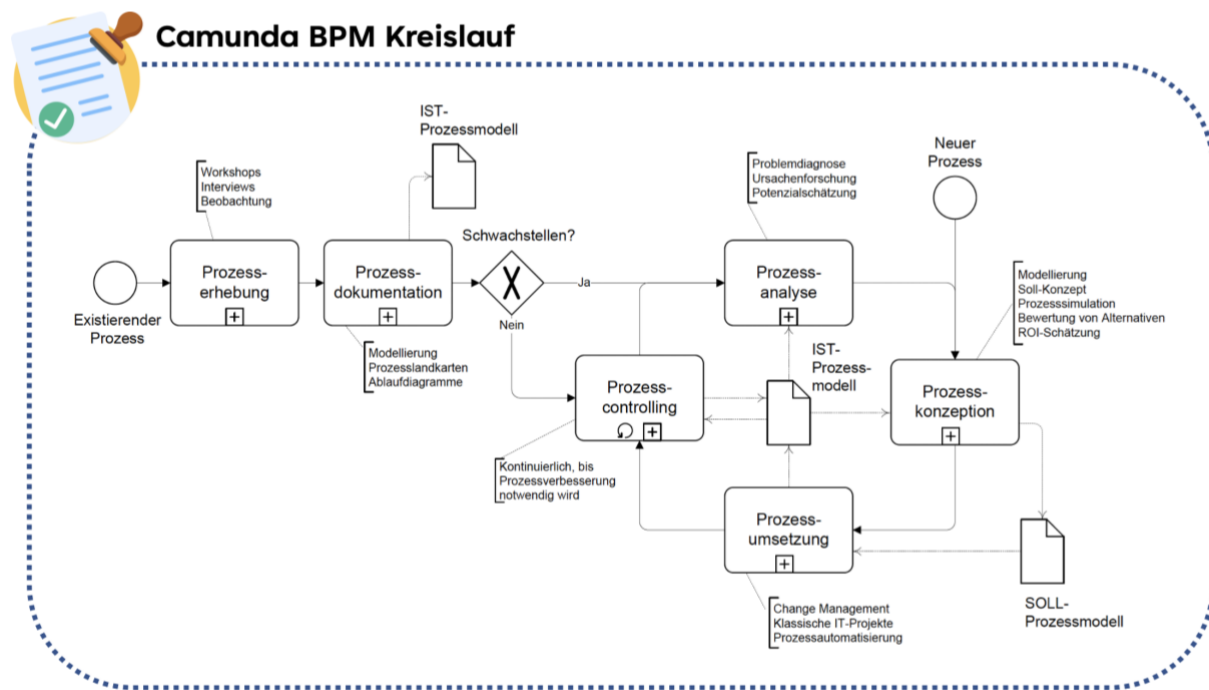


Abbildung 4: Camunda BPM-Kreis als Vorgehensmodell zum BPM ²

² Quelle: Freund und Rücker 2017, S. 3

2.3.3: Prozessmodell

Den BPM-Kreislauf zusammenfassend, kann abgeleitet werden, dass Prozessmodelle eine große Rolle im Management von Prozessen spielen. Sie finden in nahezu jeder Phase im Lebenszyklus eines Prozesses eine Verwendung.

Ein Prozessmodell kann dabei definiert werden als eine statische Beschreibung eines spezifischen Prozesses (vgl. Obermeier 2014, S. 10). Es stellt einen Prozess graphisch dar. Weiterhin beschreibt es diesen in einem konkreten Detailierungsgrad und anhand unterschiedlicher Aspekte. Zu diesen Aspekten können z.B. die einzelnen Aktivitäten, die Regeln und der Ablauf des Prozesses zählen. Ein solches Modell kann aber auch auf weitere Aspekte eingehen, wie etwa auf konkrete Akteure, oder Daten.

Darüber hinaus basiert ein Prozessmodell in der Regel auf einer Modellierungssprache. Dabei ist anzumerken, dass es zur Modellierung von Prozessen diverse solcher Sprachen gibt. Diese werden oft auch als Modellierungsnotationen, oder kurz, nur als Notationen bezeichnet. Ein Beispiel für eine solche Notation ist die BPMN. Sie steht im Hauptfokus dieser Thesis und wird ab Kapitel 5 tiefer erörtert. Um hier jedoch ein Grundverständnis aufzubauen, soll folgendes genannt werden:

2.3.4: Grundbegriffe zu Modellierungsnotationen

Jede Notation, darunter auch die BPMN, stellt eine Reihe von Notationselementen zur Verfügung. Zu diesen zählen für gewöhnlich Elemente, wie Aktivitäten, Start- und Endereignisse. Sie werden eingesetzt, um Prozesse zu modellieren, und somit auf eine konkrete Art zu beschreiben. Darüber hinaus verfügt jede gängige Notation über eine Syntax und Semantik. Ergänzend dazu besitzen einige auch eine sog. Ausführungssemantik. Die drei Begriffe sind dabei wie folgt zu erklären:

Syntax

Die Syntax beschreibt die konkreten Regeln einer Notation (vgl. Freund und Rücker 2017, S. 17). Dies bedeutet, sie geht darauf ein, wie ein Element, z.B. eine Aktivität, graphisch dargestellt wird und wie es, in Kombination mit anderen Elementen, regelkonform eingesetzt werden kann. Ferner definiert sie, welche spezifischen Informationen für das Element hinterlegt werden können. Diese können dabei sowohl von fachlicher Art sein, wie etwa eine Beschreibung des Elements, als auch von technischer Art, wie eine Information, die zur Ausführung in einer Engine relevant ist.

Semantik

Die Semantik ist zu verstehen als die inhaltliche Bedeutung eines Notationselements (vgl. Freund und Rücker 2017, S. 17). Sie definiert, wie ein konkretes Element, etwa eine Aktivität, zu deuten ist – und zwar zum Beispiel als eine Form von Arbeit.

Ausführungssemantik

Die Ausführungssemantik erweitert die eigentliche Semantik, durch eine technische Perspektive. Sie beschreibt, wie spezifische Elemente aus technischer Sicht auszuführen, respektive zu deuten sind (vgl. OMG 2011, S. 425). Folglich ermöglicht sie es, dass Prozessmodelle durch eine Software interpretiert und somit automatisiert, bzw. gesteuert werden können.

3: Grundlagen – zur künstlichen Intelligenz

Dieses Kapitel behandelt die Grundlagen zum Themengebiet der Künstlichen Intelligenz. Es legt einen Fokus auf das Teilgebiet des Machine Learnings. Innerhalb von diesem geht es primär auf das Supervised Learning ein. Die Theorie zu diesem erörtert es anhand eines konkreten Modells: Dem sog. Entscheidungsbaum. Zudem vermittelt das Kapitel die Grundlagen zum Deep Learning und dem PMML-Standard. Insgesamt soll somit ein Grundverständnis zur KI geschaffen werden, um diese im Verlauf der Arbeit im Kontext zur Prozessautomatisierung betrachten zu können.

3.1: Künstliche Intelligenz – als Oberbegriff

Der Begriff der **Künstlichen Intelligenz** (KI) kann nur schwer einheitlich definiert werden. Er kann jedoch verstanden werden als ein interdisziplinäres Themengebiet, welches vor allem im Bereich der Informatik angesiedelt ist (vgl. Kirste und Schürholz 2019, S. 21). Dieses befasst sich damit, menschliches Denken und Lernen über Software abzubilden. Ziel dadurch ist es, Maschinen, bzw. Software eine Form von Intelligenz zu verleihen (vgl. Döbel et al. 2018, S. 8). Durch diese sollen sie befähigt werden, aus Erfahrungen zu lernen, sich ein Wissen anzueignen und hochkomplexe Aufgaben ausführen können, welche bisher nicht, oder nur durch Menschen lösbar waren.

Innerhalb des Themengebietes der KI gibt es viele Teilgebiete. Unter diesen stehen gegenwärtig vor allem das **Machine Learning** und das **Deep Learning** im Fokus von Forschung und Wirtschaft.

Im Zuge dieser Thesis wird die KI ausschließlich mit Fokus auf diese beiden Gebiete betrachtet. Sie bilden die Grundlage für die meisten IT-Systeme, respektive Algorithmen, welche als moderne KI interpretiert, und zunehmend in der Praxis verwendet werden (vgl. Kersting et al. 2019, S. 8). Daher wird in den folgenden Unterkapiteln genauer auf diese beiden Gebiete eingegangen.

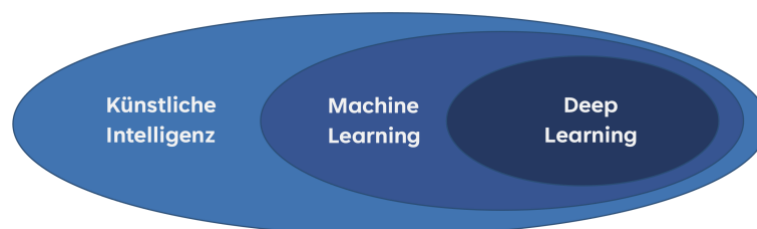


Abbildung 5: Zusammenhang zwischen KI, Machine Learning und Deep Learning

3.2: Machine Learning – als Teilgebiet der KI

Das **Machine Learning** (ML) ist ein Teilgebiet der Künstlichen Intelligenz. Es befasst sich damit, lernfähige Systeme zu entwickeln, die für konkrete Problemstellungen eingesetzt werden können. Dazu setzt das ML spezifische Algorithmen ein, welche sich auf Grundlage von Daten, konkretes Wissen aneignen und dieses zur Lösung der Probleme nutzen (vgl. Döbel et al. 2018, S. 9).

Kontext zur klassischen Programmierung

In diesem Hinblick kann das ML als ein Gegenentwurf zur klassischen Programmierung gedeutet werden. In Letzterer wird die Logik zur Lösung eines Problems stets manuell programmiert. Dazu muss diese Logik eindeutig beschrieben werden, und ist nach der Implementierung statisch. Dies bedeutet, sie kann nur durch weitere manuelle Arbeit angepasst werden.

Diese Aspekte bergen einige Herausforderungen, da sich die Logik zur Lösung eines Problems im Laufe der Zeit ändern kann, oder sich oft nicht eindeutig, bzw. vollständig beschreiben lässt. Dies kann daran liegen, dass ein jeweiliges Problem von sehr vielen Parametern abhängig ist, es viele Varianten gibt, diese sehr individuell, oder zum Teil noch nicht bekannt sind. Als Beispiel für ein solches Problem kann, wie von Kersting et al. 2019, ein Spam-Filter in einem E-Mail-Programm genannt werden. Dieser soll Spam-Mails identifizieren – z.B. anhand von deren Inhalt. Insgesamt gibt es jedoch extrem viele und regelmäßig neue Formen von Spam-Mails, an welche die Lösung angepasst werden muss (vgl. Kersting et al. 2019, S. 8). Dies kann durch klassische Entwicklung kaum, bzw. wenn überhaupt, nur mit viel Aufwand und hoher Komplexität umgesetzt werden.

Grundidee des ML im Kontext zur klassischen Programmierung

Das Machine Learning hingegen, respektive deren Algorithmen, können diese Probleme lösen. Sie analysieren Daten, um in diesen konkrete Muster zu identifizieren – z.B. wann eine Mail als Spam gilt. Ziel ist es, auf Basis dieser Muster, allgemeingültige Modelle zu erlernen, um diese auf eine konkrete Problemstellung, wie das bereits genannte Erkennen der Spam-Mails anzuwenden.

3.2.1: Daten im Kontext des Machine Learnings

Die Grundzutat, welche benötigt wird, um ein allgemeingültiges ML-Modell zu erlernen, sind also immer Daten. Diese müssen in einer möglichst aussagekräftigen Form vorliegen. Dies bedeutet, sie müssen zum einen in einer großen Menge vorhanden sein, sowie zum anderen möglichst viele unterschiedliche Fälle abdecken. Dabei muss berücksichtigt werden, dass Daten grundsätzlich in zwei Arten unterteilt werden können: In strukturierte- und unstrukturierte Daten (vgl. Herrmann 2021, S. 32). Diese beiden Arten werden in der Folge kurz erörtert – da je nach Art nur bestimmte ML-Algorithmen zur Verarbeitung jener Daten geeignet sind:

Strukturierte Daten sind Daten, bzw. Datensätze, die gemäß einem klar definierten Datenmodell formatiert und gespeichert werden. Ein Beispiel für ein solches Modell ist eine Tabelle in einer Datenbank, die eine feste Anzahl von Spalten umfasst (vgl. Müller und Lenz 2013, S. 78).

Unstrukturierte Daten hingegen werden in ihrer jeweiligen Rohform gespeichert. Sie werden nicht gemäß einem definierten Datenmodell umgewandelt. Beispiele für solche Daten sind etwa Bild- und Audiodateien, oder der Inhalt von Textdateien, wie einer PDF (vgl. Döbel et al. 2018, S. 24).

3.2.2: Lernstrategien des Machine Learnings

Zusammenfassend kann Machine Learning also interpretiert werden, als ein Oberbegriff für eine Reihe von Algorithmen, die eine Software dazu befähigen, auf Basis von Daten, konkrete Modelle zu erlernen, um mit diesen spezifische Problemstellungen zu lösen.

Die Algorithmen können dabei in verschiedene Kategorien unterteilt werden. Diese werden häufig als sogenannte Lernstrategien bezeichnet. Diese Strategien unterscheiden sich primär durch zwei Aspekte voneinander. Zum einen dadurch, wie aus den Daten ein Modell gebildet wird – und zum anderen dadurch, für welche Zwecke sie eingesetzt werden können (vgl. Döbel et al. 2018, S. 25).

Die drei bekanntesten dieser Lernstrategien sind dabei das **Supervised-**, das **Unsupervised-** und das **Reinforcement Learning**. Die beiden letzten sind im Rahmen dieser Arbeit jedoch nicht weiter relevant. Für zusätzliche Informationen zu diesen wird daher auf Quellen, wie „Döbel et al. 2018, S. 25ff“ verwiesen – und in der Folge nur auf das **Supervised Learning** eingegangen.

3.2.3: Supervised Learning

Das **Supervised Learning** (SL) ist eine Lernstrategie, mittels der ein ML-Modell für eine konkrete Problemstellung trainiert wird – und zwar mit Daten, die als Beispiele zur Lösung dieses Problems vorbereitet sind. Diese sind meistens historische Daten zur Problemstellung. Sie bestehen aus einzelnen Datensätzen, die jeweils eine Menge von Eingabewerten und ein zugehöriges Ergebnis besitzen. Das Letztere wird dabei für gewöhnlich als Zielvariable bezeichnet. Ein SL-Modell soll im Hinblick auf diese Variable trainiert werden.

Ziel der Algorithmen ist es, auf Basis der Beispiel-Datensätze ein Muster zu identifizieren, durch welches – in Abhängigkeit zu konkreten Eingabedaten – der Wert einer Zielvariable vorhergesagt werden kann (vgl. Welsch et al. 2018, S. 371). Folglich ist ein Supervised Learning Modell als ein Vorhersagemodell zu interpretieren. Dieses soll auf bisher unbekannte Situationen angewendet werden können. In diesem Kontext ist es vor allem für zwei Szenarien geeignet. Zu diesen zählen sog. Regressions- und Klassifikationsaufgaben (vgl. Döbel et al. 2018, S. 26). Diese beiden Typen von Aufgaben sind im Allgemeinen, bzw. anhand von Abbildung 2, wie folgt zu beschreiben:

Eine **Regressionsaufgabe** bezweckt die Prognose eines spezifischen Wertes. Dieser ist numerisch, respektive kontinuierlich. Ein Beispiel für eine Regressionsaufgabe ist die Prognose des Umsatzes eines Eisverkäufers, in Abhängigkeit eines aktuellen Wochentages und der Temperatur.

Eine **Klassifikationsaufgabe** hingegen zielt darauf ab, einzelne Datensätze in konkrete, logisch-abgegrenzte Klassen einzuordnen. Ihr Ergebnis ist daher kategorisch. Ein Beispiel für eine solche Aufgabe ist eine Kategorisierung von Bankkunden, die einen Dispokredit angefragt haben. Diese können, unter Berücksichtigung von Informationen, zu ihrem Schufa-Status, sowie Einkommen in kreditwürdige und nicht kreditwürdige Kunden unterteilt werden.

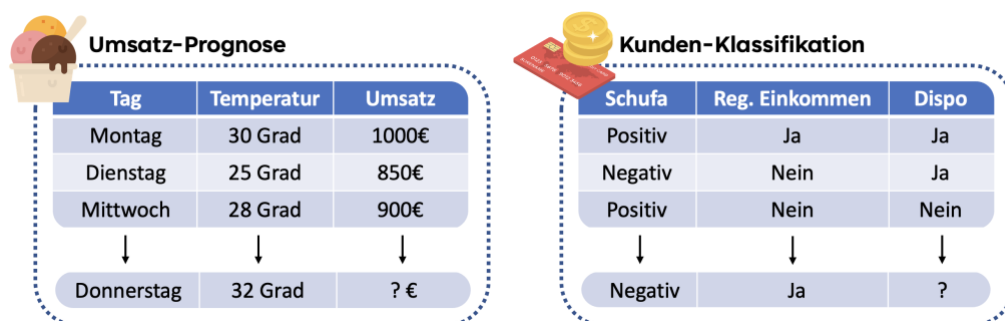


Abbildung 6: Beispiele für lösbare Problemstellungen im Supervised Learning

3.2.4: Training von Supervised Learning Modellen

Zusammengefasst verfolgt das Supervised Learning also das Ziel konkrete Vorhersagen für bisher unbekannte Situationen zu treffen. Diese sind aus technischer Sicht zu verstehen, als Datensätze, welche nicht in den Beispieldaten vorhanden sind. Folglich wurden sie nicht zum initialen Training eines Modells verwendet.

Um im Hinblick auf diese Situationen dennoch gute Vorhersagen, mit möglichst wenigen Fehlern zu treffen, muss ein Modell allgemeingültig sein – d.h. es darf die Beispiele nicht nur auswendig lernen, sondern muss diese abstrahieren. Folglich sollte ein Modell auf diese Eigenschaft geprüft werden, bevor es produktiv genutzt wird. Daher ist zum Training dieser Modelle ein strukturiertes Vorgehen von Nöten. Ein solches kann wie das in Abb. 7 dargestellte Vorgehen aufgebaut sein.

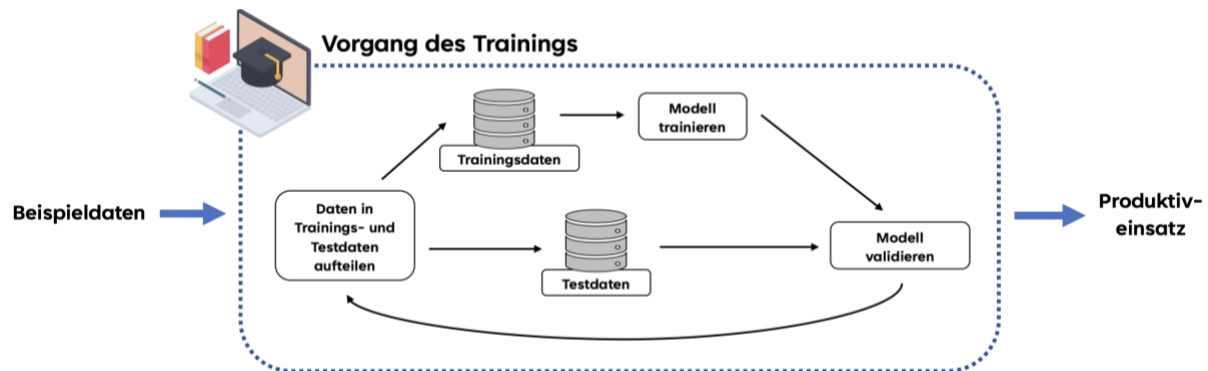


Abbildung 7: Schritte zur Erstellung eines Supervised Learning Modells

Gemäß diesem werden die Datensätze, mittels welchen ein SL-Algorithmus ein Modell erlernen soll immer in zwei alleinstehende Datenpools aufgeteilt: In sogenannte Trainings- und Testdaten. Die Einordnung der Datensätze in die Pools ist dabei zufällig. Nicht zufällig ist jedoch die Menge der Daten in diesen. Zwar gibt es keine festen Regeln – in der Praxis werden die Daten allerdings meist in 70% Trainings- und 30% Testdaten unterteilt (vgl. Kersting et al. 2019, S. 24).

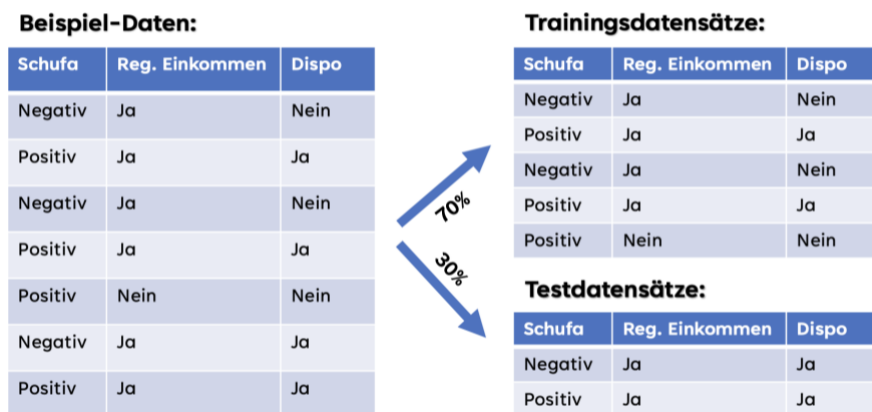


Abbildung 8: Aufteilung von Beispieldaten in Trainings- und Testdaten

Die Trainingsdaten sind dabei als jene Daten zu verstehen, aus welchen ein ML-Algorithmus ein Modell erlernt. Die Testdaten hingegen, sind Daten, mittels welchen das erlernte Modell auf seine Vorhersage-Qualität getestet wird. Zeigt sich in der Validierung des Modells, dass dieses nicht geeignet ist, um die Ergebnisse der Testdaten vorherzusagen, so kann dies dazu führen, dass der Lernvorgang von neuem beginnt – z.B. mit mehr Daten, oder einer anderen Aufteilung dieser.

Um diesen Vorgang exemplarisch zu veranschaulichen, kann auf die Klassifikationsaufgabe aus Abb. 6 zurückgegriffen werden. Eine Menge von Daten zu dieser könnte, wie in Abb. 8 dargestellt in Trainings- und Testdaten unterteilt werden. Auf Basis dieser Daten kann sodann ein konkretes ML-Modell erstellt werden – bspw. ein sogenannter Entscheidungsbaum.

3.2.5: Entscheidungsbaum

Ein Entscheidungsbaum ist ein Modell aus der Familie des SL. Es wird im Rahmen dieser Thesis praktisch eingesetzt. Mit einem solchen Baum lassen sich – wie es sich aus dem Namen ableiten lässt – konkrete Entscheidungen treffen (vgl. Russel und Norvig 2012, S. 814). Ergo wird er primär für Klassifikationen eingesetzt. In der Theorie ist er aber auch für Regressionsaufgaben geeignet.

3.2.5.1: Aufbau eines Entscheidungsbaums

Im Kern ist ein Entscheidungsbaum zu deuten als ein Graph, welcher aus einer Menge von Knoten und Kanten besteht. Dieser verfolgt das Ziel, mittels logischen Abfolgen von Entscheidungen, den Wert einer Zielvariable vorherzusagen. Faktisch könnte ein solcher Baum also eingesetzt werden, um vorherzusagen, ob ein konkreter Kunde einer Bank einen Dispokredit erhalten soll, oder nicht. Ein exemplarischer Baum zu dieser Problemstellung ist in Abbildung 9 dargestellt.¹

Abgeleitet aus diesem, ist jeder Knoten in einem solchen Baum als eine spezifische Entscheidung zu deuten. An jeder von diesen wird der Wert eines konkreten Attributes abgefragt. Jeder Knoten besitzt dabei eine Menge von Kanten, welche die unterschiedlichen Optionen einer Entscheidung darstellen. Diese werden repräsentiert, durch die möglichen Werte des zu prüfenden Attributes.

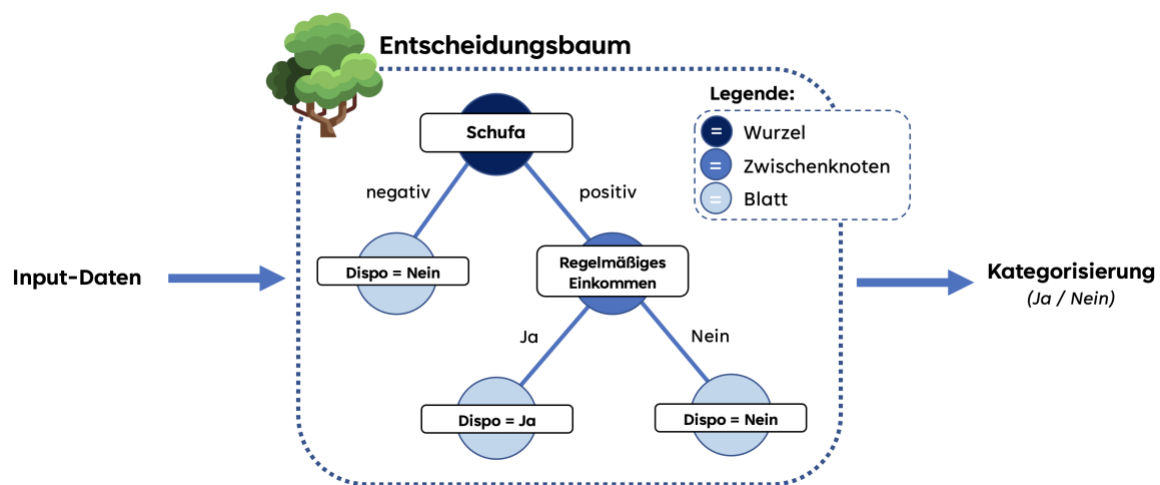


Abbildung 9: Exemplarischer Entscheidungsbaum zur Kategorisierung von Bankkunden

Insgesamt sind die Knoten dabei in einer Hierarchie angeordnet. Sie verzweigen sich, ausgehend von einem sog. Wurzelknoten, über mehrere Entscheidungsebenen hinweg immer weiter. Deshalb ist das Modell in metaphorischer Hinsicht mit den Zweigen eines Baumes vergleichbar – wodurch sich auch sein Name ableitet (vgl. Kersting et al. 2019, S. 113).

Außerdem bilden sich durch diese Hierarchie eine Menge von Entscheidungspfaden. Diese geben jeweils an, unter welchen Umständen ein konkretes Ergebnis eintritt. Zudem sind die Pfade in der Regel eindeutig (vgl. Müller und Lenz 2013, S. 102). Das heißt, sie schließen sich gegenseitig aus. Angewendet auf den exemplarischen Baum in Abbildung 9 gibt es drei solcher Pfade.

3.2.5.2: Funktionsweise eines Entscheidungsbaums

Um eine konkrete Vorhersage zu treffen, muss ein geeigneter Datensatz an das Modell übergeben werden (siehe Abb. 10). Ziel ist es, auf Basis von diesem, einen konkreten Pfad im Baum zu finden, durch welchen ein Ergebnis zur Problemstellung abgeleitet werden kann.

Um dies zu erreichen, wird der Baum mittels diesem Datensatz – beginnend an der Wurzel – von oben nach unten durchlaufen (vgl. Morik 2018, S. 22). Dabei wird, wie im vorherigen Unterkapitel angeführt, an jedem aktiven Knoten, der Wert eines konkreten Attributes abgefragt. Je nach Wert des Attributes wird einer zutreffenden Kante zum nächsten Knoten gefolgt. Dieser Vorgang wird

¹ Für Grundlagen zur Graphentheorie, bzw. Knoten und Kanten, siehe: Domschke et al. 2015, S. 71ff

so lange wiederholt, bis ein Knoten erreicht wird, der keine Folgeknoten besitzt. Ein solcher wird in aller Regel als Blatt bezeichnet (vgl. Kersting et al. 2019, S. 115). Aus diesem kann das Ergebnis des Baumes – im Falle von Abbildung 10 eine Kategorisierung – abgelesen werden.

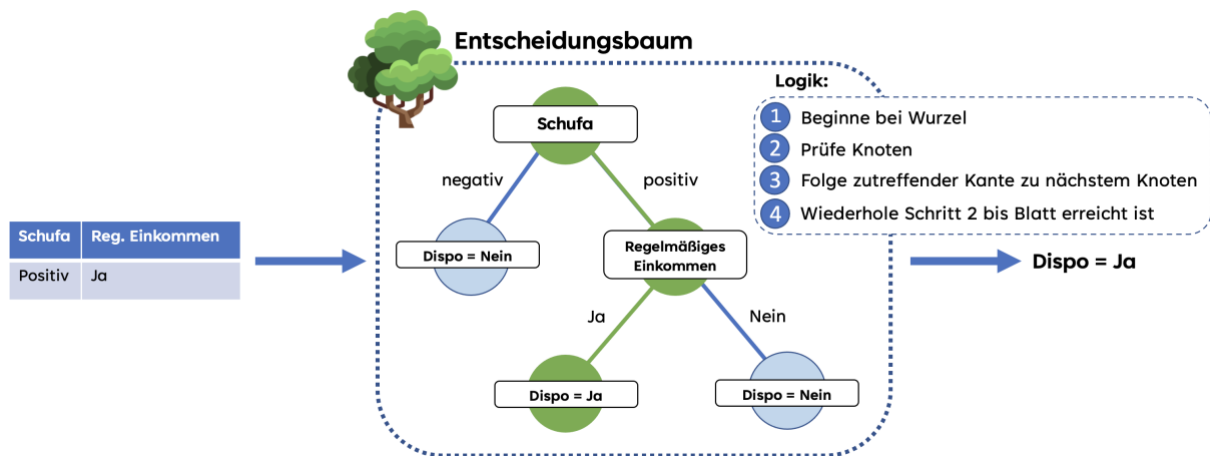


Abbildung 10: Funktionsweise eines Entscheidungsbaums

3.2.5.3: Training eines Entscheidungsbaums

Entsprechend der Grundidee des SL wird ein Entscheidungsbaum, bzw. werden die Regeln eines solchen, durch konkrete Algorithmen, aus einer Menge von Trainingsdaten erlernt. Das Ziel dieser Algorithmen ist es, die Daten – im Hinblick auf die Zielvariable – in möglichst homogene und in der Größe aussagekräftige Gruppen zu unterteilen (vgl. Müller und Lenz 2013, S. 103). Dabei soll jede dieser Gruppen einen konkreten Pfad im Entscheidungsbaum darstellen (siehe Kap. 2.2.3.1).

Um dies zu erreichen, folgen die Algorithmen einem simplen Grundkonzept: Sie teilen die Daten, anhand von konkreten Attributen, mit einem rekursiven Vorgehen, in immer kleinere Teilmengen auf (vgl. Géron 2018, S. 171). Der Startpunkt ist dabei stets der Wurzelknoten.

An diesem wird für jedes einzelne Attribut in den Trainingsdaten berechnet, in welche Teilmengen dieses die Daten – im Hinblick auf die Zielvariable – zerlegen kann (siehe Abb. 11). Jenes Attribut, dass die Mengen erstellt, die am homogensten sind, wird dann zur Teilung genutzt. Am Beispiel von Abbildung 11 ist dies das Attribut “Schufa”. Mittels diesem können die Daten in zwei neue Teilmengen aufgeteilt werden.

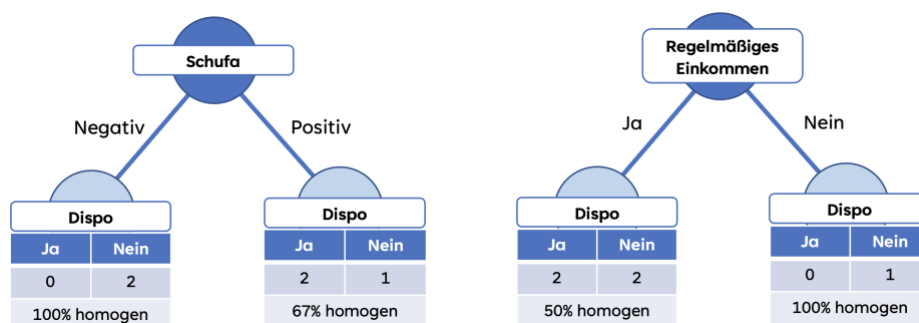


Abbildung 11: Training eines Entscheidungsbaums durch Extraktion von Zusammenhängen

Jede dieser Teilmengen stellt dabei einen neuen Knoten im Modell dar. Für jeden von diesen wird der oben beschriebene Vorgang rekursiv wiederholt – und zwar so lange, bis keine noch kleineren

Teilmengen mehr gebildet werden können. Insgesamt werden also, über mehrere Ebenen hinweg, immer weitere Knoten generiert, bis der Baum, wie jener in Abb. 12, nicht mehr erweiterbar ist.

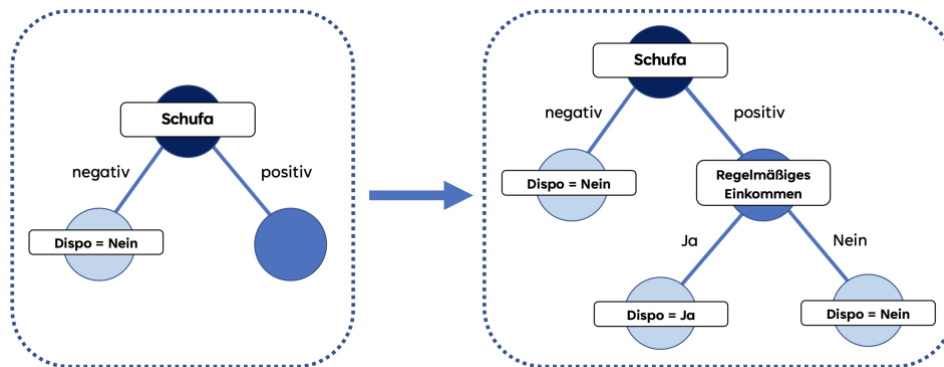


Abbildung 12: Schritte zur Erstellung eines exemplarischen Entscheidungsbaumes

3.2.6: Validierung von Supervised Learning Modellen

Liegt nach dem Training ein einsatzbereites ML-Modell vor, so sollte dieses – wie in Kapitel 3.2.4 erörtert – bezüglich seiner Qualität überprüft werden. Um dies zu ermöglichen, wird das Modell mit jedem Datensatz aus den Testdaten durchlaufen. Dadurch entsteht eine neue Datenbasis, in welcher für jeden Testdatensatz eine spezifische Vorhersage vorliegt (siehe Abb. 13).

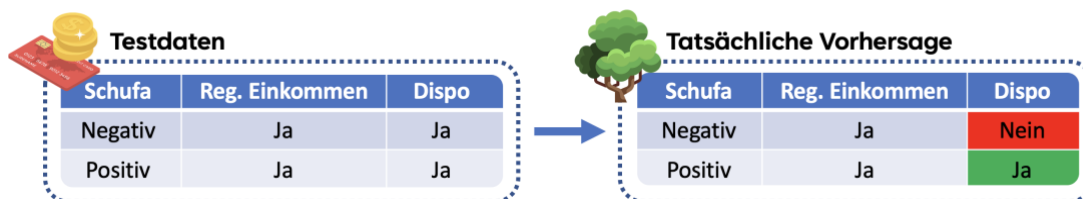


Abbildung 13: Überprüfung des Entscheidungsbaumes mittels Testdaten

Auf Basis dieser Daten können verschiedenste Auswertungen erstellt werden. Diese ermöglichen es, die Qualität der Modelle quantitativ zu bewerten. Für das Szenario der Klassifikation könnte z.B. eine sogenannte Konfusionsmatrix erstellt werden. Diese stellt die tatsächlichen Werte einer Zielvariable mit den vom Modell vorhergesagten Werten in eine Beziehung (siehe Abb. 14). Damit ermöglicht sie es, unterschiedliche Kennzahlen zu berechnen. Eine von diesen ist beispielsweise die Genauigkeit des Modells. Diese liegt – für die Testdaten im vorliegenden Beispiel – bei 50%.

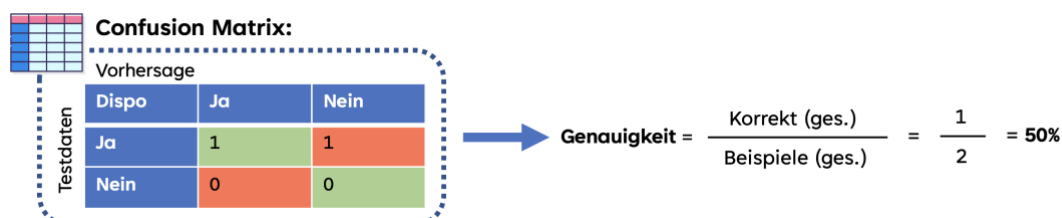


Abbildung 14: Berechnung der Modellqualität mittels einer Konfusionsmatrix

Für eine Zielvariable, welche nur zwei Werte besitzt, ist dies sehr schlecht. Faktisch könnte eine Klasse genauso gut per Zufall gewählt werden – und die Genauigkeit würde ebenfalls im Bereich um 50% liegen. Dementsprechend kann aus der Kennzahl abgeleitet werden, dass der vorliegende

Baum nicht zur Klassifikation geeignet ist – bzw. dass die Daten nicht zum Lernen geeignet sind. Daher sollten diese auf ihre Eignung geprüft werden. Dabei sollte evaluiert werden, inwiefern die Daten mit der Zielvariable korrelieren und aus fachlicher Sicht geeignet sind, um das Problem zu lösen. Je nach Resultat muss der Baum sodann mit neuen, oder mehr Daten trainiert werden.

Insgesamt zeigt sich dadurch exemplarisch, dass ein ML-Modell viele aussagekräftige Trainings- und Testdaten benötigt, um ein hochwertiges Vorhersagemodell zu erlernen. Im hier vorliegenden Beispiel wird jedoch bewusst darauf verzichtet, um eine Anschaulichkeit zu gewährleisten.

3.2.7: Deep Learning

Ein weiteres Teilgebiet der KI ist das **Deep Learning**. Dieses ist zu deuten als eine spezielle Form des Machine Learnings, in welcher sog. künstliche neuronale Netze (KNN) eingesetzt werden (vgl. Greplova und Fischer 2020, S. 2). Im Allgemeinen gilt sie als die performanteste, bzw. modernste Art des maschinellen Lernens (vgl. Kersting und Tresp 2019, S. 32). Faktisch ist das Deep Learning daher einer der Hauptgründe für den gegenwärtigen Boom der KI. Es kann für eine Vielzahl von Szenarien eingesetzt werden. Dabei ist es sehr häufig die technische Grundlage für moderne KI-Lösungen – auch für jene, die im Rahmen dieser Arbeit eingesetzt werden.

Im Allgemeinen gibt es jedoch viele Algorithmen, bzw. Formen von neuronalen Netzen. Diese sind sehr komplex und nur bedingt relevant für das Verständnis dieser Arbeit. Daher wird in der Folge nur auf die Grundidee, sowie den Aufbau dieser Netze, und nicht auf deren mögliche Formen und technische Details eingegangen. Für ein tieferes Verständnis zum Themengebiet wird jedoch auf Quellen wie „Döbel et al. 2018, S. 36ff“ und „Kersting et al. 2019, S. 149ff“ verwiesen.

3.2.7.1: Grundidee von künstlichen neuronalen Netzen

Die Grundidee von künstlichen neuronalen Netzen ist es, die Funktionsweise eines menschlichen Gehirns zu imitieren (vgl. Döbel et al. 2018, S. 34). Dieses besteht u.a. aus einer riesigen Menge von miteinander verbundenen Neuronen, welche Reize, bzw. Informationen verarbeiten – mit der Absicht, konkrete Ergebnisse zu erzielen (vgl. Kersting et al. 2019, S. 151–152). Ein KNN soll diese Struktur, mittels Algorithmen, technisch abbilden. Damit soll es – analog zum ordinären Machine Learning – für spezielle Problemstellungen trainiert und eingesetzt werden können.

3.2.7.2: Aufbau eines künstlichen neuronalen Netzes

Orientiert an dem eben beschriebenen Aufbau besteht ein künstliches neuronales Netz (KNN) aus einer Menge von Knoten, welche für gewöhnlich auch als künstliche Neuronen bezeichnet werden. Diese sind – wie in Abbildung 15 exemplarisch dargestellt – über sogenannte Kanten miteinander verbunden. Die Kanten versorgen ein Neuron mit dem Output, bzw. Daten eines anderen Neurons.

Jedes der Neuronen ist dabei als eine konkrete Recheneinheit des Netzwerks, respektive als eine mathematische Formel zu verstehen. Es nimmt über die jeweiligen Kanten konkrete Input-Daten entgegen und berechnet mittels diesen einen Output. Dabei ist zu berücksichtigen, dass jede der Kanten eine Gewichtung besitzt. Diese gibt an, welche Bedeutung eine Verbindung zwischen zwei Neuronen hat. Je stärker sie ist, desto größer ist der Einfluss, den ein Neuron auf den Output eines anderen Neurons hat. Im Rahmen des Trainings eines KNN werden daher diese Gewichtungen so

lange angepasst, bis im Hinblick auf das gesamte Netz, ein möglichst optimales Ergebnis, bzw. eine hohe Genauigkeit in den Vorhersagen erzielt wird (vgl. Döbel et al. 2018, S. 39).

Wie in Abb. 15 sichtbar, sind die Neuronen dabei im Allgemeinen in unterschiedlichen Schichten angeordnet. Insgesamt können diese in drei Arten unterteilt werden – in Eingabe-, Ausgabe- und versteckte Schichten (vgl. Buxmann und Schmidt 2019, S. 14).

Die **Eingabeschicht** ist dabei stets die erste Schicht eines KNN. Sie umfasst Neuronen, welche die Daten entgegennehmen, die vom Netz zur Lösung einer Problemstellung verarbeitet werden (vgl. Kersting et al. 2019, S. 159). Auf diese folgt immer eine sogenannte **versteckte Schicht**. Ein KNN kann unzählige dieser Schichten besitzen. In diesen findet die eigentliche Verarbeitung der Daten statt. Dazu werden diese – wie oben angedeutet – unter Berücksichtigung der Gewichtungen der Kanten verarbeitet und über die Schichten hinweg weitergegeben. Die **Ausgabeschicht** hingegen ist immer die letzte Schicht. Diese umfasst Neuronen, die mit der äußersten versteckten Schicht verbunden sind und folglich das Ergebnis zur Problemstellung bereitstellen. Ein solches kann z.B. eine Klassifikation, oder eine Wahrscheinlichkeitsmatrix sein.

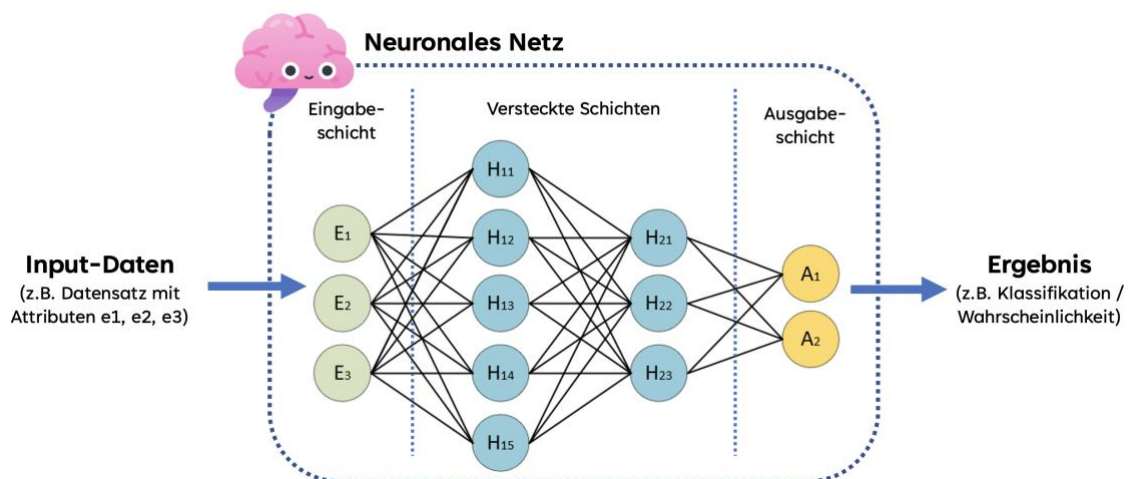


Abbildung 15: Grundlegender Aufbau eines künstlichen neuronalen Netzes ²

3.2.7.3: Der Weg vom neuronalen Netz zum Deep Learning

Zusammenfassend zeichnen sich künstliche neuronale Netze also durch eine hohe Flexibilität aus. Sie können aus einer frei wählbaren Zahl von versteckten Schichten und Neuronen bestehen. Dies ermöglicht es, dass nahezu jegliche Formen von mathematisch abbildbaren Problemstellungen durch KNN gelöst werden können. Hierbei ist immer dann von **Deep Learning** die Rede, wenn ein Netz aus mehreren versteckten Schichten besteht (vgl. Morik 2018, S. 23).

Insgesamt bieten KNN, bzw. das Deep Learning also einen wesentlich höheren Funktionsumfang als die traditionellen Algorithmen des ML. Dabei gilt generell: Je komplexer ein Problem ist, dass mit einem neuronalen Netz gelöst werden soll, desto mehr versteckte Schichten benötigt dieses.

Daneben hat sich gezeigt, dass KNN sowohl bei der Verarbeitung von strukturierten als auch von unstrukturierten Daten sehr gute Ergebnisse liefern können (vgl. Morik 2018, S. 23). Bisherige SL-Modelle hingegen sind primär für Szenarien geeignet, in welchen strukturierte Daten verarbeitet werden. Daher werden KNN oft in Anwendungsszenarien eingesetzt, die sich mit unstrukturierten Daten befassen. Zu diesen zählen z.B. die Klassifikation von Bildern, die Übersetzung von Texten,

² Quelle: Windisch 2021

die Spracherkennung, oder die Extraktion von Informationen aus Dokumenten. Für diese sind die klassischen Modelle des ML, wie z.B. ein Entscheidungsbaum kaum geeignet.

3.3: PMML als Standard

Die **Predictive Model Markup Language** – kurz PMML – ist ein herstellerunabhängiger Standard, mittels dem Machine Learning Modelle in einem spezifischen Format abgebildet werden können. Faktisch ermöglicht er eine einheitliche Interpretation von ML-Modellen. Ziel dadurch ist es, dass diese Modelle nahtlos zwischen verschiedensten Tools ausgetauscht, und durch diese, für jeweils bestimmte Zwecke, eingesetzt werden können (vgl. Streibich und Zeller 2019, S. 112).

Auf Basis von diesem Standard ist es beispielsweise denkbar, dass ein konkretes ML-Modell, wie etwa der Entscheidungsbaum aus Kap. 3.2.5.1, mit einem bestimmten Tool erlernt, in das PMML-Format exportiert, und folgend darauf, in ein anderes Tool importiert wird. Durch dieses andere Tool wiederum kann das Modell z.B. technisch implementiert, bzw. produktiv eingesetzt werden.

Um dieses Ziel zu erreichen, bildet die PMML ein ML-Modell in einem spezifischen XML-Format, bzw. in einer XML-Datei ab (vgl. Baars und Kemper 2021, S. 309). Diese Datei wird für gewöhnlich als PMML-Modell bezeichnet. Sie besteht aus einer Menge von Komponenten (siehe Abb. 16). Jede von diesen wird durch konkrete XML-Tags repräsentiert. Je nach Art des vorliegenden ML-Modells können einige dieser Tags variieren – vor allem jene zum eigentlichen ML-Modell.

Mittels welchen XML-Tags dieses abgebildet werden kann, ist für das Verständnis jedoch vorerst irrelevant. Anzumerken ist allerdings, dass die PMML eine Vielzahl von Modelltypen aus dem ML unterstützt (siehe Streibich und Zeller 2019, S. 113). Beispiele für diese Typen sind unter anderem die vorher erörterten Entscheidungsbäume, sowie neuronale Netze.

Einige andere XML-Tags hingegen existieren immer. Zu diesen zählen bspw. das **Data Dictionary** und das **Mining Schema**. Ersteres enthält dabei generelle Informationen, zu allen Attributen, die vom Modell verwendet werden – etwa zu deren Namen, sowie Datentypen. Das Mining Schema hingegen erweitert das Data Dictionary. Es beinhaltet Informationen, die einen speziellen Bezug zum Typen des Modells haben. Sollen mit diesem etwa Klassifikationen vorhergesagt werden, so gibt das Schema z.B. für jedes Attribut an, ob dieses ein Input- oder ein Output-Attribut ist.



```
<?xml version="1.0"?>
<PMML version="4.4"
  xmlns="http://www.dmg.org/PMML-4_4"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">

  ... Metadaten ...

  <DataDictionary> ... </DataDictionary>
  <MiningSchema> ... </MiningSchema>

  ... ML-Modell (Tag abhängig vom Typ - z.B.) ...
  <TreeModel> ... </TreeModel>

</PMML>
```

Abbildung 16: Basisstruktur eines PMML-Modells

4: Fusion – von Prozessen und KI

Im folgenden Kapitel werden die Grundlagen zu Prozessen und der KI kombiniert. Es wird erörtert, für welche Zwecke die KI im Kontext zu Prozessen eingesetzt werden kann. Dabei wird der Fokus auf den Bereich der Prozessautomatisierung gelegt. Außerdem wird untersucht, welche Arten von KI-Lösungen es gibt und welche Herausforderungen beim Einsatz von diesen auftreten können.

4.1: Möglichkeiten zum Einsatz von KI in Unternehmen

Im Rückblick auf die Einleitung werden der KI, in der öffentlichen Debatte, sehr große Potentiale zugerechnet. Diese reichen von der Entwicklung von Produkten und Dienstleistungen, bis hin zur Schaffung komplett neuartiger Geschäftsmodelle. Doch auch abseits dieser Debatte werden sich durch den Einsatz von KI einige Chancen erhofft. So sehen viele Unternehmen in der Technologie vor allem eine Möglichkeit, um ihre Prozesse zu optimieren (vgl. Reder 2021, S. 19).

Mit Blick auf dieses Ziel können Unternehmen die KI auf verschiedenen Ebenen einsetzen. Je nach Ebene stehen dabei signifikant unterschiedliche Aufgaben, sowie Ziele im Fokus. Insgesamt kann der Einsatz allerdings in zwei Arten unterschieden werden: In einen Einsatz auf einer operativen Ebene und einen Einsatz auf einer Managementebene.

Einsatz auf Managementebene

Auf einer Managementebene werden KI-Lösungen genutzt, um die Prozesse eines Unternehmens, im Hinblick auf konkrete Problemstellungen, zu analysieren, bzw. zu überwachen. Folglich haben die Lösungen im Kontext zu einem Prozess eine passive Rolle. Dies bedeutet, sie tragen nicht zur Durchführung von Prozessen bei. Stattdessen werden die Lösungen in den verschiedenen Phasen, im Lebenszyklus eines Prozesses eingesetzt (siehe Kap. 2.3.2). Im Rahmen von diesen erfüllen sie jeweils konkrete Aufgaben. Durch diese tragen sie zum Management, sowie zur kontinuierlichen Verbesserung der Prozesse eines Unternehmens bei.

Ein spezifisches Werkzeug, in dem KI-Technologien oft eingesetzt werden, ist das Process Mining. Dieses kann etwa im Zuge einer Prozesserhebung verwendet werden, um auf Basis von konkreten Prozessdaten, ein Ist-Prozessmodell abzuleiten. Weiterhin kann es aber auch zur Prozessanalyse genutzt werden, um z.B. Schwachstellen in einem Prozess zu identifizieren. Daneben kann es im Prozesscontrolling eingesetzt werden, um die Ausführung von Prozessen zu überwachen.⁵

Einsatz auf operativer Ebene

Die zweite Möglichkeit zum Einsatz von KI, im Kontext zu Prozessen, ist jene auf einer operativen Ebene. Auf dieser setzen Unternehmen spezifische KI-Lösungen direkt in ihren Prozessen ein. Dies bedeutet, sie werden im Rahmen der einzelnen Instanzen eines Prozesses für konkrete Aufgaben genutzt. So werden sie bspw. verwendet, um bestimmte Schritte in einem Prozess automatisch auszuführen, oder um Menschen bei der Durchführung von diesen zu unterstützen. Faktisch haben die Lösungen also eine aktive Rolle und partizipieren an der Durchführung eines Prozesses.

Insgesamt kann der Einsatz von KI auf dieser Ebene daher als ein konkretes Mittel zur – in Kapitel 2.2 erörterten – Prozessautomatisierung verstanden werden. Auf dieser liegt im Rahmen dieser

⁵ Die Grundlagen zum Process Mining sind für diese Arbeit nicht von Relevanz. Für weitere Informationen zu diesen wird daher lediglich auf „van der Aalst 2016“ verwiesen.

Thesis ein Hauptfokus. Deshalb wird in der Folge erörtert, für welche Anwendungsszenarien KI-Lösungen im Kontext zur Prozessautomatisierung eingesetzt werden können.

4.2: Einsatz von KI zur Prozessautomatisierung

Das Spektrum an Anwendungsszenarien, in welchen konkrete KI-Lösungen, bzw. ML-Modelle zur Prozessautomatisierung eingesetzt werden können, ist nahezu endlos. Im Grunde genommen gibt es so viele Szenarien, dass eine eigene Arbeit über diese verfasst werden könnte. Grundsätzlich können diese jedoch in verschiedene Kategorien unterteilt werden. Um ein Grundverständnis über den Einsatz von KI in dieser Domäne zu erhalten, wird daher auf einige dieser Kategorien, sowie praktische Beispiele zu diesen eingegangen.

Klassische Einsatzszenarien

Am längsten üblich und somit am weitesten verbreitet ist der Einsatz von KI für diverseste Arten von Klassifikationen und Regressionen. Die Grundlagen zu diesen werden in Kapitel 3.2.3 erörtert.

Im Hinblick auf Erstere kann die KI dabei für fast alle Arten von operativen, bzw. regelbasierten Entscheidungen eingesetzt werden. Das wohl bekannteste Beispiel für eine solche ist – aus Sicht einer Bank – die Bewertung der Kreditwürdigkeit von potentiellen Kreditkunden (vgl. Lauterjung 2020, S. 269). Ebenfalls gängig ist der Einsatz zur Betrugsbekämpfung. So klassifizieren Banken häufig die Transaktionen ihrer Kunden (vgl. Abdelkafi et al. 2019, S. 15) und Versicherungen die Schadensmeldungen dieser (vgl. Spindler und Kögel 2020, S. 3ff). Beide verfolgen damit das Ziel, etwaige Betrugsfälle zu identifizieren – und somit proaktiv zu vermeiden.

Auch im Kontext zu Regressionen kann die KI für unzählige Szenarien eingesetzt werden. So wird sie insbesondere für Aufgaben verwendet, in welchen operative Berechnungen, bzw. Vorhersagen erstellt werden müssen. Diese können in den unterschiedlichsten Funktionen eines Unternehmens angesiedelt sein. Zu diesen zählen beispielsweise die Bereiche der Beschaffung, der Logistik und des Vertriebs. In der Beschaffung werden KI-Lösungen z.B. eingesetzt, um Wiederbeschaffungs- oder Lieferzeiten abzuschätzen. In der Logistik hingegen werden sie oft zum Zweck der optimalen Routenplanung verwendet (vgl. Reder 2021, S. 13). Im Vertrieb wiederum werden sie vielfach herangezogen, um etwa Vertriebszahlen zu prognostizieren, oder um optimale Verkaufspreise zu bestimmen (vgl. Bitkom 2017, S. 45).

Neue Möglichkeiten durch das Deep Learning

Das Deep Learning sorgt – wie in Kapitel 3.2.6 angemerkt – für den derzeitigen Boom im Bereich der KI. Dabei ist zu konstatieren, dass alle Szenarien aus dem, oben beschriebenen, klassischen ML auch mit dem Deep Learning umsetzbar sind. Allerdings eröffnen sich auch eine Vielzahl von neuen Anwendungsszenarien, welche bisher nicht umsetzbar waren.

So ist das Deep Learning z.B. dazu geeignet, konkrete Texte zu analysieren, um deren Bedeutung zu verstehen. Daher wird es bspw. im Kundensupport eingesetzt, um E-Mails auf Basis von deren Inhalt zu klassifizieren – mit dem Ziel, je nach Kategorie der Anfrage, etwaige Support-Prozesse auszulösen (vgl. Bitkom 2017, S. 44).

Auf einer ähnlichen Grundidee bauen auch Chatbots auf. Ein solcher ist zu verstehen als eine KI-basierte Software, mit der ein Mensch in Echtzeit kommunizieren kann – und zwar per Text oder Sprache (vgl. Kusber 2017, S. 231-232). Im Vergleich zu einer Textanalyse versteht der Bot einen Text aber nicht nur, sondern kann selbst auch, konkrete, inhaltlich-verständliche Texte erzeugen. In der Theorie kann ein Chatbot an beinahe jeglichen Stellen in Prozessen eingesetzt werden, an

welchen mit bestimmten Akteuren kommuniziert werden soll. Auch Chatbots werden daher stark im Kundenservice eingesetzt (vgl. Bitkom 2017, S. 45). Sie sind in aller Regel direkt in Webseiten, bzw. Apps eingebettet – z.B. in jene von Online-Shops (vgl. Bitkom 2017, S. 46). Dabei ist jedoch zu beobachten, dass sich ihr Einsatz vorwiegend auf simple Szenarien beschränkt (vgl. Abdelkafi et al. 2019, S. 16). Bei Bedarf wird oft ein Mensch zugeschaltet. Trotzdem können Unternehmen mit Chatbots schon heute einen Großteil ihrer Kundeninteraktionen automatisch abwickeln – laut einer Studie bis zu 85% davon (vgl. Bitkom 2017, S. 45).

Des Weiteren eignet sich das Deep Learning auch zur Bilderkennung. Im Kontext zu dieser setzen z.B. Autoversicherungen KI-Lösungen ein, um in ihren Schadensregulierungsprozessen, auf Basis von Bildern eines Fahrzeugs, die Höhe und Plausibilität eines konkreten Versicherungsschadens zu bewerten (vgl. Abdelkafi et al. 2019, S. 45).

Außerdem wird Deep Learning häufig mit dem Bereich des sog. Document Understanding in eine Beziehung gebracht. Dieses befasst sich mit der Analyse, bzw. der Extraktion von Informationen aus Dokumenten. Diese nehmen in Prozessen eine wesentliche Rolle ein – denn in nahezu jedem Prozess werden Dokumente entweder verarbeitet, oder erstellt. Daher liegen sie in beachtlichen Mengen vor, und können folglich für eine Vielzahl von Szenarien eingesetzt werden. So benutzen einige Unternehmen KI-Lösungen etwa zum Zweck der automatisierten Buchhaltung – indem sie Belege analysieren, und diese, in der Folge, auf Basis der extrahierten Informationen automatisch verbuchen (vgl. Bitkom 2017, S. 43-44).

Faktisch zeigt sich also, durch jegliche dieser Beispiele, dass die Künstliche Intelligenz ein großes Potential zum Einsatz in Prozessen aufweist. Dabei wird insbesondere sichtbar, dass sie genutzt werden kann, respektive schon heute genutzt wird, um Tätigkeiten zu automatisieren, die bisher von Menschen durchgeführt werden.

4.3: KI als Make-Or-Buy Entscheidung

Um diese Potentiale zu schöpfen kann ein Unternehmen zwei Ansätze verfolgen. Es kann eine KI-Lösung entweder selbst entwickeln, oder diese über konkrete Anbieter beschaffen. Somit ist der Einsatz von KI – wie nahezu jede andere IT-Entscheidung – als eine Make-Or-Buy-Entscheidung zu verstehen. Die beiden Alternativen dieser Entscheidung zeichnen sich dabei wie folgt aus:

Buy-Entscheidung

Alles in allem gibt es schon heute, für diverseste Szenarien, eine Vielzahl von KI-Lösungen, welche am Markt verfügbar sind. Diese sind oft als Cloud-Produkte erhältlich, und können über konkrete Schnittstellen (APIs) genutzt werden. Sie bieten den Vorteil, dass sie, für gewöhnlich ein fertiges ML-Modell zur Verfügung stellen, oder den Vorgang des Trainings stark vereinfachen. Dabei ist anzumerken, dass es je nach Problemstellung viele unterschiedliche Anbieter gibt. Folglich kann problemstellungsübergreifend nur schwer ein Überblick über den ganzen Markt gegeben werden. Es gibt jedoch auch mehrere Anbieter, die generische Plattformen für das ML bereitstellen. Diese werden bspw. in „*Hanussek et al. 2021*“ vorgestellt und untersucht.

Make-Entscheidung

Im Zuge dieser Entscheidung entwickelt ein Unternehmen eine KI-Lösung selbstständig. Folglich muss es das ML-Modell explizit trainieren. Dazu benötigt es, je nach Problemstellung, eine große Menge von Daten. Darüber hinaus muss ein Unternehmen ggf. eine zugehörige Applikationslogik implementieren, eine geeignete Infrastruktur bereitstellen und das fertige System warten. Jeder

dieser Aspekte verursacht einen Aufwand, der berücksichtigt werden sollte. Nichtsdestotrotz ist auch die Eigenentwicklung nicht selten. Gängig zu dieser sind dabei Programmiersprachen, wie Python und Frameworks, wie TensorFlow. Es können aber auch spezielle ML-Tools, wie etwa die Software KNIME eingesetzt werden. Mittels dieser kann ein ML-Modell – ohne die Notwendigkeit konkreter Programmierung – über eine graphische Benutzeroberfläche erstellt werden.

4.4: Herausforderungen beim Einsatz von KI

Im Rückblick auf die letzten beiden Unterkapitel kann argumentiert werden, dass der Einsatz von KI in Prozessen viele Potentiale birgt – und einige Mehrwerte erzeugen kann. Es wird jedoch auch ersichtlich, dass diese, in der Regel, ohne erhebliche Anstrengungen, nicht ausgeschöpft werden können. Dies zeigt sich vor allem dadurch, dass durch die Entwicklung, Einführung oder Wartung einer KI-Lösung ein beachtlicher Aufwand verursacht wird. Doch damit noch nicht genug: Neben dem Aspekt des Aufwands muss berücksichtigt werden, dass durch den Einsatz einer KI in einem Prozess einige weitere spezifische Herausforderungen entstehen können. Im Fokus stehen dabei insbesondere die vier, welche in der Folge erörtert werden.

4.4.1: Transparenz & Akzeptanz

Eine der Hauptherausforderungen beim Einsatz von KI-Lösungen in Prozessen ist – einer Studie aus dem Jahr 2021 zufolge – eine potentiell fehlende Akzeptanz durch die Stakeholder (vgl. Reder 2021, S. 26). Diese können aus verschiedensten Gründen kritisch gegenüber dem Einsatz sein. So können die Mitarbeiter z.B. abgeneigt sein, da sie Angst haben könnten, dass jene KI ihre Arbeit in Zukunft vollständig übernimmt. Dies kann in etwa dazu führen, dass die Mitarbeiter die Lösung nicht wie gewünscht einsetzen. Somit kann der Mehrwert, welcher durch den Einsatz der Lösung erzielt werden könnte, nicht vollständig ausgeschöpft werden. Faktisch muss es also für jegliche Stakeholder transparent sein und offen kommuniziert werden, wie und zu welchen Zwecken eine KI-Lösung eingesetzt wird.

4.4.2: Integration

Das Ziel, welches durch den Einsatz einer KI in einem Prozess verfolgt wird, ist es, eine bestimmte Problemstellung zu lösen. Durch die Lösung dieser soll ein spezifischer, sowie möglichst optimaler Mehrwert erzielt werden. Dabei muss jedoch beachtet werden, dass dieser, für gewöhnlich, erst dann geschöpft werden kann, wenn die jeweilige KI vollständig in den zugrundeliegenden Prozess integriert ist – denn beide Objekte sind jeweils abhängig voneinander.

Exemplarisch kann dies durch den Entscheidungsbaum verdeutlicht werden, der in Kapitel 3.2.5 erörtert wird. Dieser kann als ein Klassifikator verstanden werden, welcher in einem Prozess zur Genehmigung eines Dispokredits eingesetzt werden kann. In diesem kann er verwendet werden, um zu bestimmen, ob ein konkreter Kunde, der eine Anfrage für einen solchen Kredit gestellt hat, kreditwürdig ist. Das jeweilige Ergebnis kann, im Prozess, als Entscheidungsgrundlage verwendet werden, auf Basis welcher die Anfrage akzeptiert oder abgelehnt wird. Damit jedoch eine solche Vorhersage erzeugt werden kann, ist das ML-Modell auf konkrete Daten angewiesen – zum einen auf Daten zum Einkommen und zum anderen auf Daten zum SCHUFA-Status des Kunden. Erstere

können in der Anfrage selbst enthalten sein. Letztere hingegen müssen im Zuge der Durchführung des Prozesses von der SCHUFA abgefragt werden.

Alles in allem kann daraus abgeleitet werden, dass eine KI nie als Insellösung verstanden werden darf. Ihr Einsatz sollte – wie es für jegliche andere IT-Systeme auch gelten sollte – nicht isoliert sein. Stattdessen muss er sowohl auf einer fachlichen als auch auf einer technischen Ebene in die jeweiligen Prozesse integriert sein. Faktisch müssen konkrete Lösungen gefunden werden, mittels welchen eine solche Integration möglich ist. Diese müssen in der Lage sein, den Einsatz einer KI-Lösung fachlich abzubilden. Darüber hinaus müssen sie aber auch geeignet sein, diesen technisch zu steuern, und die Ergebnisse aus dem Einsatz der Lösung für konkrete Zwecke zu verarbeiten.

4.4.3: Qualität der Ergebnisse

Darüber hinaus muss berücksichtigt werden, dass die Vorhersagen eines ML-Modell in aller Regel fehlerhaft sein können. Dies liegt daran, da sie mit historischen Daten trainiert werden – jedoch Vorhersagen für unbekannte Situationen treffen sollen. Zu diesem Zweck erlernen sie Muster aus den historischen Daten und verallgemeinern diese. Nichtsdestotrotz muss berücksichtigt werden, dass diese Daten immer nur einen Teilausschnitt der Realität darstellen. Sie sind beschränkt auf bestimmte Parameter. Der komplette Kontext einer Situation ist durch sie nicht bekannt. Faktisch werden Parameter, die nicht in den Daten enthalten sind, jedoch relevant wären, vernachlässigt. Zudem können sich die Muster in den Daten im Laufe der Zeit verändern.

Daher kann gefolgert werden, dass die Vorhersagen einer KI immer eine Unsicherheit aufweisen. Dadurch kann es vorkommen, dass die Ergebnisse der KI nicht mit der Realität korrelieren. Wird auf Basis eines solchen falschen Resultats eine spezifische Handlung ausgeführt, so kann dies negative Folgen für ein Unternehmen haben. Stuft eine Bank z.B. die Kreditwürdigkeit von einem Kunden falsch ein und lehnt deshalb dessen Kreditanfrage ab, so verliert sie Zinseinnahmen, die sie potentiell hätte einnehmen können.

Daher sollten auch im Produktivbetrieb konkrete Maßnahmen ergriffen werden, um kontinuierlich zu prüfen, ob die Resultate einer KI zufriedenstellend sind. Dabei ist es, je nach Szenario, sowohl denkbar, dass jeder einzelne Einsatz geprüft wird, oder nur Stichproben gezogen werden. Je nach Entscheidung wäre es folglich möglich, dass falsche Vorhersagen im Einzelfall korrigiert werden können – oder dass ein Monitoring über die Qualität der Ergebnisse erstellt wird. Letzteres könnte genutzt werden, um zu evaluieren, ob das Modell geeignet ist, oder neu trainiert werden muss.

4.4.4: Erklärbarkeit

Zudem kann es eine Herausforderung sein, dass es für Menschen meist nicht nachvollziehbar ist, aus welchen Gründen eine KI, repräsentiert durch ein ML-Modell, ein konkretes Ergebnis, wie z.B. eine Klassifikation prognostiziert (vgl. Buxmann und Schmidt 2019, S. 17). Im Vergleich zu den anderen Herausforderungen beim Einsatz von KI-Lösungen in Prozessen ist sie jene, über die in der Forschung am meisten diskutiert wird – und zwar unter dem Begriff „Explainable AI“.

Je nach Szenario kann das Wissen bezüglich der Gründe, für eine Entscheidung eines ML-Modells, unwichtig sein (vgl. Schaaf et al. 2021, S. 11). Insbesondere jedoch in Prozessen ist es oft kritisch, und stellt daher eine Herausforderung für Unternehmen dar (vgl. Reder 2021, S. 27). Das Wissen kann dabei einen wesentlichen Einfluss auf den Ausgang eines Prozesses haben, und diesen etwa monetär beeinflussen. Lehnt eine Bank, z.B. aufgrund von einer fehlerhaften Klassifikation, den

Kreditantrag eines solventen Kunden ab, so erleidet diese einen monetären Nachteil. Kann jedoch der Grund zur Entscheidung nachvollzogen werden, besteht die Möglichkeit diese zu korrigieren.

Weiterhin kann das Wissen aber auch in rechtlichen, bzw. ethischen Aspekten relevant sein – vor allem, wenn eine KI-Entscheidung eine direkte Auswirkung auf einen Menschen hat (vgl. Schaaf et al. 2021, S. 11). Dies ist etwa der Fall, wenn eine KI-Lösung im Personalwesen – oder genauer gesagt, zur Auswahl von Mitarbeitern im Recruiting-Prozess verwendet wird (vgl. Buxmann und Schmidt 2019, S. 17). In diesem muss nachvollziehbar sein, wie eine Entscheidung getroffen wird, um sicherzustellen, dass durch die Lösung – aus ethischen Gesichtspunkten – keine spezifischen Personengruppen benachteiligt werden.

Kontext zur Herausforderung – symbolische & subsymbolische Modelle:

Die Herausforderung gründet darin, dass ein Großteil der ML-Modelle, die in modernen KI-Lösungen eingesetzt werden, sog. **subsymbolische** Modelle sind. Diese bieten sehr gute Vorhersage-Ergebnisse, und sind für diverseste Anwendungsszenarien geeignet. Allerdings bilden sie ihr Wissen in hochkomplexen Formen ab, welche für Menschen nur schwer oder gar nicht verständlich sind. Dementsprechend erlauben sie kaum, bis gar keine Rückschlüsse über die erlernten Muster – und somit auch nicht über den Vorgang der Entscheidungsfindung einer KI (vgl. Döbel et al. 2018, S. 92). Exemplarisch für ein Modell von diesem Typ sind KNN. Diese repräsentieren ihr Wissen durch eine Vielzahl von Gewichtungen (siehe Kap. 3.2.7.2).

Das Gegenstück zu einem solchen KNN sind die **symbolischen** Modelle. Diese bilden ihr Wissen explizit und verständlich ab – und erlauben folglich konkrete Rückschlüsse zur Entscheidungsfindung (vgl. Döbel et al. 2018, S. 92). Ein Beispiel für ein solches Modell ist der Entscheidungsbaum. Dieser stellt erlernte Zusammenhänge mittels einer Menge von Knoten und Kanten, respektive Pfaden dar (siehe Kap. 3.2.5.1). Folglich scheint ein symbolisches Modell viele Vorteile zu bieten. Jedoch sind diese häufig, im Vergleich zu subsymbolischen Modellen, entweder in ihrer Vorhersage-Genauigkeit schlechter oder für etwaige Einsatzszenarien nicht geeignet. Ergo werden sie seltener eingesetzt.

5: Grundlagen – zur BPMN und DMN

Das letzte Kapitel zusammenfassend birgt der Einsatz von KI in Prozessen also große Potentiale, aber auch große Herausforderungen. Eine potentielle Lösung für diese Herausforderungen und somit eine Möglichkeit zur Automatisierung von Prozessen mittels KI kann die BPMN sein. Daher wird in diesem Kapitel auf Grundlagen zur BPMN und zur Automatisierung von BPMN-basierten Prozessen eingegangen. Zudem wird die DMN – als eine Ergänzung zur BPMN – untersucht.

5.1: Business Process Model and Notation

Die **Business Process Model and Notation** (BPMN) ist eine Methode zur Modellierung sowie zur Ausführung von Prozessen (vgl. Gadatsch 2020, S. 127). Sie besteht aus zwei Komponenten – aus einer Notation, sowie einem sog. Metamodell. Des Weiteren ist die BPMN der ISO-Standard zur Prozessmodellierung und wird verwaltet durch die Object Management Group (OMG).⁶

5.1.1: BPMN als Modellierungsnotation

Im Kern ist die BPMN zu verstehen als eine spezifische Modellierungssprache, die exklusiv für die Abbildung von Prozessen entwickelt ist (vgl. Freund und Rücker 2017, S. 28). Sie stellt eine Reihe von Notationselementen zur Modellierung von diesen bereit. Eine Übersicht über jegliche dieser Elemente ist Anhang 1 zu entnehmen. Im Hinblick auf diese ist zu erwähnen, dass jedes von ihnen eine konkrete graphische Darstellung und eine durch die Semantik definierte Bedeutung besitzt. Zudem können die Elemente in vier Kategorien unterteilt werden: In Flusselemente, verbindende Objekte, Swimlanes und Artefakte (siehe Abb. 17).⁷

In den folgenden Unterkapiteln wird auf wichtigen Notationselemente aus diesen vier Kategorien eingegangen – bzw. auf jene, welche für diese Thesis von Relevanz sind. Zuvor soll allerdings ein Grundverständnis zur Modellierung und den Elementen im Allgemeinen aufgebaut werden.

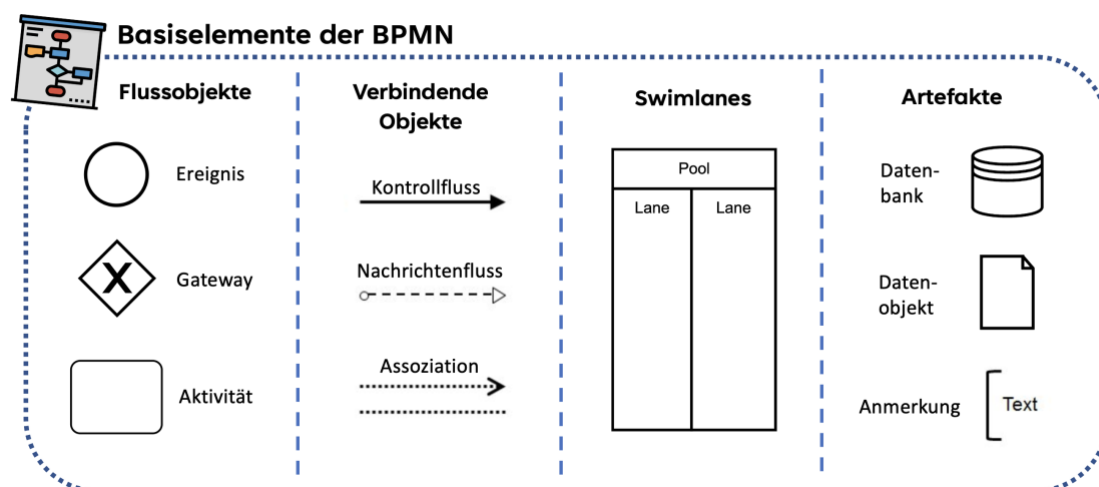


Abbildung 17: Ausschnitt der relevanten Notationselemente der BPMN

⁶ Die Object Management Group (OMG) ist eine Organisation, die sich mit der Entwicklung von herstellerunabhängigen Technologie-Standards beschäftigt – siehe <https://www.omg.org/>

⁷ Für relevante Grundlagen zu Modellierungssprachen – siehe Kapitel 2.4

Grundverständnis zur Modellierung mit der BPMN

Um dieses zu vermitteln, werden die wichtigsten Elemente der BPMN erörtert – und zwar anhand des Stellenausschreibungsprozesses, welcher in Abb. 18 dargestellt ist. Exemplarisch aufgezeigt an diesem, ist ein BPMN-basierter Prozess immer ähnlich aufgebaut:

Im Kern besteht er stets aus einer Menge von **Flusselementen**. Diese sind die zentralen Bausteine eines Prozesses. Zu den Flusselementen zählen Aktivitäten, Ereignisse und Gateways. Aktivitäten sind hierbei zu verstehen als Arbeit, welche im Zuge des Prozesses ausgeführt werden soll. Eine solche Arbeit kann z.B. das Melden eines Personalbedarfs sein. Ereignisse hingegen beschreiben Zustände, welche im Laufe der Prozessausführung eintreten, oder ausgelöst werden. Ein solcher Zustand ist bspw. jener, dass ein Mitarbeiter benötigt wird. Ein Gateway wiederum, repräsentiert eine Form von Ablauflogik. Eine solche kann etwa sein, dass eine Stellenausschreibung korrigiert werden muss, wenn sie bei der Prüfung etwaige Fehler aufweist.

Die Flusselemente werden durch Kontrollflüsse miteinander verbunden. Kontrollflüsse gehören zu den **verbindenden Objekten**. Diese werden verwendet, um Notationselemente in eine Beziehung zueinander zu stellen. Insofern ergibt sich durch die verbindenden Elemente die zeitlich-logische Abfolge eines Prozesses (vgl. Freund und Rücker 2017, S. 34).

Strukturiert werden kann diese Abfolge durch **Swimlanes**. Zu den Swimlanes zählen der Pool und die Lane. Ein Pool ist dabei zu verstehen als ein separater Prozess – eine Lane hingegen als ein Akteur in diesem. Im Stellenausschreibungsprozess sind die Lanes daher die Fachabteilung und die Personalabteilung.

Außerdem stellt die BPMN sog. **Artefakte** bereit. Diese dienen zur Ergänzung der Prozessmodelle mit zusätzlichen Informationen. Artefakte können z.B. einfache Kommentare sein – wie etwa eine Information darüber, auf welchen Portalen eine Ausschreibung veröffentlicht werden soll.

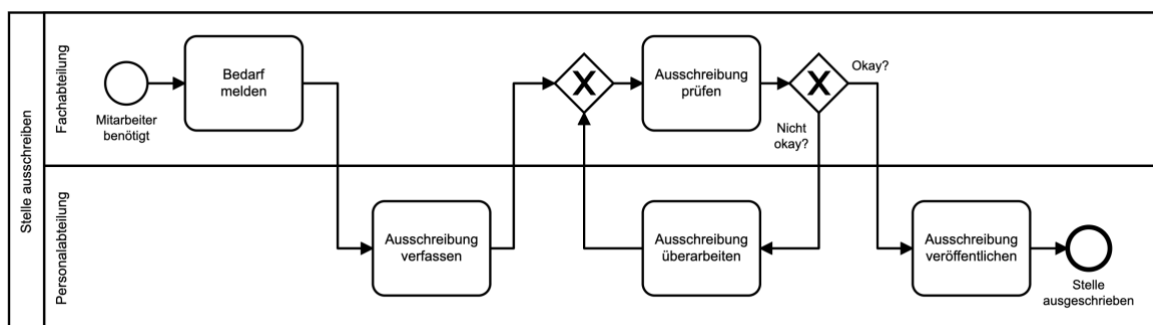


Abbildung 18: BPMN-basierter Prozess zur Stellenausschreibung ⁸

5.1.2: BPMN als Notation mit Fokus auf Automatisierung

Durch die Limitierung auf Prozesse verfolgt die BPMN das Ziel, diese möglichst einheitlich, sowie verständlich, für alle relevanten Stakeholder abzubilden: Vom Manager, der einen Überblick über den Prozess erhalten will, über den Mitarbeiter, der ihn ausführt, bis hin zum IT-Experten, der ihn in IT-Systeme implementiert. Daher bietet die BPMN die Möglichkeit, Prozesse auf verschiedenen Detaillierungsgraden zu modellieren.

Folglich kann die BPMN für zahlreiche Zwecke eingesetzt werden. Auf einer fachlichen Ebene kann sie etwa zur Dokumentation, Analyse oder Kommunikation von Prozessen dienen. Doch vor allem

⁸ Quelle: Allweyer 2015, S. 16

auf technischer Ebene bietet sie viele Einsatzmöglichkeiten und Mehrwerte – denn im Vergleich zu anderen Modellierungsnotationen legt die BPMN einen klaren Fokus auf die Automatisierung von Prozessen (vgl. Freund und Rücker 2017, S. 7).

Im Hinblick auf diesen Fokus soll die Notation als eine gemeinsame Sprache zwischen der IT und den Fachabteilungen in einem Unternehmen verstanden werden (vgl. Freund und Rücker 2017, S. XVII). Sie verfolgt das Ziel, einen Prozess – mit einem einzelnen Modell – sowohl zu beschreiben als auch zu automatisieren.

5.1.2.1: Automatisierung durch Process Engines

Der Fokus der BPMN auf die Automatisierung zeigt sich vor allem dadurch, dass es möglich ist, BPMN-Prozessmodelle, durch den Einsatz von sogenannten Process Engines, zu automatisieren, bzw. digital zu steuern. Das Konzept einer solchen Engine wird im Zuge von Kapitel 5.2 im Detail erörtert. Grundsätzlich ist sie jedoch zu verstehen, als eine konkrete Software, welche in der Lage ist, ein BPMN-basiertes Prozessmodell technisch zu interpretieren – und folglich auszuführen. Zu diesem Zweck implementiert sie die Ausführungssemantik und das Metamodell der BPMN.

5.1.2.2: Technische Konfiguration der BPMN-Modelle

Ein BPMN-Modell wird dabei stets in Form einer XML-Datei gespeichert (vgl. Freund und Rücker 2017, S. 226). Um es zu automatisieren, müssen die einzelnen Notationselemente aus dem Modell technisch konfiguriert werden. Durch die Konfiguration wird eine technische Ausführungslogik für die Elemente hinterlegt, bzw. auf eine solche verwiesen. Faktisch wird also beschrieben, wie ein bestimmtes Element, dass in einem Prozessmodell abgebildet ist, ausgeführt werden soll.

Die Grundlage für die Konfiguration bildet dabei das sog. Metamodell der Notation. Dieses ist zu interpretieren als ein spezifisches Datenmodell. Es beschreibt die Elemente der BPMN als einzelne Entitäten (siehe z.B. OMG 2011, S. 151). Folglich gibt es an, aus welchen Attributen ein jeweiliges Element besteht. Zu den Attributen eines jeden Elements zählen dabei, in aller Regel, mindestens eine ID und ein Name. Neben diesen kann es allerdings auch unzählige weitere Attribute besitzen. Diese sind dabei meist technischer Art – sprich relevant für die Ausführung des Elements.

Im Rahmen der technischen Konfiguration können, respektive müssen für diese Attribute aus dem Metamodell spezifische Informationen hinterlegt werden (siehe Abb. 19). Diese werden ebenfalls in der XML-Datei gespeichert. Die Ausführungssemantik der Notation nutzt diese Informationen. Sie beschreibt, wie die jeweiligen Elemente durch eine Engine ausgeführt werden müssen – bspw. indem eine konkrete Funktion, welche durch die Konfiguration hinterlegt ist, aufgerufen wird.

Dabei ist jedoch zu berücksichtigen, dass jede Process Engine, im Hinblick auf die Möglichkeiten zur Ausführung der Prozessschritte, einen unterschiedlichen Funktionsumfang besitzen kann. Dies ist begründet durch die Flexibilität der BPMN – denn diese schränkt nicht ein, welche spezifischen Systeme, Technologien, oder Programmiersprachen eingesetzt werden können, um einen Schritt in einem Prozess durchzuführen. Faktisch sind die Möglichkeiten zur Ausführung, und somit auch zur Konfiguration eines Attributs, immer abhängig von den Funktionen einer Process Engine (vgl. Freund und Rücker 2017, S. 238). Darüber hinaus kann es je nach Engine einige weitere Attribute geben, welche zum Zweck der Automatisierung verwaltet werden können.

Ein vereinfachtes Beispiel für eine solche technische Konfiguration ist in Abbildung 19 dargestellt. Diese zeigt exemplarisch – für die Engine des Unternehmens Camunda, die im Verlauf der Thesis

praktisch eingesetzt wird – welche Attribute für einen sog. Service Task gepflegt werden können. Letzterer wird im Rahmen von Kapitel 5.1.3.2 tiefer untersucht.

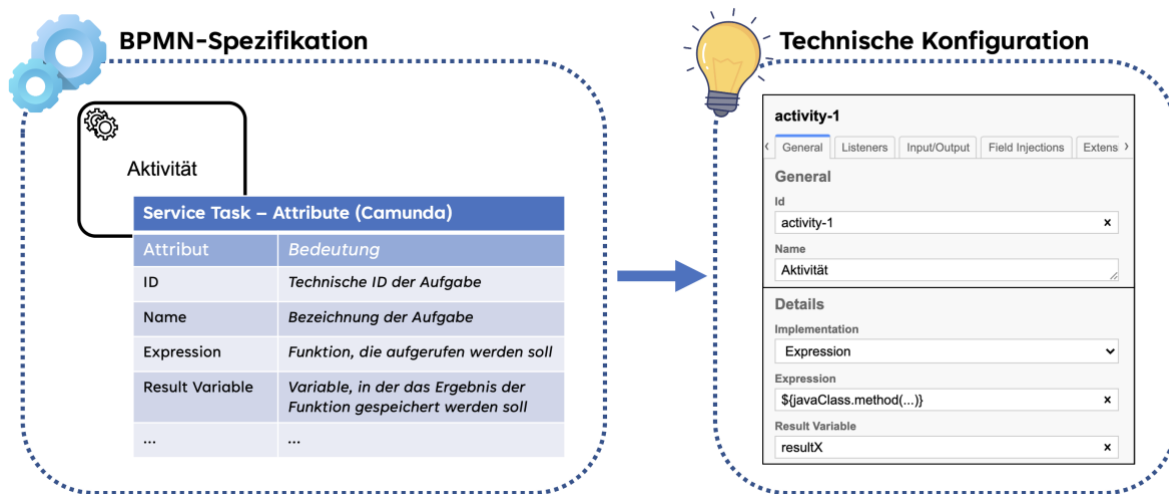


Abbildung 19: Automatisierung eines Prozesses durch Konfiguration der Elemente

5.1.3: Die Notation im Detail

Bevor jedoch auf die Process Engines eingegangen wird, werden in der Folge einige Elemente der BPMN im Detail erörtert – beginnend mit den Flusselementen, respektive Aktivitäten:

5.1.3.1: Aktivitäten

Aktivitäten zählen zu den Flusselementen der BPMN. Sie sind zu deuten als eine Form von Arbeit, die im Zuge eines Prozesses ausgeführt werden soll. Die Ausführung dieser Arbeit benötigt Zeit. Zur Modellierung von Aktivitäten stellt die BPMN diverse Notationselemente zur Verfügung. Diese variieren vor allem in ihrem Detaillierungsgrad. Sie können grundsätzlich in zwei Typen unterteilt werden – in atomare und in nicht atomare Aktivitäten (vgl. OMG 2011, S. 163, siehe Abb. 20).



Abbildung 20: Unterscheidung von Aktivitäten in atomar und nicht-atomar

Eine atomare Aktivität, ist dabei zu deuten als ein spezifischer Arbeitsschritt, welcher nicht noch genauer beschrieben werden kann, oder soll. Auf den atomaren Aktivitäten, repräsentiert durch das Element „**Aufgabe**“, wird im folgenden Unterkapitel der Fokus liegen.

Eine nicht atomare Aktivität hingegen ist komplexerer Natur. Sie kann durch weitere Aktivitäten noch genauer beschrieben werden. Ein Beispiel für eine solche Aktivität ist ein Unterprozess. Auf Aktivitäten dieser Art wird im Rahmen dieser Thesis jedoch nicht weiter eingegangen. Für mehr Informationen zu diesen wird auf Quellen, wie „*Freund und Rücker 2017, S. 79ff*“ verwiesen.

5.1.3.2: Aufgaben

Aufgaben sind der Kern eines BPMN-basierten Prozesses. Im technischen Jargon werden sie auch als Tasks bezeichnet. Sie beschreiben die einzelnen Schritte eines Prozesses, welche tatsächlich durchgeführt werden sollen. In der Notation können Aufgaben sogenannte Typen besitzen. Diese werden im Prozessmodell durch entsprechende Symbole visuell dargestellt. Ein Typ ist hierbei zu verstehen als eine konkrete Art, bzw. Form einer Aufgabe. Dieser kann z.B. eingesetzt werden, um die Aufgaben in einem Modell anschaulicher zu machen. Primär hat ein Typ allerdings einen technischen Zweck. Er beeinflusst, wie eine zum Typ zugehörige Aufgabe, aus technischer Sicht, durch eine Engine, ausgeführt werden muss. Ergo besitzt jeder Typ, im Kontext zum Metamodell, unterschiedliche Attribute, sowie eine dedizierte Ausführungssemantik (siehe OMG 2021, S. 156).

Die Notation stellt im Standard sieben Typen von Aufgaben bereit (siehe Abb. 21). Auf diese wird in der Folge eingegangen. Zuvor soll jedoch erwähnt werden, dass es auch die Möglichkeit gibt, eigene Typen von Aufgaben zu erstellen, und die BPMN mit diesen zu erweitern (vgl. OMG 2011, S. 158). Auf diese Möglichkeit wird in diesem Kapitel allerdings nicht im Detail eingegangen. Das Wissen über ihre Existenz ist jedoch im Kontext zur KI für Kapitel 6 relevant.

Manueller Task

Ein manueller Task ist eine Aufgabe, welche alleinig von Menschen ausgeführt wird und komplett außerhalb einer Softwareanwendung abläuft. Sie wird von keiner Process Engine verwaltet.

- **z.B.** - Einwerfen einer Rechnung in einen Postkasten.

Benutzer Task

Ein Benutzer Task wird ebenfalls ausschließlich durch Menschen bearbeitet. Der Unterschied zum manuellen Task ist jedoch, dass er digital abgebildet ist. Ein Benutzer Task wird also über eine Software unterstützt und über eine Engine verwaltet. Dabei wird dem Akteur ein Interface – z.B. ein Formular – bereitgestellt, über welches er den Task bearbeiten und abschließen kann.

- **z.B.** - die Genehmigung eines Urlaubsantrages über ein HR-Tool.

Skript Task

Ein Skript Task ist eine Aufgabe, welche – wie ihr Name andeutet – ein Skript beinhaltet. Dieses ist im Prozessmodell hinterlegt. Wird der Task ausgelöst, so wird das Skript direkt von der Engine ausgeführt. Folglich muss es in einer Sprache vorliegen, die von dieser interpretiert werden kann. Das Skript kann dabei verschiedenste Zwecke erfüllen, wie:

- **z.B.** - Öffnen einer Datei, als Vorbereitung für die nächste Aktivität

Service Task

Ein Service Task ist eine Aufgabe, welche automatisiert durch eine beliebige Software ausgeführt wird. Eine solche kann bspw. ein spezifisches System, wie ein ERP-System, oder ein Web-Service sein. Unabhängig von der Art der Lösung, wird ein Service Task stets durch die Engine gesteuert. Wird er angestoßen, so ruft diese – über eine im Task konfigurierte Schnittstelle – die Funktion einer Software auf, welche die Aufgabe logisch ausführt (vgl. Dumas et al. 2018, S. 374).

- **z.B.** - Automatisierte Erstellung einer Rechnung durch ein SAP-System.
- **z.B.** - Automatisierte Bonitätsprüfung eines Kunden durch SCHUFA Web-Service.

Nachrichten-Empfangender Task

Auch der Nachrichten-Empfangende Task wird stets durch eine Engine gesteuert. Er ist zu deuten als eine Aufgabe, im Zuge welcher auf eine Nachricht von einem externen Akteur gewartet wird. Diese gilt als beendet, sobald die Nachricht eingetroffen ist (vgl. OMG 2011, S. 161) .

- **z.B.** – Warten auf die Zahlung eines Kunden, nach dem Versand einer Rechnung

Nachrichten-Sendender Task

Der Nachrichten-Sendende Task ist das Gegenstück zum Nachrichten-Empfangenden Task. Aus einer Perspektive der Automatisierung ist er sehr ähnlich zum Service Task. Er ist zu verstehen als eine Aufgabe, die durch eine Software ausgeführt wird. Diese sendet eine konkrete Nachricht an einen externen Akteur. Der Task gilt als beendet, sobald die Nachricht versendet wurde (vgl. OMG 2011, S. 159).

- **z.B.** – Senden einer Rechnung an einen Kunden, nach dem Versand einer Bestellung

Geschäftsregel Task

Ein Geschäftsregel Task ist eine Aufgabe, in der eine automatisierte Entscheidung getroffen wird. Folglich wird auch dieser Task durch eine Engine gesteuert. Die Entscheidung selbst wird jedoch nicht durch sie durchgeführt. Stattdessen kann die Process Engine eine sog. Business Rule Engine aufrufen (vgl. OMG 2011, S. 163). In dieser wird die Entscheidung durch die Auswertung einer konkreten Entscheidungslogik getroffen. Eine solche kann z.B. durch die DMN abgebildet werden, die in Kapitel 5.2 behandelt wird. Folglich können Fragestellungen automatisiert werden, wie:

- **z.B.** – Entscheidung, ob Versandkosten für eine Bestellung anfallen
- **z.B.** – Entscheidung, ob Bonitätsprüfung für einen Kunden durchgeführt werden muss

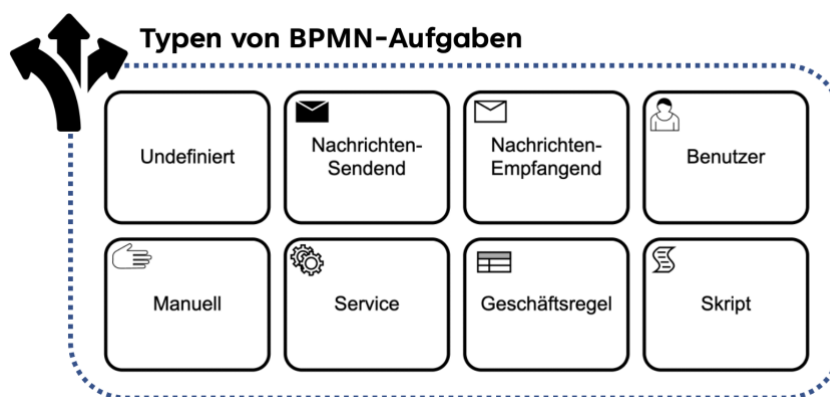


Abbildung 21: Unterschiedliche Typen von BPMN-Aufgaben

5.1.3.3: Ereignisse

Ereignisse zählen, wie die Aktivitäten, zu den Flusselementen der BPMN. Im technischen Jargon werden sie oft auch als Events bezeichnet. Aus einer fachlichen Sicht sind sie dabei zu verstehen als konkrete Geschehnisse, welche einen spezifischen Zustand beschreiben (vgl. Gadatsch 2020, S. 128). Sie repräsentieren keine Arbeit und haben folglich auch keine Dauer (vgl. Allweyer 2015, S. 64). Stattdessen sind sie auf einen konkreten Zeitpunkt bezogen und gelten als abgeschlossen, sobald der beschriebene Zustand eingetreten ist.

Alles in allem steuern, bzw. beeinflussen Ereignisse den zeitlich-logischen Ablauf von Prozessen. Sie sind in der Lage, Prozesse zu starten, zu unterbrechen und zu beenden. Mit Bezug auf diese Eigenschaften unterscheidet die BPMN Ereignisse in drei grundlegende Arten (vgl. OMG 2011, S. 233; siehe Abb. 22). Diese drei Arten sind wie folgt zu beschreiben:

Startereignis

Ein Startereignis illustriert den Beginn eines Prozesses. Es beschreibt, unter welchen Umständen eine Prozessdurchführung ausgelöst werden kann. Ein Prozess hat mindestens ein Startereignis. Wird eines von diesen ausgelöst, so wird eine neue Instanz des entsprechenden Prozesses erstellt.

Endereignis

Das Endereignis ist das Gegenstück zum Startereignis. Es markiert das Ende eines Prozesses. Ein Prozess benötigt mindestens ein Endereignis. Das Endereignis beschreibt, mit welchem Resultat eine Prozessdurchführung endet. Folglich hat es eine direkte Beziehung zum Output.

Zwischenereignis

Zwischenereignisse treten während dem Ablauf einer Prozessinstanz auf. Sie können entweder im Prozessfluss, oder angeheftet an Aktivitäten modelliert werden. Je nach ihrer Art unterbrechen Zwischenereignisse entweder den kompletten Prozess, oder konkrete Aktivitäten, an welchen sie angeheftet sind. Sind sie von letzterer Art, können sie zudem in zwei weitere Kategorien eingeteilt werden – in unterbrechende, sowie nicht unterbrechende Ereignisse. Unterbrechende Ereignisse stoppen die Aktivität, an die sie angeheftet sind. Sie setzen den Prozess über ihren ausgehenden Kontrollfluss fort. Nicht unterbrechende Ereignisse hingegen starten einen neuen Teilprozess und setzen die eigentliche Aktivität fort.

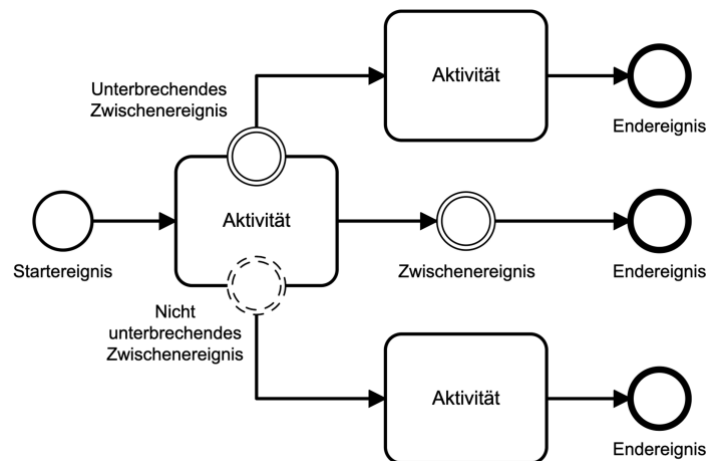


Abbildung 22: BPMN – Arten von modellierbaren Ereignissen

Analog zu Aktivitäten können auch Ereignisse spezifische Typen besitzen. Diese beschreiben die Art, sowie das Verhalten eines Ereignisses. Eine Übersicht über alle Typen von Ereignissen, welche die BPMN zur Verfügung stellt, ist Anhang 1 zu entnehmen. In der Folge wird allerdings nicht im Detail auf diese eingegangen, da sie für die Thesis nicht von hoher Relevanz sind. Es soll jedoch erwähnt werden, dass sie im Kern in die zwei folgenden Kategorien eingeteilt werden können:

Ausgelöste Ereignisse

Ausgelöste Ereignisse sind Ereignisse, die von einem betrachteten Prozess ausgelöst werden (vgl. Freund und Rücker 2017, S. 52). Sie werden an externe Akteure, die außerhalb dieses Prozesses

stehen, gesendet. Ausgelöste Ereignisse werden durch ausgefüllte Symbole dargestellt. Ein Typ eines solchen Ereignisses ist das Nachrichten-Sendende Ereignis. Dieses beschreibt, dass eine Nachricht an einen externen Akteur versendet wurde. Ein praktisches Beispiel hierfür ist, aus der Sicht eines Bewerbers, das Versenden einer Bewerbung an ein Unternehmen (siehe Abb. 23).

Eingetretene Ereignisse

Eingetretene Ereignisse sind das Gegenstück zu ausgelösten Ereignissen. Der Prozess löst diese nicht selbst aus, sondern wartet auf ihr Eintreten (vgl. Freund und Rücker 2017, S. 52). Dieses ist abhängig von Faktoren, wie Zeit oder externen Akteuren. Eingetretene Ereignisse werden durch nicht ausgefüllte Symbole visualisiert. Ein exemplarischer Typ eines solchen Ereignisses ist das Nachrichten-Empfangende Ereignis. Dieses gibt wieder, dass eine konkrete Nachricht empfangen wurde. Ein praktisches Beispiel hierfür ist, aus der Sicht eines Bewerbers, das Empfangen einer Empfangsbestätigung, nach dem Versand seiner Bewerbung an ein Unternehmen (siehe Abb. 23).

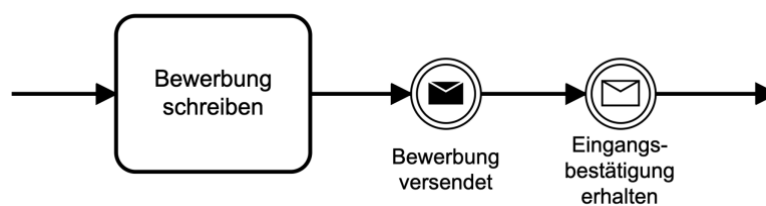


Abbildung 23: Unterscheidung von Ereignissen in eingetretene und ausgelöste Ereignisse

5.1.3.4: Gateways

Mit den bisher erörterten Flusselementen ist ein Prozess, in einer verständlichen Weise, nur linear modellierbar. Dies bedeutet, er ist lediglich eine Folge von Aktivitäten und Ereignissen. Als eine solche Folge würde ein Prozess jedoch immer gleich ablaufen – was kaum der Praxis entspricht. In dieser ist es üblich, dass Prozesse in unterschiedlichen Varianten ablaufen. In diesen Varianten können die Ablaufreihenfolgen variieren, bestimmte Aktivitäten parallel ablaufen, oder auch nur unter konkreten Umständen notwendig sein (vgl. Freund und Rücker 2017, S. 34). Zur Abbildung solcher Situationen stellt die BPMN ein drittes Flusselement bereit – das Gateway.

Ein Gateway bildet die Ablauflogik in einem Prozess ab. Es kann eingesetzt werden, um einen Prozess zu verzweigen und um verschiedene Zweige von diesem wieder zusammenzuführen (vgl. Gadatsch 2020, S. 131). Analog zu den Ereignissen hat auch ein Gateway keine konkrete Dauer. Es prüft lediglich die jeweilige Ablauflogik und führt den Prozess in Abhängigkeit zu dieser fort (vgl. Allweyer 2015, S. 26). Da die Logik in ihrer Form variieren kann, definiert die BPMN diverse Arten von Gateways (siehe Abb. 24).

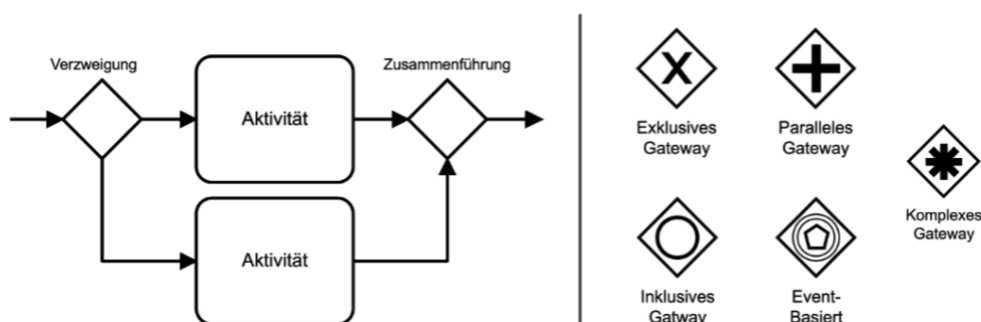


Abbildung 24: Zweck und Arten von Gateways in der BPMN

Im Folgenden werden lediglich zwei dieser Gateways näher beschrieben – das Exklusive und das Parallele Gateway. Diese werden in den späteren Kapiteln praktisch eingesetzt. Für detailliertere Informationen zu den anderen Arten von Gateways wird auf „Allweyer 2015, S. 25ff“ verwiesen.

Exklusives Gateway

Ein exklusives Gateway ist zu verstehen als ein Entscheidungspunkt in einem Prozess. An diesem wird ein Prozess, auf Basis konkreter Bedingungen, über genau einen von mehreren verfügbaren Pfaden fortgesetzt. Die Bedingungen sind dabei je Pfad definiert und schließen sich gegenseitig aus. Wird das Gateway ausgeführt, so werden jegliche der Bedingungen geprüft, und dem Pfad gefolgt, dessen Bedingung zutrifft. Alles in allem kann es daher eingesetzt werden, um Abfolgen von Prozessschritten abzubilden, die nur unter speziellen Umständen ausgeführt werden sollen. Exemplarisch kann dies am Beispiel eines Verkaufsprozesses aufgezeigt werden. In diesem kann z.B. definiert sein, dass eine Bonitätsprüfung für einen Kunden durchgeführt werden muss, sofern dessen Warenkorb einen Wert von 500€ übersteigt (siehe Abb. 25).

Paralleles Gateway

Das Parallele Gateway wird genutzt, um einen Prozess, in mehrere Abfolgen von Flusselementen aufzuteilen. Es setzt einen Prozess zeitgleich, sprich parallel über alle ausgehenden Kontrollflüsse fort. Der Begriff „Parallel“ impliziert dabei allerdings nicht, dass etwaige Tätigkeiten gleichzeitig durchgeführt werden müssen (vgl. Freund und Rücker 2017, S. 38). Er bedeutet, dass diese nicht hintereinander, sondern unabhängig voneinander ausgeführt werden können. Wird das Gateway zur Zusammenführung eingesetzt, führt es den Prozess fort, sobald alle parallelisierten Stränge abgeschlossen sind. Ein Beispiel für den Einsatz des Gateways bietet der in Abb. 25 dargestellte Ausschnitt aus einem Stellenausschreibungsprozess.

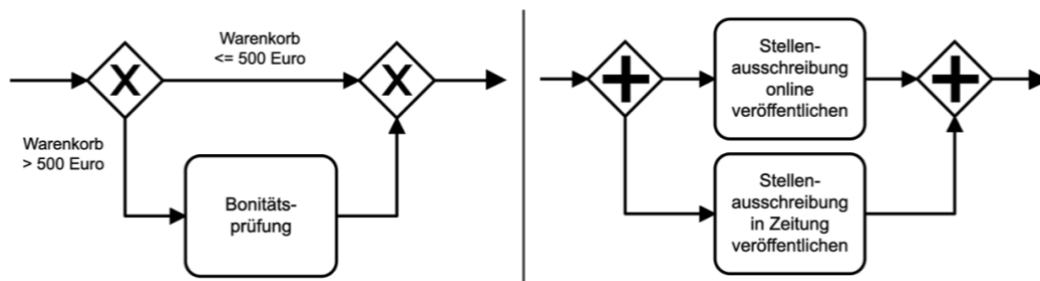


Abbildung 25: Beispiele für die Verwendung von exklusiven und parallelen Gateways

5.1.3.5: Swimlanes

Swimlanes sind Notationselemente, welche zur Strukturierung und zur Abgrenzung von Prozessen dienen. Sie gehören nicht zu den Flusselementen, sondern bilden eine weitere Elementkategorie. Zu dieser zählen laut der BPMN-Spezifikation zwei Elemente – der Pool und die Lane (vgl. OMG 2011, S. 28). Im Hinblick auf diese Elemente ist anzumerken, dass diese lediglich eine fachliche Bedeutung aufweisen. Sie besitzen keine Ausführungssemantik, und haben ergo keinen Einfluss auf die Automatisierung eines Prozesses. Nichtsdestotrotz sind sie für diese Thesis relevant. Ihr Zusammenhang ist daher – anhand von Abbildung 26 – wie folgt zu beschreiben:

Ein **Pool** repräsentiert einen abgegrenzten Prozess, oder einen unabhängigen Teilnehmer, der an diesem beteiligt ist. Ein Prozessmodell kann mehrere Pools besitzen. Im Vertriebsprozess eines Unternehmens könnten sowohl das Unternehmen selbst als auch der Kunde einen Pool darstellen. Die Interaktionen zwischen Pools können dabei über sog. Nachrichtenflüsse abgebildet werden.

Pools können optional durch **Lanes** tiefer konkretisiert werden. Ein Pool kann aus mehreren Lanes bestehen. Eine Lane ist dabei zu verstehen, als ein Akteur, welcher dem Pool direkt untergeordnet ist. Ein solcher Akteur kann etwa eine Person, eine Rolle, eine Abteilung, oder sogar ein IT-System sein (vgl. Gadatsch 2020, S. 131). Folglich wird durch Lanes der funktionsübergreifende Charakter von Prozessen sichtbar. Im eben angeführten Vertriebsprozess können die Lanes z.B. Funktionen, wie der Vertrieb oder das Lager sein.

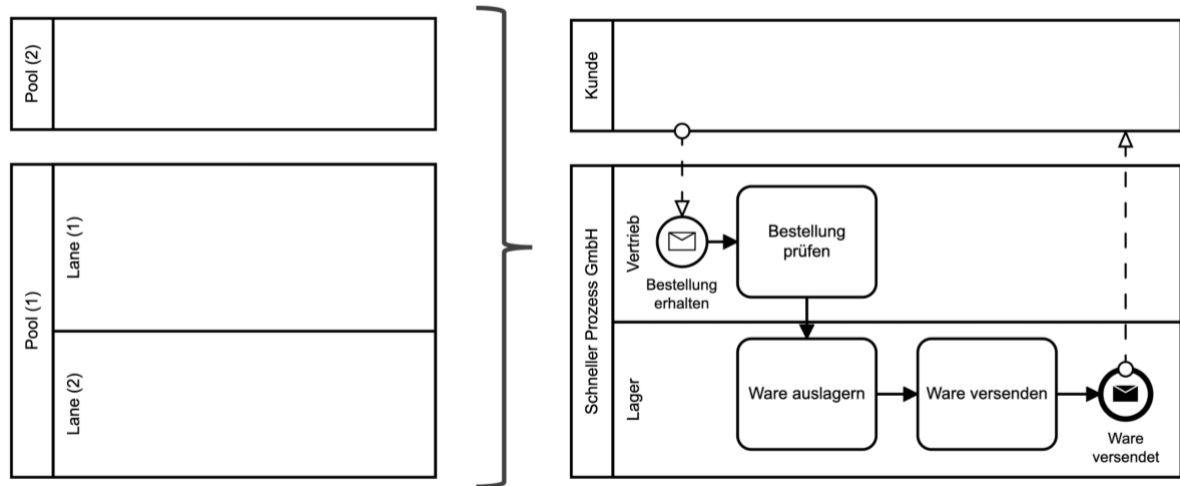


Abbildung 26: Nutzung von Swimlanes zur Strukturierung eines Vertriebsprozesses

5.1.3.6: Artefakte

Die bisherigen Kapitel zusammenfassend kann also abgeleitet werden, dass im Fokus der BPMN der zeitlich-logische Ablauf von Prozessen steht. Dieser wird über spezifische Flusselemente, die durch Kontrollflüsse verbunden sind, abgebildet – und durch Swimlanes strukturiert.

Allerdings ist diese zeitlich-logische Abfolge nicht der einzige Aspekt, der in einem Prozessmodell abgebildet werden kann. Neben diesem kann ein Prozess auch mit sog. Artefakten angereichert werden. Diese sind als eine weitere Kategorie von BPMN-Notationselementen zu verstehen.

Artefakte ergänzen konkrete Notationselemente in einem BPMN-Prozessmodell mit zusätzlichen Informationen (vgl. Allweyer 2015, S. 156). Sie sind dabei über Assoziationen mit dem jeweiligen Element verbunden. Durch ihren Einsatz wird das Ziel verfolgt, einen Prozess bspw. noch genauer zu beschreiben, oder ihn leichter verständlich zu machen. Dabei sind Artefakte allerdings weder ein Teil der zeitlich-logischen Abfolge, noch haben sie einen Einfluss auf diese (vgl. Debevoise und Taylor 2016, S. 106). Folglich besitzen sie auch keine Ausführungssemantik.

Die BPMN stellt im Standard mehrere Artefakte bereit. Zu diesen zählen die sog. Anmerkung, die Datenbank und das Datenobjekt. In der Folge wird auf diese Artefakte jeweils kurz eingegangen:

Anmerkung

Eine Anmerkung ist zu verstehen als eine ergänzende Information zu einem Notationselement in einem Prozessmodell (vgl. Freund und Rücker 2017, S. 102). Sie ist als ein Freitext formuliert und wird über eine Assoziation an das jeweilige Element angehängt. Sie wird in der Regel verwendet, um komplexe Schritte in einem Prozess genauer zu beschreiben, oder um weitere Informationen zu deren Durchführung zu dokumentieren (siehe Abb. 27).

Datenobjekt & Datenbank

Da Prozesse in der Regel digital ablaufen, werden durch ihre Ausführung viele Daten erstellt, bzw. verarbeitet. Die Daten repräsentieren dabei stets konkrete Objekte, wie eine Bestellung, und sind in bestimmten Orten, wie etwa einer Datenbank abgelegt (vgl. Allweyer 2015, S. 134). Um diese Elemente abzubilden, können die Artefakte Datenobjekt und Datenbank genutzt werden. Je nach Art des Einsatzes besitzen diese dabei eine unterschiedliche Bedeutung. Im Beispiel von Abb. 27 geben sie etwa an, aus welcher Quelle die Daten für eine konkrete Aktivität bezogen werden, und welches Objekt im Zuge des Prozesses verarbeitet wird. Insgesamt machen sie den Prozess somit transparenter und stellen ihn – im Kontext zu den verwendeten Daten – detaillierter dar.

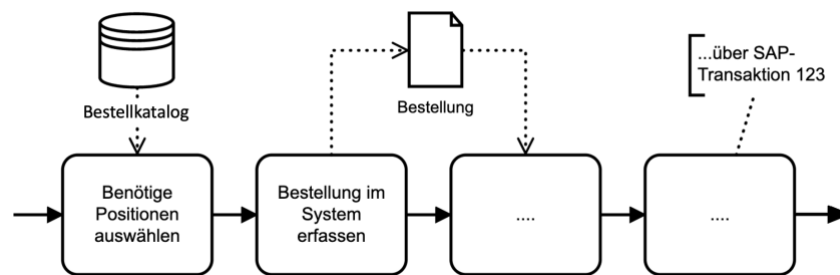


Abbildung 27: Daten und Anmerkungen als beispielhafte Artefakte der BPMN

5.2: Process Engine

Eine Process Engine ist zu interpretieren als eine Software zur Teil- bzw. Vollautomatisierung von Prozessen (vgl. Leymann 2018). Sie ist für gewöhnlich eine Komponente in einem sog. Business Process Management System (BPMS). Eine Architektur eines solchen ist in Abb. 28 dargestellt.

Ein BPMS, zeichnet sich dadurch aus, dass ein Prozess und dessen Ausführungslogik nicht durch klassische Softwareentwicklung implementiert wird. Stattdessen wird er durch ein Prozessmodell abgebildet (vgl. Scheer 2020, S. 80). Dieses wird in einem sogenannten **Process Model Repository** gespeichert. Zudem muss es in einer konkreten Notation modelliert sein, welche ein Metamodell und eine Ausführungssemantik besitzt.

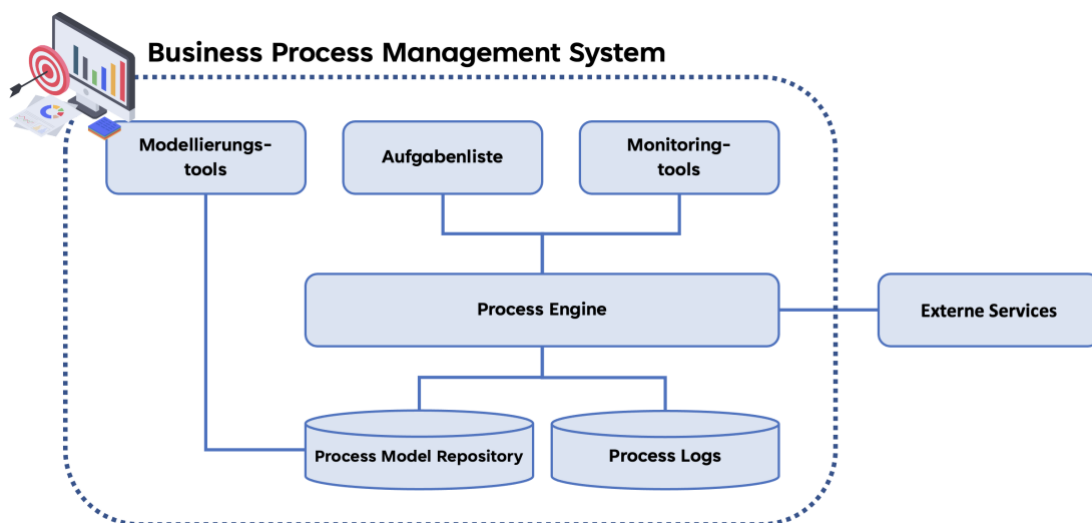


Abbildung 28: Exemplarische Architektur eines Business Process Management Systems⁹

⁹ In Anlehnung an: Dumas et al. 2018, S. 348

Spezialisierung auf eine Notation

Faktisch kann also abgeleitet werden, dass ein BPMS immer nur Prozessmodelle einer konkreten Notation unterstützt. Dazu implementiert es das Metamodell und die Ausführungssemantik der jeweiligen Notation. Im Rahmen dieser Thesis ist diese Notation stets die BPMN – in der Theorie kann es jedoch auch BPM-Systeme geben, welche andere Notationen unterstützen.

Aufgrund der Spezialisierung ist eine Process Engine in der Lage, Prozessmodelle, welche mittels der unterstützten Notation modelliert sind, technisch zu interpretieren. Folglich kann sie genutzt werden, um Prozesse auf Basis von diesen Modellen auszuführen, zu steuern und zu kontrollieren. Ihr Fokus liegt dabei auf dem zeitlich-logischen Ablauf der Prozesse. Im Kontext zu diesem führt sie diverse Aufgaben aus. Diese werden in der Folge beschrieben und durch Abb. 29 verdeutlicht.

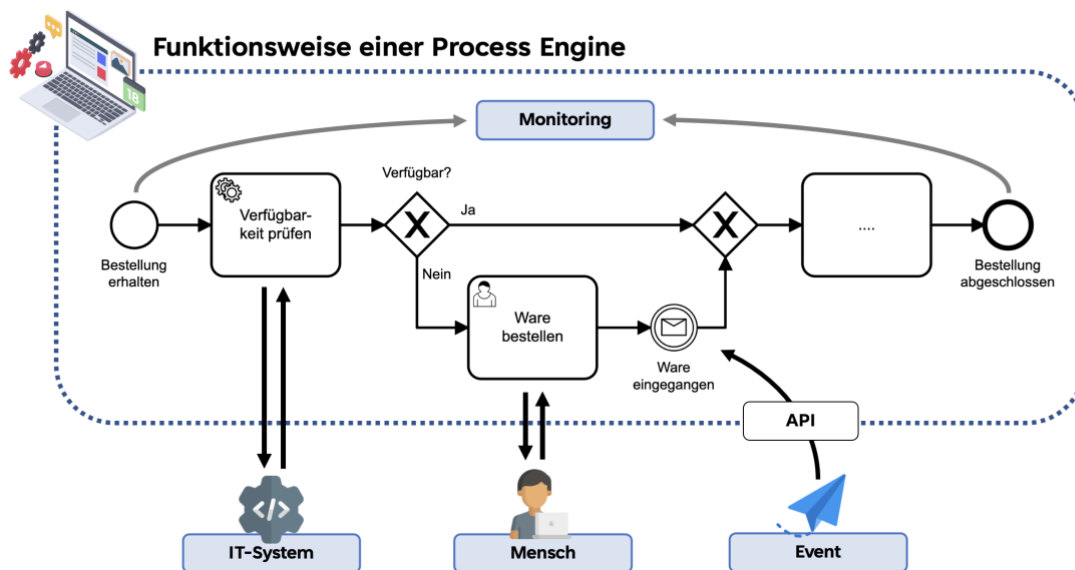


Abbildung 29: Funktionsweise einer Process Engine ¹⁰

5.2.1: Funktionsweise einer Process Engine

Eine Process Engine führt Prozesse strikt gemäß einem Prozessmodell aus. Sie erstellt, auf Basis dieses Modells, neue Instanzen des Prozesses, sobald ein gültiges Starterereignis eintritt. Jede der Instanzen ist dabei eindeutig identifizierbar und verfügt über eigene Daten (vgl. Debevoise und Taylor 2016, S. 23). Diese Daten werden für gewöhnlich als Prozessvariablen bezeichnet und von der Engine in sog. **Process Logs** gespeichert. Sie können im Zuge einer Prozessausführung erstellt, oder der Engine von außen übergeben werden. Alles in allem sind sie dabei für unterschiedlichste Zwecke relevant. Ein solcher kann z.B. die Durchführung einer spezifischen Aktivität sein.

Innerhalb der einzelnen Prozessinstanzen automatisiert eine Engine den Übergang zwischen den Prozessschritten (vgl. Becker et al. 1999, S. 4). Dabei überwacht sie kontinuierlich den Status der Prozesse. In Abhängigkeit zu diesem Status, und orientiert am Prozessmodell, steuert die Engine den Kontrollfluss, wartet auf Ereignisse, durchläuft Gateways und löst bei Bedarf die zeitlich-logisch gestaffelten Aktivitäten eines Prozesses aus.

Die Engine selbst führt in der Regel jedoch keine Aktivitäten aus. Sie koordiniert Aktivitäten, auf Basis des Modells, an spezifische Akteure, welche hierfür zuständig sind. Ein Akteur kann dabei sowohl ein Mensch als auch eine Software sein. Im Kontext zur BPMN wird der Akteur über den

¹⁰ In Anlehnung an: Freund und Rücker 2017, S. 6

Typ einer Aufgabe definiert (siehe Abb. 29). Folglich weist eine Engine eine Aktivität entweder, über eine **Aufgabenliste**, einem menschlichen Prozessteilnehmer zu, oder ruft über spezifische Schnittstellen **externe Services** auf (vgl. Freund und Rücker 2017, S. 6). Diese Services wiederum führen die Aufgabe automatisch, bzw. synchron oder asynchron aus. Ferner können mittels der Engine die Daten bereitgestellt werden, welche die Akteure benötigen, um die jeweiligen Schritte durchzuführen (vgl. zur Mühlen und Hansmann 2012, S. 370). Die Informationen darüber, welche Daten dies sind, können ebenfalls im Prozessmodell hinterlegt sein.

Wird ein Prozessschritt abgeschlossen, so verarbeitet die Engine dessen Resultat. Dabei speichert sie gegebenenfalls konkrete Daten, welche im Zuge des Schrittes erzeugt werden. Zudem steuert sie den weiteren Verlauf des Prozesses. Darüber hinaus loggt eine Engine jedoch auch spezifische Informationen zu relevanten Ereignissen im Ablauf eines Prozesses. Zu diesen zählen in der Regel jegliche Statusübergänge im Prozess und dessen Aktivitäten. Ein solcher Übergang ist dabei z.B. der Start, oder das Ende einer Aktivität (vgl. zur Mühlen und Hansmann 2012, S. 371–372).

All diese Daten werden in der Regel in einem historischen Kontext, sprich dauerhaft gespeichert. Sie werden nach der Ausführung der Instanz nicht gelöscht. Dadurch sollen die Daten, im Kontext zum Lebenszyklus des Prozesses, für die Phase des Prozesscontrollings nutzbar gemacht werden. So können, mittels der Nutzung von **Monitoring-Tools**, Auswertungen zu den Prozessen erstellt und ihr jeweiliger Ablauf im Nachhinein nachvollzogen werden können.

5.2.2: Exemplarische Process Engines

Insgesamt kann also abgeleitet werden, dass eine Process Engine jegliche Aufgaben im Kontext zur Steuerung eines Prozesses übernimmt. Damit stellt es ein hochflexibles System mit geringem Aufwand zur Prozessimplementierung und -anpassung dar.

Ein Unternehmen, dass eine Process Engine einsetzen will, um von diesen Vorteilen zu profitieren, muss diese allerdings nicht selbst entwickeln – denn auf dem freien Markt gibt es bereits viele solcher Lösungen. Diese werden zum Teil zwar nur über Lizenzen angeboten, sind oft aber auch als Open-Source Lösungen frei verfügbar. Eine sehr bekannte Process Engine, die auf die BPMN spezialisiert ist, ist die des Unternehmens Camunda. Dieses bietet seine Softwarelösung sowohl als Open-Source- als auch als lizenzbasiertes Cloud-Produkt an. Im praktischen Teil dieser Arbeit wird diese Engine zur Automatisierung von konkreten Prozessen exemplarisch angewendet.¹¹

5.3: Decision Model and Notation

Die **Decision Model and Notation** (DMN) ist eine Methode zur Modellierung und Ausführung von Entscheidungen. Sie ist der BPMN in vielen Punkten ähnlich. Sie wird, wie die BPMN, von der OMG verwaltet. Zudem besteht sie ebenfalls aus einer Notation und einem Metamodell.

Die DMN ist, analog zur BPMN, im Kern als eine Modellierungssprache zu interpretieren. Ihr Fokus liegt alleinig auf Entscheidungen. Diese stellt sie in Entscheidungsmodellen dar. Ein vollständiges DMN-basiertes Entscheidungsmodell beschreibt eine Entscheidung dabei auf zwei Ebenen – auf einer sog. Decision Requirement Ebene und einer Ebene der Entscheidungslogik. Beide Ebenen werden in den Kapiteln 5.3.1 und 5.3.2 ausführlicher erörtert.

¹¹ Für mehr Informationen zu Camunda – siehe: <https://www.camunda.org/>

Durch ihre Beschränkung auf Entscheidungen verfolgt die DMN das Ziel, diese verständlich und transparent zu modellieren (vgl. OMG 2021, S. 9). Folglich kann sie für diverse Zwecke eingesetzt werden. Zu diesen zählen etwa die Dokumentation, oder Analyse von Entscheidungen.

Der Fokus der DMN liegt jedoch, wie der in der BPMN, auf technischen Zwecken – und dabei vor allem auf der Automatisierung von Entscheidungen. Diese ist möglich, da DMN-Modelle ebenfalls durch geeignete Softwarelösungen technisch interpretierbar sind. Sie werden ebenfalls in einem XML-Format abgebildet und können auf Basis einer Ausführungssemantik gesteuert werden. Die Lösungen, welche hierfür geeignet sind, werden in der Regel als Business Rule Engines oder DMN-Engines bezeichnet. Auf sie wird im Rahmen von Kapitel 5.4 tiefer eingegangen.

5.3.1: Decision Requirement Diagram

Die erste Abstraktionsebene, auf der ein DMN-basiertes Entscheidungsmodell eine Entscheidung beschreibt, ist die Decision Requirements Ebene. Auf dieser Ebene wird die Gesamtstruktur einer Entscheidung abgebildet (vgl. Figl et al. 2018, S. 1). Dies erfolgt, indem die Entscheidung in sog. Decision Requirement Diagrammen (DRD) graphisch modelliert wird.

Ein DRD setzt sich dabei aus einer Menge von Notationselementen zusammen, welche die DMN bereitstellt. Die wichtigsten dieser Elemente sind die Decision, das Business Knowledge Model, die Knowledge Source und Input Data. Diese vier Hauptelemente werden über Konnektoren, die als Requirement Links bezeichnet werden, in Beziehungen gesetzt. Zu diesen Konnektoren zählen das Information-, das Knowledge- und das Authority Requirement.

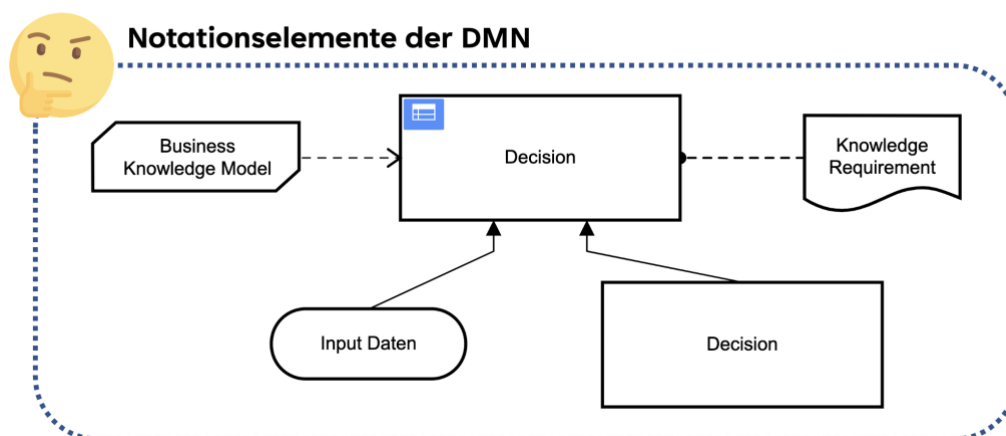


Abbildung 30: Notationselemente und Abhängigkeiten in einem abstrakten DMN-Modell

5.3.1.1: Input Data

Das Element „**Input Data**“ repräsentiert Informationen, welche zum Treffen einer Entscheidung benötigt werden. Diese Informationen liegen in der Regel in Form von Daten vor. Deren Ursprung liegt jedoch nicht innerhalb der jeweiligen Entscheidung (vgl. Debevoise und Taylor 2016, S. 54). Sie werden der Entscheidung von außerhalb übergeben – etwa von einem Prozess, durch den die Entscheidung ausgeführt wird. Die Daten existieren folglich bereits vor der Entscheidung. Sie sind zum Beispiel in Datenbanken gespeichert.

In einem DRD werden die Input Data Elemente über sogenannte „**Information Requirements**“ mit Decisions verbunden. Dabei beschreibt ein Input Data Element die benötigten Informationen oft

nicht im Detail, sondern mit einem groben Abstraktionsgrad. Das heißt, sie werden hauptsächlich als komplette Geschäftsobjekte, wie ein Kunde, eine Bestellung, oder eine Rechnung modelliert (vgl. Decision Management Solutions 2016, S. 6). Die Details dieser Objekte, sprich ihre Attribute, werden graphisch nicht dediziert dargestellt.

5.3.1.2: Knowledge Source

Eine „**Knowledge Source**“ ist ein Element, welches eingesetzt werden kann, um auf Faktoren zu verweisen, die einen Einfluss auf die Ausführung und die Logik einer Entscheidung haben. Solche Faktoren können z.B. Gesetze, interne Regelungen, Best-Practices, oder Individuen sein (vgl. OMG 2021, S. 11-12). Eine Knowledge Source kann über „**Authority Requirements**“ mit einem oder mehreren Elementen verbunden werden, die eine ausführbare Entscheidungslogik in sich tragen. Solche Elemente sind entweder eine Decision oder ein Business Knowledge Model. Die Knowledge Source selbst trägt jedoch keine ausführbare Logik in sich. Sie hat einen rein dokumentierenden Charakter (vgl. Decision Management Solutions 2016, S. 8).

5.3.1.3: Business Knowledge Model

Ein „**Business Knowledge Model**“ (BKM) bildet eine Logik ab, auf deren Basis eine Entscheidung getroffen wird. Es besitzt eine konkrete, spezifizierte und ausführbare Entscheidungslogik. Diese kann in unterschiedlichen Formen beschrieben sein, welche in Kapitel 5.3.2. eingegrenzt werden. Ein BKM wird ausgeführt, um Entscheidungen zu treffen. Dabei erhält es eine Menge von Inputs, verarbeitet diese auf Basis der Logik und gibt ein Ergebnis als Output zurück. Weiterhin zeichnet sich das BKM dadurch aus, dass es wiederverwendbar ist und somit in mehreren Entscheidungen verwendet werden kann (vgl. Debevoise und Taylor 2016, S. 55). Das BKM kann über „**Knowledge Requirements**“ mit Decisions oder mit anderen BKM's verbunden werden.

5.3.1.4: Decision

Eine „**Decision**“ ist ein Element, welches eine einzelne Entscheidung repräsentiert. Dieses kann, analog zum BKM, eine eigene Entscheidungslogik besitzen, oder auf ein solches referenzieren. In einem DRD kann es mehrere Decisions geben, die zueinander in Beziehungen stehen (siehe Abb. 30). Aus allen Einzelentscheidungen in einem DRD ergibt sich sodann eine Gesamtentscheidung.

Um eine solche zu treffen, benötigt eine jede Decision konkrete Daten (vgl. Decision Management Solutions 2016, S. 6). Daher nimmt sie jeweils eine Menge von Inputs entgegen. Ein Input für eine Decision kann dabei entweder durch das Element Input Data modelliert werden, oder der Output einer anderen Decision sein (vgl. OMG 2021, S. 14). Unter Einbezug der Inputs führt die Decision folgend darauf die Entscheidungslogik aus und gibt einen Output zurück.

5.3.2: Entscheidungslogik

Die zweite Abstraktionsebene, auf welcher die DMN eine Entscheidung betrachtet, ist die Ebene der Entscheidungslogik. Auf dieser Ebene wird, wie der Name ableiten lässt, die Logik definiert, auf Grundlage welcher eine Entscheidung tatsächlich getroffen wird. Folglich ist diese Ebene eher

technischer Art. Sie beschreibt, wie eine Entscheidung implementiert ist. Die jeweilige Logik wird dabei, wie im letzten Kapitel angedeutet, durch Decisions oder BKM's abgebildet.

Laut der DMN-Spezifikation kann eine Entscheidungslogik in verschiedensten Formen abgebildet werden. Die einzige Voraussetzung ist, dass die Logik durch eine Funktion abgebildet sein muss. Als praktische Beispiele für Formen, welche diese Anforderungen erfüllen, nennt die Spezifikation u.a. Geschäftsregeln, Entscheidungstabellen und analytische Modelle (vgl. OMG 2021, S. 16).

Kontext zur künstlichen Intelligenz

Zu den analytischen Modellen zählen dabei laut der Spezifikation vor allem PMML-Modelle (vgl. OMG 2021, S. 8, 16, 51). Diese werden im Zuge von Kapitel 3.3 erörtert. Sie bilden ein ML-Modell in einer standardisierten und technisch interpretierbaren Form ab. Faktisch ermöglicht die DMN also eine Einbindung, bzw. einen direkten Einsatz von konkreten ML-Modellen. Diese Möglichkeit wird jedoch nur sehr selten angewendet, da die meisten DMN-Engines hierzu nicht geeignet sind. Nichtsdestotrotz ist sie von hoher Relevanz für diese Thesis und wird in den späteren Kapiteln tiefer erörtert.

Ebenfalls eine hohe Relevanz haben sog. Entscheidungstabellen. Diese werden in der Praxis am häufigsten eingesetzt (vgl. Figl et al. 2018, S. 2). Daher wird im Folgenden auf sie eingegangen.

5.3.2.1: Entscheidungstabelle

Eine Entscheidungstabelle ist ein Konstrukt, mittels welchem die Logik einer Entscheidung, über eine Menge von Entscheidungsregeln, strukturiert und einfach dargestellt werden kann. Wie jede andere Tabelle auch, besteht sie aus mehreren Zeilen und Spalten. Dabei repräsentiert jede Zeile eine konkrete Entscheidungsregel – jede Spalte hingegen einen Input oder einen Output dieser Regeln (vgl. Calvanese et al. 2016, S. 1). Die beiden Dimensionen sind wie folgt zu beschreiben:

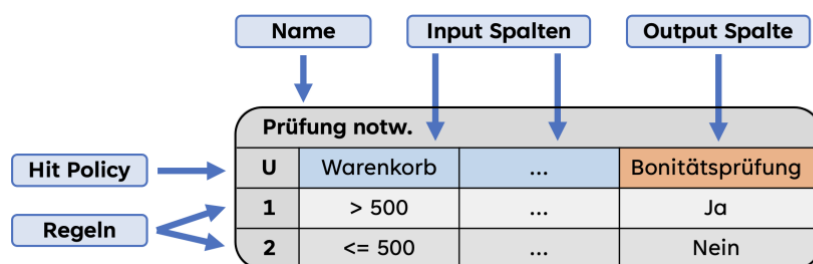


Abbildung 31: Grundlegender Aufbau einer DMN-basierten Entscheidungstabelle

Die Input- und Output-Spalten sind zu verstehen als einzelne Variablen von Datenobjekten. Eine solche könnte etwa der Wert eines Warenkorbs in einer Bestellung sein. Zudem besitzt jede Spalte einen konkreten Typ (vgl. Calvanese et al. 2016, S. 1). Somit kann sie z.B. eine Nummer oder eine Zeichenfolge repräsentieren.

Die Entscheidungsregeln hingegen sind zu verstehen als logische Ausdrücke. Sie setzen sich aus einer begrenzten Menge von logischen Bedingungen zusammen, welche mit logischen Operatoren definiert werden. Eine Regel kann für jede Input-Spalte der Entscheidungstabelle eine Bedingung besitzen. Sind alle Bedingungen einer Regel erfüllt, gibt sie einen definierten Output zurück.

Um ein Entscheidungsmodell ausführen zu können, müssen die zugehörigen Bedingungen in einer Form abgebildet werden, die von einer Business Rule Engine technisch interpretiert werden kann. Die DMN schränkt nicht ein, welche Form dies sein kann. Sie ist abhängig davon, welche Formen

eine konkrete Engine unterstützt. Die DMN-Spezifikation stellt jedoch, als einen Quasi-Standard, die „**Friendly Enough Expression Language**“ (FEEL) vor (siehe OMG 2021, S. 89ff). Diese ist in der Praxis am präsentesten und wird von allen bekannten DMN-Engines unterstützt.

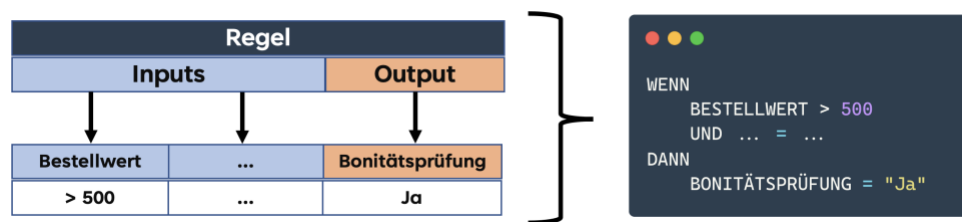


Abbildung 32: Aufbau und Logik einer Entscheidungsregel

FEEL – Friendly Enough Expression Language

Die **FEEL** ist zu verstehen als eine Sprache, zur Beschreibung von Entscheidungslogik, in Form von logischen Bedingungen (vgl. Figl et al. 2018, S. 2). Sie kann von Rechnern technisch interpretiert werden und ermöglicht somit die Automatisierung der Logik. Sie stellt eine Reihe von Operatoren und Funktionen bereit, um Bedingungen zu formulieren. Die wichtigsten dieser Sprachkonstrukte bzw. die für diese Arbeit relevanten, sind in Tabelle 1 gelistet und beschrieben. Für ausführlichere Informationen zu diesen, sowie den anderen Konstrukten der FEEL wird auf die Dokumentation des Unternehmens Camunda verwiesen.¹²

Name	Typ	Operator	Beispiel	Beschreibung
Gleich	Zeichenfolge		„Text“	Wert entspricht „Text“
Gleich	Zahl		10	Wert entspricht 10
Größer	Zahl	>	> 10	Wert ist größer als 10
Größer gleich	Zahl	>=	>= 10	Wert ist größer oder gleich 10
Kleiner	Zahl	<	< 10	Wert ist kleiner als 10
Kleiner gleich	Zahl	<=	<= 10	Wert ist kleiner gleich 10
Zwischen (mit ein- oder ausgeschlossenen Grenzwerten)	Zahl	[X..Y]]X..Y[[X..Y] [X..Y[	[0..10]]0..10[]0..10] [0..10[	Wert ist zwischen 0 und 10 Wert ist zwischen 1 und 9 Wert ist zwischen 1 und 10 Wert ist zwischen 0 und 9

Tabelle 1: Logische Vergleiche mittels FEEL-Operatoren

Hit-Policies der Entscheidungstabellen

Abgesehen von den Regeln, welche in einer konkreten Form abgebildet sein müssen, besitzt jede ausführbare Entscheidungstabelle auch eine sogenannte Hit-Policy. Diese beeinflusst, auf welche Art und Weise eine Entscheidungstabelle ausgeführt wird und wie sich der finale Output bildet.

Die DMN-Spezifikation definiert sieben verschiedene Hit-Policies. All diese Policies haben die Gemeinsamkeit, dass sie die Regeln der Entscheidungstabellen von oben nach unten durchlaufen. Sie lassen sich jedoch in zwei Kategorien gliedern: In Policies, die genau ein Ergebnis zurückgeben und in Policies, die mehrere Ergebnisse zurückgeben (vgl. Debevoise und Taylor 2016, S. 103).

¹² Ausführliche Dokumentation zur FEEL: <https://docs.camunda.io/docs/reference/feel/what-is-feel/>

In der Folge werden nicht alle Hit-Policies erörtert. Details zu diesen sind „OMG 2021, S. 71ff“ zu entnehmen. Es wird ausschließlich auf die Policy **FIRST** eingegangen, da diese für Anhang 8, bzw. die Erstellung der Prototypen von Relevanz ist.

- **FIRST:** Diese Policy prüft nacheinander, für jede Zeile einer Entscheidungstabelle, ob alle Bedingungen von dieser zutreffen. Ist dies der Fall, wird nicht mehr nach weiteren Treffern gesucht, sondern der Output der Regel zurückgegeben.

5.3.3: DMN als Ergänzung zur BPMN

Das bisherige Kapitel zur DMN resümierend kann abgeleitet werden, dass die DMN eine komplett eigenständige Notation ist. Beim Blick in die Praxis kann allerdings festgestellt werden, dass sie selten als eine solche verwendet wird – denn sie wird meistens nur in Kombination mit der BPMN eingesetzt. Auch in dieser Arbeit wird sie exklusiv in Kombination mit der BPMN betrachtet.

In diesem Kontext wird die DMN für gewöhnlich verwendet, um konkrete Entscheidungen, die in einem Prozess getroffen werden, gezielt abzubilden und zu automatisieren. Dabei wird das Ziel verfolgt, den Prozessfluss und die Entscheidungslogik voneinander abzugrenzen (vgl. OMG 2021, S. 9). Dadurch ist es möglich, dass ein Unternehmen seine Prozesse und Entscheidungen getrennt verwalten kann (vgl. Figl et al. 2018, S. 1). Zudem können Prozessmodelle erstellt werden, welche einfacher zu verstehen sind – da die oft komplexe Entscheidungslogik nicht abgebildet wird.

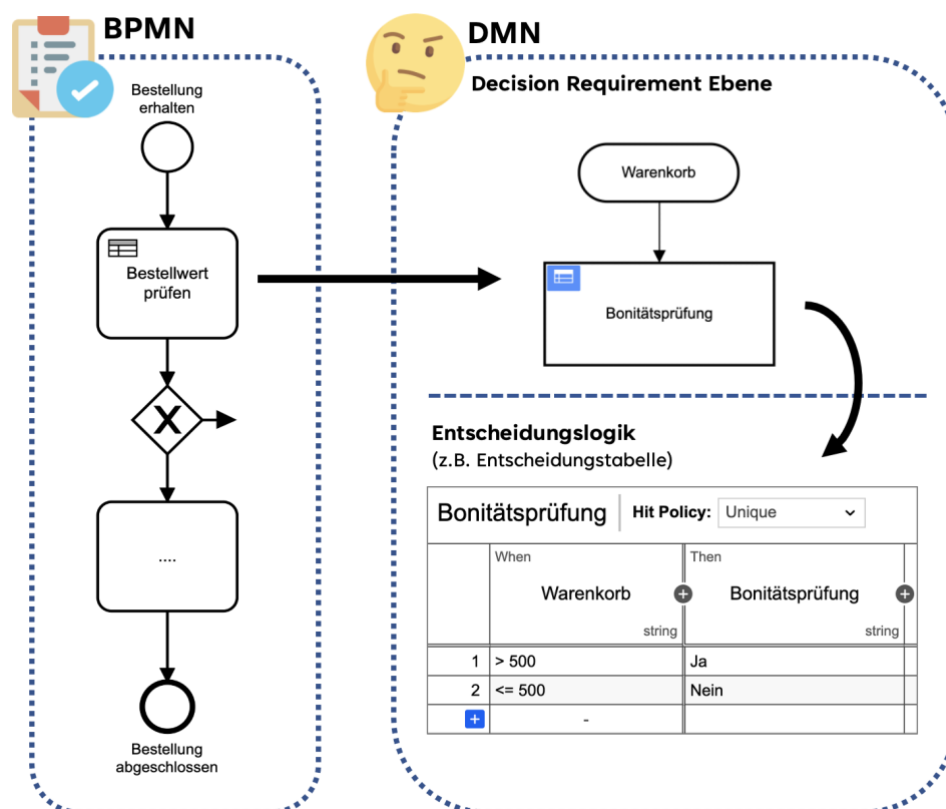


Abbildung 33: Gesamtkontext zwischen BPMN- und DMN-Modellen¹³

¹³ In Anlehnung an: Figl et al. 2018, S. 3

Ein beispielhaftes Szenario, an welchem dies verdeutlicht werden kann, ist in Abb. 33 dargestellt. Dieses stellt einen Teilausschnitt aus einem Verkaufsprozess dar. In diesem soll die Entscheidung getroffen werden, ob – in Abhängigkeit zum Wert eines Warenkorbs – eine Bonitätsprüfung für den jeweiligen Kunden durchgeführt werden muss. Abgeleitet aus diesem Beispiel, ist die einzige direkte Verbindung zwischen den Notationen ein Geschäftsregel Task (siehe Kap. 5.1.3.2). Dieser löst eine Entscheidung aus und nutzt deren Ergebnis im weiteren Prozessverlauf.

5.4: DMN-Engine

Eine **DMN-Engine** ist zu verstehen als eine Software, die zur Automatisierung von Entscheidungen eingesetzt werden kann. Sie wird im Allgemeinen häufig auch als eine sog. **Business Rule Engine** bezeichnet. Ihr Grundkonzept ist sehr ähnlich zu dem einer Process Engine (siehe Kapitel 5.2).

Orientiert an einer solchen Process Engine basieren auch die Funktionen einer DMN-Engine auf konkreten Modellen – in diesem Fall, auf DMN-basierten Entscheidungsmodellen. Diese sind in einem XML-Format abgebildet. Eine DMN-Engine ist in der Lage ein solches Modell technisch zu interpretieren. Diese Fähigkeit besitzt sie, da sie exklusiv auf die Notation spezialisiert ist. Sie unterstützt die DMN in nativer Form, indem sie sich an der Spezifikation dieser (vgl. OMG 2021, S. 371) orientiert. Dabei implementiert sie – ähnlich zu einer Process Engine – Aspekte, wie die Ausführungssemantik und das Metamodell der DMN.

5.4.1: Funktionsweise einer DMN-Engine

Eine DMN-Engine führt Entscheidungen strikt gemäß einem zugrundeliegenden DMN-Modell und den Regeln der Notation aus. Ihre grundsätzliche Funktionsweise ist in Abbildung 34 skizziert. Sie erstellt, sobald ein Entscheidungsvorgang ausgelöst wird, auf Basis des Modells, eine Instanz der Entscheidung. Der Auslöser ist dabei für gewöhnlich von externer Art. Dieser kann z.B. ein Prozess, bzw. eine Aktivität sein, im Rahmen welcher eine Entscheidung getroffen werden soll. Von diesem Auslöser werden – auf Basis des jeweiligen Entscheidungsmodells – konkrete Daten an die Engine übergeben, die zum Treffen der Entscheidung notwendig sind.

Mittels dieser Daten und dem Wissen zur DMN führt die Engine die einzelnen Notationselemente des Entscheidungsmodells, respektive deren Entscheidungslogik aus. Orientiert an Kapitel 3.4.2 kann die Logik dabei z.B. durch eine Entscheidungstabelle, aber auch durch komplexere Formen, wie ein analytisches Modell repräsentiert werden. Welche konkreten Formen unterstützt werden ist jedoch abhängig von der jeweiligen DMN-Engine (siehe Kapitel 5.4.2).

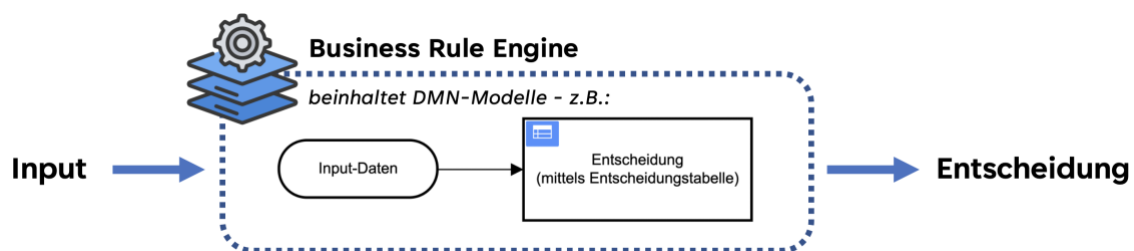


Abbildung 34: Grundkonzept und Funktionsweise einer Business Rule Engine

Das Ergebnis aus der Ausführung eines DMN-Modells ist eine konkrete Entscheidung. Diese wird für etwaige Folgehandlungen an den Auslöser zurückgegeben. Insgesamt kann eine DMN-Engine

also automatisiert Entscheidungen treffen. Darüber hinaus speichert sie, wie eine Process Engine, in der Regel verschiedene Daten zur Ausführung der Entscheidungen. Somit macht sie es für eine Nachbetrachtung nachvollziehbar, aus welchem Grund, im Zuge einer Entscheidung eine konkrete Entscheidungsoption gewählt ausgewählt wird. Ferner ermöglicht die DMN-Engine damit diverse weitere Auswertungen zu den Entscheidungen.

5.4.2: Überblick über DMN-Engines

Auch DMN-Engines sind in einer ausreichenden Zahl am Markt verfügbar. Dabei ist anzumerken, dass viele von diesen direkt in einer BPMN-Engine eingebettet sind. Denn aufgrund der engen Verbindung zwischen den beiden Notationen besitzt jede Process Engine für gewöhnlich eine solche DMN-Engine. Dies gilt auch für die Lösung des Unternehmens Camunda.

Für die meisten Anwendungszwecke sind diese eingebetteten Engines völlig ausreichend. Jedoch muss beachtet werden, dass sie in ihrem Funktionsumfang meist limitiert sind. Dies bedeutet, sie unterstützen teils nicht alle Aspekte der DMN. So unterstützt z.B. die DMN-Engine, welche in die BPMN-Engine von Camunda integriert ist, nur Entscheidungstabellen. Daher ist es nicht möglich, dass analytische Modelle, wie ein PMML-Modell, direkt durch diese ausgeführt werden können.

Daher gibt es auch einige Standalone Lösungen, die einen signifikant größeren Funktionsumfang abdecken. Zu diesen Lösungen zählt unter anderem die Business Rule Engine „**Drools**“. Diese wird im Praxisteil der Thesis – in Kapitel 7 – eingesetzt, um eine spezifische KI-basierte Entscheidung zu automatisieren.¹⁴

¹⁴ Für mehr Informationen zu Drools, siehe: <https://www.drools.org/>

6: Entwurf – Einsatz von KI in BPMN-basierten Prozessen

Im Folgenden wird die in Kapitel 4 eingeleitete Fusion zwischen Prozessen und der KI fortgesetzt. Es wird untersucht, wie die KI und die BPMN miteinander kombiniert werden können. Dabei liegt ein Fokus auf der Fragestellung, wie der Einsatz von KI durch die BPMN abgebildet und darüber hinaus automatisiert werden kann. Zudem wird erörtert, ob und wie die BPMN dabei helfen kann, die in Kapitel 4.4 erörterten Herausforderungen beim Einsatz einer KI in Prozessen zu lösen.

6.1: Rollen einer KI im Prozesskontext

Das vierte Kapitel zusammenfassend kann erörtert werden, dass die KI ein sehr großes Potential zum Einsatz in Prozessen besitzt. Sie kann für nahezu unzählige Anwendungsszenarien eingesetzt werden. Einige von diesen Szenarien werden in Kapitel 4.2 exemplarisch diskutiert.

Werden diese aus einer abstrakten Perspektive begutachtet, so kann argumentiert werden, dass eine KI-Lösung immer eine spezifische Arbeit ausführt. Das Ziel dieser Arbeit ist es, eine konkrete Problemstellung zu lösen. Darüber hinaus zeigt sich aber auch, dass die Lösung – im Kontext zu einem Prozess – vor allem zwei Rollen annehmen kann.

Im Hinblick auf diese ist eine KI-Lösung entweder als ein **Orchestrator** oder als ein **Prozessakteur** zu verstehen. Je nach Rolle hat die KI dabei eine unterschiedliche Beziehung zum Prozess – und führt andersartige Aufgaben aus. Demzufolge hat sie, abhängig zur Rolle, auch ein variierendes Verhältnis zur BPMN. Daher werden diese Rollen in der Folge jeweils näher betrachtet.

6.1.1: Künstliche Intelligenz als Orchestrator

Nimmt eine KI-Lösung die Rolle eines Orchestrators ein, so kann sie im Kontext zu einem Prozess als ein sog. Intermediär verstanden werden. Als ein solcher positioniert sich die Lösung zwischen einem Prozesskunden und einer Menge von Prozessen (siehe Abb. 35).¹⁵

Ihre Aufgabe ist es, zwischen dem Kunden und den Prozessen, eine spezifische Problemstellung zu lösen. Ein Beispiel für eine solche Problemstellung ist die Bearbeitung von Kundenanfragen im Kundenservice. Zur Lösung von dieser setzen Unternehmen – entnommen aus Kapitel 4.2 – häufig sogenannte Chatbots ein. Diese nehmen textbasierte Nachrichten von einem Kunden entgegen, klassifizieren deren Inhalt und lösen je nach Ergebnis der Kategorisierung etwaige Prozesse aus.

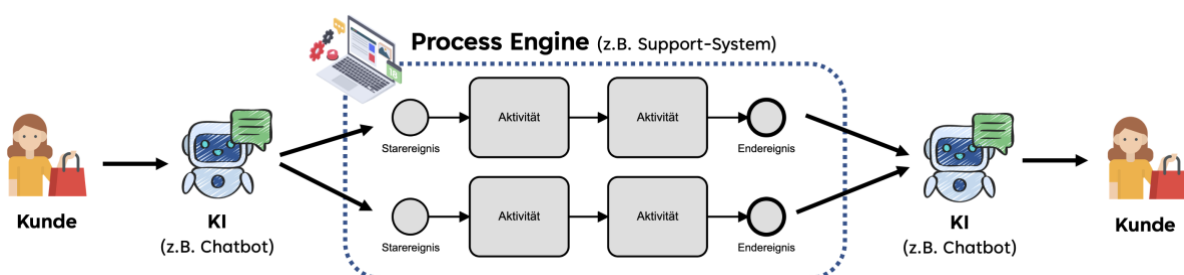


Abbildung 35: Künstliche Intelligenz als Orchestrator bzw. Intermediär im Prozess

¹⁵ Für Informationen zur grundlegenden Struktur eines Prozesses – siehe Kapitel 2.1

Folglich führt ein Orchestrator, wie ein solcher Chatbot, häufig eine Arbeit aus, welche außerhalb eines eigentlichen Prozesses liegt. Diese Arbeit ist dabei als prozessübergreifend zu beschreiben. Um sie auszuführen, benötigt ein Orchestrator ein Wissen über einen oder mehrere Prozesse. Mit diesem kann er – je nach Form der Implementierung – z.B. konkrete Prozesse auslösen, mit Daten versorgen, deren Status abfragen, oder deren Output an den Kunden zurückgeben.

Da er allerdings keinem konkreten Prozess zugeordnet ist, kann der Einsatz eines Orchestrators kaum durch die BPMN abgebildet und automatisiert werden. Daher wird in der Folge nicht weiter auf die Rolle, bzw. auf Szenarien, in welchen eine KI diese Rolle einnimmt, eingegangen.

6.1.2: Künstliche Intelligenz als Prozessakteur

Nimmt eine KI-Lösung hingegen die Rolle eines Prozessakteurs ein, so ist diese direkt, bzw. aktiv an der Ausführung eines Prozesses beteiligt. Ihre Aufgabe ist es, innerhalb dieses Prozesses, eine spezifische Problemstellung zu lösen. Diese bezieht sich dabei auf eine oder mehrere Aktivitäten in eben jenem Prozess (siehe Abb. 36).

Für solche Problemstellungen gibt es zahlreiche Beispiele. Eines von diesen ist – entnommen aus Kapitel 4.2 – die Bestimmung der Kreditwürdigkeit eines Kunden in einem Kreditvergabeprozess. Für diese kann z.B. ein Klassifikator eingesetzt werden. Auf Basis der Ergebnisse von diesem kann der jeweilige Kredit entweder gewährt oder abgelehnt werden. Ein anderes Beispiel hingegen ist das Übertragen von Daten aus Belegen in ein Buchhaltungssystem, zum Zweck der Buchhaltung. Für dieses können KI-Lösungen aus der Domäne des sog. Document Understandings verwendet werden. Diese wurden in Kapitel 4.2 betrachtet. Sie können spezifische Rechnungspositionen aus Belegen auslesen und somit zur automatisierten Buchhaltung eingesetzt werden.

Abstrakt betrachtet führt eine KI – als Prozessakteur – also immer eine Arbeit aus, welche direkt in einem Prozess positioniert ist. Im Kontext der BPMN kann diese Arbeit daher als eine konkrete Aktivität verstanden werden. Eine solche kann grundsätzlich durch die BPMN abgebildet werden. Zudem kann ihre Ausführung, aus technischer Sicht, durch eine Engine angestoßen werden.

Daher liegt in der Folge auf dieser Rolle der alleinige Fokus. Im Hinblick auf diesen wird erörtert, wie die KI und die BPMN miteinander kombiniert werden können. Dabei wird – mit Blick auf die Problemstellung der Thesis – zuerst untersucht, wie der Einsatz einer KI in einem Prozessmodell modelliert und durch eine geeignete Process Engine automatisiert werden kann. Folgend darauf wird analysiert, ob und wie die Notation, bei der Bewältigung der Herausforderungen, welche in Kapitel 4.2 beschrieben sind, helfen kann.

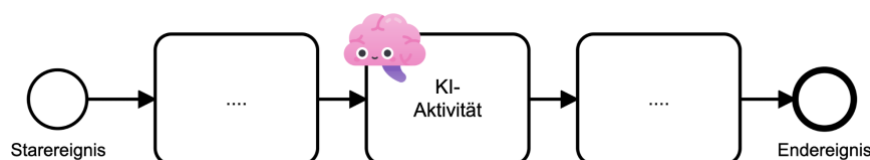


Abbildung 36: Künstliche Intelligenz als aktiv am Prozess beteiligter Akteur

6.2: Modellierung eines KI-Einsatzes in der BPMN

Um eine Aktivität, in welcher eine KI-Lösung eingesetzt wird, in einem Prozessmodell abzubilden, kann das BPMN-Notationselement „**Aufgabe**“ verwendet werden. Dieses wird in Kapitel 5.1.3.2

beschrieben. Es repräsentiert einen spezifischen Arbeitsschritt in einem Prozess, der von einem festgelegten Akteur ausgeführt werden kann.

In seiner Rohform – sprich als nicht typisierter Task – lässt sich aus dem Element allerdings nicht ableiten, dass zur Durchführung der Aufgabe eine KI-Lösung eingesetzt wird. Außerdem kann die Aufgabe in dieser Form nicht automatisiert werden. Um diese Probleme zu lösen, kann sie jedoch mit sogenannten Typen annotiert werden. Diese werden ebenfalls im Rahmen von Kapitel 5.1.3.2 erörtert. Sie stellen jeweils eine spezielle Form einer Aufgabe dar – z.B. einen Service Task.

6.2.1: Typen zur Modellierung eines KI-Einsatzes

Theoretisch betrachtet kann der Einsatz einer KI durch diverse solcher Typen modelliert werden. In der Folge wird daher auf vier exemplarische Typen, welche im Besonderen dafür geeignet sind, eingegangen. Diese sind in Abbildung 37 visuell dargestellt.

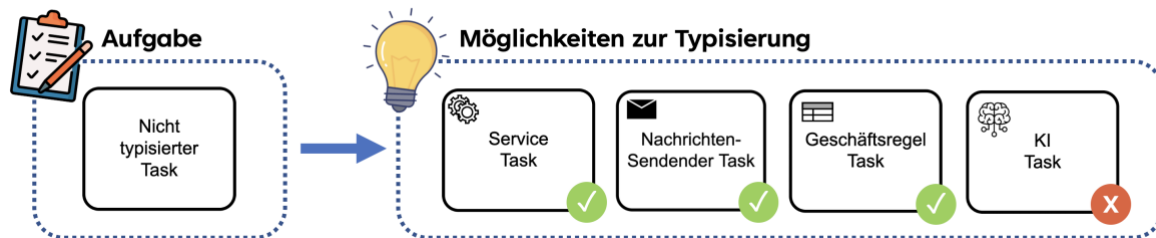


Abbildung 37: Möglichkeiten zur Typisierung einer Aufgabe in der eine KI eingesetzt wird

6.2.1.1: Modellierung als Service Task

Jede KI-Lösung kann als eine konkrete Art einer Software interpretiert werden. Mit Blick auf die Typen, welche der BPMN-Standard zur Verfügung stellt, ist es daher nur logisch, dass der Einsatz einer solchen Lösung als ein Service Task modelliert werden kann. Dieser ist, laut der Definition in Kapitel 5.1.3.2, zu verstehen als eine Aufgabe, die automatisiert durch eine beliebige Software ausgeführt wird. Diese Anforderung erfüllt jede KI-Lösung. Dabei ist es egal, ob diese, im Hinblick auf eine Make-Or-Buy Entscheidung, von einem Anbieter beschaffen, oder eigenentwickelt ist.

6.2.1.2: Modellierung als Nachrichten-Sendender Task

Der Nachrichten-Sendende Task ist sehr ähnlich zum Service Task. Orientiert an seiner Definition aus Kapitel 5.1.3.2 ist er zu verstehen als eine Aufgabe, im Zuge welcher eine Nachricht an einen externen Akteur gesendet wird. Ein solcher kann dabei von diverser Art sein – und somit z.B. auch durch eine Software repräsentiert werden. Folglich kann ein solcher Akteur logisch gefolgert auch eine KI-Lösung sein. Die Nachricht, die an diese gesendet wird, könnte wiederum eine Anfrage zum Treffen einer Vorhersage sein. Daher kann abgeleitet werden, dass auch dieser Aufgabentyp eingesetzt werden kann, um den Einsatz einer beliebigen KI-Lösung abzubilden.

Insgesamt muss jedoch berücksichtigt werden, dass der Task als abgeschlossen gilt, sobald die Nachricht versendet wurde. Demzufolge wird nicht auf das Ergebnis der KI gewartet. Um dieses zu erhalten, muss es durch ein Nachrichten-Empfangendes Ereignis im Prozessmodell abgebildet werden (siehe Abb. 38). Die Theorie zu einem solchen wird im Zuge von Kapitel 5.1.3.3 diskutiert.

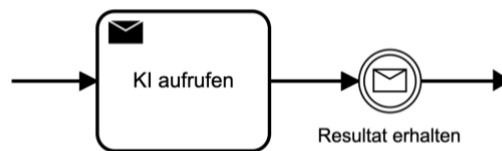


Abbildung 38: Modellierung eines KI-Einsatzes als Nachrichten-Sender Task

6.2.1.3: Modellierung als Geschäftsregel Task

Wird die Funktionsweise von gängigen KI-Einsatzszenarien näher untersucht, so kann abgeleitet werden, dass eine KI häufig Vorhersagen trifft. Diese können meist als konkrete Entscheidungen verstanden werden. Dies lässt sich am Beispiel einer Klassifikation verdeutlichen. In einer solchen entscheidet sich ein ML-Modell dazu – auf Grundlage von erlernten Regeln – einen vorliegenden Datensatz in eine spezifische Klasse einzuordnen.

Daher ist es logisch ableitbar, dass der Einsatz einer KI-Lösung auch über einen Geschäftsregel Task modelliert werden kann. Ein solcher ist dabei, gemäß der Definition aus Kapitel 5.1.2.2, zu interpretieren als eine Entscheidung, welche auf Basis eines Entscheidungsmodells, automatisch durch eine Business Rule Engine getroffen wird.

6.2.1.4: Modellierung als KI-Task

Mit dem Service Task, dem Nachrichten-Sendenden Task und dem Geschäftsregel Task wurden bisher nur Typen erörtert, welche die BPMN im Standard zur Verfügung stellt. Generell bietet die Notation allerdings auch die Möglichkeit eigene Aufgabentypen zu erstellen (siehe Kap. 5.2.1.22). Ergo ist es eine vierte Möglichkeit – um den Einsatz einer KI in einer Aufgabe abzubilden – einen eigenen Typen für diesen Zweck zu definieren. Dieser könnte z.B. als KI-Task bezeichnet werden, und eine generische Aufgabe repräsentieren, in der, zur Lösung einer konkreten Problemstellung, eine KI verwendet wird.

6.3: Automatisierung eines KI-Einsatzes in der BPMN

Insgesamt wurde nun also erörtert, wie der Einsatz einer KI-Lösung mittels der BPMN modelliert werden kann. Jedoch ist es allein durch diese Modellierung noch nicht möglich, den Einsatz dieser Lösung in einer Engine zu automatisieren. Damit dies erreicht werden kann, müssen die einzelnen Elemente im jeweiligen BPMN-Prozessmodell konfiguriert – und die Logik zur Ausführung dieser technisch implementiert werden. Die Notwendigkeit hierzu, sowie die theoretischen Grundlagen zu diesem Vorgang werden in Kapitel 5.1.2.1 behandelt.

Im Fokus der technischen Konfiguration des Modells steht dabei die Aktivität, bzw. die Aufgabe, in welcher die KI-Lösung eingesetzt wird. Rückblickend auf das vorherige Kapitel kann diese über eine Aufgabe abgebildet werden, die entweder als ein Service-, ein Nachrichten-Sender-, ein Geschäftsregel-, oder ein KI-Task abgebildet ist. Je nach tatsächlichem Typ des Tasks, gibt es diverse Möglichkeiten zu dieser technischen Konfiguration. Diese sind abhängig von der Process Engine, die zur Automatisierung eingesetzt wird, und vom Metamodell des jeweiligen Typen.

Da der in Kapitel 6.2.1.4 analysierte KI-Task kein Teil des BPMN-Standards ist, gibt es für diesen logischerweise keine Entität im Basis-Metamodell. Er muss dem Metamodell explizit hinzugefügt werden. Folglich kann der Task je BPMN-Anwender unterschiedlich definiert werden. Dies führt

dazu, dass keine generischen Erkenntnisse zur Automatisierung einer solchen Aufgabe abgeleitet werden können. Daher wird in der Folge nur auf die drei anderen Aufgabentypen eingegangen, die Teil des BPMN-Standards sind. Im Hinblick auf diese werden exemplarische Möglichkeiten zur Konfiguration und zur Automatisierung in einer Process Engine erörtert.

6.3.1: Automatisierung mittels Service Task

In der Theorie kann ein Service Task – um eine spezifische Aufgabe auszuführen – eine beliebige Software aufrufen (siehe Kapitel 5.1.3.2). Abstrakt betrachtet, könnte folglich abgeleitet werden, dass er auf diverseste Weisen konfiguriert und technisch implementiert werden kann.

In der Praxis ist dies jedoch kaum der Fall – denn in dieser sind die Möglichkeiten zur technischen Implementierung eines Service Tasks stets beschränkt. Sie sind abhängig vom Funktionsumfang der Engine, die zur Automatisierung des Prozesses eingesetzt wird. Eine solche kann verschiedene Funktionen zur Verfügung stellen, um eine Logik zu definieren, welche im Rahmen eines Service Tasks ausgeführt werden soll.

Eine von diesen Funktionen ist dabei – für gewöhnlich – jene, dass die Logik zur Ausführung eines Service Tasks durch eigenentwickelte Funktionen hinterlegt werden kann. Diese Funktionen sind dabei, je nach verwendeter Engine, in einer konkreten Programmiersprache, wie z.B. der Sprache Java definiert. Mittels dieser Möglichkeit kann eine KI-Lösung, wie z.B. ein ML-Modell entweder direkt in jener Funktion eingebettet – oder über spezielle Schnittstellen aufgerufen werden.

Außerdem gibt es in einigen Engines die Möglichkeit, über vordefinierte Schnittstellen, spezifische Systeme direkt aufzurufen. Solche Schnittstellen existieren jedoch – wenn überhaupt – häufig nur für unternehmensweit nutzbare Systeme, wie z.B. ein ERP-System. Da KI-Lösungen allerdings kaum dieser Charakteristik entsprechen gibt es in aller Regel keine Schnittstellen für sie.

Folglich kann eine KI-Lösung, in einem Service Task, für gewöhnlich nur mittels einer individuell entwickelten Funktion in einen BPMN-basierten Prozess integriert werden. Ein Beispiel hierfür ist in Abbildung 39 dargestellt. Dieses zeigt den Aufruf einer KI über eine eigenentwickelte Funktion. Diese ist in der Programmiersprache Java abgebildet. Da die Ausführungslogik folglich nicht im Prozessmodell definiert ist, muss die Funktion die KI-Lösung mit den Daten versorgen, die zum Einsatz von dieser benötigt werden. Diese können im Rahmen der individuellen Ausführungslogik z.B. aus den Prozessvariablen einer jeweiligen Instanz, oder aus anderen unternehmensinternen, sowie externen Quellen bezogen werden. Der eigentliche Task wird dabei so konfiguriert, dass er die Funktion auslöst und ihr Ergebnis in einer Prozessvariable speichert.

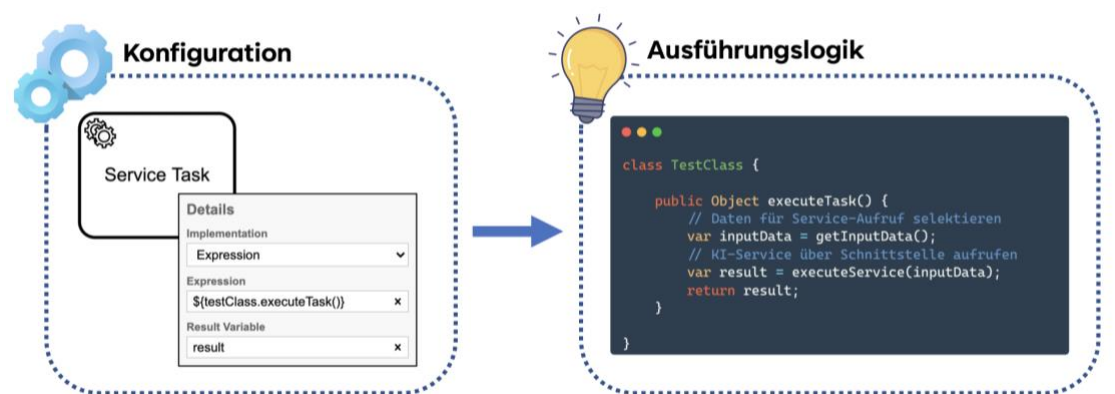


Abbildung 39: Technische Implementierung eines Service Tasks

6.3.2: Automatisierung mittels Nachrichten-Sendendem Task

Ähnliches gilt für einen Nachrichten-Sendenden Task. Auch bei diesem sind die Möglichkeiten zur Konfiguration und zur technischen Implementierung, abhängig von den Funktionen, welche eine Process Engine zur Verfügung stellt. Da er jedoch sehr ähnlich zu einem Service Task ist, können für gewöhnlich die gleichen Funktionen wie bei einem solchen genutzt werden.

Dementsprechend kann die Logik zum Einsatz einer KI also auch bei einem Task von diesem Typ durch eine individuell entwickelte Funktion abgebildet werden. Der zugehörige Task wiederum kann so konfiguriert werden, dass er diese Funktion ausführt (siehe Abb. 40). Alles in allem muss dabei jedoch berücksichtigt werden, dass – orientiert an der Theorie – nicht auf das Ergebnis der KI gewartet wird. Stattdessen muss der Empfang, wie es in Kapitel 6.2.1.2 angemerkt wird, durch ein Nachrichten-Empfängendes Ereignis abgebildet werden. Daher endet der Task nicht damit, dass das Ergebnis der KI-Lösung in z.B. einer Prozessvariable gespeichert wird. Dies wird – wenn nötig – erst mit dem Empfangen des Ereignisses durchgeführt.

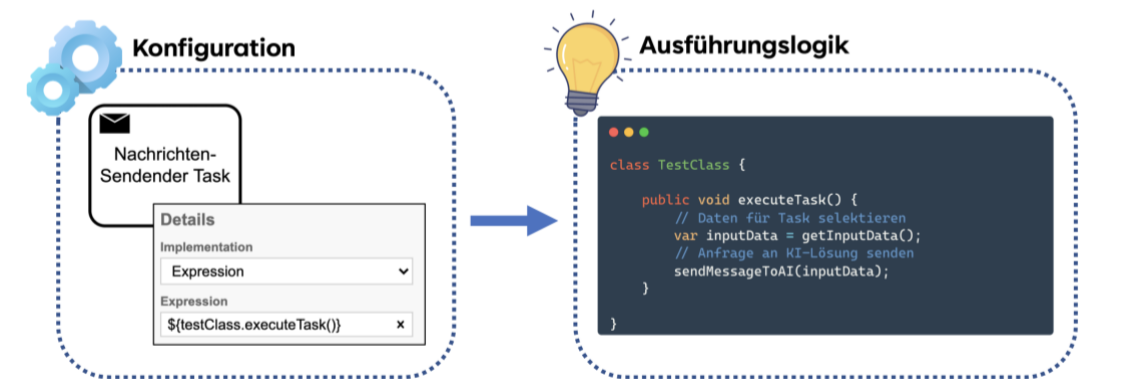


Abbildung 40: Technische Implementierung eines Nachrichten-Sendenden Tasks

6.3.3: Automatisierung mittels Geschäftsregel Task

Analog zu den anderen analysierten Aufgabentypen, sind auch die Möglichkeiten zur technischen Implementierung eines Geschäftsregel Tasks abhängig von den Funktionen einer Process Engine. Dabei können – da er ebenfalls sehr ähnlich zu einem Service Task ist – in der Regel die gleichen Funktionen eingesetzt werden, wie bei einem solchen. Für gewöhnlich stellt eine Engine aber auch weitere Funktionen bereit, die explizit geeignet sind, um Entscheidungen zu automatisieren.

Am geläufigsten ist dabei die Unterstützung der DMN, durch die Bereitstellung von DMN-Engines. Denn mit dieser Notation können, wie in Kapitel 5.3 erörtert wird, Entscheidungen modelliert und automatisiert werden. Somit kann im Zuge eines Geschäftsregel Tasks eine DMN-Engine genutzt werden, um ein bestimmtes DMN-Modell auszuwerten.

Im Hinblick auf diese Möglichkeit sind primär zwei Varianten denkbar, inwiefern eine KI-Lösung, aus technischer Sicht, in ein solches DMN-Modell integriert und folglich ausgeführt werden kann. Diese werden in den folgenden beiden Unterkapiteln näher erörtert.

6.3.3.1: Variante 1 – PMML-Modell in Business Knowledge Model

Eine KI-Lösung kann – in der Theorie – nativ in ein DMN-Modell eingebunden werden. Die einzige Voraussetzung hierfür ist, dass sie in Form eines sog. PMML-Modells vorliegt. Die Grundlagen zu

diesem werden in Kapitel 3.3 erörtert. Im Allgemeinen ist es zu verstehen als eine Datei, welche ein ML-Modell in einer standardisierten Form abbildet.

In der DMN hingegen wird ein PMML-Modell als ein analytisches Modell interpretiert, das eine spezifische Entscheidungslogik in sich trägt. Als ein solches kann es, in einem DMN-Modell, durch ein Business Knowledge Model abgebildet werden. Dieses wiederum kann durch eine Decision aufgerufen und somit durch eine geeignete DMN-Engine automatisiert ausgeführt werden.¹⁶

Um also ein PMML-Modell in einem BPMN-basierten Prozess zu verwenden, muss der zugehörige Geschäftsregel Task so konfiguriert werden, dass er jene Decision aus dem DMN-Modell auslöst. Diese trifft sodann eine automatisierte Entscheidung, indem sie die Entscheidungslogik, welche im Business Knowledge Model, bzw. dem PMML-Modell abgebildet ist, ausführt (siehe Abb. 41). Im Hinblick auf diese Möglichkeit muss jedoch berücksichtigt werden, dass – wie in Kapitel 5.3.2 angemerkt – einige DMN-Engines die Einbindung von PMML-Modellen nicht unterstützen.

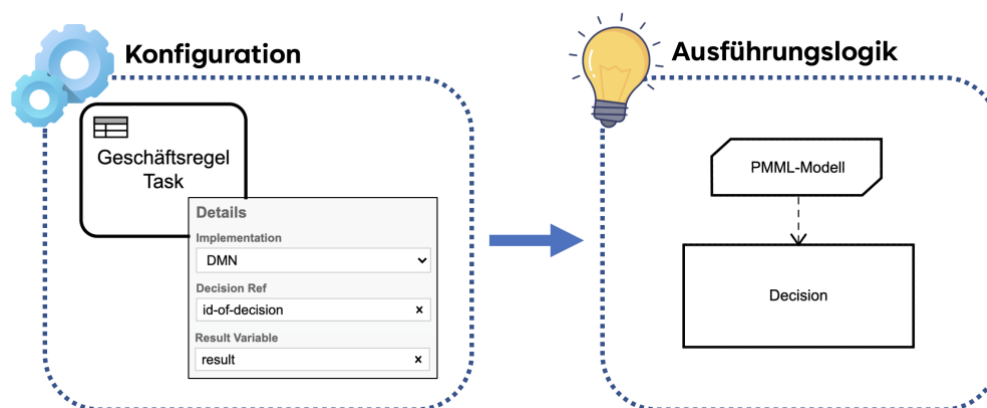


Abbildung 41: Technische Implementierung zum Einsatz eines PMML-Modells

6.3.3.2: Variante 2 – Entscheidungstabelle

Eine weitere Möglichkeit, um eine KI-Lösung in ein DMN-Modell einzubinden, kann es daher sein, ein ML-Modell in eine andere Form zu überführen, die in jeder gängigen DMN-Engine verwendet werden kann. Eine solche ist z.B. eine Entscheidungstabelle. Die Grundlagen zu dieser werden in Kapitel 5.3.2.1 behandelt. Gemäß diesen ist sie zu interpretieren als ein Modell, welches sich aus einer Menge von Entscheidungsregeln zusammensetzt. Diese wiederum werden in der Form einer Tabelle strukturiert.

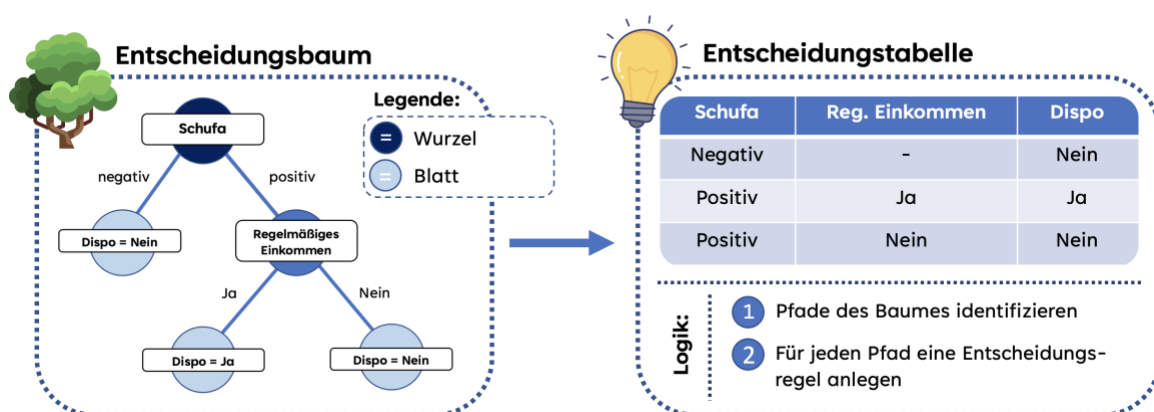


Abbildung 42: Transformation eines Entscheidungsbaums in eine Entscheidungstabelle

¹⁶ Für weitere Informationen zu den angesprochenen Notationselementen – siehe Kapitel 5.1.3

Ein ML-Modell, das geeignet erscheint, um in eine solche Form transformiert zu werden, ist der sogenannte Entscheidungsbaum. Dieser wird im Zuge von Kapitel 3.2.5 untersucht. In der Theorie ist er einer Entscheidungstabelle sehr ähnlich – denn auch ein solcher Baum besteht aus einer Menge von Entscheidungsregeln.

Der Unterschied zwischen den beiden Modellen liegt folglich nur in ihrer Strukturierung. Während eine Entscheidungstabelle ihre Regeln, in, jeweils unzusammenhängenden, einzelnen Zeilen einer Tabelle abbildet, so werden sie in einem Entscheidungsbaum – ausgehend von einer Wurzel – in einer Hierarchie angeordnet. In dieser Hierarchie ist jeder Pfad von der Wurzel zu einem Blatt als eine einzelne Entscheidungsregel zu verstehen.

Folglich müssen – damit ein solcher Entscheidungsbaum in die Form einer Entscheidungstabelle überführt werden kann – jegliche dieser Pfade identifiziert werden. Folgend darauf kann für jeden der Pfade eine explizite Regel in einer Entscheidungstabelle angelegt werden (siehe Abb. 42).

Die Entscheidungstabelle, die durch dieses Vorgehen aufgebaut wird, kann wiederum durch eine DMN-Engine ausgeführt werden. Um sie in einem BPMN-basierten Prozess einzusetzen, muss ein Geschäftsregel Task so konfiguriert werden, dass er eine Decision auslöst, welche eben diese Entscheidungstabelle beinhaltet (siehe Abb. 43).

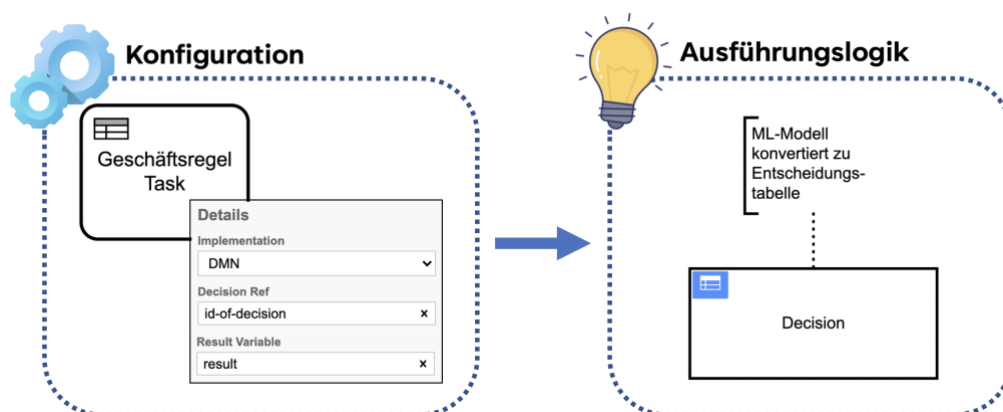


Abbildung 43: Technische Implementierung einer DMN-Entscheidungstabelle

6.4: Analyse der Herausforderungen

Alles in allem wurde nun also aufgezeigt, dass der Einsatz einer KI, auf diverse Weisen durch die BPMN abgebildet und durch eine Process Engine automatisiert werden kann. Noch nicht erörtert wurde jedoch die Fragestellung, ob und wie die Notation bei der Lösung der Herausforderungen helfen kann, welche beim Einsatz von KI-Lösungen in Prozessen auftreten können. Für einige von diesen Herausforderungen, bzw. für jene, welche im Rahmen von Kapitel 4.4 diskutiert werden, wird diese Fragestellung in der Folge beantwortet.

6.4.1: Transparenz & Akzeptanz

Die erste Herausforderung, welche in Kapitel 4.4 erörtert wird, ist jene der potentiell fehlenden Akzeptanz zum Einsatz einer KI, aufseiten bestimmter Stakeholder. Um diese zu lösen, könnte es eine mögliche Herangehensweise sein, den Einsatz der jeweiligen KI-Lösung für jeden relevanten Stakeholder transparent zu machen und offen zu kommunizieren.

Um dieses Ziel zu erreichen kann die BPMN, aus theoretischer Sicht, einen Beitrag leisten. Sie legt einen alleinigen Fokus auf die Abbildung von Prozessen. Durch diesen soll es ermöglicht werden, Prozessmodelle zu entwerfen, die sowohl detailreich als auch verständlich sind (siehe Kap. 5.1.2).

Gemäß der Theorie kann daher die Hypothese aufgestellt werden, dass mittels des Einsatzes der BPMN also Prozessmodelle erstellt werden können, in welchen der Einsatz einer KI graphisch und möglichst transparent abgebildet ist. Durch welche BPMN-Elemente der Einsatz dabei modelliert werden kann, wird in Kapitel 6.2 untersucht. Noch nicht erörtert wurde allerdings, inwiefern das Prozessmodell, respektive der Einsatz einer KI mit weiteren Informationen angereichert werden kann. Mit Blick auf diese Frage ist vor allem der Einsatz von Notationselementen denkbar, welche den Swimlanes und den Artefakten zugerechnet werden. Deshalb werden diese, in der Folge, im Kontext zum Einsatz einer KI, näher betrachtet.

6.4.1.1: Einsatz von Swimlanes

Orientiert an der Argumentation aus Kapitel 6.2.1 kann eine KI-Lösung als ein spezifischer Akteur in einem Prozess verstanden werden. Als ein solcher kann sie, in der BPMN, durch sog. Swimlanes abgebildet werden. Zu diesen zählen der Pool und die Lane. Die Grundlagen zu diesen Elementen werden im Zuge von Kapitel 5.1.3.5 erörtert. Sie sind zu deuten als strukturierende Elemente, die einen abgeschlossenen Prozess, sowie konkrete Akteure in diesem abbilden.

Insgesamt muss jedoch berücksichtigt werden, dass eine KI, je nach Perspektive, sowohl als ein interner, aber auch als ein externer Akteur verstanden werden kann. Dies hat zur Folge, dass sie, je nach ihrer Art, auf eine unterschiedliche Weise dargestellt werden kann (siehe Abb. 44-45).

KI als interner Akteur

Wird eine KI-Lösung als ein interner Akteur verstanden, so kann sie mittels einer Lane abgebildet werden (siehe Abb. 44). Diese kann entweder ein Teil einer übergeordneten Lane oder eines Pools sein. In der Lane ist der eigentliche Einsatz einer KI abgebildet. Dieser kann, wie in Kapitel 6.1.2 erörtert, etwa über einen Service Task abgebildet werden. Der Task ist dabei über konventionelle Kontrollflüsse mit den anderen Flusselementen eines Prozesses verbunden.

Die Modellierungsvariante kann z.B. eingesetzt werden, wenn explizit dargestellt werden soll, für welche Schritte in einem Prozess eine KI zuständig ist. Dabei muss jedoch beachtet werden, dass die Variante impliziert, dass die KI-Lösung im Einflussbereich des jeweiligen Prozesses, bzw. des Unternehmens, oder der Process Engine liegt. Dies kann bspw. der Fall sein, wenn die KI-Lösung eigenentwickelt ist, oder über eine Web-Schnittstelle synchron angesprochen werden kann.

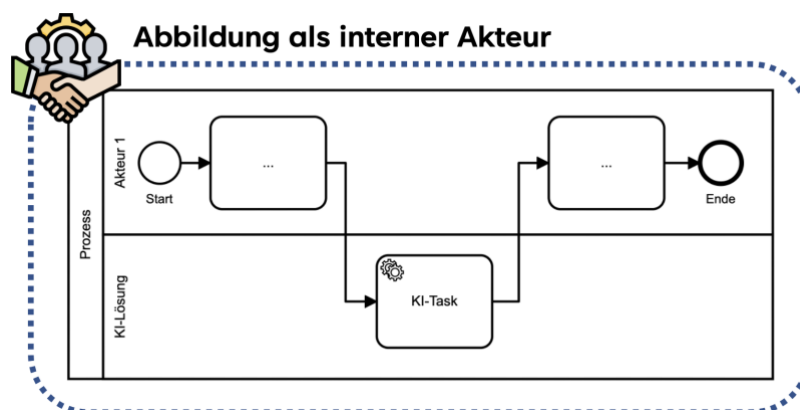


Abbildung 44: Abbildung einer KI-Lösung als interner Prozessakteur

KI als externer Akteur

Ist eine KI hingegen als ein externer Akteur zu verstehen, so kann sie mittels einem eigenen Pool abgebildet werden (siehe Abb. 45). Dieser ist als ein eigenständiger Prozess zu deuten. Daher ist es – sofern ein vollständiges Prozessmodell erstellt werden soll – nicht ausreichend, dass in eben diesem Pool nur der Einsatz der KI-Lösung abgebildet ist. Stattdessen verfügt der Pool über einen kompletten Prozessfluss, mit mindestens einem Start- und Endereignis. Ergänzend dazu kann er über Nachrichtenflüsse mit anderen Pools interagieren. Aus diesen heraus kann, je nach Szenario, der eigentliche Einsatz der KI ausgelöst werden.

Alles in allem kann diese Variante für die gleichen Zwecke, wie die Erste eingesetzt werden. Dabei muss jedoch beachtet werden, dass sie im Vergleich zur ersten Variante anders zu interpretieren ist. Sie impliziert, dass eine KI-Lösung nicht im Einflussbereich eines Prozesses, respektive eines Unternehmens oder der Engine liegt, die diesen ausführt. Dies kann bspw. der Fall sein, wenn die KI-Lösung von einem externen Anbieter – z.B. über die Cloud - bereitgestellt wird, oder wenn die Ausführung asynchron ist. In diesem Fall muss auf das Ergebnis der KI gewartet werden.

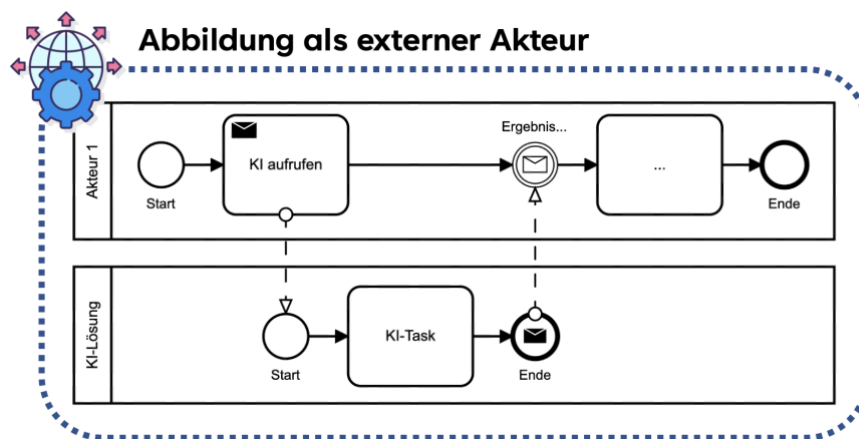


Abbildung 45: Abbildung einer KI-Lösung als externer Prozessakteur

6.4.1.2: Einsatz von Artefakten

Auch Elemente aus der Kategorie der Artefakte können eingesetzt werden, um ein Prozessmodell, in welchem der Einsatz einer KI abgebildet ist, mit zusätzlichen Informationen zu ergänzen. Dabei stehen vor allem die Elemente Datenbank, Datenobjekt, sowie Artefakt im Fokus (siehe Abb. 46). Die theoretischen Grundlagen zu diesen werden im Zuge von Kapitel 5.1.3.6 definiert.

Einsatz von Datenobjekt & Datenbank

Die ersten beiden sind im Kontext zur KI relevant, da für den Einsatz einer solchen stets konkrete Daten benötigt werden. Mit den beiden Elementen können diese explizit und transparent in einem Prozessmodell abgebildet werden. Dabei kann das Element Datenobjekt eingesetzt werden, um darzustellen, welche konkreten Daten, bzw. Objekte, die in Form von Daten repräsentiert werden, eine KI als Input verarbeitet, oder als Output erzeugt. Ein solches Objekt kann z.B. eine Rechnung sein. Die Datenbank hingegen kann verwendet werden, um abzubilden, aus welcher spezifischen Datenquelle jene Daten bezogen werden, oder an welchem Ort sie gespeichert werden.

Einsatz von Anmerkungen

Das letztere Artefakt wiederum ist relevant, da KI-Lösungen sehr komplex sind. Es ist zu deuten als ein simpler Kommentar, welcher an ein anderes Element angehängt werden kann. Im Kontext

zu einer KI kann er genutzt werden, um den Einsatz dieser mit textuellen Kontextinformationen anzureichern, die nicht allein aus dem Notationselement ableitbar sind. Diese können diversester Art sein. Sie können kaum eingegrenzt werden. Eine Anmerkung könnte dementsprechend bspw. eine Information darüber sein, von welchem Anbieter die Lösung stammt, die eingesetzt wird.

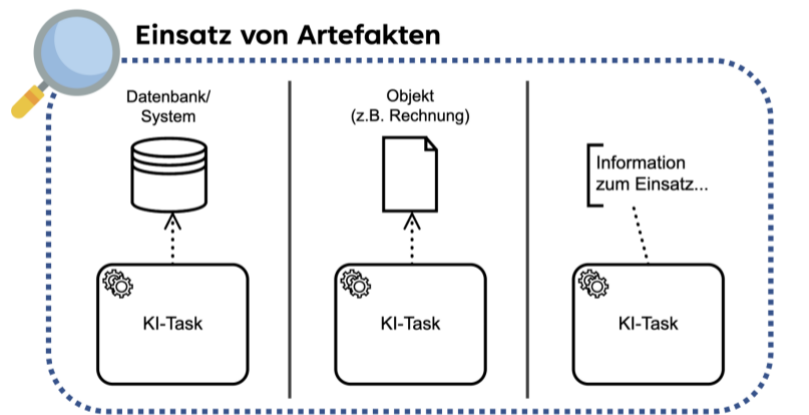


Abbildung 46: Einsatz von Artefakten zur Kontextualisierung eines KI-Einsatzes

6.4.2: Integration

Die Herausforderung der Integration wird in Kapitel 4.4.2 behandelt. Sie befasst sich damit, dass der Einsatz einer KI erst dann einen optimalen Mehrwert erzeugt, wenn die jeweilige KI-Lösung vollständig in einen Prozess integriert ist (siehe Kapitel 4.4.2). Daher müssen spezielle Lösungen gefunden werden, mittels welchen eine solche Integration, sowohl aus einer technischen als auch aus einer fachlichen Sicht, möglich ist.

Die BPMN, in Kombination mit einer geeigneten Process Engine, stellt in der Theorie eine solche Lösung dar. So kann die Notation verwendet werden, um den Einsatz einer KI aus einer fachlichen Sicht zu modellieren. Wie dies konkret möglich ist, wurde im Zuge von Kapitel 6.2 und dem letzten Unterkapitel diskutiert. Durch eine Process Engine wiederum kann der Einsatz der KI-Lösung aus einer technischen Perspektive in den Prozess integriert werden. Dazu muss das jeweilige BPMN-Prozessmodell konfiguriert und die zugehörige Ausführungslogik implementiert werden. Das Vorgehen, bzw. die Möglichkeiten hierzu werden in Kapitel 6.3 erörtert.

Bisher nicht analysiert wurde jedoch, wie die BPMN, sowie entsprechende Process Engines dabei helfen können, die Ergebnisse einer KI für konkrete Zwecke zu verarbeiten. Denn erst durch diese Weiterverwendung kann der Einsatz einer KI einen realen Mehrwert erzeugen. Daher wird diese Fragestellung in der Folge untersucht. In der Theorie sind dabei vor allem zwei abstrakte Zwecke denkbar, für welche das Ergebnis aus dem Einsatz einer KI eingesetzt werden kann:

6.4.2.1: Einsatz in Folgeschritten

Zum einen kann das Ergebnis aus dem Einsatz einer KI in den weiteren Schritten eines Prozesses verwendet werden. In diesem Fall wird ein Mehrwert direkt in einer Aktivität geschaffen – durch die Weiterverarbeitung des Ergebnisses in eben dieser.

Exemplarisch kann ein solches Einsatzszenario verdeutlicht werden am, in Kapitel 4.2 erörterten, Einsatz einer Document Understanding Lösung, in einem Buchhaltungsprozess. Rückblickend auf dieses wird dargelegt, dass eine solche KI-Lösung eingesetzt werden kann, um spezifische Daten

aus zu verbuchenden Belegen zu extrahieren. Der Mehrwert durch den Einsatz der Lösung besteht darin, dass diese Daten eingesetzt werden können, um einen jeweiligen Beleg im Folgeschritt des Prozesses automatisch zu verbuchen.

Szenarien dieser Art können in der BPMN durch Aktivitäten, bzw. durch einzelne Tasks abgebildet werden. Die theoretischen Grundlagen zu diesen werden in Kapitel 5.1.3.2 behandelt. Sie können in diversen Typen vorliegen – etwa als Benutzer- oder als Service Task. Je nach Typ besitzen sie eine konkrete technische Implementierung. Im Rahmen dieser kann das Ergebnis aus dem Einsatz einer KI-Lösung auf eine jeweils benötigte Art und Weise verarbeitet werden.

Ein abstraktes Beispiel hierzu ist in Abbildung 47 dargestellt. Dieses zeigt einen BPMN-basierten Prozess, in welchem das Ergebnis aus dem Einsatz einer KI durch einen Service Task verarbeitet wird. Dieser kann, analog zu dem Service Task aus Kapitel 6.3.1, eine individuell programmierte Logik ausführen. Im Zuge dieser wird, mit dem Ergebnis, eine bestimmte Funktion aufgerufen.

Damit dies jedoch möglich ist, muss die Engine auf die jeweiligen Ergebnisse, aus der Anwendung der KI, zugreifen können. Daher muss bei der technischen Implementierung des KI-Einsatzes stets sichergestellt werden, dass die Ergebnisse in einer spezifischen Datenquelle gespeichert werden. Eine solche könnte z.B. eine Prozessvariable sein.

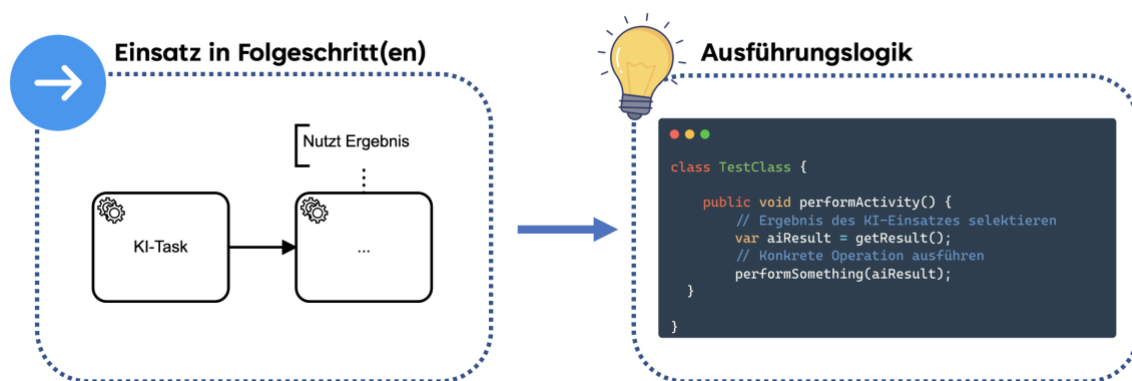


Abbildung 47: Verwendung des Ergebnisses aus einem KI-Einsatz in einer Folgeaktivität

6.4.2.2: Einsatz zur Prozesssteuerung

Zum anderen kann ein Ergebnis aus dem Einsatz einer KI-Lösung aber auch zur Steuerung eines Prozesses eingesetzt werden. In einem solchen Fall hat es einen direkten Einfluss auf den Verlauf, bzw. die Ausführungslogik eines Prozesses. Dies kann einen Mehrwert erzeugen, wenn auf Basis eines konkreten Resultats z.B. abgeleitet werden kann, welche spezifischen Schritte, im weiteren Verlauf des Prozesses, für eine vorliegende Instanz ausgeführt werden müssen und welche nicht. Faktisch ermöglicht dieser Einsatzzweck also die intelligente Steuerung von Prozessen.

Ein Beispiel hierfür bietet der in Kapitel 3.2.5.3 entworfene Entscheidungsbaum. Dieser ist als ein Klassifikator zu deuten, welcher in einem Prozess zur Beantragung eines Dispokredits eingesetzt werden kann. Bewertet dieses Modell einen Kunden als kreditwürdig, so könnte in der Folge eine Aktivität ausgeführt werden, in welcher die Kreditlinie freigeschaltet wird. Wird er hingegen als nicht kreditwürdig eingestuft so ist dies nicht nötig. Stattdessen könnte jedoch eine Nachricht an den Kunden gesendet werden, in welcher er über die Ablehnung informiert wird.

In der BPMN kann ein solches Szenario durch Gateways abgebildet werden. In aller Regel handelt es sich dabei um exklusive Gateways. Diese werden im Rahmen von Kapitel 5.1.3.4 beschrieben. Sie sind zu verstehen als Verzweigungen in einem Prozessmodell. An diesen wird ein Prozess, auf Basis bestimmter Bedingungen, über genau einen von mehreren verfügbaren Pfaden fortgesetzt.

Die jeweiligen Bedingungen sind dabei, je Pfad, im Prozessmodell konfiguriert. Sie beziehen sich in aller Regel auf eine Prozessvariable. Wird ein Gateway durch eine Engine ausgeführt, so prüft diese jegliche Bedingungen und folgt jener Bedingung, welche zutreffend ist.

Um das Ergebnis der KI in einer Process Engine zur Prozesssteuerung einzusetzen, muss es folglich in einer konkreten Prozessvariable gespeichert werden. Darüber hinaus müssen in den möglichen Folgepfaden – mit Referenz auf diese Variable – die Bedingungen zur Ausführung der jeweiligen Pfade konfiguriert werden. Ein Beispiel hierzu ist in Abbildung 48 dargestellt.

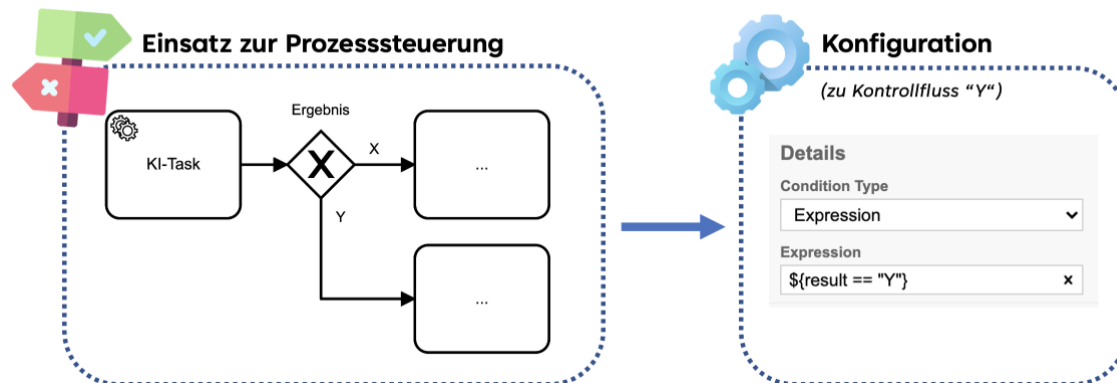


Abbildung 48: Verwendung des Ergebnisses aus einem KI-Einsatz zur Prozesssteuerung

6.4.3: Qualität der Ergebnisse

Die dritte Herausforderung, welche im Rahmen von Kapitel 4.4 begutachtet wird, ist jene, dass die Vorhersagen einer KI stets eine gewisse Unsicherheit besitzen. Dies bedeutet, dass einige von diesen nicht mit der Realität übereinstimmen. Sie können fehlerhaft sein. Werden sie trotz einem Fehler für bestimmte Zwecke eingesetzt, wie z.B. für jene, die in Kapitel 6.4.2 aufgelistet werden, so kann dies negative Auswirkungen auf das Ergebnis des jeweiligen Prozesses und somit auch auf das Unternehmen haben (siehe Kap. 4.4.3).

Um in der Lage zu sein mit den Fehlern umzugehen, und um die negativen Folgen zu vermeiden, müssen die Ergebnisse aus dem Einsatz einer KI kontinuierlich überprüft werden. Der Prüfer ist dabei in aller Regel ein menschlicher Akteur. Dieser muss ein tiefgreifendes Fachwissen über die Problemstellung besitzen, für welche die KI-Lösung eingesetzt wird.

Überprüfung mittels Benutzer Task

Die Überprüfung einer Vorhersage durch einen solchen Akteur ist grundsätzlich als eine Aktivität zu deuten. Als eine solche kann sie durch einen Benutzer Task abgebildet, und durch den Einsatz einer Process Engine automatisiert gesteuert werden. Daher kann abgeleitet werden, dass auch diese Herausforderung durch den Einsatz der BPMN und einer Engine gelöst werden kann.

Grundsätzlich gibt es jedoch mehrere Szenarien, wie ein solcher Benutzer Task zur Prüfung eines Ergebnisses verwendet werden kann. Diese können grob betrachtet in zwei Herangehensweisen unterschieden werden. Diese werden in der Folge näher erörtert.

6.4.3.1: Gezielte Prüfung der Ergebnisse

Die erste der beiden Herangehensweisen kann als gezielt beschrieben werden. Bei dieser wird im Vorhinein klar festgelegt, unter welchen Umständen das Ergebnis einer KI-Lösung überprüft wird.

Alles in allem sind diverse Szenarien denkbar, welche dieser Charakteristik entsprechen. Abstrakt betrachtet können diese, in aller Regel, in drei Varianten unterschieden werden. Diese sind in den Abbildungen 49 und 50 dargestellt. Sie werden in der Folge jeweils kurz erörtert:

Allgemeine Prüfung

Die erste Variante kann definiert werden als eine Allgemeine Prüfung. Bei dieser wird nach jedem Einsatz einer KI-Lösung das Ergebnis dieser geprüft. Am Beispiel des Prozesses zur Genehmigung eines Dispokredits, wird also nach jedem Einsatz des ML-Modells überprüft, ob die Klassifikation eines Kunden konsistent ist. Ein solches Szenario kann abgebildet werden, indem auf den Einsatz einer KI-Lösung stets ein Benutzer Task folgt (siehe Abb. 49). Die Anwendung dieses Szenarios kann sinnvoll sein, wenn die Vorhersage-Genauigkeit der KI-Lösung eher gering ist. Sie kann aber auch eingesetzt werden, wenn das Ergebnis einer KI, in jeglichen Fällen, eine hohe Auswirkung auf das Ergebnis eines Prozesses hat – oder bei Fehlern starke negative Folgen verursacht.

Prüfung in Abhängigkeit zum Ergebnis

Die zweite Variante hingegen ist zu beschreiben als eine Prüfung in Abhängigkeit zum Ergebnis. Bei dieser wird das Resultat einer KI nicht nach jedem Einsatz von dieser überprüft, sondern nur wenn es einen konkreten Wert aufweist. Ein solches Szenario wäre z.B. gegeben, wenn im Prozess zur Genehmigung eines Dispokredits, die Klassifizierung nur dann überprüft wird, wenn der Kunde als nicht-kreditwürdig eingestuft wird. Im Hinblick auf die BPMN kann es durch den Einsatz eines exklusiven Gateways modelliert werden (siehe Abb. 49). Dieses kann so konfiguriert werden, dass es die Prüfung nur dann auslöst, wenn die Vorhersage einen konkreten Wert aufweist. Der Einsatz dieser Variante ist denkbar, sofern bestimmte Folgen vermieden werden sollen, die nur in diesem Fall eintreten können. Am Beispiel des Dispokredits wäre eine solche Folge etwa der Verlust von Zinseinnahmen durch die Ablehnung des Kredits. Eine solche kann verursacht werden, wenn ein Kunde als nicht-kreditwürdig klassifiziert wird, obwohl er eigentlich kreditwürdig wäre.

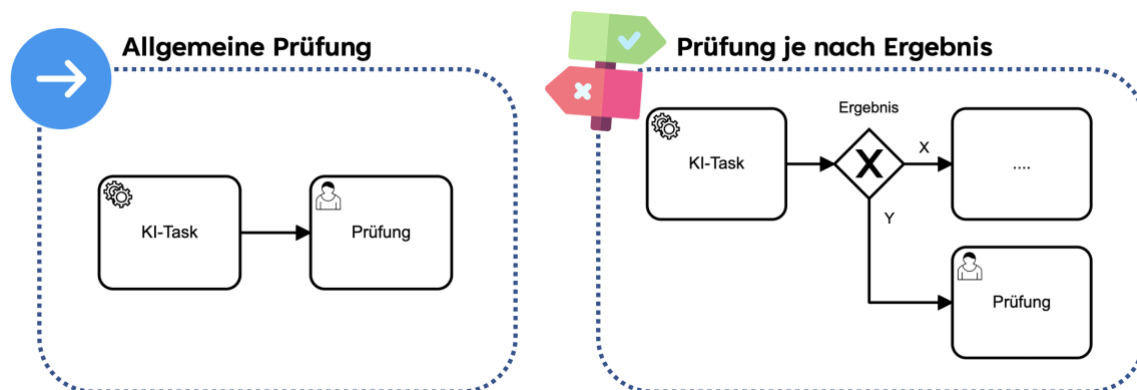


Abbildung 49: Allgemeine Möglichkeiten zur Kontrolle des Ergebnisses einer KI

Prüfung in Abhängigkeit zum Fall

Die dritte Variante wiederum ist eine Prüfung in Abhängigkeit zum jeweils vorliegenden Fall. Bei einer solchen wird eine Prüfung nur dann ausgeführt, wenn eine bestimmte Bedingung erfüllt ist, welche sich auf den Ausführungskontext eines Prozesses bezieht. Am Beispiel des Prozesses zur Genehmigung eines Dispokredits kann eine solche Bedingung etwa die Höhe dieses Kredits sein.

Um ein solches Szenario abzubilden ist zuallererst eine Relevanzprüfung notwendig. Eine solche kann, wie in Abbildung 50 dargestellt ist, z.B. durch einen Geschäftsregel Task, bzw. durch eine Entscheidungstabelle repräsentiert werden. Das Ergebnis der Tabelle kann wiederum durch ein

exklusives Gateway verarbeitet werden. Dieses kann so konfiguriert werden, dass es die Prüfung nur dann auslöst, wenn diese im Hinblick auf die Relevanzprüfung notwendig ist (siehe Abb. 50).

Ein solches Szenario kann sinnvoll sein, wenn die Folgen, die eine inkorrekte Vorhersage mit sich bringt, stark fallabhängig sind. Auch dies lässt sich am Beispiel eines Dispokredits verdeutlichen. Wird dieser gewährt, obwohl ein Kunde nicht-kreditwürdig ist, so steigen die Konsequenzen eines potentiellen Zahlungsausfalles logischerweise mit der Höhe des Kredits. Folglich könnte mittels dieser Variante definiert werden, dass eine Prüfung obligatorisch ist, wenn der Kreditbetrag über einem bestimmten Limit liegt.

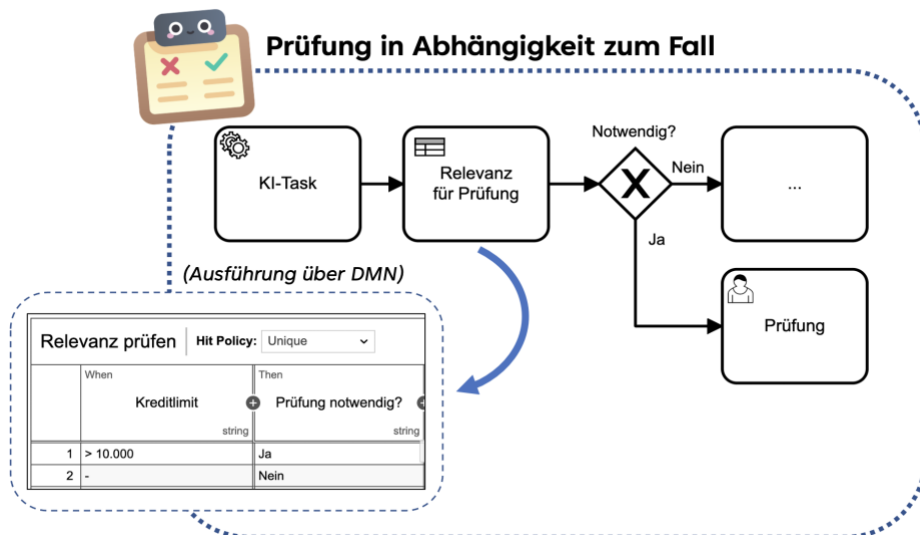


Abbildung 50: Fallabhängige Kontrolle des Ergebnisses einer KI

6.4.3.2: Zufällige Prüfung der Ergebnisse

Die zweite Herangehensweise ist das Gegenstück zur ersten. Bei dieser ist es nicht klar definiert, unter welchen Umständen das Ergebnis einer KI überprüft werden muss. Stattdessen werden die Prüfungen zufällig durchgeführt. Ein Beispiel, wie eine solche zufällige Prüfung ausgestaltet sein kann, ist in Abbildung 51 dargestellt. In diesem werden laufend Stichproben aus einem Prozess gezogen – und für eben diese eine Überprüfung durchgeführt.

Um ein solches Szenario abzubilden, muss für jede Instanz eines Prozesses bestimmt werden, ob diese eine solche Stichprobe ist. Dazu kann z.B. ein Service Task eingesetzt werden, welcher eine Funktion ausführt, die eine zufällige Zahl zwischen null und eins generiert. Diese Zufallszahl kann als Entscheidungsgrundlage dienen. Sie kann in einer Prozessvariable gespeichert werden. Diese wiederum kann durch ein exklusives Gateway ausgewertet werden. Das Gateway kann dabei so konfiguriert werden, dass es für jede Prozessinstanz, bei welcher die Zufallszahl größer ist als ein bestimmter Schwellenwert, eine Überprüfung auslöst.

Die Anwendung eines solchen Szenarios kann vor allem dann sinnvoll sein, wenn, im Allgemeinen, die Qualität und die Tauglichkeit einer KI-Lösung überwacht werden soll. Denn während bei den anderen Szenarien immer nur bestimmte Fälle im Fokus stehen und primär das Ziel verfolgt wird negative Konsequenzen zu vermeiden, können bei einer zufälligen Prüfung Rückschlüsse über die KI-Lösung als Ganzes gezogen werden. Exemplarisch kann mit einem solchen Szenario also auch das ML-Modell aus dem Prozess zur Genehmigung eines Dispokredits überwacht werden.

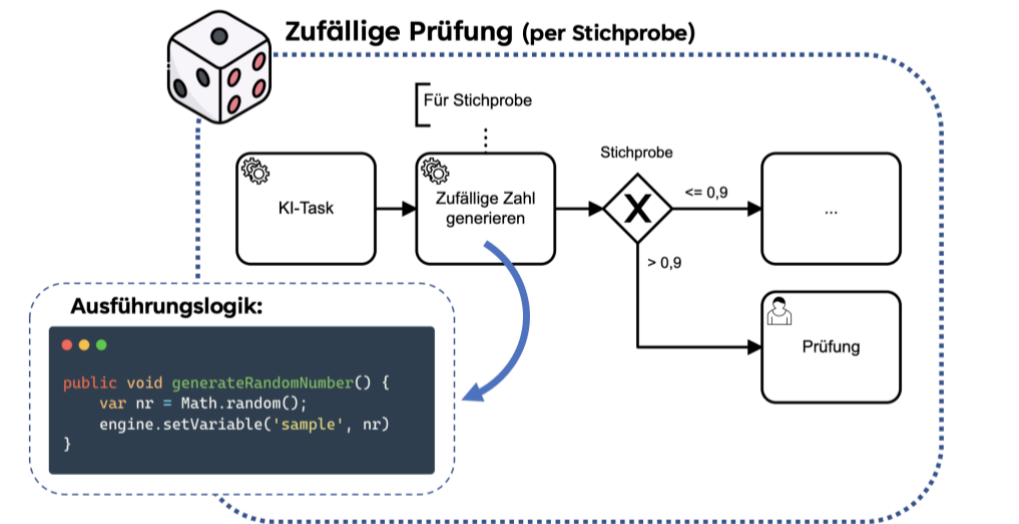


Abbildung 51: Zufällige Prüfung der Ergebnisse aus dem Einsatz einer KI

6.4.4: Erklärbarkeit

Die letzte Herausforderung, die in Kapitel 4.4 erörtert wird, ist jene, dass es für Menschen oftmals nicht nachvollziehbar ist, aus welchen Gründen eine KI eine spezifische Vorhersage erzeugt. Dies liegt daran, dass viele der Lösungen, welche heutzutage eingesetzt werden, auf subsymbolischen Modellen basieren (siehe Kap. 4.4.4). Diese bilden ihr Wissen in hochkomplexen Formen ab, die für Menschen kaum interpretierbar sind. Ferner ist es nicht sichergestellt, dass die Gründe für die Vorhersagen nachvollziehbar gespeichert werden – selbst, wenn die Modelle verständlich sind.

Faktisch ist der Entscheidungsvorgang einer KI-Lösung häufig kaum erklärbar. Da das Wissen zu den Gründen einer Entscheidung in Prozessen aber oft benötigt wird, müssen konkrete Lösungen hierfür gefunden werden. Sonst ist es möglich, dass bestimmte KI-Lösungen in einigen Szenarien nicht eingesetzt werden können, oder keine Akzeptanz erfahren. Dies wiederum führt dazu, dass ein potentieller Mehrwert, der durch den Einsatz der KI erzeugt werden könnte, verloren geht.

Auf den ersten Blick kann die BPMN nur bedingt helfen, um diese Herausforderung zu bewältigen. Sie kann ausschließlich gelöst werden durch den Einsatz von ML-Modellen, Neuronalen Netzen, oder weiteren KI-Techniken, die eine Auskunft über die Gründe für ihre jeweiligen Entscheidungen geben. Diese wiederum können jedoch durch die BPMN orchestriert werden. Ferner ist es möglich, dass eine Process Engine den Grund für die Vorhersagen speichert. Insgesamt sind dabei diverse Herangehensweisen denkbar. Diese werden in der Folge näher betrachtet:

6.4.4.1: Orchestration transparenter Modelle

Eine erste Möglichkeit, um die Herausforderung zu lösen, ist es ein ML-Modell einzusetzen, das den Grund für eine Prognose explizit erkenntlich macht. Ein solches könnte z.B. ein symbolisches Modell, wie ein Entscheidungsbaum sein. Es kann aber auch ein anderes, subsymbolisches Modell sein, das für diesen Zweck angepasst ist. Analog zum Vorgehen in Kapitel 6.3.2 kann der Einsatz des Modells durch einen Service Task abgebildet werden. Dieser Task wiederum kann flexibel konfiguriert werden. Er kann auf eine Funktion verweisen, durch welche das Modell aufgerufen und der Grund für das Resultat ermittelt wird. Abschließend kann durch die Funktion sowohl der

Grund als auch das Ergebnis gespeichert werden. Die Daten können dabei z.B. in einer konkreten Datenbanktabelle, oder einer Prozessvariable hinterlegt werden (siehe Abb. 52).

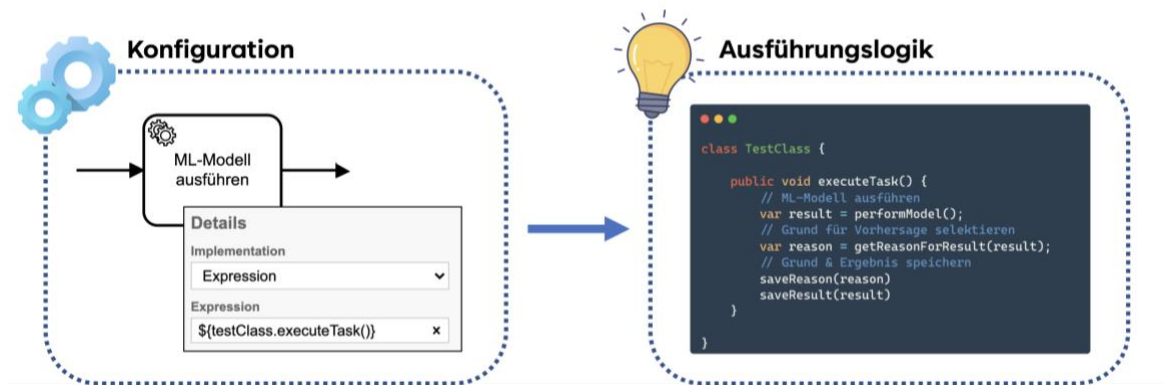


Abbildung 52: Orchestration eines transparenten ML-Modells

6.4.4.2: Orchestration verschiedener Modelle durch Ensemble Learning

Eine zweite Herangehensweise zur Lösung der Herausforderung könnte hingegen das sogenannte Ensemble Learning darstellen. Dieses ist zu interpretieren als ein spezielles Lernverfahren aus der Familie des Maschinellen Lernens. Bei eben diesem werden mehrere ML-Modelle, welche jeweils auf unterschiedlichen Lernalgorithmen basieren, eingesetzt, um eine spezifische Problemstellung zu lösen (vgl. Aunkofer 2017).¹⁷

Das Ensemble Learning wird primär mit dem Ziel eingesetzt, Vorhersagen zu treffen, welche eine möglichst hohe Genauigkeit aufweisen. Die Grundidee dabei ist, dass die Nutzung von mehreren Modellen zu besseren Ergebnissen führt – da ein jedes Modell anders trainiert wird, sein Wissen in unterschiedlichen Formen abbildet und verschiedene Vor- sowie Nachteile besitzt. Insgesamt basiert eine Vorhersage also nicht nur auf einem Modell, sondern wird aus dem Durchschnitt von mehreren Modellen abgeleitet (vgl. Aunkofer 2017).

Grundsätzlich ist es jedoch auch denkbar das Vorgehen des Ensemble Learnings anzupassen. Es könnte auch verwendet werden, um die Vorhersagen eines subsymbolischen Modells zu erklären. Dazu müsste simultan zu jenem subsymbolischen-, ein symbolisches Modell eingesetzt werden. Analog zum Vorgehen im letzten Unterkapitel könnte dieses Modell sodann verwendet werden, um die Gründe für eine konkrete Entscheidung zu dokumentieren.

Kontext zur BPMN

Die BPMN kann im Kontext zu diesem Lösungsansatz für zwei Szenarien eingesetzt werden. Zum einen, um den Einsatz der ML-Modelle in einem konkreten Prozessmodell abzubilden – und zum anderen, um eben dieses Modell durch eine Process Engine zu steuern. Alles in allem kann dabei ein paralleles Gateway eingesetzt werden. Dieses wiederum kann den Einsatz der ML-Modelle auslösen, der z.B. über Service Tasks abgebildet sein kann (siehe Abb. 53).

Das Problem bei diesem Vorschlag ist jedoch, dass es vorkommen kann, dass die beiden Modelle unterschiedliche Ergebnisse produzieren. Dies führt dazu, dass das symbolische ML-Modell nicht erklären kann, warum die Vorhersage des subsymbolischen Modells zustande kommen könnte.

Dieses Problem wiederum könnte allerdings durch den Einsatz der BPMN gelöst werden. So kann, angelehnt an der fallabhängigen Prüfung aus Kapitel 6.4.3.1, ein exklusives Gateway eingesetzt

¹⁷ Für mehr Informationen zum Ensemble Learning – siehe: Aunkofer 2017

werden, um zu prüfen, ob die Ergebnisse der Modelle gleich sind. Ist dies nicht der Fall kann ein Benutzer Task ausgelöst werden. In diesem könnte ein Mitarbeiter eine eigene Vorhersage treffen und den Grund für diese explizit hinterlegen.

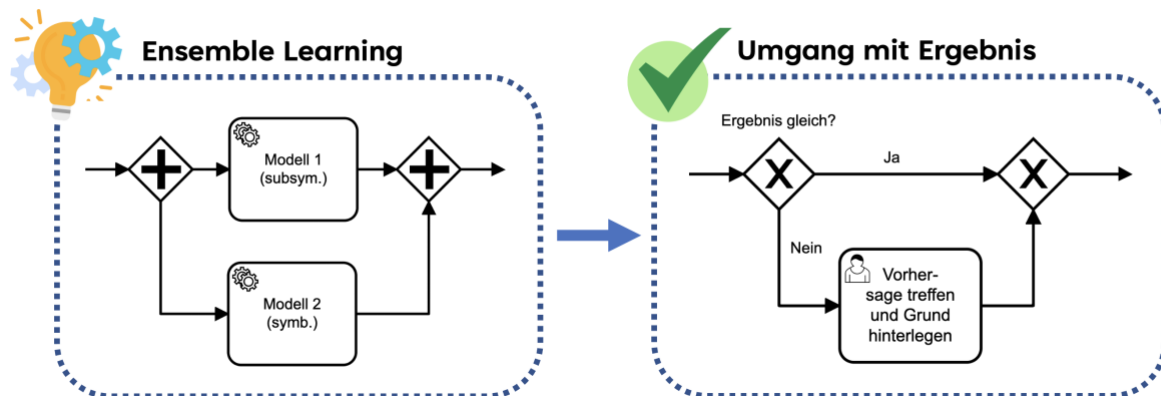


Abbildung 53: Erklärbarkeit der Vorhersagen einer KI durch Ensemble Learning

6.4.4.3: Umwandlung in DMN-Entscheidungstabelle

Eine dritte Möglichkeit dagegen ist es, ein ML-Modell in eine verständlichere Form zu überführen. Im Kontext zur BPMN ist eine solche Form z.B. eine Entscheidungstabelle. Die Umwandlung eines ML-Modells in eine solche Tabelle wird im Zuge von Kapitel 6.3.3.2 anhand des Beispiels eines Entscheidungsbaums diskutiert. Zudem wird sie in Anhang 8 und 9 exemplarisch ausgeführt.

In eben dieser Form kann ein ML-Modell sehr transparent dargestellt werden. Der Vorgang einer Entscheidung ist direkt nachvollziehbar, da das Wissen jener KI über einzelne Regeln abgebildet wird. Außerdem kann die Entscheidungstabelle, wie es im oben genannten Kapitel erörtert wird, durch nahezu jede DMN-Engine ausgeführt werden.

Darüber hinaus bietet die Engine für gewöhnlich den Vorteil, dass jeder Entscheidungsvorgang in der Datenbank dieser aufgezeichnet wird. So speichert bspw. auch die Engine des Unternehmens Camunda, für jede Entscheidung, die über eine Entscheidungstabelle ausgeführt wird, bestimmte Daten. Zu diesen zählen die ID der zutreffenden Regel, sowie der Name und der Wert der Output-Variable (siehe Abb. 54). Faktisch ist es, über die Verbindung mit der Entscheidungstabelle, direkt nachvollziehbar, welche Regel des Modells angewendet wird, um eine Vorhersage zu tätigen.

	WHERE	ORDER BY	id_			
	dec_inst_id_		rule_id_		var_name_	text_
1	83d9cd01-d43f-11ec-ad9b-ee6a33bb42d9		DecisionRule_169		CREDITWORTHY	NO
2	83d9cd03-d43f-11ec-ad9b-ee6a33bb42d9		DecisionRule_63		CREDITWORTHY	YES

Abbildung 54: Log-Daten zur Ausführung einer DMN-Entscheidungstabelle

7: Entwurf – Prototypischer Einsatz von KI in BPMN-Prozessen

In diesem Kapitel werden die theoretischen Überlegungen zur Kombination der KI mit der BPMN und einer Process Engine, welche im vorigen Kapitel erörtert wurden, exemplarisch in die Praxis überführt. Dazu werden für zwei ausgewählte Prozesse spezifische Prototypen erstellt. In diesen wird der Einsatz einer jeweiligen KI-Lösung durch eine Process Engine gesteuert.

7.1: Technisches Setting zur Automatisierung der Prozesse

Um die Prototypen zu erstellen, wird die Process Engine des Unternehmens Camunda eingesetzt. Diese wird in Kapitel 5.2.2 erwähnt. Für die Implementierung der Ausführungslogik der einzelnen Prozessschritte werden hingegen spezifische Web-Services programmiert. Diese basieren auf der Programmiersprache Kotlin, in Kombination mit dem Framework Spring Boot.

7.2: Prototyp 1 – Automatisierung mittels Service Task

Der erste Prozess, welcher exemplarisch durch den Einsatz einer KI-Lösung und die Nutzung der BPMN automatisiert werden soll, ist ein Prozess zur Bearbeitung von Eingangsrechnungen. Dieser findet für gewöhnlich in jedem Unternehmen statt. In einer nicht durch eine Engine gesteuerten und stark vereinfachten Variante kann er wie in Abbildung 55 dargestellt ablaufen.

Ausgangslage – Beschreibung des zu automatisierenden Prozesses

Gemäß dieser, wird eine Instanz des Prozesses gestartet, sobald ein Unternehmen eine Rechnung erhält. Der Kanal, über den diese Rechnung eingeht, ist nicht weiter beschrieben. Dies bedeutet, sie kann z.B. per Post oder Mail eintreffen. Um sie zu begleichen, werden konkrete Informationen benötigt – insbesondere solche bzgl. dem Rechnungsbetrag, den Rechnungspositionen und der Überweisungsadresse des Empfängers. Folglich müssen diese Daten, in einem ersten Schritt des Prozesses, durch einen Mitarbeiter, aus der Rechnung ausgelesen und in ein konkretes System übertragen werden. Ein solches könnte bspw. ein ERP-System sein. Im zweiten Schritt kann die Rechnung auf Basis von diesen Informationen – im gleichen System – durch einen weiteren, oder den gleichen Mitarbeiter beglichen werden.

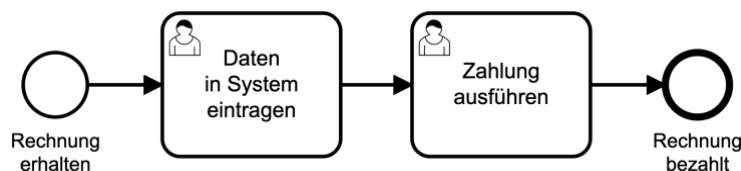


Abbildung 55: Ausgangslage – Prozess zur Bearbeitung von Eingangsrechnungen

Alles in allem kann daher erörtert werden, dass der Prozess ein hohes Automatisierungspotential besitzt – denn jegliche der Schritte werden von Mitarbeitern ausgeführt. Die Digitalisierung dieser Schritte ist jedoch kaum möglich, da nicht alle Rechnungen in einem digitalen Format vorliegen. Demzufolge muss es ein erstes Ziel eines Automatisierungsvorhabens sein, dass alle Rechnungen in einem digitalen Format vorliegen müssen, bevor sie vom Prozess verarbeitet werden können. Ein solches könnte z.B. das PDF-Format sein. Wie diese Voraussetzung geschaffen werden kann,

wird im Zuge dieser Thesis nicht erörtert. Ist sie jedoch gegeben, so kann untersucht werden, wie die Aktivitäten im Allgemeinen, durch den Einsatz einer Engine, automatisiert werden können:

Einsatz einer KI-Lösung

Im Hinblick auf diese Fragestellung ist zu erwähnen, dass die erste Aktivität – das Auslesen der Informationen aus einer Rechnung – aus einer Perspektive der Automatisierung sehr komplex ist. Der Grund dafür ist, dass der Inhalt der PDF-basierten Rechnungsdokumente, welcher durch eine Folge von Zeichen repräsentiert wird, von einer Software nicht sinnvoll interpretiert werden kann. Hierzu fehlt der Software in der Regel das Wissen über den Kontext der Rechnung. Ohne diesen ist sie nicht in der Lage, eine Zeichenfolge als z.B. eine Rechnungsnummer zu identifizieren. Ferner ist es eine Herausforderung, dass jedes Rechnungsdokument verschieden aufgebaut sein kann. Dies bedeutet, dass eine relevante Information, wie etwa der Rechnungsbetrag, stets an anderen Stellen positioniert sein kann. Alles in allem ist es also nahezu unmöglich, die Aktivität durch einfache Regeln, bzw. eine statische Logik zu automatisieren.

Stattdessen können jedoch KI-Lösungen aus der Familie des Document Understandings genutzt werden. Die Grundidee dieser ist es spezifische Informationen aus unstrukturierten Dokumenten zu extrahieren und diese in einer strukturierten Form auszugeben. In dieser können sie durch eine Software interpretiert und für spezifische Zwecke weiterverarbeitet werden (siehe Abb. 56). Um dieses Ziel zu erreichen, verwenden die KI-Lösungen konkrete ML-Modelle, respektive Neuronale Netze, welche darauf trainiert sind, bestimmte Felder, wie bspw. eine Rechnungsnummer, in den Dokumenten zu identifizieren.

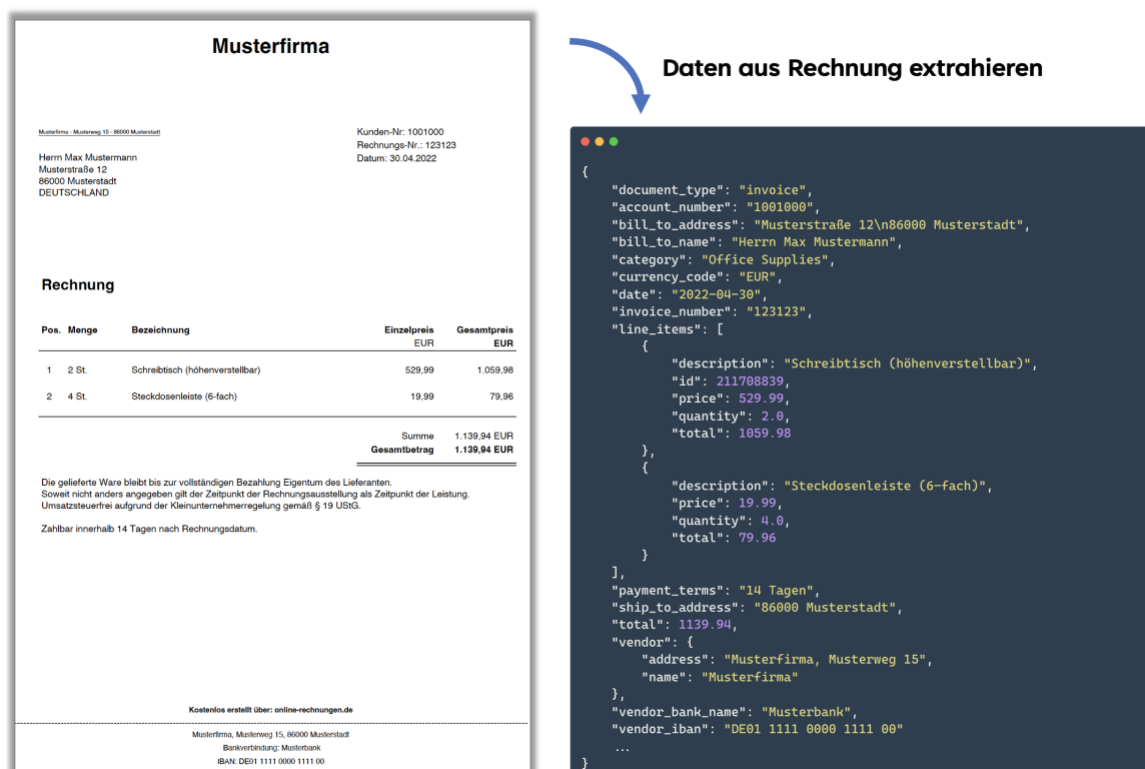


Abbildung 56: Grundidee zur Extraktion von Daten aus einer Rechnung

Für die Anwendung dieser Technik auf Rechnungen gibt es auf dem Softwaremarkt bereits einige Lösungen. Daher ist es nicht notwendig, eine eigene KI für diesen Anwendungsfall zu entwickeln. Eine dieser Lösungen ist die „Invoice OCR API“ des Unternehmens „veryfi“. Diese stellt eine API

zur Verfügung, an welche Rechnungen gesendet werden können. Wie diese genau genutzt werden kann, wird in Anhang 2 beschrieben. Es soll jedoch erwähnt werden, dass sie die Rechnungen in Echtzeit verarbeitet und die extrahierten Informationen in einem Format, wie dem JSON-Format zurückgibt (siehe Abb. 56).¹⁸

Überprüfung der Ergebnisse der KI

Wird die KI in der Aktivität verwendet, muss jedoch berücksichtigt werden, dass ihre Ergebnisse fehlerhaft sein können. Faktisch könnte es – theoretisch betrachtet – vorkommen, dass sie z.B. den Rechnungsbetrag, oder die Überweisungsadresse falsch extrahiert. Dies könnte dazu führen, dass eine Zahlung an ein falsches Konto, oder mit einem fehlerhaften Betrag, überwiesen wird. Die Korrektur dieses Fehlers kann zu einem hohen Aufwand und einer erneuten Ausführung des Prozesses führen. Ergo kann diskutiert werden, ob es sinnvoll ist, den Prozess mit einem weiteren Schritt zu ergänzen, in welchem ein Mitarbeiter das Ergebnis einer Extraktion überprüft und bei Bedarf ausbessert. Die Varianten zur Abbildung eines solchen Schrittes werden in Kapitel 6.3.2 erörtert. Im Hinblick auf diese kann sich dazu entschlossen werden, eine sog. allgemeine Prüfung durchzuführen. Im Rahmen dieser wird die Prüfung für jede einzelne Rechnung ausgeführt.

Traditionelle Automatisierung

Das Ausführen der Zahlung hingegen kann durch traditionelle Softwareentwicklung automatisiert werden. Denn es läuft stets nach dem gleichen Prinzip ab. Grob beschrieben wird ein konkreter Rechnungsbetrag, der sich aus einer Menge von Rechnungspositionen ergibt, an ein bestimmtes Konto überwiesen, welches in der Rechnung hinterlegt ist. Die benötigten Informationen werden in der vorherigen Aktivität aus der jeweiligen Rechnung extrahiert. Demzufolge liegen sie in einer strukturierten Form vor. Folglich kann die Zahlung, durch eine einfache Logik, in einer Software abgebildet werden.

Automatisierung mittels der BPMN und einer Process Engine

Damit der Prozess exemplarisch durch eine Engine automatisiert werden kann, muss zuallererst, durch den Einsatz der BPMN ein Soll-Prozessmodell erstellt werden. Werden dabei jegliche bisher untersuchten Aspekte berücksichtigt, so kann dieses, wie in Abbildung 57 dargestellt, modelliert werden. Bevor dieses jedoch durch eine Engine gesteuert werden kann, muss es konfiguriert, und die zugehörige Ausführungslogik implementiert werden. Der Vorgang hierzu stellt die eigentliche prototypische Automatisierung des Prozesses dar, und wird in Anhang 3 dokumentiert. Alles in allem kann der automatisierte Prozess – im Hinblick auf diese – wie folgt beschrieben werden:

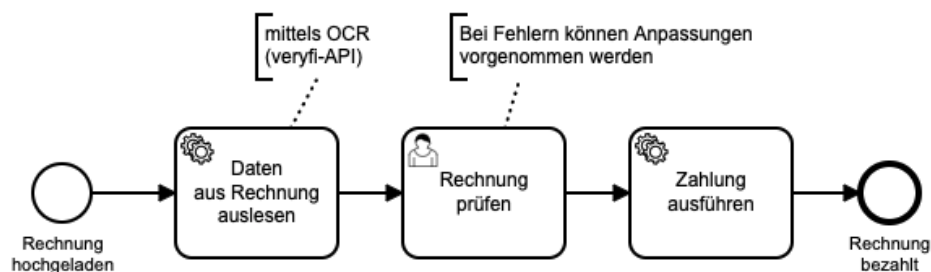


Abbildung 57: Prototyp – Automatisierter Prozess zur Bearbeitung von Eingangsrechnungen

¹⁸ Für mehr Informationen zum Unternehmen – siehe: <https://www.veryfi.com/>

Der Prozess wird ausgelöst, wenn eine Rechnung, welche in Form von einem PDF vorliegen muss, an eine konkrete Schnittstelle geschickt wird. Im ersten Schritt werden sodann, durch den Aufruf der „Invoice OCR API“ die relevanten Informationen aus dieser Rechnung extrahiert. Der Einsatz der KI-Lösung ist dabei als ein Service Task modelliert. Auf ihn folgt ein Benutzer-Task. Im Zuge von diesem soll ein Mitarbeiter prüfen, ob die extrahierten Daten korrekt sind. Dazu werden ihm, über einen Dialog, sowohl die Rechnung als auch die extrahierten Daten zur Verfügung gestellt. Letztere kann der Mitarbeiter bei Bedarf korrigieren. Folgend darauf wird, im letzten Schritt, auf Basis der verifizierten, bzw. korrigierten Daten, die Zahlung ausgeführt. Nach diesem Schritt, der ebenfalls als ein Service Task abgebildet ist, endet der Prozess.

7.3: Prototyp 2 – Automatisierung mittels Geschäftsregel Task

Der zweite Prozess hingegen, der prototypisch durch den Einsatz einer KI-Lösung, in Kombination mit der BPMN und einer Process Engine automatisiert werden soll, ist ein Prozess zur Bearbeitung von Kreditanfragen. Im Vergleich zum ersten Prozess ist er branchenspezifisch. Er wird lediglich von Unternehmen aus der Finanzbranche, bzw. Banken ausgeführt. Dabei kann er – analog zum ersten Beispiel – ebenfalls in einer Form vorliegen, in welcher er nicht durch eine Engine gesteuert wird. In einer solchen könnte er beispielsweise, wie in Abbildung 58 dargestellt ablaufen.

Ausgangslage – Beschreibung des zu automatisierenden Prozesses

Orientiert an dieser, wird eine neue Instanz des Prozesses gestartet, wenn eine Kreditanfrage von einem Kunden eingeht. Diese enthält konkrete Informationen – z.B. solche zur Höhe des Kredits, zum Einkommen des Kunden und zu dessen Kreditsicherheiten. Der Antrag wird im ersten Schritt des Prozesses an einen Mitarbeiter weitergeleitet. Dieser überprüft, auf Basis der Informationen, die in der Anfrage enthalten sind, ob der Kunde kreditwürdig ist. Faktisch trifft er also, auf Basis seiner Erfahrung, oder spezifischer Regeln eine Entscheidung. Das Resultat dieser gibt er in eine konkrete Software ein. Je nach Ergebnis müssen in der Folge sodann unterschiedliche Aufgaben ausgeführt werden. Ist der Kunde kreditwürdig, so muss ein zuständiger Mitarbeiter im nächsten Schritt ein Kreditangebot erstellen und dieses verschicken. Ist er hingegen nicht kreditwürdig, so muss der Mitarbeiter den Kunden über die Ablehnung seiner Anfrage informieren.

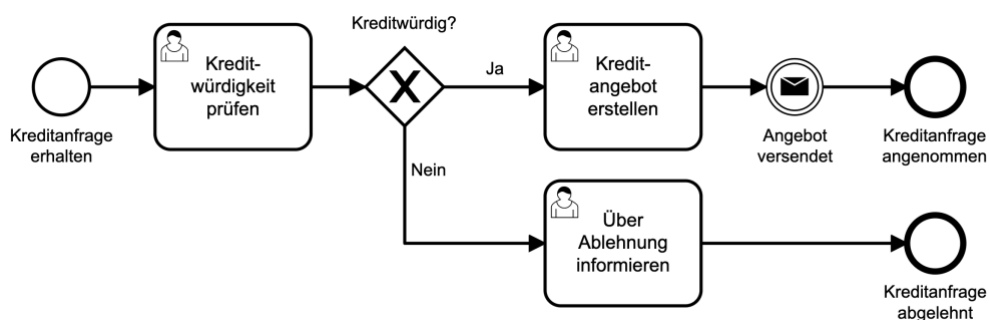


Abbildung 58: Ausgangslage – Prozess zur Bearbeitung von Kreditanfragen

Im Großen und Ganzen kann argumentiert werden, dass auch dieser Prozess ein hohes Potential zur Automatisierung aufweist. Denn auch in diesem werden alle Prozessschritte durch Mitarbeiter ausgeführt. In der Theorie könnten sie jedoch durch traditionelle Softwareentwicklung und den Einsatz von konkreten KI-Lösungen automatisiert werden.

Traditionelle Automatisierung

Für die traditionelle Automatisierung geeignet sind dabei insbesondere zwei Schritte – zum einen der Schritt zum Erstellen des Kreditangebots und zum anderen jener zum Informieren des Kunden über eine Ablehnung. Beide Schritte besitzen eine immer gleiche Logik, welche in einer Software abgebildet werden kann. So muss etwa im Rahmen des ersten Schrittes, jedes Mal, auf Basis der Anfrage und einer spezifischen Berechnungslogik, ein Kreditangebot berechnet werden. Folgend darauf muss dieses an den Kunden versendet werden. Im Zuge des zweiten Schrittes hingegen muss eine Nachricht bzgl. der Ablehnung verfasst und an den Kunden verschickt werden. Diese kann dabei z.B. auf einer Vorlage basieren, welche mit Daten zur Anfrage angereichert wird.

Einsatz einer KI-Lösung

Nicht geeignet für die traditionelle Automatisierung ist hingegen der Schritt zur Überprüfung der Kreditwürdigkeit eines Kunden. Dieser ist aus einer Perspektive der Automatisierung, analog zum vorher erörterten Auslesen von Daten aus einer Rechnung, erheblich komplexer. Die Tätigkeit ist dabei als eine konkrete Entscheidung, bzw. als eine Klassifikationsaufgabe zu verstehen. Sie wird auf Basis einer Vielzahl von Regeln und Informationen getroffen. Diese können zueinander jeweils in Beziehungen stehen und einen spezifischen Einfluss auf das Resultat der Entscheidung haben.

Als eine solche Entscheidung kann das Überprüfen der Kreditwürdigkeit – in der Theorie – durch ein ML-Modell, respektive durch einen Klassifikator automatisiert werden (siehe Abb. 59). Dabei muss jedoch erwähnt werden, dass sowohl die Daten als auch die Kriterien, die zum Treffen der Entscheidung verwendet werden, von Bank zu Bank unterschiedlich sein können. Folglich gibt es für diesen Anwendungsfall keine generischen Lösungen auf dem Markt. Daher muss im Zuge der prototypischen Automatisierung des Prozesses ein eigenes ML-Modell entwickelt werden.

Um ein solches zu erstellen, werden konkrete historische Daten zur Problemstellung benötigt. Die Beschaffung, die Vorbereitung, sowie weitere Details zu diesen werden in Anhang 4 beschrieben. Diese Daten werden sodann in der Folge verwendet, um einen Entscheidungsbaum zu trainieren. Der Vorgang, bzw. technische Details hierzu werden in Anhang 5 dokumentiert.

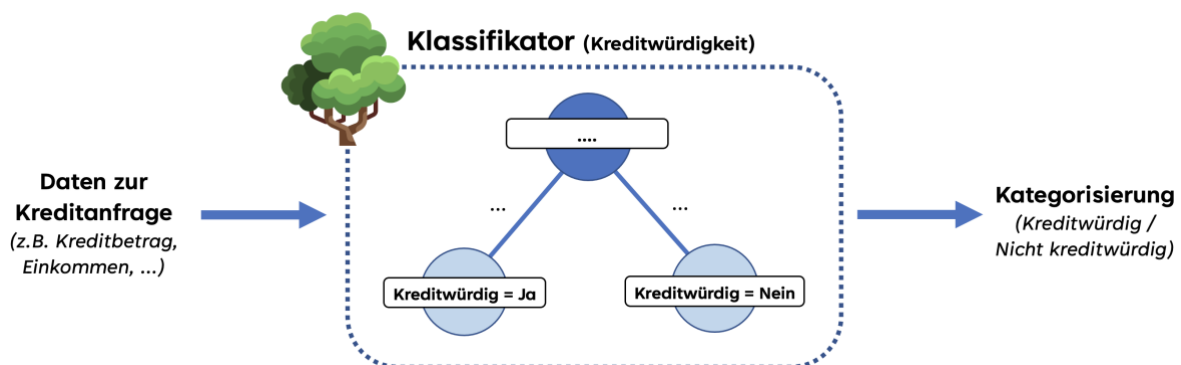


Abbildung 59: Klassifikator zur Bewertung der Kreditwürdigkeit von Bankkunden

Überprüfung der Ergebnisse einer KI

Wird der Entscheidungsbaum produktiv eingesetzt, so muss – wie auch beim Auslesen der Daten aus einer Rechnung – berücksichtigt werden, dass dessen Vorhersagen nur zu einer bestimmten Wahrscheinlichkeit korrekt sind. Dementsprechend kann es passieren, dass einige Vorgänge nicht richtig klassifiziert werden. Dies kann dazu führen, dass ein theoretisch kreditwürdiger Kunde als nicht-kreditwürdig eingestuft wird – und vice versa. Dies hat entweder zur Folge, dass die Bank

potentielle Zinserträge nicht erwirtschaftet, oder ein wesentlich höheres Risiko eingeht, dass ein Kunde einen Kredit nicht tilgen kann. Beide Fälle haben monetär negative Auswirkungen.

Daher kann auch hier diskutiert werden, inwiefern es sinnvoll sein kann, einen zusätzlichen Schritt in den Prozess einzufügen, in welchem die Klassifikation durch einen Mitarbeiter überprüft wird. Die Möglichkeiten zu einer solchen Ergebnisprüfung werden in Kapitel 6.4.3 diskutiert. Im Hinblick auf diese könnte ebenfalls eine allgemeine Prüfung ausgeführt werden, um die Konsequenzen zu vermeiden. Da diese jedoch bereits im Rahmen des ersten Prototyps implementiert wird, wird in diesem Prototyp eine ergebnisabhängige Prüfung eingesetzt. Sie wird nur dann ausgeführt, wenn die Kreditwürdigkeit eines Kunden negativ bewertet wird.

Automatisierung mittels der BPMN und einer Process Engine

Um den Prozess prototypisch in einer Process Engine zu automatisieren muss – wie im vorherigen Beispiel – zuerst, mit der BPMN, ein Prozessmodell erstellt werden. Werden dabei alle erörterten Aspekte einbezogen, so kann dieses, wie in Abbildung 60 dargestellt, modelliert werden. Um das Modell in einer Process Engine zu nutzen, muss es anschließend konfiguriert und die zugehörige Ausführungslogik technisch implementiert werden. Der Vorgang hierzu – sprich die tatsächliche, prototypische Automatisierung des Prozesses – wird ab Anhang 7 dokumentiert. Alles in allem kann der automatisierte Prozess jedoch wie folgt beschrieben werden:

Eine Instanz des Prozesses wird gestartet, sobald eine neue Kreditanfrage, über eine bestimmte, technisch implementierte Schnittstelle eingeht. Im ersten Schritt wird sodann, auf Grundlage der Daten aus der Anfrage, die Kreditwürdigkeit eines jeweiligen Kunden bewertet. Zu diesem Zweck wird das vorher erörterte, und eigens entwickelte ML-Modell eingesetzt. Da die Problemstellung als eine Entscheidung zu deuten ist, wird der Einsatz des ML-Modells über einen Geschäftsregel Task abgebildet. Kommt dieser zum Schluss, dass ein Kunde kreditwürdig ist, so soll im nächsten Schritt ein Service Task ausgelöst werden. Dieser erstellt ein Kreditangebot, und sendet dieses per Mail an den Kunden. Wird ein Kunde hingegen als nicht-kreditwürdig klassifiziert, so soll das Ergebnis der KI durch einen Mitarbeiter überprüft werden. Die Prüfung ist dabei als ein Benutzer-Task abgebildet. Bestätigt der Mitarbeiter im Rahmen von diesem das Ergebnis des ML-Modells, so soll anschließend ein Nachrichten-Sendender Task ausgelöst werden. In diesem wird eine Mail an den Kunden gesendet, in welcher er über die Ablehnung seiner Anfrage informiert wird. Kommt der Mitarbeiter bei der Prüfung jedoch zum Schluss, dass die Klassifikation der KI nicht schlüssig ist, so kann er eine Korrektur vornehmen, und den betroffenen Kunden als kreditwürdig einstufen. Dies hat zur Folge, dass keine Ablehnung verschickt wird. Stattdessen wird in der Folge – wie für jede andere positive Prüfung – ein Kreditangebot erstellt und an den Kunden versendet.

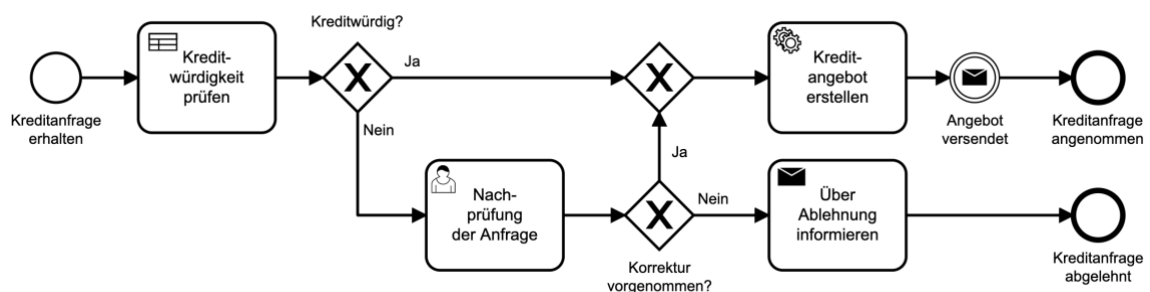


Abbildung 60: Prototyp - Automatisierter Prozess zur Bearbeitung von Kreditanfragen

Technische Implementierung des Geschäftsregel Tasks

Da der Einsatz des ML-Modells als Geschäftsregel Task abgebildet ist, muss beachtet werden, dass dessen technische Implementierung, wie es in Kapitel 6 beschrieben wird, auf zwei Weisen vorgenommen werden kann (siehe Kapitel 6.3.2).

Im Zuge der prototypischen Automatisierung werden beide Varianten implementiert – wobei für jede der Varianten ein eigenes Prozessmodell existiert, und die Ausführungslogik jeweils über ein anderes DMN-Entscheidungsmodell abgebildet wird (siehe Abb. 61).

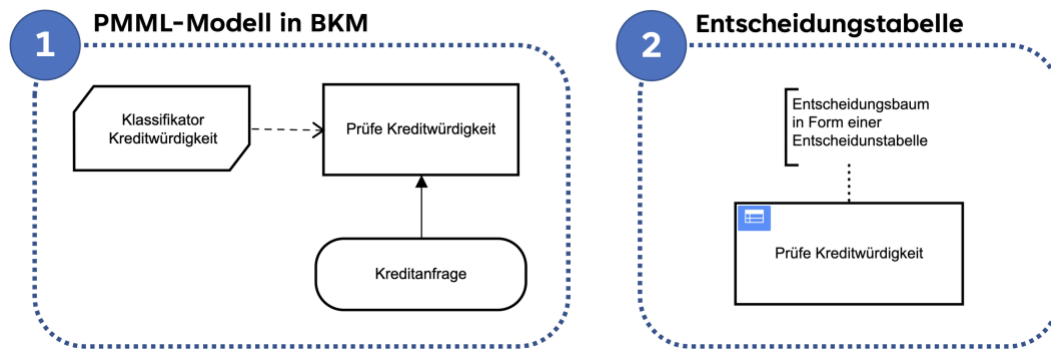


Abbildung 61: Varianten zur Technischen Implementierung des Klassifikators

7.3.1: Variante 1 – Einsatz eines PMML-Modells

Die erste Variante des Prozesses wird in Anhang 7 dokumentiert. In dieser wird die Prüfung der Kreditwürdigkeit eines Kunden durch den Einsatz des Klassifikators in Form eines PMML-Modells automatisiert. In dieser Form kann der Baum, repräsentiert durch ein Business Knowledge Model, nativ in ein DMN-Modell integriert werden (siehe Kap. 5.3.2). Wird dieses über den in Abbildung 60 erörterten Geschäftsregel Task in den prototypischen Prozess integriert, so wird das PMML-Modell aufgerufen, wenn die zugehörige Decision und folglich das BKM ausgelöst werden.

7.3.2: Variante 2 – Einsatz einer Entscheidungstabelle

Die zweite Variante hingegen ist in den Anhängen 8 und 9 dokumentiert. In dieser Variante wird der Entscheidungsbaum, der in der Form eines PMML-Modells abgebildet ist, in die Form einer Entscheidungstabelle überführt. Die Grundidee hierzu wird in Kapitel 6.3.3.2 diskutiert. Im Zuge von Anhang 8 wird eine Lösung implementiert, um diese Umwandlung durchzuführen. Diese wird sodann in Anhang 9 eingesetzt. Die resultierende Entscheidungstabelle kann über eine konkrete Decision in den Geschäftsregel Task aus Abbildung 60 eingebunden werden. Sie wird ausgeführt, wenn der zugehörige BPMN-Task ausgelöst wird und die Decision anspricht.

8: Evaluation – Bewertung der Prototypen

Im Zuge dieser Thesis werden die Prozesse, die in Kapitel 7 erörtert wurden, durch die Erstellung von Prototypen, automatisiert. Dazu wird die Camunda Process Engine eingesetzt und eine Web-Applikation entwickelt, in welcher die Ausführungslogik der Prozesse abgebildet ist. Der Vorgang zur Implementierung dieser Applikation, bzw. zur Automatisierung der Prozesse in der Engine ist in den Anhängen 3-9 dokumentiert. In diesem Kapitel werden die Prototypen evaluiert. Dabei wird insbesondere auf den Einsatz der KI-Lösungen eingegangen, sowie auf die Erkenntnisse zur Kombination dieser Lösungen mit der BPMN und einer Process Engine.

8.1: Prototyp 1 – Automatisierung mittels Service Task

Durch den ersten Prototyp wird ein Prozess zur Zahlung von Eingangsrechnungen automatisiert. Dieser ist nicht direkt wertschöpfend – zur Ausführung von jeglichen unternehmerischen Aufgaben jedoch notwendig. Die Rechnungen, welche in diesem verarbeitet werden, liegen immer in Form einer PDF vor. Um die Informationen, welche für die Bezahlung einer Rechnung relevant sind, aus einer solchen zu extrahieren, wird eine KI-Lösung eingesetzt – die „**veryfi Invoice API**“.

Integration der KI-Lösung in den Prozess

Die Implementierung des Prozesses und die Integration der KI-Lösung in diesen, ist in Anhang 3 dokumentiert. Im Hinblick auf diese ist zu erwähnen, dass der Gesamtaufwand zum Einsatz der KI sehr gering ist. Bei dieser handelt es sich um eine cloudbasierte Softwarelösung. Diese umfasst ein ML-Modell, welches auf jegliche Arten von Rechnungen trainiert ist. In aller Regel muss dieses nicht angepasst werden. Darüber hinaus ist die technische Implementierung sehr einfach. Um die Lösung anzusprechen kann eine Bibliothek genutzt werden, welche vom Anbieter zur Verfügung gestellt wird (siehe Anhang 2). Folglich ist es nicht notwendig, dass eine eigene Logik entwickelt werden muss, um die API aufzurufen. Stattdessen kann die Bibliothek in die Ausführungslogik der Aktivität eingebunden und in dieser verwendet werden. Diese wiederum kann sodann durch den Service Task, durch welchen die Aktivität im Prozessmodell abgebildet ist, aufgerufen werden.

Nutzen des KI-Einsatzes

Insgesamt wird durch den Einsatz der Lösung, der durch eine Process Engine gesteuert wird, ein Schritt im Prozess automatisiert, welcher zuvor durch einen Mitarbeiter durchgeführt wurde. Alles in allem kann dadurch sowohl ein quantitativer als auch qualitativer Mehrwert erzeugt werden.

Quantitativ gesehen werden vor allem Mitarbeiterressourcen frei, welche bisher durch den Schritt gebunden waren. Da diese stets begrenzt sind, konnte durch den Schritt ein potentieller Engpass entstehen. Durch die Automatisierung kann dieser aufgelöst und die freien Ressourcen für andere Zwecke eingesetzt werden. Außerdem kann die Zeit, welche zur Ausführung des Schrittes benötigt wird, signifikant reduziert werden. Dies liegt vor allem daran, dass eine Software eine spezifische Aufgabe, wie jene Klassifikationsaufgabe, für gewöhnlich wesentlich schneller lösen kann als ein Mensch. Zudem ist sie in aller Regel sehr flexibel skalierbar, und folglich – im Vergleich zu einem Mitarbeiter – kaum limitiert. Faktisch kann der Schritt, durch die Einbindung der KI in eine Engine, in nahezu Echtzeit ausgeführt werden, ohne auf freie Ressourcen warten zu müssen. Des Weiteren können die Kosten sinken, welche durch den Prozessschritt verursacht werden. Der Grund hierfür ist, dass die Gesamtkosten für eine Software – auf einen bestimmten Zeitraum bezogen – in aller Regel geringer sind als die Kosten für die Mitarbeiterressourcen. Ferner ist es aber auch denkbar,

dass die Anzahl der Fehler sinkt, die durch den Schritt verursacht werden – sofern angenommen werden kann, dass das ML-Modell im Gesamten weniger Fehler verursacht als ein Mensch.

Aus einer qualitativen Sicht hingegen hat die Automatisierung insbesondere zur Folge, dass die Zufriedenheit der Mitarbeiter steigen kann. Dies ist dadurch zu begründen, dass dem Mitarbeiter eine sehr monotone und repetitive Arbeit abgenommen wird. Eine solche kann auf Dauer zu einer Unzufriedenheit der jeweiligen Mitarbeiter führen. Sofern die neue Tätigkeit attraktiver ist, kann dieser entgegengewirkt werden.

Prüfung als Zusatzaufwand

Gleichermaßen muss jedoch auch beachtet werden, dass durch die Prüfung des Ergebnisses der KI ein neuer Schritt in den Prozess eingefügt wird. Dieser soll durch einen Mitarbeiter bearbeitet werden. Durch den Schritt sollen potentielle Konsequenzen vermieden werden, welche entstehen können, wenn auf der Basis einer fehlerhaften KI-Vorhersage eine konkrete Handlung ausgeführt wird – wie die Überweisung des Rechnungsbetrages an ein falsches Konto. Diese Prüfung erzeugt jedoch einen hohen Zusatzaufwand, da sie für jede Instanz des Prozesses ausgeführt wird. Daher kann der Zeitvorteil, der durch den Einsatz der KI im ersten Prozessschritt erzeugt wird, nicht voll ausgeschöpft werden, und gegebenenfalls ein neuer Engpass entstehen. Außerdem ist der Schritt sehr monoton, weshalb auch die potentiell höhere Mitarbeiterzufriedenheit nicht erreicht werden könnte. Dementsprechend sollte diskutiert werden, ob die Prüfung in allen Fällen, im Hinblick auf die Konsequenzen, wirtschaftlich sinnvoll ist, oder ob eine andere Lösung gefunden werden sollte. Eine tiefere Untersuchung dieser Fragestellung findet im Rahmen von Kapitel 9.2.3 statt.

Automatisierung weiterer Schritte als Folge

Ferner wird mit der Ausführung der Zahlung ein weiterer Schritt im Prozess automatisiert, welcher bisher durch einen Mitarbeiter bearbeitet wurde. Dessen Automatisierung ist allerdings lediglich symbolisch. Es ist keine reelle Ausführungslogik hinterlegt, durch welche eine Zahlung tatsächlich ausgeführt wird. Dementsprechend kann keine objektive Aussage über den Gesamtaufwand der Implementierung getätigt werden. Können jedoch zur Entwicklung der Ausführungslogik, wie bei der Einbindung der KI-Lösung, konkrete vordefinierte Bibliotheken verwendet, oder existierende Lösungen in den Prozess integriert werden, so kann angenommen werden, dass der Aufwand zur Implementierung des Schrittes ebenfalls gering ist.

Dabei hat – unabhängig von der endgültigen Form – auch diese Automatisierung zur Folge, dass die Mitarbeiterressourcen, die bisher an den Prozess gebunden sind, frei werden. Zudem gilt auch für diesen Schritt, dass die Zeit, die für dessen Ausführung benötigt wird, verringert werden kann. Zur Erfüllung des Schrittes werden dabei die Daten genutzt, welche von der KI aus der jeweiligen Rechnung extrahiert wurden. Diese werden, im Hinblick auf den Prototypen, in einer Datenbank gespeichert, und können durch eine simple Abfrage selektiert werden. Insgesamt zeigt dies, dass die KI-Lösung, durch den Einsatz der BPMN, sowie der Process Engine, vollständig in den Prozess integriert ist – und ihr Ergebnis zu konkreten Zwecken weiterverarbeitet.

Gesamtfazit zum Prototyp

Im Allgemeinen zeigt sich also, dass die Automatisierung des Prozesses, durch den Einsatz einer KI, in Kombination mit der BPMN und einer Process Engine, einige Mehrwerte heben kann. Dabei wird jedoch auch ersichtlich, dass einige Potentiale, welche durch die einzelnen Schritte gehoben werden, im Hinblick auf den kompletten Prozess nicht realisiert werden können. So werden durch die Automatisierung der Datenextraktion und der Zahlung z.B. einige Mitarbeiterressourcen frei. Da diese limitiert sind, können potentielle Engpässe vermieden werden. Zudem könnten die freien Ressourcen für unternehmerisch sinnvollere, bzw. für wertschöpfende oder kreative Tätigkeiten

eingesetzt werden. Da einige Mitarbeiter allerdings für die Prüfung der KI-Ergebnisse eingeplant werden müssen, verbleiben diese, zumindest zum Teil, im Prozess. Des Weiteren könnten sowohl die Durchlaufzeit als auch die Kosten des Prozesses verringert werden. Auch hier ist die Prüfung jedoch kontraproduktiv, denn sie verlängert den Prozess. Ohne den Prüfschritt wäre der Prozess vollständig automatisiert. Ferner verursacht er, durch den Einsatz der Mitarbeiter, neue Kosten.

8.2: Prototyp 2 – Automatisierung mittels Geschäftsregel Task

Durch den zweiten Prototyp wird ein Prozess zur Bearbeitung von Kreditanfragen automatisiert. Dieser ist grundsätzlich als wertschöpfend zu interpretieren. Durch die Ausführung des Prozesses, bzw. durch die damit verbundene Vergabe von Krediten wird ein Wert generiert. Dieser zeichnet sich aus, durch die Einnahme von Zinserträgen. Um in diesem Prozess die Kreditwürdigkeit eines potentiellen Kunden zu prüfen, wird ein eigens entwickeltes ML-Modell eingesetzt.

Training des Modells

Der Vorgang zur Erstellung des ML-Modells ist in den Anhängen 4 und 5 dokumentiert. Um es zu trainieren, wird eine aussagekräftige Menge von historischen Daten benötigt. Für den Prototypen werden diese von einer externen Quelle bezogen – der Datenplattform „Kaggle“. Für den Einsatz in einem produktiven Umfeld wären diese Daten nicht geeignet – da die Regeln, um zu bewerten, ob eine Person kreditwürdig ist, von Bank zu Bank unterschiedlich sein können. Faktisch würden primär Daten benötigt werden, welche aus einer verifizierten, bzw. internen Quelle stammen. Um jedoch grundlegende Erkenntnisse zum Training und Einsatz eines eigenentwickelten ML-Modells zu sammeln, sind die Daten völlig ausreichend.

So lässt sich aus dem Prototyp z.B. ableiten, dass durch den Vorgang der Modellbildung ein hoher Aufwand erzeugt wird. Zusätzlich zum eigentlichen Training, sowie zum Test des Modells umfasst diese auch eine Datenselektion und eine Datenvorbereitung, die viel Arbeit in Kauf nehmen kann. Ferner verursacht das Modell auch im Produktivbetrieb einen kontinuierlichen Aufwand. Es muss gewartet, regelmäßig überprüft, und bei Bedarf angepasst, bzw. erneut trainiert werden. Dabei müssen für letzteres ggf. die oben genannten Schritte wiederholt werden.

Integration der KI-Lösung in den Prozess

Das trainierte ML-Modell wird in zwei Varianten in den Prozess integriert. Im Rahmen der ersten Variante wird es in der Form eines PMML-Modells, welches von einigen wenigen DMN-Engines interpretiert werden kann, in ein DMN-Modell eingebunden. Zu diesen DMN-Engines zählt unter anderem die Engine „Drools“. Der Vorgang zur Einbindung und zum Einsatz dieser im Prozess ist in Anhang 7 dokumentiert. Im Hinblick auf diesen Vorgang ist zu bemerken, dass die technische Implementierung der Variante, im Vergleich zum Training des Modells, einen deutlich geringeren Aufwand umfasst. Dies liegt daran, da die DMN-Engine, wie auch die KI im ersten Prototyp, eine vordefinierte Bibliothek zur Verfügung stellt. Diese kann von Entwicklern verwendet werden, um ein DMN-Modell direkt auszuführen. Folglich muss im Prototyp nur eine Logik abgebildet werden, um die Daten zu selektieren, welche die Engine zum Treffen der Entscheidung benötigt. Außerdem muss aber auch das Ergebnis des DMN-Modells verarbeitet werden. Die finale Logik kann sodann durch die Konfiguration eines zugehörigen Geschäftsregel Tasks aufgerufen werden.

Im Zuge der zweiten Variante hingegen wird dasselbe ML-Modell in eine Entscheidungstabelle transformiert – und als eine solche zur Durchführung der Aufgabe eingesetzt. Die Einbindung der Tabelle in die Process Engine ist dabei sehr simpel. Sie muss lediglich in eine Decision aus einem DMN-Modell eingebunden werden. Folgend darauf muss der zugehörige Geschäftsregel Task so

konfiguriert werden, dass er diese auslöst und ihr Ergebnis in einer spezifischen Prozessvariable speichert. Die eigentliche Komplexität der Variante liegt dafür jedoch in der Umwandlung. Diese ist logisch betrachtet hochkomplex und verursacht einen großen Aufwand. Um sie durchzuführen, wird – beschrieben durch Anhang 8 – ein eigener Algorithmus, bzw. eine Applikation entwickelt.

Nutzen des Einsatzes

Alles in allem wird der Einsatz des ML-Modells also durch eine Process Engine gesteuert. Dadurch wird, analog zum ersten Prototypen, ein Schritt im Prozess automatisiert, der bisher durch einen Mitarbeiter ausgeführt wurde. Folglich ist auch der Mehrwert, welcher durch die Automatisierung erzielt werden kann, sehr ähnlich zum ersten Prototyp.

So werden aus quantitativer Sicht ebenfalls die Mitarbeiterressourcen frei, welche bisher an den Schritt gebunden waren. Faktisch kann auch hier ein potentieller Engpass behoben und die freien Ressourcen für andere Tätigkeiten eingesetzt werden. Darüber hinaus gilt auch für diesen Schritt, dass die Zeit, welche für dessen Ausführung benötigt wird, signifikant sinkt. Gleiches kann daher auch für die Kosten gelten, welche durch den Schritt erzeugt werden.

Qualitativ betrachtet kann hingegen garantiert werden, dass die Einstufung der Kreditwürdigkeit eines Kunden immer objektiv vorgenommen wird. Im Ausgangszustand des Prozesses kann dies nie endgültig sichergestellt werden. Wird die Entscheidung durch einen Mitarbeiter getroffen, so kann in dessen Entscheidung immer eine gewisse Subjektivität einfließen – etwa aufgrund seiner Erfahrung. Wird jedoch das ML-Modell eingesetzt, so wird jede der Entscheidungen datenbasiert getroffen. Dabei werden für alle Entscheidungen dieselben Entscheidungsregeln angewendet.

Prüfung als Zusatzaufwand

Ferner muss beachtet werden, dass auch bei diesem Prototyp die Vorhersagen der KI überprüft werden. Diese Prüfung wird dabei ebenfalls durch einen Mitarbeiter durchgeführt. Allerdings wird sie nicht für jede Instanz vorgenommen, sondern lediglich, wenn der Kunde als nicht kreditwürdig eingestuft wird. Dementsprechend benötigt auch dieser Schritt einige Ressourcen – jedoch nicht so viele, wie der erste Prototyp. Nichtsdestotrotz sollte auch für dieses Beispiel diskutiert werden, ob die Prüfung wirtschaftlich ist – oder ob ggf. sogar weitere Fälle überprüft werden sollten. So könnte es z.B. sinnvoll sein, auch Fälle zu prüfen, in welchen ein Kunde als kreditwürdig eingestuft wird und der gewünschte Kreditbetrag sehr hoch ist. Ist die Klassifikation in einem solchen Fall falsch, so kann diese bei einem Zahlungsausfall hohe monetäre Konsequenzen haben. Eine tiefere Untersuchung der Fragestellung findet auch im Zuge von Kapitel 9.2.3 statt.

Automatisierung weiterer Schritte als Folge

Auch die restlichen Schritte des Prozesses werden im Zuge der prototypischen Automatisierung durch die BPMN abgebildet und in einer Engine automatisiert. Ihre Automatisierung ist jedoch, wie auch schon im letzten Prototyp, nur symbolisch. Dies bedeutet, es ist keine Ausführungslogik hinterlegt, durch die eine Absage versendet, oder ein Kreditangebot erstellt und versendet wird. Daher kann auch in diesem Fall keine vollständige Aussage über den Aufwand getroffen werden, der bei der Implementierung der Schritte anfällt. Es kann allerdings gefolgert werden, dass auch durch die Automatisierung dieser Schritte, die Mitarbeiterressourcen, die bisher an den Prozess gebunden waren, frei werden. Weiterhin sinkt auch bei diesen Schritten die Zeit, welche für deren Durchführung benötigt wird. Ferner können auch die Kosten des Prozesses reduziert werden.

Abschließend zeigt sich, dass die KI-Lösung auch bei diesem Prototyp – durch die Kombination mit der BPMN und einer Engine – vollständig in den Prozess integriert ist. Denn das Ergebnis des ML-Modells wird für einen bestimmten Zweck weiterverarbeitet. In diesem Fall ist dieser Zweck die Steuerung der Ablauflogik. Durch diese werden die jeweils relevanten Folgeschritte ausgelöst.

Gesamtfazit zum Prototyp

Im Fazit kann daher abgeleitet werden, dass auch die Automatisierung des zweiten Prozesses, durch den Einsatz eines ML-Modells, der BPMN und einer Process Engine einen nennenswerten Mehrwert erzeugt. So werden durch die Automatisierung der Prozessschritte, welche bisher durch Mitarbeiter ausgeführt wurden, einige Mitarbeiterressourcen frei. Demzufolge können etwaige Mitarbeiterengpässe verhindert und die freien Ressourcen für andere Zwecke eingesetzt werden. Ein solcher könnte z.B. die Kundenbetreuung sein. Zudem sinkt die Durchlaufzeit des Prozesses signifikant – zumindest, wenn der Kunde vom ML-Modell als kreditwürdig klassifiziert wird. In diesem Fall läuft der Prozess ohne menschlichen Eingriff, respektive vollständig softwarebasiert ab. Dank der Steuerung durch die Engine wird er in nahezu Echtzeit, bzw. in wenigen Sekunden abgefertigt. Diese minimale Reaktionszeit kann die Zufriedenheit der Kunden stark erhöhen, da sie sofort eine Rückmeldung zu ihrer Anfrage erhalten. Ergänzend dazu gibt es je nach Variante jedoch noch einige weitere Erkenntnisse. Diese werden in der Folge kurz erörtert.

8.2.1: Variante 1 – Spezielle Erkenntnisse zum Einsatz des PMML-Modells

Die Beschreibung aus dem letzten Unterkapitel, sowie Anhang 7 zusammenfassend, wird im Zuge der ersten Variante, das trainierte ML-Modell, in Form eines PMML-Modells, in ein DMN-Modell integriert. Folgend darauf wird dieses DMN-Modell durch eine geeignete Engine ausgeführt.

Aus einer fachlichen Sicht heraus erweist sich, dass dieses Vorgehen den Mehrwert hat, dass die Entscheidung, welche durch das ML-Modell getroffen wird, transparent gemacht wird. Der Grund dafür ist, dass die Entscheidung modelliert werden muss, um in die DMN-Engine eingebunden zu werden. Da das resultierende Modell graphisch dargestellt werden kann, ist somit sichtbar, wie die Gesamtentscheidung getroffen wird und welche Daten für diese benötigt werden.

Wird die Implementierung dagegen aus einer technischen Perspektive bewertet, so muss erwähnt werden, dass die Nutzung der DMN nicht zwangsläufig notwendig ist, um das zugrundeliegende PMML-Modell auszuführen. Um es auszuwerten könnte auch eine sog. PMML-Engine eingesetzt werden. Alles in allem wird durch das Vorgehen also ein zusätzlicher Aufwand erzeugt. Statt das PMML-Modell also direkt anzusprechen, muss ein DMN-Modell erstellt werden, in welchem das ML-Modell integriert ist. Zudem muss eine externe DMN-Engine in den Prozess integriert werden, und alle Modelle sowie der Code hierzu gewartet werden. Ergo sollte also erörtert werden, wann es sinnvoll ist, die DMN einzusetzen. Diese Fragestellung wird in Kapitel 9.3.2 diskutiert.

8.2.2: Variante 2 – Spezielle Erkenntnisse zum Einsatz der Entscheidungstabelle

Im Rahmen der zweiten Variante hingegen, die ebenfalls im vorherigen Unterkapitel beschrieben wird und zudem in Anhang 9 dokumentiert ist, wird das PMML-Modell in die Form einer DMN-Entscheidungstabelle transformiert. Diese wiederum wird durch eine DMN-Engine ausgeführt.

Theoretisch betrachtet bringt diese Variante den Vorteil, dass die Entscheidungsregeln des ML-Modells transparent, bzw. verständlich dargestellt werden. Für den Prototyp kann dies allerdings nicht gänzlich bestätigt werden. Die Umwandlung ist zwar möglich – insgesamt zeigt sich jedoch, dass die finale Entscheidungstabelle immer noch sehr komplex ist. Sie umfasst zwölf Spalten mit Input-Werten, eine Spalte mit Output-Werten und 220 Regeln. Darüber hinaus gibt es durch die Umwandlung keinen reellen Erkenntnisgewinn – denn die Entscheidungsregeln werden aus einem Entscheidungsbaum extrahiert. Dieser ist als ein symbolisches Modell zu deuten. Faktisch waren

die Regeln in diesem bereits transparent. Sie werden lediglich, wie in Kapitel 6.3.3.2 beschrieben, in eine andere Form überführt. Im Vergleich zur Abbildung als nativer Entscheidungsbaum zeigt sich jedoch, dass diese Form den Vorteil bringt, dass die Engine speichert, welche Regel im Zuge einer Entscheidung ausgewählt wird. Faktisch kann immer nachvollzogen werden, weshalb eine bestimmte Entscheidung getroffen wurde.

9: Evaluation – Nutzen der Kombination von KI und BPMN

Rückblickend auf die initiale Zielsetzung, soll durch diese Thesis untersucht werden, inwiefern der Einsatz einer KI, in Kombination mit der BPMN sowie einer Process Engine einen Nutzen erzeugen kann. Im Rahmen von diesem Kapitel wird diese Fragestellung final beantwortet. Die Grundlagen zur Beantwortung sind dabei die Entwürfe aus Kapitel 6 und die Bewertung der Prototypen aus Kapitel 8. Mit diesen wird die Fragestellung aus zwei Blickwinkeln betrachtet. Dabei wird zuerst evaluiert, welche allgemeinen Erkenntnisse zur Kombination gewonnen werden können. Folgend darauf wird bewertet, wie die BPMN bei der Lösung der Herausforderungen helfen kann, die in Kapitel 4.4 erörtert werden. Abschließend wird sodann ein Gesamtfazit zum Nutzen gezogen.

9.1: Allgemeine Kombination

Alles in allem lässt sich durch die Thesis darlegen, dass die Künstliche Intelligenz für eine Vielzahl von Anwendungsszenarien eingesetzt werden kann (siehe Kapitel 4.2). Zudem wird belegt, dass sie sowohl theoretisch als auch praktisch, mit der BPMN und Process Engines kombiniert werden kann. Dabei lassen sich unter anderem die folgenden Erkenntnisse ableiten:

9.1.1: Nutzen des Einsatzes einer KI

Durch die entworfenen Prototypen lässt sich belegen, dass die KI insbesondere eingesetzt werden kann, um monotone sowie repetitive Aufgaben zu übernehmen, welche bisher von menschlichen Prozessakteuren ausgeführt werden. Dabei erweist sich, dass der Einsatz einer KI-Lösung sowohl einen qualitativen als auch einen quantitativen Nutzen erzeugen kann.

So werden aus einer quantitativen Perspektive eine bestimmte Menge von Mitarbeiterressourcen frei, die bisher an etwaige Prozessschritte gebunden waren. Folglich können potentielle Engpässe aufgelöst und die freien Ressourcen für andere Zwecke eingesetzt werden, die wiederum einen höheren Nutzen erzielen. Ein solcher Zweck könnte z.B. eine Aufgabe im direkten Kundenkontakt, oder eine anderweitige wertschöpfende Tätigkeit sein. Daneben zeigt sich, dass die Durchlaufzeit der Prozesse reduziert werden kann und auch die Kosten dieser sinken können.

Darüber hinaus kann aus den quantitativen Aspekten auch ein qualitativer Nutzen resultieren. So können die geringere Durchlaufzeit und mehr freie Mitarbeiterressourcen etwa dazu führen, dass die Kundenzufriedenheit steigt. Ferner kann abgeleitet werden, dass eine KI stets Entscheidungen trifft, welche datenbasiert sind. Demzufolge sind sie frei von der Subjektivität eines menschlichen Entscheiders.

Im Fazit muss jedoch erwähnt werden, dass diese Vorteile nie generalisierbar sind. Der wirkliche Mehrwert ist stets abhängig vom zugrundeliegenden Anwendungsszenario, sowie der fachlichen und der technischen Integration der Lösung in die relevanten Systeme.

Potential der (Voll-)Automatisierung durch eine Process Engine

Nichtsdestotrotz kann geschlussfolgert werden, dass die KI in Kombination mit der BPMN, sowie einer Process Engine, ein erhebliches Potential zur Automatisierung von Prozessen besitzt. Durch die Möglichkeit, einzelne Schritte in einem Prozess zu automatisieren und diese softwarebasiert zu steuern, kann ein ganzer Prozess vollautonom ausgeführt werden. Ist dieser wiederum in eine

Anwendung eingebettet, die in der Lage ist, den Prozess zu starten und dessen Ergebnis in einer geeigneten Form anzuzeigen, so kann einem Kunden ein 24/7-Service bereitgestellt werden. Aber auch wenn dies nicht der Fall ist, erzeugt die Engine einen Nutzen. So kann der Einsatz einer KI zumindest gesteuert, bzw. automatisch ausgeführt und die Folgeschritte ausgelöst werden.

9.1.2: Möglichkeiten zum KI-Einsatz

Damit das Potential jedoch ausgeschöpft werden kann, sollte der Einsatz einer KI-Lösung, durch die Modellierung einer Aufgabe, in einem Prozessmodell abgebildet werden – und dieses Modell in eine geeignete Process Engine eingebunden werden. Dabei wird im Zuge von Kapitel 6.2 und 6.3 untersucht, welche konkreten Typen von Aufgaben hierzu geeignet sind. Diese werden in der Folge, im Hinblick auf die Modellierung und die Automatisierung, evaluiert.

9.1.2.1: Service Task

Zuallererst wird der Service Task als geeignet befunden. Orientiert an der Definition aus Kapitel 5.1.3.2 ist er zu deuten als eine konkrete Aufgabe, die durch eine beliebige Software ausgeführt wird. Folglich bietet der Task den wesentlichen Vorteil, dass durch ihn, aus einer fachlichen Sicht, der Einsatz von jeglichen Arten von KI-Lösungen abgebildet werden kann. Dabei ist es irrelevant, ob die Lösung von einem Unternehmen selbst entwickelt, oder von einem spezifischen externen Anbieter beschafft wird. Der Task kann für beide Fälle eingesetzt werden.

Auch aus einer technischen Perspektive erweist sich der Service Task als extrem flexibel. So zeigt sich auf Basis der theoretischen Analyse und dem Einsatz im ersten Prototyp, dass es diverseste Möglichkeiten geben kann, um eine Ausführungslogik für einen Service Task zu hinterlegen. Eine von diesen ist, dass eine individuelle Ausführungslogik entwickelt werden kann, die im Zuge der Aufgabenausführung ausgelöst wird. Durch diese Möglichkeit gibt es – aus einem theoretischen Blickwinkel – schier unendliche Möglichkeiten zur Ausgestaltung der Ausführungslogik.

Nichtsdestotrotz sollte beim Einsatz eines Service Tasks berücksichtigt werden, dass dieser sehr generisch ist. Es gibt nahezu unendlich viele Arten von Software. Deswegen ist es – sofern dieses Wissen relevant ist – nur schwer ersichtlich, ob im Rahmen eines Service Tasks eine KI eingesetzt wird. Denn in der Theorie kann in diesem Task jede beliebige Software verwendet werden.

9.1.2.2: Nachrichten-Sendender Task

Als zweiter geeigneter Typ wird der Nachrichten-Sendende-Task genannt. Angelehnt an seiner Definition aus Kapitel 5.1.3.2 ist er zu interpretieren als eine spezifische Aufgabe, im Zuge welcher eine Nachricht an einen externen Akteur gesendet wird. Ein solcher könnte logisch betrachtet z.B. eine KI sein. Aus dieser Beschreibung lässt sich ableiten, dass der Nachrichten-Sendende-Task ebenfalls sehr generisch ist. Daher kann er, wie der Service Task, eingesetzt werden, um sowohl den Einsatz von eigenentwickelten als auch von fremdbeschafften KI-Lösungen abzubilden.

Ferner ist der Nachrichten-Sendende-Task auch aus einem technischen Blickwinkel sehr flexibel. Auch für ihn gibt es unterschiedlichste Möglichkeiten zur Konfiguration, bzw. zur Definition einer Ausführungslogik. Zu diesen zählt erneut die Möglichkeit eine individuell programmierte Logik zu

hinterlegen. Dementsprechend sind auch bei diesem Task die Optionen zur Ausgestaltung einer praktischen Ausführungslogik nahezu endlos.

Insgesamt sollte jedoch beachtet werden, dass auch der Nachrichten-Sendende-Task bestimmte Nachteile aufweist. So ist es auch bei diesem Aufgabentyp nur schwer ersichtlich, ob im Rahmen einer konkreten Aufgabe eine KI angesprochen wird. Darüber hinaus ist bei dieser Variante die Modellierung des Nachrichten-Sendenden-Tasks in der Regel nicht ausreichend. Der Task endet, sobald die Nachricht versendet wurde. Demzufolge wird nicht auf das Ergebnis des KI-Einsatzes gewartet. Ist dieses von Relevanz, muss ergo ein Nachrichten-Empfangendes Ereignis modelliert werden. Alles in allem führt dies dazu, dass die KI in der Lage sein muss, ein solches Ereignis zu versenden. Außerdem erhöht es die Komplexität im Prozessmodell.

9.1.2.3: Geschäftsregel Task

Der dritte Typ, der als geeignet befunden wird, um den Einsatz einer KI-Lösung abzubilden, ist der Geschäftsregel Task. Abgeleitet aus Kapitel 5.1.3.2 ist er zu verstehen als eine Aufgabe, im Rahmen welcher eine konkrete Entscheidung getroffen wird. Diese Charakteristik ist sehr ähnlich zu der vieler KI-Lösungen. Denn die Grundidee dieser ist es, in aller Regel, Vorhersagen zu treffen, welche als Entscheidungen interpretiert werden können. Theoretisch kann also durch diesen Typ der Einsatz einer beliebigen Lösung abgebildet werden, die dieser Charakteristik entspricht.

Jedoch gibt es eine technische Limitierung. Jede Entscheidung, die durch einen Geschäftsregel Task abgebildet wird, muss automatisch durch eine Business Rule Engine getroffen werden. Eine solche ist z.B. eine DMN-Engine. Die Theorie zu dieser wird in Kapitel 5.4 vermittelt. Um eine KI, bzw. deren zugrundeliegendes ML-Modell in eine solche DMN-Engine zu integrieren, muss jenes Modell in eine Form überführt werden, in der es durch die Engine interpretiert werden kann. Eine solche ist bspw. ein PMML-Modell oder eine Entscheidungstabelle. Diese Anforderungen erfüllen, können in aller Regel nur Lösungen, die eigenentwickelt sind. Folglich sind KI-Lösungen, die von externen Anbietern beschafft werden, meist nicht geeignet, um in den Task integriert zu werden.

Zudem muss berücksichtigt werden, dass die Umwandlung in eine Form, die für eine DMN-Engine interpretierbar ist, eine hohe Komplexität und je nach Fall, einen hohen Aufwand erzeugen kann. Dies zeigt sich vor allem anhand der Anhänge 8 und 9. Im Gegenzug bietet die DMN jedoch den Vorteil, dass sie den Vorgang einer Entscheidung sehr transparent darstellt. Denn dieser wird stets durch sogenannte Entscheidungsmodelle graphisch abgebildet.

Nichtsdestotrotz muss berücksichtigt werden, dass auch ein Geschäftsregel Task sehr generisch ist. Mit ihm können beliebige Arten von Entscheidungen abgebildet werden. Daher ist es auch bei einer oberflächlichen Betrachtung eines Geschäftsregel Tasks nicht auf Anhieb ersichtlich, ob im Zuge von diesem eine KI-Lösung eingesetzt wird.

9.1.2.4: KI-Task

Eine vierte Möglichkeit, welche geeignet scheint, um den Einsatz einer KI-Lösung abzubilden, ist gemäß Kapitel 6 die Erweiterung des BPMN-Standards um einen weiteren Typ – den „KI-Task“. Die Möglichkeit einer solchen Erweiterung wird in Kapitel 5.1.1.2 angesprochen.

Fachlich evaluiert wäre die Abbildung eines solchen KI-Tasks in einem BPMN-Prozessmodell sehr intuitiv. Da der Task ein eigenes, eindeutiges Symbol besitzt, wäre es jederzeit klar, dass im Zuge einer konkreten Aufgabe eine beliebige KI-Lösung eingesetzt wird.

Allerdings hat der Typ auch Nachteile. So widerspricht er z.B. der Grundidee des Standards. Diese will die Notation so simpel wie möglich machen, damit er branchenübergreifend eingesetzt und von jedem Anwender, unabhängig aus welchem Fachgebiet dieser stammt, verstanden werden kann. Würde für jede Art einer Software – wie eben eine KI – ein eigener Aufgabentyp existieren, so wäre dieser Anspruch nicht haltbar. Die Notation würde zu komplex werden. Außerdem muss beachtet werden, dass der Typ, weil er kein Teil des Standards ist, logischerweise keine Entität im Standard-Metamodell der Notation besitzt. Dies hat zur Folge dass er in jeglichen BPMN-Tools, wie z.B. einer Process Engine, oder einem Werkzeug zur Prozessmodellierung, nicht von Haus aus unterstützt wird. Um sie dennoch zur Modellierung oder zur Automatisierung zu nutzen, muss das Metamodell der Notation angepasst werden. Dadurch entsteht ein relevanter Aufwand.

9.1.2.5: Fazit zu untersuchten Typen

Im Resümee kann aus der Evaluation abgeleitet werden, dass – obwohl sie jeweils bestimmte Nachteile aufweisen – jeder der betrachteten Aufgabentypen eine konkrete Eignung besitzt, um den Einsatz einer KI-Lösung, im Rahmen einer Prozessaktivität abzubilden.

Liegt der Fokus jedoch auf der Thematik der Prozessautomatisierung durch eine Process Engine, so wird nur die Verwendung des Service-, des Nachrichten-Sendenden- und des Geschäftsregel Tasks empfohlen. Denn sie sind im Vergleich zum KI-Task ein Teil des BPMN-Standards. Als ein solcher existiert für sie eine Entität im Standard-Metamodell der Notation. Daher werden sie von Haus aus von jeglichen gängigen BPMN-Tools unterstützt.

Nichtsdestotrotz sollte stets abhängig vom Anwendungsszenario, sowie dem zugrundeliegenden Ziel entschieden werden, welcher Typ fachlich oder technisch am besten geeignet ist.

9.1.3: Daten als Grundvoraussetzung

Unabhängig davon, welcher Aufgabentyp schlussendlich gewählt wird, zeigt sich durch die Arbeit zudem, dass Daten eine fundamentale Rolle für den Einsatz einer KI-Lösung spielen. Sie können aus verschiedenen Quellen bezogen werden. Diese können sowohl von unternehmensinterner als auch externer Art sein. Dabei ist es unter anderem abhängig von der Art der Lösung, in welcher Menge und für welchen Zweck jene Daten benötigt werden. Wird die Lösung von einem externen Anbieter beschafft, so existiert häufig bereits ein vortrainiertes ML-Modell. Folglich werden nur Daten benötigt, um diese KI einzusetzen. Muss das ML-Modell jedoch angepasst werden, oder wird es – wie in Anhang 5 – komplett eigenständig erstellt, so nehmen die Daten eine wesentlich größere Rolle ein. In diesem Fall werden sie nicht nur zum Einsatz des ML-Modells, sondern auch zum Training und Test von diesem benötigt. Ergo müssen die Daten, wie in Kapitel 3.2.1 erörtert, in einer aussagekräftigen Menge vorliegen und möglichst viele unterschiedliche, bzw. relevante Fälle abdecken. Ansonsten kann entweder gar keines, oder kein qualitativ hochwertiges Modell entwickelt werden.

Vorteile durch den Einsatz einer Process Engine

Insgesamt kann aus dem Entwurf und den Prototypen abgeleitet werden, dass der Einsatz einer Engine in diesem Kontext einige Vorteile bieten kann. So ermöglicht sie es, durch die Entwicklung und die Ausführung von individueller Prozesslogik, dass die Daten, welche zum Einsatz einer KI-Lösung benötigt werden, flexibel verwaltet werden können. Dementsprechend können sie z.B. je

nach Bedarf, in einer Prozessvariable, aber auch in jeglichen anderen Datenquellen gespeichert, oder aus einer solchen bezogen werden.

Zudem kann eine Process Engine aber auch nützlich sein, wenn ein Unternehmen noch überhaupt keine Daten besitzt, welche zum Training eines ML-Modells verwendet werden können – jedoch plant, in Zukunft eine KI-Lösung einzusetzen, die individuell trainiert werden muss. In diesem Fall kann die Engine verwendet werden, um die Daten zu einem konkreten Schritt im Prozess, welcher potentiell durch den Einsatz einer KI automatisiert werden könnte, zu sammeln. Die Grundidee hierzu ist in Abbildung 62 visuell dargestellt. Sie kann auf diverse Weisen technisch implementiert werden – bspw. indem die Daten in konkreten Prozessvariablen und somit in der Datenbank der Engine gespeichert werden.

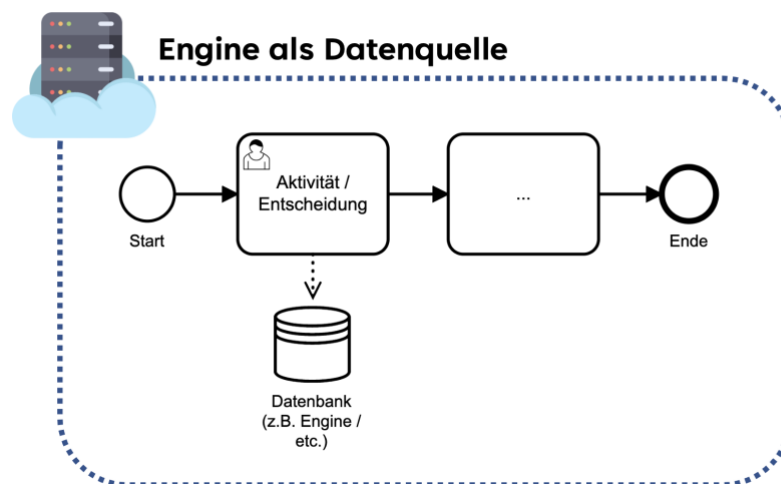


Abbildung 62: Process Engine als Datenquelle für ML-Modell Training

9.1.4: Aufwand und Komplexität der Implementierung

Sind geeignete Daten vorhanden, so steht dem Einsatz einer KI jedoch kaum mehr etwas im Weg. Dann können die notwendigen Schritte in die Wege geleitet werden, um eine KI in den jeweiligen Prozess zu integrieren. Der Aufwand und die Komplexität, welche dabei entstehen sind vor allem von zwei Faktoren abhängig:

Art der KI-Lösung

Zum einen sind sie abhängig von der Art der jeweiligen Lösung. Dabei erweist sich, dass der Weg der Eigenentwicklung zu einem hohen Aufwand und Komplexität führt. Dies zeigt sich vor allem, wie es in den Anhängen 4 und 5 ersichtlich ist, neben der technischen Implementierung, anhand der Datenvorbereitung und des Trainings einer KI-Lösung. Des Weiteren wird ein kontinuierlicher Aufwand durch die Wartung und den Betrieb der KI-Lösung verursacht. Weiterhin entsteht aber auch ein Aufwand durch die Entwicklung und die Wartung der Ausführungslogik, durch welche die KI angesprochen wird.

Bei einem System, welches dagegen von einem externen Anbieter beschafft wird, ist der Aufwand in aller Regel geringer. Der Grund dafür ist, dass gar kein, oder nur ein geringer Aufwand für das Training des ML-Modells anfällt. Zudem muss die Lösung selbst nicht gewartet werden – sondern ausschließlich die Ausführungslogik, durch welche diese eingebunden wird. Der Aufwand und die Komplexität, um diese zu programmieren, ist dabei grundsätzlich abhängig von den verfügbaren Implementierungsoptionen. Stellt der Anbieter – wie im ersten Prototyp, bzw. in Anhang 4 – eine

konkrete Bibliothek, oder eine simple API bereit, durch welche die Lösung angesprochen werden kann, so ist die Integration meist sehr einfach. Ist dies nicht der Fall, so kann die Implementierung jedoch auch sehr aufwendig werden.

Art des Aufgabentyps

Ferner hat aber auch der Aufgabentyp, der zur Abbildung des KI-Einsatzes eingesetzt wird, einen Einfluss auf den Aufwand und die Komplexität – denn dieser begrenzt die verfügbaren Optionen zur Ausgestaltung der Ausführungslogik. So hat ein Service Task für gewöhnlich eine extrem hohe Flexibilität. Er kann in aller Regel eine individuell programmierte Ausführungslogik auslösen. Im Rahmen von dieser können, angelehnt an den Prototypen, existierende Bibliotheken verwendet werden, um den Implementierungsaufwand möglichst gering zu halten (siehe Anhang 4).

Wird hingegen ein Geschäftsregel Task eingesetzt, und die Ausführungslogik des Tasks durch ein DMN-Modell abgebildet, dass durch eine DMN-Engine ausgewertet werden soll, so beschränken sich die Implementierungsmöglichkeiten auf den Funktionsumfang jener DMN-Engine. In diesem Fall kann es notwendig sein, dass die jeweilige KI-Lösung, die in den Schritt eingebunden werden soll, umgewandelt werden muss, bevor sie in das DMN-Modell eingebunden werden kann. Alles in allem wird daher vor allem durch die Umwandlung der KI-Lösung ein hoher Aufwand sowie Komplexität erzeugt (siehe Anhang 8). Nichtsdestotrotz sollte diese nie allein betrachtet werden. Auch die eigentliche Implementierung (siehe Anhang 9) und die Wartung der Modelle verursacht einen zusätzlichen Aufwand und Komplexität.

9.2: Kombination zur Lösung der KI-spezifischen Herausforderungen

Insgesamt ist es jedoch nicht nur möglich, den tatsächlichen Einsatz einer KI durch die BPMN und eine Process Engine abzubilden. Denn neben der Erörterung dieser Thematik wird durch die Thesis ebenfalls belegt, dass die beiden Lösungen dabei helfen können, die Herausforderungen zu lösen, welche in Kapitel 4.4 untersucht werden und beim Einsatz einer KI in Prozessen auftreten können. Dabei lassen sich – je einzelner Herausforderung – die folgenden Erkenntnisse ableiten:

9.2.1: Transparenz & Akzeptanz

Die erste Herausforderung, die untersucht wird, umfasst das Problem, dass der Einsatz von KI-Lösungen oft nicht transparent ist. Dieser Aspekt wiederum kann in einer mangelnden Akzeptanz der Lösungen durch konkrete Stakeholder resultieren. Ohne die Akzeptanz kann der Einsatz einer KI zur Automatisierung eines Prozesses jedoch kaum zum gewünschten Mehrwert führen.

Da die BPMN eine graphische Abbildung von Prozessen ermöglicht, wird im Rahmen von Kapitel 6.4.1 die Hypothese aufgestellt, dass sie dazu geeignet sein könnte, diese Herausforderung zu lösen. Denn durch ihre Nutzung kann der Einsatz einer KI abgebildet werden. Dazu können diverse Typen von Aufgaben eingesetzt werden. Zu diesen Typen zählen der Service-, der Nachrichten-Sendende-, der Geschäftsregel- und der sog. KI-Task. Diese werden in Kapitel 6.2.1 theoretisch untersucht und in Kapitel 9.1.2 evaluiert.

Eignung der Aufgabentypen

Im Hinblick auf diese Typen kann geschlussfolgert werden, dass jeder dieser eine grundsätzliche Eignung besitzt, um den Einsatz einer KI-Lösung abzubilden. Im Allgemeinen ist jedoch in Bezug

auf den Service-, den Nachrichten-Sendenden- und den Geschäftsregel Task zu beachten, dass diese jeweils sehr generisch sind. Ohne weitere Informationen, die bspw. in der Beschreibung des jeweiligen Tasks hinterlegt sind, ist es kaum ersichtlich, dass im Zuge ihrer Ausführung eine KI-Lösung verwendet wird. Dies ist jedoch kein abschließender Grund, diese nicht einzusetzen.

Der KI-Task hingegen ist aus fachlicher Sicht geradezu perfekt geeignet, um die Herausforderung zu lösen. Er ist sehr intuitiv und kann über ein geeignetes Symbol explizit anzeigen, dass im Zuge seiner Ausführung eine KI eingesetzt wird. Er hat jedoch den Nachteil, dass er kein Teil des BPMN-Standards ist – und daher nicht von Haus aus zur Modellierung eingesetzt werden kann. Er muss nachträglich in das jeweilige Tool, das zur Modellierung verwendet wird, eingepflegt werden.

Weitere Notationselemente

Darüber hinaus wird in Kapitel 6.2.1 diskutiert, dass Swimlanes und Artefakte verwendet werden können, um den Einsatz einer KI mit weiteren Kontextinformationen anzureichern. So zeigt sich, dass eine KI, durch die Abbildung in einem eigenen Pool als externer und durch die Abbildung in einer Lane als interner Prozessakteur modelliert werden kann. Ergänzend dazu kann durch ein Datenobjekt angezeigt werden, welche Daten im Zuge eines Prozessschrittes verarbeitet werden. Des Weiteren kann durch eine Datenbank abgebildet werden, an welchem Ort bestimmte Daten gespeichert, oder aus welcher Datenquelle diese bezogen werden. Die Anmerkung hingegen kann eingesetzt werden, um den Einsatz einer KI mit einem Kommentar anzureichern.

Daher kann also abgeleitet werden, dass es möglich ist, den Einsatz einer KI-Lösung mit weiteren Informationen zu ergänzen. Dabei muss jedoch berücksichtigt werden, dass für das Verständnis der Datenbank, des Datenobjektes, des Pools und der Lane, ein fachliches Wissen zur Notation erforderlich sein kann. Dieses kann nicht von allen Stakeholdern einer Organisation vorausgesetzt werden. Daher ist der Einsatz der Elemente zur Lösung der Herausforderung nur bedingt möglich.

Fazit zur Herausforderung

Insgesamt kann also geschlussfolgert werden, dass die Notation zur Lösung der Herausforderung nur beschränkt geeignet ist. So kann sie zwar als Werkzeug eingesetzt werden, um den jeweiligen Prozess in seiner grundlegenden Form graphisch zu modellieren. Dieser Aspekt wird jedoch in den meisten Fällen nicht ausreichend sein, um eine völlige Transparenz und Akzeptanz zu erreichen.

Stattdessen könnte dies ein Anwendungsszenario für das sogenannte Change-Management sein. Im Groben kann dieses beschrieben werden als ein Managementkonzept zur Planung, Steuerung und Kontrolle von tiefgreifenden Veränderungen in einer Organisation – wobei häufig der Faktor Mensch im Mittelpunkt steht (vgl. Chies 2016, S. 9). Ein Beispiel für eine solche Veränderung ist etwa der Einsatz von neuen Technologien, wie der Künstlichen Intelligenz.¹⁹

Da das Change-Management allerdings ein eigenes Themengebiet darstellt, das außerhalb des Rahmens dieser Thesis liegt, wird nicht weiter auf es eingegangen. Nichtsdestotrotz soll erwähnt werden, dass das Prozessmodell ggf. ein Hilfsmittel für die Methode sein kann.

9.2.2: Integration

Die zweite Herausforderung befasst sich damit, dass eine KI-Lösung vollständig in einen Prozess integriert werden muss – sowohl auf einer fachlichen als auch auf einer technischen Ebene. Erst wenn dies gegeben ist, kann der volle Mehrwert durch den Einsatz dieser Lösung erzeugt werden.

¹⁹ Für grundlegende Informationen zum Change-Management – siehe: Chies 2016, S. 9-15

Wie der eigentliche Einsatz einer KI-Lösung in den Prozess integriert werden kann, wird in Kapitel 6.2 sowie 6.3 erörtert und in Kapitel 9.1.2. sowie 9.1.4 evaluiert. Dabei wird jedoch nicht darauf eingegangen, wie das Ergebnis aus dem Einsatz der Lösung, aus einer fachlichen und technischen Sicht, weiterverwendet werden kann. Denn vor allem, bzw. erst durch die Weiterverwendung des Ergebnisses kann ein tatsächlicher, respektive ein optimaler Mehrwert erzielt werden.

Die Fragestellung hierzu wird im Zuge von Kapitel 6.4.2 in der Theorie untersucht. Hierbei werden zwei Zwecke identifiziert, für die das Ergebnis aus dem Einsatz einer KI eingesetzt werden kann: Zur Prozesssteuerung und zur Verarbeitung in Folgeschritten. Der erste Einsatzzweck wird durch den ersten Prototyp praktisch implementiert – der zweite durch den Zweiten.

Nutzen der Engine

Insgesamt kann daher auf Basis der Prototypen belegt werden, dass durch den Einsatz der BPMN und einer geeigneten Process Engine, das Ergebnis einer KI sowohl zur Prozesssteuerung als auch in Folgeaktivitäten verwendet werden kann. Folglich kann der jeweilige optimale Mehrwert, aus dem Einsatz einer KI-Lösung, durch eine Engine ausgeschöpft werden. Dabei zeigt sich vor allem beim Zweck der Prozesssteuerung, dass dieser einfach abzubilden ist. Es ist lediglich notwendig, dass das Ergebnis aus dem Einsatz einer KI in einer konkreten Prozessvariable gespeichert wird. Folgend darauf kann in den ausgehenden Kontrollflüssen eines exklusiven Gateways jeweils auf diese referenziert werden. Für die Verwendung des Resultats der KI in den Folgeaktivitäten eines Prozesses kann hingegen keine klare Aussage zum Implementierungsaufwand gegeben werden. Der Grund dafür ist, dass die zugehörige Aufgabe jeden Typ annehmen kann, welchen die BPMN zur Verfügung stellt. Ergo kann er z.B. sowohl ein Service- als auch ein Benutzer-Task sein.

Potential zur Automatisierung weiterer Schritte

Weiterhin zeigt sich anhand der Prototypen, dass es sich beim Einsatz einer Engine anbietet, auch andere Schritte in einem Prozess zu automatisieren, die keinen direkten Kontext zum Einsatz der KI haben. So werden z.B. im zweiten Prototyp das Erstellen und Versenden des Kreditangebots, aber auch das Versenden einer Absage symbolisch automatisiert.

Dabei erweist sich, wie in den Kapiteln 8.1 und 8.2 bewertet wird, dass auch die Automatisierung dieser Schritte einen konkreten Mehrwert erzeugt. Ein solcher kann, analog zum Nutzen aus dem Einsatz einer KI, sehr divers sein. Gängige Beispiele sind jedoch eine verkürzte Durchlaufzeit, mehr freie Ressourcen, oder weniger Kosten zur Durchführung der Schritte.

Alles in allem zeigt sich also, dass durch den Einsatz einer Process Engine, und die damit mögliche Automatisierung weiterer Schritte, ein wesentlich größerer Nutzen erzielt werden kann, als wenn die KI-Lösung isoliert eingesetzt wird.

9.2.3: Qualität der Ergebnisse

Als dritte Herausforderung wird angeführt, dass die Vorhersagen einer KI-Lösung stets mit einer gewissen Unsicherheit behaftet sind. Dies bedeutet, sie können im Einzelfall fehlerhaft sein. Führt ein Unternehmen auf Basis einer solchen falschen Vorhersage eine spezifische Handlung aus, so kann dies zu negativen Konsequenzen führen.

Damit diese vermieden werden können, wird im Zuge von Kapitel 6.4.3 erörtert, dass ein Prozess, in dem eine KI-Lösung eingesetzt wird, um einen Benutzer Task erweitert werden kann. In diesem wiederum kann durch einen Mitarbeiter das Ergebnis der KI überprüft werden. Des Weiteren wird angeführt, dass dieser Benutzer Task, je nach Notwendigkeit der Prüfung, in diversen Varianten

abgebildet werden kann: Zu eben diesen zählen eine allgemeine-, eine ergebnisabhängige-, eine fallabhängige- und eine zufällige Prüfung (siehe Kapitel 6.4.3).

Aufwand versus Nutzen der Prüfung

Das diese Varianten durch die BPMN abgebildet und durch eine Process Engine gesteuert werden können, wird in Kapitel 6.4.3 erwiesen. Weiterhin zeigt sich im Zuge der Prototypen, dass sie auch in der Praxis geeignet sind, um das Resultat einer KI zu prüfen und bei Bedarf zu korrigieren.

Zudem kann geschlussfolgert werden, dass der Aufwand zur Implementierung der Varianten für gewöhnlich recht gering ist. Sollen alle Prozessinstanzen geprüft werden, so müssen lediglich das Ergebnis der KI und die Daten, auf Basis welcher dieses getroffen wurde, bereitgestellt werden. Da diese in aller Regel in einer Datenbank oder in spezifischen Prozessvariablen gespeichert sind, kann sehr einfach darauf zugegriffen werden. Das Ergebnis der Prüfung wiederum muss nur in einer weiteren Prozessvariable gespeichert werden, und kann dann durch die Engine verarbeitet werden. Sollen dagegen nur bestimmte oder zufällige Instanzen geprüft werden, muss zusätzlich zur eigentlichen Prüfung ein Gateway abgebildet werden. Durch dieses kann die Relevanz der Prüfung festgestellt werden. Diese Relevanz kann dabei, vor dem Gateway, durch einen Service Task, oder eine DMN-Entscheidungstabelle bestimmt werden.

Alles in allem muss jedoch beachtet werden, dass der ausschlaggebende Aufwand nicht durch die Implementierung, sondern durch die Ausführung der Prüfung verursacht wird. Im Zuge von dieser werden Mitarbeiterressourcen benötigt. Die Menge dieser ist abhängig von der Häufigkeit, in welcher der jeweilige Prozess ausgeführt wird – aber auch von der jeweils gewählten Variante. So erweist sich vor allem die allgemeine Prüfung als sehr aufwendig, da sie für jede Instanz eines Prozesses durchgeführt wird. Insgesamt wird allerdings nicht nur ein Aufwand verursacht – denn durch den Aufwand entstehen logischerweise immer auch Kosten.

Folglich sollte eine Prüfung nur dann durchgeführt werden, wenn sie begründet, bzw. sinnvoll ist. Dies kann etwa der Fall sein, wenn eine KI-Lösung neu eingeführt wird und noch keine Erfahrung bezüglich der Qualität der Ergebnisse vorliegt. Vorwiegend stehen bei dieser Fragestellung aber monetäre Aspekte im Fokus. Um im Hinblick auf diese zu bewerten, ob ein Prüfschritt sinnvoll ist, muss berechnet werden, ob die Kosten zur nachträglichen Korrektur von einzelnen Fehlern billiger sind als die Gesamtkosten der Prüfung. Ist dies der Fall, sollte sie eher vermieden werden.

Nutzen der Engine

Ist die Prüfung jedoch sinnvoll, erweist sich der Einsatz der BPMN, respektive einer Process Engine als sehr nutzbringend. Der Grund dafür ist vor allem die hohe Flexibilität der Notation, sowie der zugehörigen Engines. So kann der Task zur Prüfung der Ergebnisse einer KI, durch eine Anpassung des Prozessmodells, jederzeit ein- oder ausgebaut werden. Dies kann insbesondere dann nützlich sein, wenn eine KI-Lösung, wie oben angesprochen, neu eingeführt wird. In diesem Fall kann die Prüfung mit minimalem Aufwand aus dem Prozess eliminiert werden, sobald bewiesen ist, dass die Qualität der Lösung zufriedenstellend ist.

Doch nicht nur auf der Ebene des gesamten Prozesses ist die Flexibilität von Vorteil. Auch auf der Ebene der einzelnen Schritte erzeugt sie einen Mehrwert. So können bspw. bei der fallabhängigen Prüfung, die Kriterien zur Bestimmung der Relevanz einer Prüfung jederzeit abgeändert werden. Dazu muss lediglich das DMN-Modell, bzw. die entsprechende Entscheidungstabelle angepasst werden. Dasselbe gilt für die zufällige Prüfung. Bei dieser Variante werden, anhand des Wertes einer Zufallszahl, der an einem exklusiven Gateway gegen einen bestimmten Schwellwert geprüft wird, stichprobenartige Prüfungen durchgeführt. Demzufolge kann die Häufigkeit der Prüfungen flexibel angepasst werden. Dazu müssen an den ausgehenden Kontrollflüssen des Gateways, die

Bedingungen zum Durchlaufen des Flusses an den neuen Schwellwert angepasst werden. Je höher dieser ist, desto häufiger wird die Prüfung ausgeführt.

9.2.4: Erklärbarkeit

Die letzte Herausforderung, die erörtert wird, befasst sich mit der Problematik, dass es für einen Menschen oftmals nicht nachvollziehbar ist, wie eine KI-Lösung eine konkrete Entscheidung trifft. Dabei wird angeführt, dass die Herausforderung ausschließlich gelöst werden kann, indem ML-Modelle, bzw. konkrete ML-Techniken eingesetzt werden, welche eine Auskunft über die Gründe für ihre jeweiligen Entscheidungen geben – und daher als erklärbare Modelle bezeichnet werden können. Beispiele für solche werden in den Kapiteln 6.4.4.1–6.4.4.3 gegeben.

Demzufolge kann die Hypothese aufgestellt werden, dass die BPMN und eine Process Engine nur bedingt dabei helfen können, die Herausforderung zu lösen. Insgesamt zeigt sich jedoch, dass sie in mehreren Punkten zumindest einen Teil zur Lösung beisteuern können. So wird z.B. aus einer fachlichen Sicht erklärbar, wie mit dem Ergebnis einer KI umgegangen wird. Aber auch aus einer technischen Perspektive erzeugen die Lösungen einen Nutzen. So können sie zum einen eingesetzt werden, um die erklärbaren KI-Lösungen zu steuern – und zum anderen, um die Gründe für deren Entscheidungen zu speichern. Weiterhin zeigt sich, dass der Einsatz der BPMN und einer Process Engine, je nach Ausprägung der erklärbaren KI, jeweils einen unterschiedlichen Nutzen, bzw. Vor- und Nachteile aufweist. Daher wird in der Folge jeweils einzeln auf diese eingegangen:

Erklärbarkeit durch transparente Modelle

Die Lösung der Herausforderung durch die Nutzung transparenter Modelle wird in Kapitel 6.4.4.1 thematisiert. Dabei zeigt sich, dass der Einsatz einer solchen Lösung, aus einer technischen Sicht, nahezu identisch zum Einsatz einer ordinären KI ist. Es kann ebenfalls ein Service Task eingesetzt werden, mittels dem die Lösung angesprochen wird. Der einzige Unterschied ist, dass zusätzlich zur Nutzung der Lösung, auch der Grund für das Ergebnis identifiziert und gespeichert wird. Da dies jedoch lediglich eine kleine Erweiterung darstellt, ist auch diese Variante nur wenig komplex.

Alles in allem muss jedoch berücksichtigt werden, dass nur wenige ML-Modelle für diesen Zweck geeignet sind. Viele bestehende KI-Lösungen nutzen solche Modelle jedoch noch nicht. Faktisch kann es notwendig sein, dass die Modelle ausgetauscht werden müssen. Nutzt ein Unternehmen allerdings eine KI, welche von einem externen Anbieter stammt, so hat es für gewöhnlich keinen Einfluss auf das Modell. Deshalb kann geschlussfolgert werden, dass der Einsatz der BPMN sowie einer Engine, in dieser Variante meist nur dann möglich ist, wenn die KI-Lösung eigenentwickelt ist oder wird. Denn in diesem Fall kann der Modell-Typ frei gewählt werden.

Erklärbarkeit durch Ensemble Learning

Das Erklären einer KI-Vorhersage durch den Einsatz der Ensemble-Technik wird dagegen im Zuge von Kapitel 6.4.4.2 diskutiert. Die Technik bezweckt die Lösung der Herausforderung durch den Einsatz mehrerer ML-Modelle, wobei mindestens eines von diesen erklärbar ist. Auch bei diesem Lösungsansatz stellt sich heraus, dass die Einbindung dieser Modelle ähnlich zur Einbindung einer ordinären KI ist. Denn auch bei diesem Ansatz können z.B. Service Tasks verwendet werden. Der wesentliche Unterschied ist jedoch, dass mehrere Modelle in den Prozess integriert und parallel für den gleichen Zweck ausgeführt werden. Außerdem muss eine Lösung dafür gefunden werden, durch welche das Problem behandelt wird, dass es vorkommen kann, dass die unterschiedlichen Modelle voneinander abweichende Ergebnisse produzieren. Eine solche könnte, wie es in Kapitel 6.4.4.2 erörtert wird, die Erweiterung des Prozesses durch einen Benutzer Task sein.

Diese Aspekte beachtend kann geschlussfolgert werden, dass der Aufwand und die Komplexität zur Implementierung dieses Lösungsansatzes höher sind als jener des Vorherigen. Zudem werden jedoch auch im Produktivbetrieb Mitarbeiterressourcen benötigt, um den eben angesprochenen Benutzer Task auszuführen. Zu guter Letzt zeigt sich, dass auch diese Variante in aller Regel nur geeignet ist, wenn eine KI eigenentwickelt ist. Der Grund hierfür ist, dass zur Anwendung dieser, mehrere ML-Modelle unterschiedlicher Art benötigt werden, welche für denselben Zweck trainiert sind. Eine externe Lösung stellt allerdings meistens nur ein Modell für einen Zweck bereit.

Erklärbarkeit durch Umwandlung in andere Form

Die Lösung der Problemstellung durch die Transformation eines ML-Modells in einer andere Form wird dagegen in Kapitel 6.4.4.3 erörtert. Dabei wird ermittelt, wie ein Entscheidungsbaum, in der Theorie, in die Form einer DMN-Entscheidungstabelle umgewandelt werden kann. Dies wiederum wird im Rahmen des zweiten Prototyps exemplarisch getestet und in Kapitel 8.2 evaluiert.

Insgesamt zeigt sich dabei, dass die Umwandlung des Baums in eine DMN-Entscheidungstabelle durchführbar ist. Dadurch kann bewiesen werden, dass es möglich ist, die Ergebnisse einer KI zu erklären. Der Grund dafür ist, dass sie in einer transparenten Form abgebildet wird, in der jede Regel durch eine einzelne Zeile repräsentiert wird. Ferner wird durch eine Engine implizit geloggt, welche Regel bei einer jeweiligen Entscheidung zutrifft. Ergänzend dazu erweist sich, auf Basis von Anhang 11, dass das Modell, durch die Umwandlung keine Genauigkeit verliert.

Diese Erkenntnisse können jedoch nicht eins zu eins auf andere ML-Modelle übertragen werden. Denn wie in Kapitel 8.2.2 erörtert, repräsentiert ein Baum sein Wissen bereits in Form von Regeln. Faktisch werden diese lediglich in eine andere Form gebracht. Somit kann zwar geschlussfolgert werden, dass die Herausforderung in der Theorie gelöst werden kann, sofern ein Modell in diese Form umgewandelt werden kann. Jedoch wird der damit verbundene Gewinn an Transparenz, in der Praxis – je nach ML-Modell – gegebenenfalls mit einem Verlust an Genauigkeit einhergehen.

Ferner zeigt sich, dass eine solche Umwandlung hochkomplex ist, und die Programmierung einer Software, mittels welcher diese umgesetzt werden kann, daher einen hohen Aufwand verursacht. Zudem stellt sich heraus, dass auch dieser Lösungsansatz nur bei einer Eigenentwicklung der KI-Lösungen geeignet ist. Der Grund dafür ist, dass zur Umwandlung des ML-Modells ein expliziter Zugriff auf das Wissen von diesem benötigt wird.

9.3: Gesamtfazit zur Kombination

Auf Basis der allgemeinen und der herausforderungsbezogenen Erkenntnisse kann abschließend ein Gesamtfazit zum Nutzen gezogen werden, der erzeugt werden kann, wenn der Einsatz einer KI mittels der BPMN und einer Process Engine abgebildet wird. Dieser wird in der Folge im Detail betrachtet – und beantwortet die initiale Fragestellung dieser Thesis.

9.3.1: Nutzen der BPMN und einer Process Engine

Im Hinblick auf diese Fragestellung kann konkludiert werden, dass der Einsatz einer KI in einem Prozess, und die Kombination dieser KI mit der BPMN und einer Process Engine einige Potentiale heben kann. Die relevantesten Gründe dafür sind insbesondere die Folgenden:

Generische Art, Offenheit und Flexibilität der BPMN

Ein erster wichtiger Grund ist die generische Art, die Offenheit und die Flexibilität des Standards. Durch diese Aspekte ist es auf einer fachlichen Ebene möglich, dass der Einsatz einer KI-Lösung, durch diverse Typen von BPMN-Tasks abgebildet werden kann (siehe Kapitel 9.1.2).

Ebenso wichtig sind jedoch auch die Folgen dieser Aspekte auf technischer Ebene. Denn aufgrund dieser steht es den Anbietern der Engines frei, nahezu beliebige Möglichkeiten anzubieten, durch welche die Ausführungslogik der Prozessschritte abgebildet werden kann. Die mächtigste dieser ist jene, dass ein Anwender eine individuelle Logik programmieren kann, welche sodann von der Engine exekutiert wird. Diese Möglichkeit wird in der Regel von jeder Engine unterstützt. Sie sorgt für eine extrem hohe Flexibilität und ermöglicht es beliebige KI-Lösungen, oder alleinstehende ML-Modelle in einen Prozess zu integrieren, respektive einzusetzen. Eine weitere Möglichkeit, die ebenfalls von nahezu jeder Engine unterstützt wird, ist die Nutzung der DMN. Diese ist relevant, da sie es ebenfalls ermöglicht, den Einsatz einer KI zu automatisieren. Die Vor- sowie Nachteile dieser Möglichkeit werden daher im nächsten Kapitel thematisiert.

Bevor jedoch auf diese eingegangen wird, soll erwähnt werden, dass die oben genannten Aspekte nicht die einzigen Vorteile beim Einsatz einer KI, in Kombination mit den beiden Lösungen sind.

Fokus auf kompletten Prozess

So ist es ein weiterer wesentlicher Vorteil, dass nicht nur einzelne Prozessschritte, sondern ganze Prozesse im Fokus der BPMN stehen. So können durch den Einsatz der Notation, ganze Prozesse in BPMN-basierten Prozessmodellen abgebildet und mit dem Ziel der Automatisierung technisch konfiguriert werden. Die Aufgabe der Process Engine wiederum ist es, die Prozesse zu steuern. Dabei orientiert sie sich am jeweiligen Prozessmodell. Somit wird sichergestellt, dass die Schritte in einer validen Reihenfolge ausgeführt werden.

Insgesamt ist es also möglich, dass die Lösungen verwendet werden, um den Einsatz einer KI zu automatisieren – aber auch, um deren Ergebnis weiterzuverarbeiten. Dies zeigt, dass, durch den Einsatz der BPMN und einer Engine, eine KI vollständig in einen Prozess integriert werden kann.

Außerdem stellt sich heraus, dass es sich anbietet, in einem Prozess, in dem der Einsatz einer KI-Lösung automatisiert wird, stets auch weitere Aktivitäten zu automatisieren. Dadurch kann der Automatisierungsgrad von einem Prozess noch weiter erhöht werden. Des Weiteren kann je nach Charakteristik des Schrittes, sowie dem Ziel der Automatisierung, ein zusätzlicher Nutzen erzeugt werden. Daher kann geschlussfolgert werden, dass durch den Einsatz einer Process Engine, nicht nur der Mehrwert aus dem Einsatz der KI ausgeschöpft werden kann, sondern auch ein Mehrwert aus der Automatisierung der anderen Schritte.

Lösung für Herausforderungen beim Einsatz einer KI in Prozessen

Abschließend zeigt sich aber auch, dass die BPMN und der Einsatz von zugehörigen Engines dabei helfen können, die KI-spezifischen Herausforderungen zu lösen, welche im Rahmen dieser Thesis diskutiert werden. So stellt die BPMN z.B. ein Werkzeug dar, dass dabei helfen kann, graphische und verständliche Prozessmodelle zu entwerfen. Diese wiederum können in anderen Disziplinen, wie dem Change-Management eingesetzt werden, um den Einsatz einer KI-Lösung transparent zu kommunizieren – mit dem Ziel, eine Akzeptanz für diese zu erreichen. Ferner stellt sich heraus, dass die beiden Lösungen optimal dafür geeignet sind, um eine KI vollständig in einen Prozess zu integrieren – sowohl auf einer fachlichen als auch auf einer technischen Ebene. Darüber hinaus ermöglichen sie, durch die Abbildung von konkreten Benutzer Tasks, eine Prüfung der Ergebnisse einer KI. Zu guter Letzt ermöglichen sie es aber auch, erklärbare KI-Lösungen zu steuern. Zudem sind sie in der Lage, die Gründe für deren Entscheidungen zu speichern.

Gesamtfazit zum Nutzen

Im Hinblick auf die initiale Fragestellung dieser Thesis lässt sich daher schließen, dass der Einsatz einer Künstlichen Intelligenz, zum Zweck der Prozessautomatisierung und die Kombination dieser mit der BPMN und einer Process Engine, einen beachtlichen Mehrwert erzeugen kann. Dabei muss jedoch beachtet werden, dass auch dieser Nutzen – analog zum allgemeinen Nutzen, der mittels dem Einsatz einer KI-Lösung in einem Prozess erzeugt werden kann – nicht pauschalisiert werden kann. Der Nutzen kann je nach Szenario unterschiedlich sein.

9.3.2: Nutzen der DMN und einer DMN-Engine

Alles in allem sollten die BPMN sowie die Process Engines jedoch nicht isoliert betrachtet werden. Aufgrund der engen Verzahnung der beiden Notationen sollten sie stets auch im Kontext zur DMN und den DMN-Engines betrachtet werden. Denn wie im Rahmen dieser Thesis beleuchtet wurde, kann eine KI-Lösung in ein DMN-Modell eingebunden und durch eine DMN-Engine automatisch ausgeführt werden. Die Voraussetzung dazu ist, dass sie in einer speziellen Form abgebildet wird, welche von der jeweiligen DMN-Engine technisch interpretiert werden kann. Eine solche Form ist, im Kontext zur KI, standardmäßig ein sogenanntes PMML-Modell.

Transparenz & dediziertes Management

Insgesamt stellt sich heraus, dass durch die Einbindung einer KI, auf Basis dieser Variante, eine gewisse Transparenz über den Vorgang einer Entscheidung gewonnen werden kann. Denn mittels der DMN werden diese in Entscheidungsmodellen graphisch abgebildet. Darüber hinaus kann die Entscheidungslogik explizit von der Logik eines Prozesses getrennt werden. Dies wiederum macht beide Logiken einfacher zu warten und ermöglicht ein dediziertes Management.

Abwägung zum Einsatz der DMN

Nichtsdestotrotz muss es immer gut abgewogen werden, ob eine KI in Kombination mit der DMN eingesetzt werden soll. Denn der Vorgang zur Integration dieser Lösung in ein DMN-Modell, bzw. in die Ausführungslogik des Prozesses kann sehr komplex und mit viel Aufwand verbunden sein.

So ist in aller Regel eine Umwandlung des ML-Modells in die Form von z.B. einem PMML-Modell oder einer Entscheidungstabelle notwendig. Zudem muss gegebenenfalls eine zusätzliche DMN-Engine in den Prozess integriert werden. Denn grundsätzlich besitzt zwar jede BPMN-Engine eine DMN-Engine – diese integrierten Engines unterstützen allerdings meist keine PMML-Modelle.

Soll im Zuge eines Prozessschrittes ausschließlich eine KI-Lösung eingesetzt werden, und ist dazu die Integration einer dedizierten DMN-Engine notwendig, wird daher empfohlen, die Lösung über andere Optionen in den Prozess einzubinden. Eine dieser ist der Einsatz von sog. PMML-Engines. Diese sind in der Lage, ein PMML-Modell nativ auszuwerten.²⁰

Der Einsatz der DMN hingegen ist erst dann sinnvoll, wenn eine Entscheidung getroffen wird, die sich aus mehreren Unterentscheidungen zusammensetzt. In diesem Fall hat die DMN den Nutzen, dass die einzelnen Entscheidungen, im Rahmen von einem einzigen BPMN-Task, von einer DMN-Engine ausgeführt und zu einer Gesamtentscheidung vereinigt werden. Würde das Szenario ohne die Notation abgebildet werden, müssten die einzelnen Entscheidungen, durch einzelne Tasks im Prozessmodell getroffen und abschließend zusammengeführt werden.

²⁰ Eine exemplarische PMML-Engine, welche in Java-basierte Applikationen eingebunden werden kann, ist die Folgende: <https://github.com/jpmml/jpmml-evaluator>

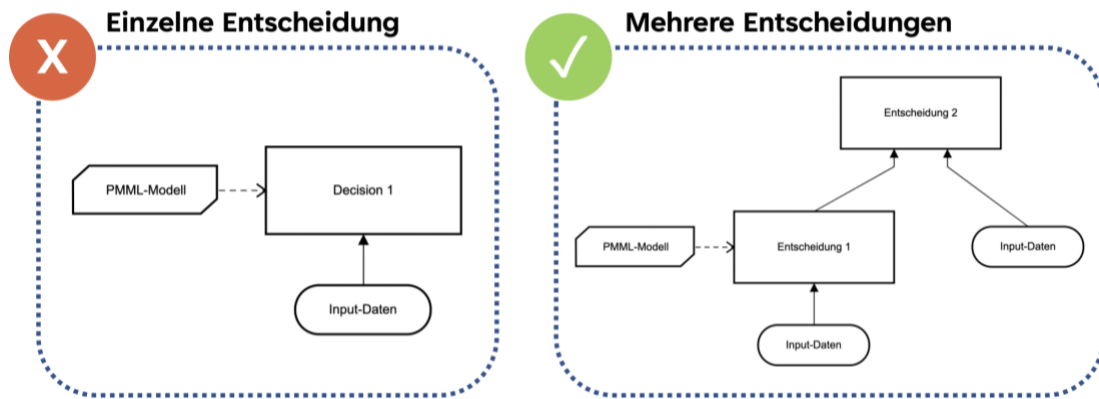


Abbildung 63: Sinnhaftigkeit der Einbindung von PMML-Modellen in DMN-Modelle

9.3.3: Eignung eines Prozesses

Im Resümee kann also gefolgert werden, dass der Einsatz von KI-Lösungen, in Kombination mit der BPMN sowie einer Process Engine einen hohen Mehrwert erzeugen kann. Darüber hinaus kann auch die Ergänzung durch die DMN und DMN-Engines in vielen Szenarien hilfreich sein.

Ungeachtet dessen muss allerdings berücksichtigt werden, dass ein solches Einsatzszenario stets einen veritablen Aufwand verursacht. So muss gegebenenfalls eine eigene KI-Lösung, respektive ein ML-Modell entworfen werden. Außerdem muss ein Prozessmodell erstellt und die zugehörige Ausführungslogik der Schritte implementiert werden. Ferner fällt ein kontinuierlicher Aufwand zur Wartung der Prozesslogik und der KI an. Weiterhin können auch zur Durchführung des Prozesses einige Mitarbeiterressourcen benötigt werden – sofern in den Prozess bspw. ein Schritt integriert wird, in welchem das Ergebnis aus dem Einsatz einer KI überprüft wird.

Alles in allem kann also geschlussfolgert werden, dass durch ein solches Vorhaben, allein wegen dem Aufwand, hohe Kosten verursacht werden. Zusätzlich dazu fallen in aller Regel jedoch auch noch Kosten für die Process Engine und die KI an – sofern diese von externen Anbietern bezogen werden. Folglich kann abgeleitet werden, dass der Einsatz einer KI, und die Kombination dieser mit der BPMN, aus einem Kosten-Nutzen-Verhältnis, nicht für alle Prozesse in einer Organisation geeignet ist. Demzufolge sollte die Automatisierung eines Prozesses, durch den Einsatz einer KI, der BPMN und einer Engine, nur für Szenarien betrachtet werden, in denen entweder ein positiver Return on Investment oder andere Wettbewerbsvorteile erzielt werden können.

10: Fazit

Die Thesis baut auf der Problemstellung auf, dass der Einsatz von KI-Lösungen in Prozessen einen hohen Mehrwert erzeugen kann. Damit dieser ausgeschöpft werden kann müssen jedoch, je nach Einsatzszenario, bestimmte Herausforderungen gelöst werden, die beim Einsatz dieser Lösungen in Prozessen auftreten können. Außerdem müssen die KI-Lösungen in die Systemlandschaft eines Prozesses eingebettet werden. Diese sind schon heute oftmals sehr komplex. Demzufolge müssen Lösungen gefunden werden, um diesen Herausforderungen zu begegnen. Als eine von diesen wird die BPMN vorgeschlagen, die im Rahmen dieser Thesis im Fokus steht.

10.1: Abschlussbetrachtung

Alles in allem sollte daher durch die Thesis untersucht werden, wie der Einsatz einer KI, durch die BPMN, aus fachlicher Sicht in einem Prozessmodell abgebildet – und aus technischer Sicht, durch eine Process Engine automatisiert werden kann. Abschließend sollte dadurch die Fragestellung beantwortet werden können, welchen Nutzen die BPMN und eine Process Engine, im Kontext zu einer KI-Lösung erzeugen können, die zum Zweck der Prozessautomatisierung eingesetzt wird.

Daher wurden zuallererst die Grundlagen zu Prozessen und der Künstlichen Intelligenz vermittelt. Auf Basis von diesen wurde diskutiert, für welche Zwecke eine KI in Prozessen eingesetzt werden kann. Dabei zeigt sich, dass die möglichen Szenarien nahezu endlos und sehr divers sind. Zudem wurden Herausforderungen identifiziert, welche auftreten können, wenn eine KI in einem Prozess eingesetzt wird. Zu diesen zählen eine fehlende Akzeptanz der Lösungen, die Notwendigkeit der Integration der Lösungen in den Prozess, die Vorhersagegenauigkeit der Modelle und der Aspekt, dass die Gründe für die Vorhersagen der Lösungen für Menschen oft nicht nachvollziehbar sind. Die Grundlagen abschließend, wurde die Theorie zur BPMN und DMN behandelt.

Folgend darauf wurden jegliche dieser theoretischen Grundlagen kombiniert, um die eigentliche Problemstellung der Thesis zu lösen. Dabei wurde erörtert, dass der Einsatz einer KI-Lösung als eine Aktivität verstanden – und somit mittels der BPMN abgebildet werden kann. Sie kann dabei entweder durch einen Service-, einen Nachrichten-Sendenden-, oder einen Geschäftsregel Task repräsentiert werden. Ferner wird erwähnt, dass für den Zweck aber auch ein neuer Aufgabentyp erstellt werden kann. Dieser könnte als KI-Task bezeichnet werden. Zudem wurden Möglichkeiten erörtert, wie die jeweiligen Tasks in einer Process Engine automatisiert werden können. Darüber hinaus wurde diskutiert, wie die BPMN und eine Engine bei der Bewältigung der oben genannten Herausforderungen helfen können – bzw. konkrete Lösungsmöglichkeiten hierzu erarbeitet.

Die Theorie wurde sodann durch die Erstellung von Prototypen praktisch getestet. Die technische Implementierung dieser Prototypen wird im Anhang der Arbeit dokumentiert.

Insgesamt kommt die Arbeit zum Schluss, dass die Kombination der BPMN und der KI zum Zweck der Prozessautomatisierung sehr sinnvoll ist. So wird anhand der Prototypen exemplarisch klar, dass durch den Einsatz einer KI ein großer Nutzen erzeugt werden kann. Dieser kann zwar nicht pauschalisiert werden – im Beispiel zeichnet er sich jedoch aus durch: Eine kürzerer Durchlaufzeit, mehr freie Mitarbeiterressourcen und geringere Kosten zur Durchführung der Prozesse.

Darüber hinaus zeigt sich, dass der Einsatz einer KI, aufgrund der generischen Art der BPMN, auf diverse Weisen fachlich abgebildet werden kann. Zudem stellt sich heraus, dass er aufgrund der Flexibilität einer Process Engine auf unterschiedliche Weisen technisch in einen Prozess integriert werden kann. Ein weiterer Vorteil der BPMN ist deren Fokus auf einen Prozess als Ganzes. Durch

diesen ist es möglich, dass das Ergebnis einer KI für konkrete Zwecke weiterverarbeitet werden kann – und auch andere Schritte im Prozess automatisiert werden können. Weiterhin stellt sich heraus, dass die BPMN und eine Engine dabei helfen können die Herausforderungen zu lösen, die beim Einsatz einer KI in Prozessen auftreten können. Im Resümee führt dies dazu, dass durch den Einsatz der beiden Lösungen, ein deutlich höherer Nutzen erzielt werden kann als beim isolierten Einsatz einer Künstlichen Intelligenz.

Daneben zeigt sich jedoch auch, dass die Automatisierung eines Prozesses durch die BPMN, und die Einbindung einer KI in die Ausführungslogik von diesem, einen hohen Aufwand erzeugt. Daher sollte sie nicht für jeden Prozess durchgeführt werden – sondern nur für Prozesse, bei denen ein positiver Return on Investment oder vergleichbare Wettbewerbsvorteile erzielt werden können.

10.2: Ausblick

Mit Blick in die Zukunft ist – wie bereits in der Einleitung erwähnt wurde – zu erwarten, dass der Einsatz von Künstlicher Intelligenz in Prozessen weiter zunehmen wird. Ebenso ist absehbar, dass auch die Prozesse selbst stetig komplexer werden. Denn aufgrund der Digitalisierung sind immer mehr Akteure, Systeme und Technologien an ihrer Durchführung beteiligt. Insgesamt werden also die Herausforderungen, die in der Problemstellung der Thesis beschrieben werden, nicht kleiner, sondern eher größer. Faktisch steigt auch der Bedarf nach einer Lösung, um diesen zu begegnen.

Im Zuge dieser Thesis haben sich die BPMN und Process Engines, welche BPMN-basierte Prozesse steuern können, als eine solche Lösung bewiesen. Daher kann erwartet werden, dass die beiden Lösungen in der Zukunft häufiger eingesetzt werden, um den Einsatz einer KI-Lösung abzubilden und zu automatisieren.

Literaturverzeichnis

- Abdelkafi, Nizar; Döbel, Inga; Meironke, Anja; Niekler, Andreas; Ries, Sonja (2019): Künstliche Intelligenz im Unternehmenskontext. Literaturanalyse und Thesenpapier. Hg. v. Fraunhofer-Zentrum für Internationales Management und Wissensökonomie IMW und Universität Leipzig.
- Allweyer, Thomas (2015): BPMN 2.0 - Business Process Model and Notation. Einführung in den Standard für die Geschäftsprozessmodellierung. 3., aktualisierte und erweiterte Aufl. Norderstedt: Books on Demand.
- Amberg, Michael; Bodendorf, Freimut; Möslin, Kathrin M. (2011): Wertschöpfungsorientierte Wirtschaftsinformatik. Berlin, Heidelberg: Springer Berlin Heidelberg (Springer-Lehrbuch).
- Aunkofer, Benjamin (2017): Ensemble Learning. Hg. v. Data Science Blog, zuletzt aktualisiert am 03.12.2017, zuletzt geprüft am 15.05.2022.
- Baars, Henning; Kemper, Hans-Georg (2021): Business Intelligence & Analytics – Grundlagen und praktische Anwendungen. Wiesbaden: Springer Fachmedien Wiesbaden.
- Becker, J.; Uthmann, C. V.; zur Muhlen, M.; Rosemann, M. (1999): Identifying the workflow potential of business processes. In: Proceedings of the 32nd Annual Hawaii International Conference on Systems Sciences, 5-8 Jan. 1999: IEEE Comput. Soc, S. 10.
- Becker, Jörg; Kahn, Dieter (2012): Der Prozess im Fokus. In: Jörg Becker, Martin Kugeler und Michael Rosemann (Hg.): Prozessmanagement. Berlin, Heidelberg: Springer Berlin Heidelberg, S. 3–16.
- Becker, Jörg; Schütte, Reinhard (2004): Handelsinformationssysteme. Domänenorientierte Einführung in die Wirtschaftsinformatik. 2. Aufl. Frankfurt am Main: Moderne Industrie (Redline-Wirtschaft).
- Bitkom (2017): Künstliche Intelligenz. Wirtschaftliche Bedeutung, gesellschaftliche Herausforderungen, menschliche Verantwortung. Unter Mitarbeit von Florian Buschbacher, Matthias Weber und Sabrina Flemming. Hg. v. Bitkom e.V. und DFKI.
- Buxmann, Peter; Schmidt, Holger (2019): Grundlagen der Künstlichen Intelligenz und des Maschinellen Lernens. In: Peter Buxmann und Holger Schmidt (Hg.): Künstliche Intelligenz. Berlin, Heidelberg: Springer Berlin Heidelberg, S. 3–19.
- Calvanese, Diego; Dumas, M; Laurson, Ülari; Maggi, F.; Montali, M.; Teinemaa, I. (2016): Semantics and Analysis of DMN Decision Tables.
- Chies, Sieglind (2016): Change Management bei der Einführung neuer IT-Technologien. Wiesbaden: Springer Fachmedien Wiesbaden.
- Debevoise, Tom; Taylor, James (2016): Prozess- und Entscheidungsmodellierung in BPMN/DMN. Eine Kurzanleitung: effektivere Prozesse durch Integration der Entscheidungsmodellierung in die Prozessmodellierung. Erste Auflage. Berlin, Wrocław: Tom Debevoise und James Taylor.
- Decision Management Solutions (2016): An Introduction to Decision Modeling with DMN. Hg. v. Decision Management Solutions. Online verfügbar unter

https://www.omg.org/news/whitepapers/An_Introduction_to_Decision_Modeling_with_DMN.pdf.

Döbel, Inga; Leis, Miriam; Vogelsang Manuel; Neustroev, Dmitry; Petzka, Henning; Rüping, Stefan et al. (2018): Maschinelles Lernen – Kompetenzen, Anwendungen und Forschungsbedarf. Förderkennzeichen: 01IS17019. Hg. v. Fraunhofer.

Domschke, Wolfgang; Drexel, Andreas; Klein, Robert; Scholl, Armin (2015): Einführung in Operations Research. 9. Aufl. 2015. Berlin, Heidelberg: Springer Berlin Heidelberg.

Dumas, Marlon; La Rosa, Marcello; Mendling, Jan; Reijers, Hajo A. (2018): Fundamentals of Business Process Management. Softcover re-print of the hardcover 2nd edition 2018. Berlin: Springer.

Figl, Kathrin; Mendling, Jan; Tokdemir, Gul; Vanthienen, Jan (2018): What we know and what we do not know about DMN. Enterprise Modelling and Information Systems Architectures (13), S. 1–16.

Freund, Jakob; Rücker, Bernd (2017): Praxishandbuch BPMN. Mit Einführung in CMMN und DMN. 5., aktualisierte Auflage. München: Hanser.

Gadatsch, Andreas (2020): Grundkurs Geschäftsprozess-Management. Analyse, Modellierung, Optimierung und Controlling von Prozessen. 9., aktualisierte und erweiterte Auflage. Wiesbaden: Springer Vieweg (Springer eBook Collection).

Géron, Aurélien (2018): Praxiseinstieg - Machine Learning mit Scikit-Learn und TensorFlow. Konzepte, Tools und Techniken für intelligente Systeme. Heidelberg: O'Reilly (Animals).

Gillenkirch, Robert (2018): Entscheidung. Hg. v. Gabler Wirtschaftslexikon. Online verfügbar unter <https://wirtschaftslexikon.gabler.de/definition/entscheidung-36360/version-259815>, zuletzt aktualisiert am 19.02.2018, zuletzt geprüft am 19.03.2022.

Greplova, Eliska; Fischer, Mark H. (2020): Machine Learning kompakt. Ein Einstieg für Studierende der Naturwissenschaften. Wiesbaden: Springer Spektrum (Springer eBook Collection).

Hanussek, Marc; Papp, Harald; Blohm, Matthias; Kintz, Maximilien; Grigorjan, Arthur; Brandt, David; Hennebold, Christoph (2021): Cloudbasierte KI-Plattformen. Chancen und Grenzen von Diensten für Machine Learning as a Service. Hg. v. Wilhelm Bauer, Thomas Bauernhansl, Marco Huber, Kraus Werner, Matthias Peissner und Thomas Renner. Fraunhofer-Institut für Arbeitswirtschaft und Organisation IAO; Fraunhofer-Institut für Produktionstechnik und Automatisierung IPA. Online verfügbar unter <http://publica.fraunhofer.de/dokumente/N-622149.html>.

Herrmann, Jörg (2021): Künstliche Intelligenz mit den Themenschwerpunkten maschinelles Lernen und künstlichen neuronalen Netzen, dargestellt anhand des Beispiels von Blended-Learning-Übungen. In: Christian Aichele und Jörg Herrmann (Hg.): Betriebswirtschaftliche KI-Anwendungen. Digitale Geschäftsmodelle auf Basis Künstlicher Intelligenz. 1. Aufl.: Springer Fachmedien Wiesbaden (Springer eBook Collection), S. 17–74.

Kersting, Kristian; Lampert, Christoph; Rothkopf, Constantin (2019): Wie Maschinen lernen. Wiesbaden: Springer Fachmedien Wiesbaden.

Kersting, Kristian; Tresp, Volker (2019): Maschinelles und Tiefes Lernen: Der Motor für „KI made in Germany“. In: *Digitale Welt* 3 (4), S. 32–34. DOI: 10.1007/s42354-019-0209-4.

- Kirste, Moritz; Schürholz, Markus (2019): Einleitung: Entwicklungswege zur KI. In: Volker Wittpahl (Hg.): Künstliche Intelligenz. Berlin, Heidelberg: Springer Berlin Heidelberg, S. 21–35.
- Kusber, Robert (2017): Chatbots – Conversational UX Platforms. In: Remigiusz Smolinski, Mariusz C. Bodek und Martin Siejka (Hg.): Innovationen und Innovationsmanagement in der Finanzbranche. Wiesbaden: Springer Gabler (SpringerLink Bücher), S. 231–244.
- Lauterjung, Sven (2020): Vom smarten Berater zur smarten Maschine. In: Rüdiger Buchkremer, Thomas Heupel und Oliver Koch (Hg.): Künstliche Intelligenz in Wirtschaft & Gesellschaft. Wiesbaden: Springer Fachmedien Wiesbaden (FOM-Edition), S. 249–274.
- Leymann, Frank (2018): Process Engine. Hg. v. Gabler Wirtschaftslexikon, zuletzt geprüft am 22.02.2022.
- Morik, Katharina (2018): Schlüsseltechnologie Maschinelles Lernen. In: *Digitale Welt* (4), S. 23–27.
- Müller, Roland M.; Lenz, Hans-Joachim (2013): Business Intelligence. Berlin, Heidelberg: Springer Berlin Heidelberg.
- Obermeier, Stefan (2014): Geschäftsprozesse realisieren. Ein praxisorientierter Leitfaden von der Strategie bis zur Implementierung. 2., aktualisierte Aufl. 2014. Wiesbaden: Springer Vieweg (Springer eBook Collection).
- OMG (2011): Business Process Model and Notation. Version 2.0. Online verfügbar unter <https://www.omg.org/spec/BPMN/2.0/PDF>.
- OMG (2021): Decision Model and Notation. Version 1.3. Online verfügbar unter <https://www.omg.org/spec/DMN/1.3/PDF>.
- Reder, Bernd (2021): Machine Learning 2021. Hg. v. Computerwoche und CIO. IDC Research Service.
- Russel, Stuart; Norvig, Peter (2012): Künstliche Intelligenz. Ein moderner Ansatz. Unter Mitarbeit von Frank Kirchner. 3., aktualisierte Auflage. München, Germany: Pearson (Pearson Studium - IT Ser).
- Schaaf, Nina; Wiedenroth, Saskia; Wagner, Philipp (2021): Erklärbare KI in der Praxis. Anwendungsorientierte Evaluation von xAI-Verfahren. Hg. v. Thomas Bauernhansl, Marco Huber und Werner Kraus. Fraunhofer-Institut für Produktionstechnik und Automatisierung IPA. Online verfügbar unter <http://publica.fraunhofer.de/dokumente/N-630667.html>.
- Scheer, August-Wilhelm (2020): Unternehmung 4.0. Vom disruptiven Geschäftsmodell zur Automatisierung der Geschäftsprozesse. 3., neu gestaltete Auflage. Wiesbaden: Springer Vieweg (Springer eBook Collection).
- Spindler, Martin; Kögel, Heinrich (2020): Erkennung von Versicherungsbetrug mit künstlicher Intelligenz. Faktenpapier. Hg. v. Bitkom e.V. (AI: Science over Fiction).
- Streibich, Karl-Heinz; Zeller, Michael (2019): Offene Plattformen als Erfolgsfaktoren für Künstliche Intelligenz. In: Peter Buxmann und Holger Schmidt (Hg.): Künstliche Intelligenz. Berlin, Heidelberg: Springer Berlin Heidelberg, S. 107–117.
- van der Aalst, Wil M. P. (2016): Process Mining. Data Science in Action. 2. Aufl. Berlin, Heidelberg: Springer Berlin Heidelberg.

Voigt, Kai-Ingo (2018): Automatisierung. Hg. v. Gabler Wirtschaftslexikon. Online verfügbar unter <https://wirtschaftslexikon.gabler.de/definition/automatisierung-27138/version-250801>, zuletzt aktualisiert am 19.02.2018, zuletzt geprüft am 04.03.2022.

Welsch, Andreas; Eitle, Verena; Buxmann, Peter (2018): Maschinelles Lernen. In: *HMD Praxis der Wirtschaftsinformatik* 55 (2), S. 366–382.

Weske, Mathias (2012): Business Process Management. Concepts, Languages, Architectures. 2. 2nd ed. 2012. Berlin, Heidelberg: Springer Berlin Heidelberg. Online verfügbar unter <http://nbn-resolving.org/urn:nbn:de:bsz:31-epflicht-1493063>.

Windisch, Sascha (2021): Intelligenz vererben - Mit genetischen Algorithmen künstliche neuronale Netze trainieren. Hg. v. adesso, zuletzt aktualisiert am 15.03.2021, zuletzt geprüft am 19.05.2022.

zur Mühlen, Michael; Hansmann, Holger (2012): Workflowmanagement. In: Jörg Becker, Martin Kugeler und Michael Rosemann (Hg.): Prozessmanagement. Berlin, Heidelberg: Springer Berlin Heidelberg, S. 367–400.

Anhang 1: BPMN Poster – alle Notationselemente der BPMN

BPMN 2.0 - Business Process Model and Notation

<http://bpmb.de/poster>

Aktivitäten

Aufgabe
Eine Aufgabe ist eine Arbeitseinheit. Ein zusätzliches markiert eine Aktivität als zugehörigen Teilprozess.

Transaktion
Eine Transaktion ist eine Gruppe von Aktivitäten, die logisch zusammen gehören. Ein Transaktionsprotokoll kann angegeben werden.

Ereignis-Teilprozess
Ein Ereignis-Teilprozess wird in einem anderen Teilprozess platziert. Er wird durch ein Startereignis ausgelöst und Teilprozess vom Ereignis aus bis zum Ende des Teilprozesses abgearbeitet oder parallel dazu ausgeführt werden.

Aufwurf-Aktivität
Eine Aufwurf-Aktivität repräsentiert einen Teilprozess oder eine Aufgabe, welche global definiert sind und im aktuellen Prozess wiederverwendet werden. Der Aufwurf eines Teilprozesses oder einer Aufgabe ist ein zusätzliches gekennzeichnet.

Aufgaben-Typen
Sie beschreiben den Charakter einer Aufgabe:

- Teilprozess
- Schleife
- Parallel
- Mehrfachausführung
- Sequenzielle
- Mehrfachausführung
- Ad-hoc
- Kompensation

Sequenzfluss
definiert die Abfolge der Ausführung.

Bedingter Fluss
enthält eine Bedingung, die definiert, wann er durchlaufen wird, und wenn nicht, werden Bedingungen nicht erfüllt.

Gateways

Exklusives Gateway
Bei einer Verzweigung wird der Fluss abhängig von einer Bedingung in genau einen ausstehenden Kanten geleitet. Bei einer Zusammenführung wird auf eine der eingehenden Kanten gewartet, um den ausgehenden Fluss zu aktivieren.

Ereignis-basiertes Gateway
Diesem Gateway folgen stets ein oder mehrere Ereignisse, die den Fluss steuern. Der Fluss wird zu dem Ereignis geleitet, das zuerst eintrifft.

Paralleles Gateway
Wenn der Sequenzfluss verzweigt wird, werden alle ausgehenden Kanten simultan aktiviert. Bei der Zusammenführung werden alle eingehenden Kanten gewartet, bevor der ausgehende Sequenzfluss aktiviert wird (Synchronisation).

Inklusives Gateway
Es werden je nach Bedingung eine oder mehrere ausgehende Kanten aktiviert bzw. eingehende Kanten synchronisiert.

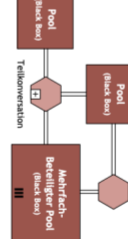
Komplexes Gateway
Verzweigungs- und Zusammenführungsgateways. Erst wenn alle nachfolgenden Ereignisse eintreffen, wird der Prozess gestartet.

Konversationen

Eine Konversation definiert einen mehrteiligen, logisch zusammenhängenden Nachrichtenaustausch. Ein zusätzliches markiert eine Teilkonversation.

Aufwurf-Konversation
Eine global definierte Konversation oder Teilkonversation. Der Aufwurf einer Konversation ist ein zusätzliches gekennzeichnet.

Konversationsdiagramm
Ein Konversationsdiagramm verknüpft Kommunikation und Teilnehmer.



Choreographien

Beteiligter A
Choreographie-Aufgabe

Beteiligter B
Choreographie-Aufgabe

Beteiligter C
Choreographie-Aufgabe



Ereignisse

	Start	Zwischen	Ende
Standard			
Ereignis-Teilprozess Unterbrechend			
Ereignis-Teilprozess Nicht-unterbrechend			
Eingetreten			
Angeheftet unterbrechend			
Angeheftet Nicht-unterbrechend			
Ausgelöst			
Standard			

Daten

Ein Datenobjekt repräsentiert Informationen, Dokumente, Emails, Briefe oder Datenströme.

Ein Datenpool repräsentiert eine Gruppe von Informationen, wie z.B. eine Liste mit Bestellungen.

Ein Dateninput ist ein externer Input für den gesamten Prozess, der von einer Aktivität gelesen wird, und die nach außen sichtbare Eingangsprozessschritte.

Ein Datenoutput ist eine Variable, die als Ergebnis eines ganzen Prozesses erzeugt wird, Prozessschritte.

Ein Datenassoziation verknüpft Datenobjekte mit Aktivitäten, Prozessen und Aufwurf-Aktivitäten. Ein Datenobjekt ist ein Ort, auf den Daten z.B. eine Datenbank oder ein Aktivitätsstrahl. Er existiert unabhängig von der Lebensdauer der Prozessinstanz.

Logos: BPM BREITEN, HfI, camunda, inubit, SIGNAVIO

Anhang 2: Nutzung der Invoice OCR API des Unternehmens verifyfi

Die **verifyfi Invoice API** ist eine cloudbasierte KI-Lösung des gleichnamigen Unternehmens **verifyfi**. Sie gehört zur Familie der Document Understanding Lösungen, und kann eingesetzt werden, um Daten aus Rechnungen zu extrahieren, welche in einem Format, wie dem PDF-Format vorliegen. In dieser Thesis wird sie in einem Prozess zur Bearbeitung von Eingangsrechnungen eingesetzt. Insgesamt gibt es dabei zwei Möglichkeiten, um mit der API zu interagieren.

Voraussetzungen zur Nutzung der API:

Um die API zu nutzen, wird ein konkreter API-Key und eine sog. Client-ID benötigt. Diese sind als Zugangsdaten zur Nutzung der API, und zur Identifikation des Nutzers zu verstehen. Sie müssen bei jeder Anfrage an die API mitgeliefert werden. Um diese initial zu erhalten, muss ein Account auf der verifyfi-Plattform angelegt werden.

Variante 1 – Nutzung der API durch direkte Ansprache der Schnittstelle

Zum einen kann die API durch den Aufruf einer konkreten Schnittstelle verwendet werden. Dieser müssen diverse Informationen übergeben werden. Zu diesen Informationen zählen vor allem die Datei, aus welcher spezifische Informationen extrahiert werden sollen, und die Zugangsdaten zur Nutzung der API.

The screenshot shows a REST client interface with a POST request to `https://api.verifyfi.com/api/v7/partner/documents/`. The request body is set to `form-data` and contains two fields: `file` (value: `Invoice.pdf`) and `file_name` (value: `invoice.pdf`). The headers section shows four headers: `CLIENT-ID` (value: `vrfiSrfioBWuRPW6MOD0m6C9wvhasIKw9pYcujm`), `AUTHORIZATION` (value: `apikey marco.schaack1:7d1da79a48a8ba920dfc02cb1cb30e7b`), `Accept` (value: `application/json`), and `Content-Type` (value: `application/x-www-form-urlencoded`).

Variante 2 – Nutzung von SDKs

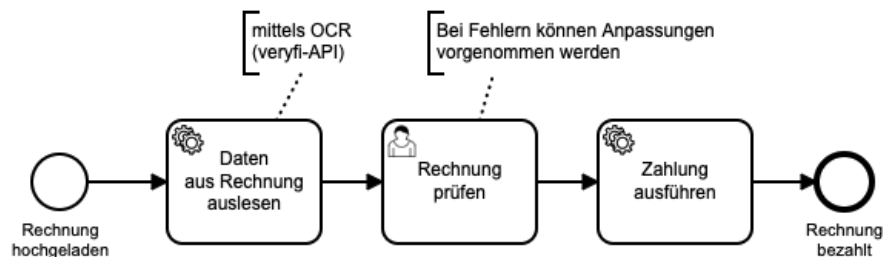
Die Abkürzung SDK steht für den Begriff „Software Development Kit“. Ein solches ist zu verstehen als eine wiederverwendbare Bibliothek, die in einer spezifischen Programmiersprache entwickelt ist, und konkrete Funktionen bereitstellt. Diese können von Entwicklern, durch die Einbindung des SDKs in ihre jeweiligen Projekte, verwendet werden. Das Ziel dadurch ist es, dass die Funktionen einer Software möglichst einfach aufgerufen und wiederverwendet werden können.

Für die Interaktion mit den Funktionen der verifyfi Invoice API existieren diverse solcher SDKs. Sie sind verfügbar in nahezu jeder gängigen Programmiersprache. Im Zuge von Anhang 4 wird eines dieser SDKs verwendet, um spezifische Informationen aus einer Rechnung zu extrahieren.¹

¹ Überblick über alle verfügbaren SDKs: <https://www.verifyfi.com/sdk/>

Anhang 3: Automatisierte Bearbeitung von Eingangsrechnungen

Um den Prozess zur Bearbeitung von Eingangsrechnungen zu automatisieren, wird die Camunda Process Engine eingesetzt. Die Engine ist dabei integriert in eine Kotlin-, bzw. eine Spring Boot basierte Web-Applikation. In dieser ist die eigentliche Logik zur Ausführung der Prozessschritte implementiert. Diese ist – im Kontext zum Prozessmodell – wie folgt zu beschreiben: ¹



Startereignis – Rechnung hochladen

Um den Prozess zu starten, wird in der Web-Applikation eine spezifische Schnittstelle entwickelt. Diese kann eine PDF entgegennehmen, in welcher eine Rechnung abgebildet sein kann. Wird über diese Schnittstelle eine Rechnung hochgeladen, so ruft die Applikation die Camunda Process Engine auf und löst eine neue Instanz des Prozesses aus. Zudem wird eine Upload-ID generiert, um in den späteren Schritten des Prozesses auf die Daten zur Prozessinstanz zugreifen zu können. Diese ID und die Rechnung werden als Prozessvariablen der Instanz gespeichert.

```
@PostMapping("/upload")
@Operation(description = "Upload a new invoice to start the payment process")
fun uploadInvoice(@RequestPart("invoice") invoice: MultipartFile): ResponseEntity<String> {
    log.debug { "A new invoice got uploaded - processing it!" }
    return ResponseEntity.ok().body(invoiceFacade.uploadInvoice(invoice))
}

fun uploadInvoice(invoice: MultipartFile): String {
    val uploadId = UUID.randomUUID().toString()
    val processVariables: MutableMap<String, Any> = HashMap()
    processVariables["uploadId"] = uploadId
    processVariables["document"] = invoice.bytes
    processService.startInstanceOfProcess(processKey, uploadId, processVariables)
    return uploadId
}
```

Aktivität - Daten aus Rechnung auslesen

Im ersten Schritt des Prozesses, der als ein Service Task abgebildet ist, sollen die Informationen, welche für die Zahlung relevant sind, durch den Einsatz der „**veryfi Invoice API**“ aus der jeweiligen Rechnung extrahiert werden. Zu diesem Zweck wird ein Service, bzw. eine Funktion in die Web-Applikation implementiert, durch welche diese API aufgerufen wird. Der Service Task wiederum wird so konfiguriert, dass er auf diese Funktion referenziert.

¹ Vollständige Implementierung verfügbar unter: <https://github.com/emaarco/bpmnandai>

```

fun extractDataFromInvoice(uploadId: String,) {
    val invoice = processService.getProcessVariable(uploadId, "document")
    var invoiceAsBase64 = Base64.getEncoder().encodeToString(invoice)
    invoiceAsBase64 = "data:application/pdf;base64,${invoiceAsBase64}"
    val jsonResponse = verifyAdapter.processInvoice(invoiceAsBase64, "invoice.pdf")
    val extractedInvoice = Invoice(uploadId, jsonResponse)
    saveInvoice(extractedInvoice)
    log.info { "Extracted data from invoice '${uploadId}'" }
}

```

Wird die Funktion ausgeführt, so bezieht diese aus den Prozessvariablen der Instanz die jeweilige Rechnung. Diese wird in der Folge an einen anderen Service übergeben. Dieser ist in der Lage mit der verify Invoice API zu interagieren. Zu diesem Zweck nutzt er die SDK für die Sprache Kotlin. Bevor diese verwendet werden kann, muss sie jedoch konfiguriert werden. Dazu müssen ihr die relevanten Zugangsdaten, wie die Client-ID und der API-Key bereitgestellt werden. Diese werden in den Umgebungsvariablen der Web-Applikation gespeichert. Ist dieser Schritt abgeschlossen, so kann eine entsprechende Funktion der SDK genutzt werden, welche die relevanten Daten aus der Rechnung extrahiert und in Form eines JSON-Objektes zurückgibt. Der Prozessschritt endet damit, dass die Daten zur Rechnung in der Datenbank der Web-Applikation gespeichert werden.

```

@Configuration
class VerifyConfig {

    @Value("${api.verify.clientId}")
    private lateinit var clientId: String

    @Value("${api.verify.authorization}")
    private lateinit var apiKey: String

    @Value("${api.verify.userName}")
    private lateinit var userName: String

    @Value("${api.verify.clientSecret}")
    private lateinit var clientSecret: String

    @Bean
    fun buildVerifyClient(): Client {
        return VerifyClientFactory.createClient(clientId, clientSecret, userName, apiKey)
    }
}

fun processInvoice(file: String, fileName: String): JSONObject {
    return try {
        val parameters = JSONObject().put("file_data", file)
        val response = verifyClient.processDocument(fileName, listOf("Invoice"), true, parameters)
        JSONObject(response)
    } catch (ex: Exception) {
        log.info { "The provided invoice could not be processed by the api: ${ex.message}" }
        getFallbackResponse()
    }
}

```

Vorbereitung – Rechnung überprüfen

Im zweiten Schritt des Prozesses soll geprüft werden, ob die KI-Lösung die Daten korrekt aus der Rechnung extrahiert hat. Daher benötigt der Mitarbeiter, welcher diese Prüfung durchführen soll, Informationen über die extrahierten Daten. Um diese zur Verfügung zu stellen, wird in der Web-Applikation eine weitere Schnittstelle entwickelt. Über diese werden die Daten zu einer konkreten Rechnung aus der Datenbank selektiert und zurückgegeben.

```

@GetMapping("/{uploadId}")
@Operation(description = "Get information to a specific invoice")
fun getSpecificInvoice(@PathVariable("uploadId") uploadId: String): ResponseEntity<InvoiceTO> {
    log.debug { "Received request to get information to invoice '$uploadId'" }
    val matchingInvoice = invoiceFacade.getInvoice(uploadId)
    return ResponseEntity.ok().body(invoiceMapper.mapToTO(matchingInvoice))
}

data class InvoiceTO(
    val uploadId: String,
    val invoiceNumber: String,
    val orderNumber: String,
    val date: String,
    val subtotal: Double,
    val shipping: Double,
    val tax: Double,
    val total: Double,
    val vendor: VendorTO,
    val cartItems: List<ShoppingCartItemTO>
)

data class VendorTO(
    val address: String,
    val email: String,
    val name: String,
    val iban: String,
)

data class ShoppingCartItemTO(
    val id: Long,
    val description: String,
    val discount: Double,
    val price: Double,
    val quantity: Double,
    val tax: Double,
    val total: Double,
    val unitOfMeasure: String,
)

```

Aktivität - Rechnung überprüfen

Die Prüfung der Rechnung wird mittels einem Benutzer Task abgebildet. Um diesen auszuführen, besitzt die Web-Applikation eine weitere Schnittstelle. Diese ist in der Lage eine korrigierte Version einer bestimmten Rechnung entgegenzunehmen.

```

@PostMapping("/approve/{uploadId}")
@Operation(description = "Approve the invoice to trigger the payment")
fun approveInvoice(
    @PathVariable("uploadId") uploadId: String,
    @RequestBody invoice: UpdatedInvoiceTO?
): ResponseEntity<InvoiceTO> {
    log.debug { "Received request to approve invoice '$uploadId': $invoice" }
    val updatedInvoice = invoice?.let { inv -> invoiceMapper.mapToUpdatedInvoice(inv) }
    val approvedInvoice = invoiceFacade.approveInvoice(uploadId, updatedInvoice)
    return ResponseEntity.ok().body(invoiceMapper.mapToTO(approvedInvoice))
}

```

Wird diese Schnittstelle aufgerufen, so lädt sie die entsprechende Rechnung aus der Datenbank. Anschließend wird die Rechnung – wenn nötig – korrigiert. Folgend darauf wird die aktualisierte Version der Rechnung in der Datenbank gespeichert und der Benutzer Task abgeschlossen.

```

fun approveInvoice(uploadId: String, updatedInvoice: UpdatedInvoice?): Invoice {
    val matchingInvoice = findInvoice(uploadId)
    updatedInvoice?.let { invoiceData -> updateAndApproveInvoice(matchingInvoice, invoiceData) }
    log.info { "Successfully approved invoice with uploadId '$uploadId'" }
    return matchingInvoice
}

data class UpdatedInvoice(
    val invoiceNumber: String,
    val orderNumber: String,
    val date: String,
    val subtotal: Double,
    val shipping: Double,
    val tax: Double,
    val total: Double,
    val vendor: Vendor,
    val cartItems: List<ShoppingCartItem>
)

```

Aktivität - Zahlung ausführen

Im letzten Schritt des Prozesses wird die Zahlung der jeweiligen Rechnung ausgeführt. Diese wird durch einen Service Task abgebildet. Dieser ruft eine Funktion der Web-Applikation auf, welche die Rechnung aus der Datenbank selektiert. Mit den Informationen zu dieser, die im Rahmen des letzten Schrittes überprüft wurden, wird eine weitere Methode aufgerufen. In dieser wiederum könnte die Logik zur tatsächlichen Durchführung der Zahlung abgebildet sein. Im Prototyp jedoch wird darauf verzichtet – und lediglich eine Log-Nachricht ausgegeben.

```
fun executePayment(uploadId: String) {  
    val matchingInvoice = findInvoice(uploadId)  
    payInvoice(matchingInvoice)  
    log.info { "Successfully executed payment for invoice '$uploadId'" }  
}
```

Anhang 4: Daten zur Erstellung eines ML-Modells zur Klassifikation von Kreditanfragen

Um ein ML-Modell zu erstellen, durch welches Kunden, welche einen Antrag für einen Kredit an eine Bank gestellt haben, in kreditwürdige und nicht kreditwürdige Kunden zu unterteilen, werden konkrete historische Daten benötigt. Damit ein allgemeingültiges Modell abgeleitet werden kann, müssen diese in einer ausreichenden Menge vorliegen.

Die Daten zum Training des ML-Modells stammen von der Data-Science Plattform Kaggle, bzw. von einem öffentlich verfügbaren Datenpool auf dieser. Dieser beinhaltet historische Daten einer Bank, in welchen die Kunden dieser, im Hinblick auf ihre Kreditwürdigkeit bewertet werden. Diese Daten können als eine CSV-Datei heruntergeladen werden. In ihrer Rohform umfasst diese Datei knapp 538.000 Zeilen. Es handelt sich dabei um Daten, die für das Supervised Learning geeignet sind. Sie besitzen eine Zielvariable – die Spalte TARGET. Diese gibt an, ob ein Kunde kreditwürdig (1) oder nicht kreditwürdig (0) ist.¹

TARGET	ID (Alle Zeilen z	Transformation	Kreditwürdig	Anzahl
0	535705		NO	1962
1	1962		YES	1962
Gesamtsumme	537667		Datensätze gesamt	3924

Datenvorbereitung & finales Datenmodell

Damit ein ausgeglichenes Modell generiert werden kann, werden die Daten vor dem eigentlichen Training angepasst. Sie werden auf knapp 4.000 Datensätze reduziert – wobei beabsichtigt wird, dass die Anzahl der Beispiele je Zielvariable identisch ist. Dazu wird ein Großteil der Datensätze, in welchen die Spalte TARGET den Wert „0“ aufweist gelöscht. Die verbleibenden Daten werden in einer neuen Excel Datei gespeichert. Die einzelnen Felder dieser sind wie folgt zu beschreiben:

	Datenbasis für ML-Modell	
	Feld	Beschreibung
	ID	ID der Kreditanfrage
	GENDER	Geschlecht des Kunden
	FIRST NAME	Vorname des Kunden
	LAST NAME	Nachname des Kunden
	EMAIL	E-Mail Adresse des Kunden
	FLAG_OWN_CAR	Kunde besitzt Auto
	FLAG_OWN_REALTY	Kunde besitzt Grundstück
	COUNT CHILDREN	Anzahl Kinder (Kunde)
	ANNUAL INCOME	Jahreseinkommen des Kunden
	EDUCATION TYPE	Bildungsgrad des Kunden
	FAMILY STATUS	Familienstatus des Kunden
	DAYS EMPLOYED	Anzahl Tage seit Beginn der aktuellen Beschäftigung
	JOB	Aktuelle Beschäftigung
	AGE	Alter des Kunden
	CREDIT AMOUNT	Gewünschter Kreditbetrag
	CREDITWORTHY	Kreditwürdig? (historisch/Zielvariable)

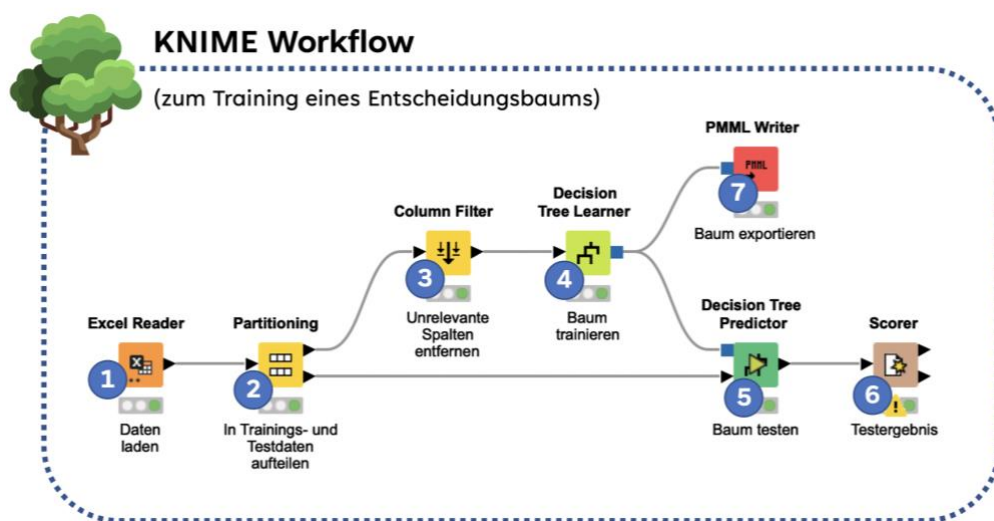
¹ Quelle der Daten: <https://www.kaggle.com/datasets/laotse/credit-card-approval>

Anhang 5: Erstellung eines ML-Entscheidungsbaums mit KNIME

Um den Entscheidungsbaum zu erstellen, durch welchen überprüft werden soll, ob ein Kunde, der eine Anfrage für einen Kredit gestellt hat, kreditwürdig ist, wird die Software **KNIME** eingesetzt. Diese ermöglicht die Erstellung von ML-Modellen über eine graphische Benutzeroberfläche. Dazu stellt sie diverse wiederverwendbare Komponenten bereit. Mittels diesen kann der Vorgang zum Training eines ML-Modells – in Form eines Workflows – abgebildet werden.¹

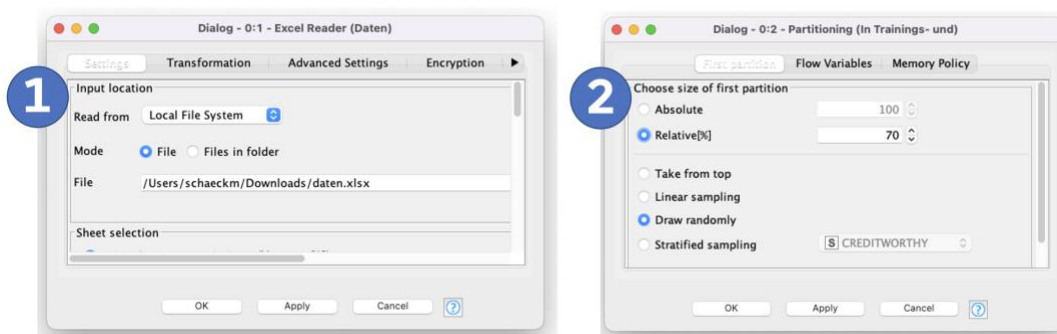
Workflow zum Training des prototypischen Entscheidungsbaumes:

Um das ML-Modell zu trainieren, das im Prozess zur Genehmigung eines Dispokredits eingesetzt werden soll, wird der folgende Workflow modelliert. Die einzelnen Schritte von diesem sind wie folgt zu beschreiben:



Schritt 1 & 2 – Daten laden & aufteilen

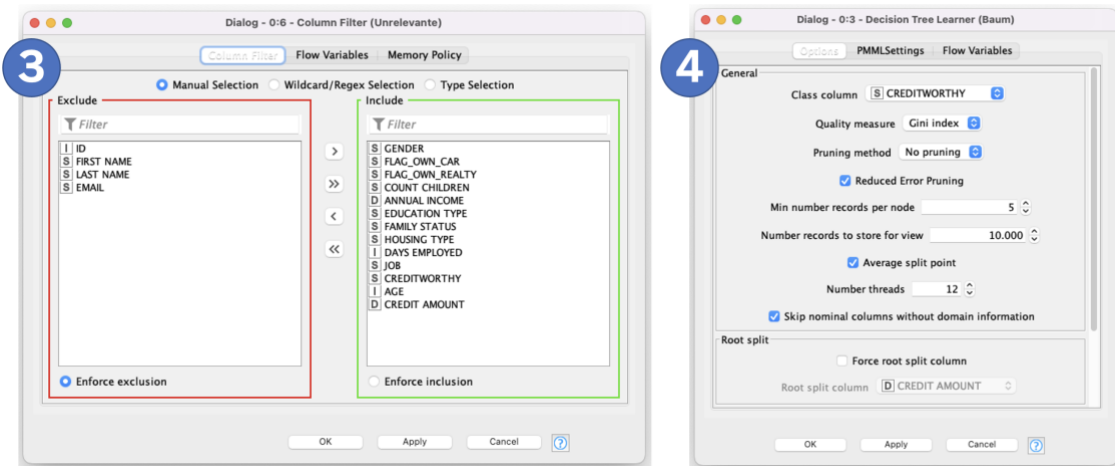
Zuallererst müssen die benötigten Daten in das Tool importiert werden. Diese liegen in Form einer Excel Datei vor. Diese wird über einen Excel Reader eingelesen. Ist der Import abgeschlossen, so werden die Daten in Trainings- und Testdaten unterteilt. Die Aufteilung basiert dabei auf einem Zufallsprinzip – wobei jedoch konfiguriert wird, dass 70% der Daten als Trainingsdaten und die restlichen 30% als Testdaten verwendet werden sollen.



¹ Mehr Informationen zu KNIME – siehe: <https://www.knime.com/>

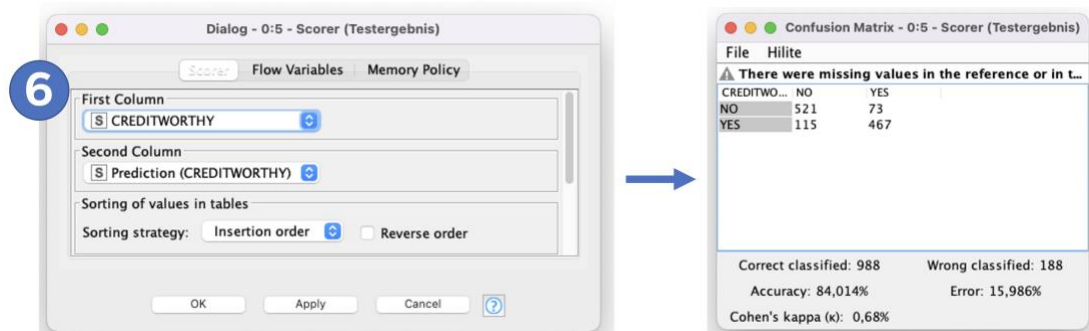
Schritt 3 & 4 – Entscheidungsbaum trainieren

Folgend darauf werden die Trainingsdaten verwendet, um den Entscheidungsbaum zu trainieren. Dazu wird ein sogenannter Decision Tree Learner eingesetzt. In diesem muss konfiguriert werden, dass das Attribut „Creditworthy“ die Zielvariable des Modells ist. Auf diese soll es trainiert werden. Zuvor wird jedoch noch ein Filter durchlaufen. Dieser entfernt Attribute, welche im Training nicht berücksichtigt werden sollen, aus den Trainingsdaten.



Schritt 5 & 6 – Entscheidungsbaum testen

Im Zuge dieser beiden Schritte wird der eben erlernte Entscheidungsbaum getestet. Dazu werden die Testdaten auf den Baum angewendet. Die Grundidee hierzu wird in Kapitel 3.2.6 untersucht. Das Ergebnis des Tests wird über eine Konfusionsmatrix abgebildet. Aus dieser kann abgelesen werden, dass der Entscheidungsbaum eine Vorhersage-Genauigkeit von ungefähr 84% besitzt.



Schritt 7 - Export als PMML

Damit der Entscheidungsbaum im Prototyp eingesetzt werden kann, wird er in Form eines PMML-Modells exportiert. Die Grundlagen zu einem solchen werden im Zuge von Kapitel 3.3 erörtert. Zu diesem Zweck wird ein sog. PMML-Writer verwendet. Dieser wiederum wird so konfiguriert, dass er das Modell in einem bestimmten Verzeichnis speichert.

Anhang 6: Exemplarisches PMML-Modell

Das PMML-Modell des Entscheidungsbaumes, welches aus KNIME exportiert wurde, ist in Form einer XML-Datei gespeichert. Der eigentliche Baum, welcher in diesem abgebildet ist, wird durch den Inhalt des XML-Tags „**TreeModel**“ repräsentiert.

Er besteht aus einer Menge von Knoten, die hierarchisch angeordnet sind. Jeder der Knoten wird dabei durch ein XML-Tag mit dem Namen „**Node**“ repräsentiert. Die Wurzel des Baumes ist die Node mit der ID „0“. Ausgehend von dieser kann jeder Knoten im Baum eine variable Menge von Unterknoten besitzen. Diese werden ebenfalls durch das XML-Tag „Node“ abgebildet. Dabei ist zu beachten, dass jeder Knoten jeweils eine Bedingung besitzt. Diese wird abgebildet durch das XML-Tag „**SimplePredicate**“. Sie gibt an, wann einem Knoten des Baumes gefolgt wird. Ein Blatt ist dann erreicht, wenn in einem Node-Tag kein weiteres Node-Tag mehr eingebettet ist.

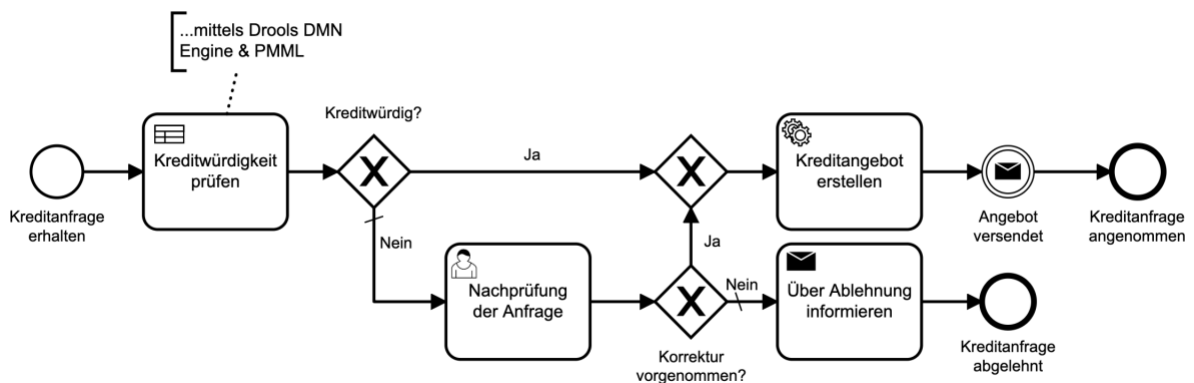
```
<?xml version="1.0" encoding="UTF-8"?>
<PMML version="4.2" xmlns="http://www.dmg.org/PMML-4_2">
  <Header copyright="schaeckm"> ... </Header>
  <DataDictionary numberOfFields="13">
    ...
  </DataDictionary>
  <TreeModel modelName="DecisionTree" functionName="classification" ... >
    <MiningSchema>
      ...
    </MiningSchema>
    <Node id="0" score="YES" recordCount="2746.0">
      <True/>
      <ScoreDistribution value="NO" recordCount="1367.0" />
      <ScoreDistribution value="YES" recordCount="1379.0" />
      <Node id="1" score="NO" recordCount="612.0">
        <SimplePredicate field="CREDIT AMOUNT" operator="greaterThan" value="262710.0" />
        <ScoreDistribution value="NO" recordCount="498.0" />
        <ScoreDistribution value="YES" recordCount="114.0" />
        ...
      </Node>
      <Node id="2" score="YES" recordCount="2134.0">
        <SimplePredicate field="CREDIT AMOUNT" operator="lessOrEqual" value="262710.0" />
        <ScoreDistribution value="NO" recordCount="869.0" />
        <ScoreDistribution value="YES" recordCount="1265.0" />
        <Node id="86" score="YES" recordCount="492.0">
          <SimplePredicate field="JOB" operator="equal" value="Laborers" />
          <ScoreDistribution value="NO" recordCount="0" />
          <ScoreDistribution value="YES" recordCount="492" />
        </Node>
        ...
      </Node>
      ...
    </TreeModel>
  </PMML>
```

Anhang 7: Automatisierte Bearbeitung von Kreditanfragen mittels einem PMML-Modell

Der Prozess zur Bearbeitung von Kreditanfragen wird in zwei Varianten automatisiert. Zum einen durch den nativen Einsatz des PMML-Modells aus Anhang 5. Zum anderen durch die Verwendung einer DMN-Entscheidungstabelle, die aus diesem PMML-Modell erzeugt werden kann. In beiden von diesen Fällen wird die Process Engine von Camunda eingesetzt. Diese ist, wie auch im ersten Prototyp, in eine Kotlin-, bzw. eine Spring Boot basierte Web-Applikation eingebettet. In dieser ist die Logik zur Ausführung der Prozessschritte implementiert.¹

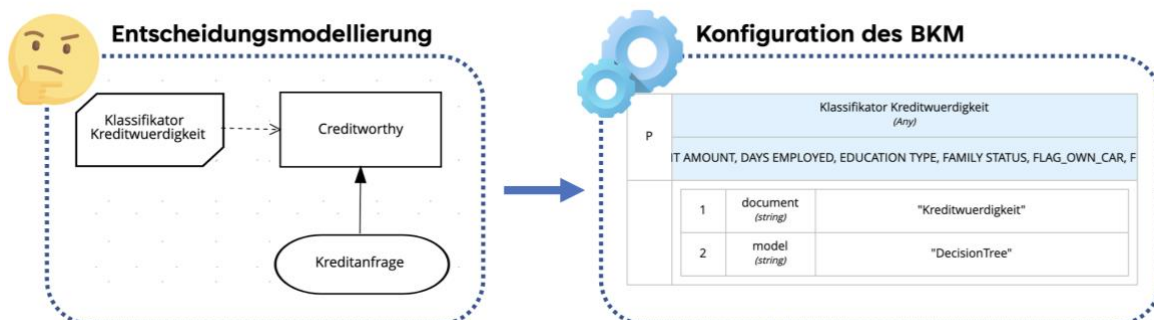
Dokumentation – Variante 1:

In diesem Anhang wird die technische Implementierung der ersten Prozessvariante dokumentiert. In dieser wird die Überprüfung der Kreditwürdigkeit eines Kunden durch den nativen Einsatz des PMML-Modells, in einem DMN-Modell automatisiert. Die einzelnen Schritte des Prozesses und das Vorgehen zur Erstellung von diesem DMN-Modell sind dabei wie folgt zu beschreiben:



Erstellung des DMN-Entscheidungsmodells

Die DMN interpretiert ein PMML-Modell als eine konkrete Form von Entscheidungslogik. Als eine solche kann sie durch ein BKM ausgewertet werden. Um ein DMN-Modell zu erstellen, in welchem ein PMML-Modell auf diese Weise integriert ist, kann das Modellierungstool „**Business Central**“ von jBPM eingesetzt werden.²



¹ Vollständige Implementierung verfügbar unter: <https://github.com/emaarco/bpmnandai>

² Die jBPM Workbench ist verfügbar unter: <https://hub.docker.com/r/jboss/jbpm-workbench>

Mittels diesem muss ein neues, bzw. ein leeres DMN-Modell erstellt werden. In diesem muss eine Decision modelliert werden, welche mit spezifischen Daten versorgt wird, und ein konkretes BKM aufruft. Das BKM wiederum muss so konfiguriert werden, dass es das PMML-Modell auswertet, welches an einem bestimmten Ort gespeichert sein muss. Sind diese Schritte durchgeführt, so kann das fertige Modell als eine XML-Datei heruntergeladen, und in der Web-Applikation, bzw. dem Projekt gespeichert werden.

Startereignis - Kreditanfrage erhalten

Damit der Prozess gestartet werden kann, wird – analog zum ersten Prototyp – eine spezifische Schnittstelle entworfen. Diese nimmt eine Kreditanfrage entgegen. Eine solche besteht dabei aus Informationen zum potentiellen Kunden, sowie zum gewünschten Kreditbetrag. Auf Basis dieser ruft die Schnittstelle eine weitere Funktion der Web-Applikation auf, welche den Prozess startet.

```
@PostMapping()
@Operation(summary = "Create a new loan request (variant: pmml)")
fun createLoanRequest(@RequestBody loanRequest: NewLoanRequestTO): ResponseEntity<LoanRequestTO> {
    log.debug { "Received request to create a new loan request: $loanRequest" }
    val createdRequest = creditFacade.createLoanRequest(loanMapper.mapToModel(loanRequest))
    return ResponseEntity.ok().body(loanMapper.mapToTO(createdRequest))
}

fun createLoanRequest(newRequest: NewLoanRequest): LoanRequest {
    val requestId = UUID.randomUUID().toString()
    val loanRequest = LoanRequest(requestId, newRequest)
    creditService.saveLoanRequest(loanRequest)
    val processVariables = mapOf(Pair("requestId", requestId))
    processService.startInstanceOfProcess(processKey, requestId, processVariables)
    log.info { "Created a new loan request: $loanRequest" }
    return loanRequest
}

data class NewLoanRequestTO(
    val applicant: ApplicantTO,
    val creditAmount: Double,
)

data class ApplicantTO(
    val firstName: String,
    val lastName: String,
    val email: String,
    val gender: String,
    val age: Int,
    val familyStatus: String,
    val nrOfChildren: String,
    val annualIncome: Double,
    val job: String,
    val daysEmployed: Int,
    val educationType: String,
    val housingType: String,
    val ownsCar: String,
    val ownsRealty: String,
)
```

Diese ist so programmiert, dass sie zuallererst eine ID für die Anfrage generiert. Durch diese kann die Kreditanfrage eindeutig identifiziert werden. Als nächstes wird die Anfrage in der Datenbank der Web-Applikation gespeichert. Folgend darauf wird die Camunda Process Engine aufgerufen, und eine neue Instanz des Prozesses gestartet. Als eine Prozessvariable wird der Instanz die eben erstellte ID übergeben. Mittels dieser kann die Instanz in den Folgeschritten identifiziert werden.

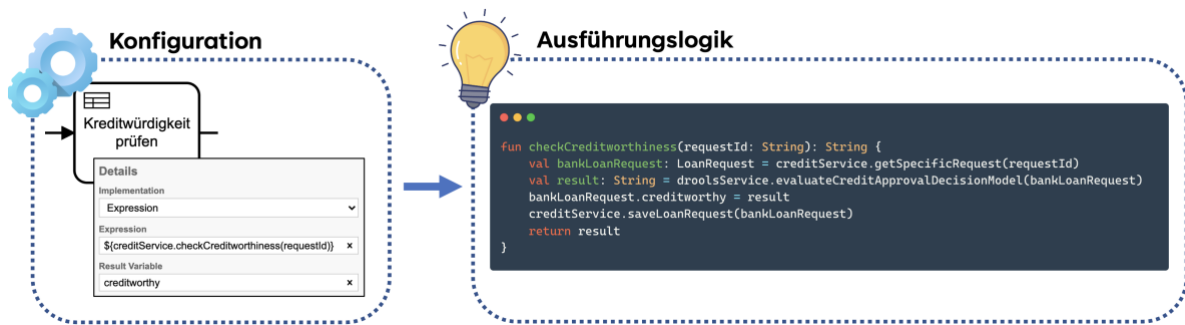
Aktivität - Kreditwürdigkeit prüfen

Die Bewertung der Kreditwürdigkeit eines Kunden ist über einen Geschäftsregel Task abgebildet. Dieser soll das DMN-Modell aus Schritt 1 ausführen. In diesem ist ein PMML-Modell integriert, welches den Entscheidungsbaum aus Anhang 4 repräsentiert. Da die Camunda DMN-Engine die Ausführung von PMML-Modellen nativ nicht unterstützt, wird für diesen Zweck die DMN-Engine „Drools“ eingesetzt.³

³ Für mehr Informationen zu Drools, siehe: <https://www.drools.org/>

Da diese DMN-Engine nicht in die BPMN Engine von Camunda integriert ist, muss sie explizit in die Web-Applikation eingebunden werden. Um sie in der Praxis zu nutzen, muss außerdem eine Ausführungslogik entwickelt werden, und der Task so konfiguriert werden, dass er diese auslöst.

Alles in allem nimmt die Logik dabei eine ID zu einer Kreditanfrage entgegen. Auf Basis von dieser selektiert sie die zugehörige Anfrage aus der Datenbank. Mittels dieser wird ein weiterer Service aufgerufen, welcher die Drools DMN-Engine nutzt, um das DMN-Modell auszuwerten.



Dieser Service wiederum lädt das DMN-Modell, welches im Rahmen dieser Variante erstellt wird. In der Folge initialisiert er die Input-Daten, die zur Auswertung der Entscheidungslogik benötigt werden. Danach führt der Service das Modell aus, verarbeitet dessen Ergebnis, und speichert es, orientiert an der Konfiguration im Prozessmodell, in einer konkreten Prozessvariable.

```
@Service
class DroolsService(private val dmnruntime: DMNRuntime) {

    @Value("${dmn.drools.model.name}")
    private lateinit var dmnruntimeModelName: String

    @Value("${dmn.drools.model.namespace}")
    private lateinit var dmnruntimeModelNamespace: String

    @Value("${dmn.drools.model.output}")
    private val dmnruntimeModelOutput: String? = null

    fun evaluateCreditApprovalDecisionModel(request: LoanRequest): String {
        val dmnruntimeModel: DMNModel = getModel()
        val isApproved = executeDecisionModel(dmnruntimeModel, request)
        return mapResult(isApproved)
    }

    /* ----- private helper methods ----- */

    private fun getModel(): DMNModel {
        return dmnruntime.getModel(dmnruntimeModelNamespace, dmnruntimeModelName)
    }

    private fun executeDecisionModel(dmnruntimeModel: DMNModel, request: LoanRequest): DMNDecisionResult? {
        val context: DMNContext = initializeContext(request)
        val dmnruntimeResult: DMNResult = dmnruntime.evaluateAll(dmnruntimeModel, context)
        return dmnruntimeResult.getDecisionResultByName(dmnruntimeModelOutput)
    }

    private fun mapResult(result: DMNDecisionResult?): String {
        return if ((result == null) || (result.result == null)) {
            "UNKNOWN"
        } else {
            result.result as String
        }
    }

    private fun initializeContext(request: LoanRequest): DMNContext {
        val dmnruntimeContext: DMNContext = dmnruntime.newContext()
        dmnruntimeContext.set("Kreditanfrage", request)
        return dmnruntimeContext
    }
}
```

Vorbereitung – Nachprüfung der Anfrage

Wenn das ML-Modell einen Kunden als nicht kreditwürdig einstuft, soll durch einen Mitarbeiter überprüft werden, ob dies zutreffend ist. Folglich benötigt ein Mitarbeiter, der eine solche Prüfung durchführt, die Informationen, auf welchen die jeweilige Entscheidung des ML-Modells basiert. Um diese bereitzustellen, wird in der Web-Applikation eine konkrete Schnittstelle implementiert. Über diese werden die Daten, die zu einer angeforderten Kreditanfrage gespeichert sind, aus der Datenbank der Web-Applikation selektiert, und zurückgegeben.

```
@GetMapping("/{requestId}")
@Operation(description = "Get a specific loan-request")
fun getLoanRequest(@PathVariable requestId: String): ResponseEntity<LoanRequestTO> {
    log.debug { "Received request to get loan-request with id '$requestId'" }
    val matchingLoanRequest = creditFacade.getSpecificLoanRequest(requestId)
    return ResponseEntity.ok().body(LoanMapper.mapToTO(matchingLoanRequest))
}
```

```
data class LoanRequestTO(
    val id: String,
    val applicant: ApplicantTO,
    val creditAmount: Double,
    var creditworthy: String,
)
```

```
data class ApplicantTO(
    val firstName: String,
    val lastName: String,
    val email: String,
    val gender: String,
    val age: Int,
    val familyStatus: String,
    val nrOfChildren: String,
    val annualIncome: Double,
    val job: String,
    val daysEmployed: Int,
    val educationType: String,
    val housingType: String,
    val ownsCar: String,
    val ownsRealty: String,
)
```

Aktivität – Nachprüfung der Anfrage

Die tatsächliche Prüfung wird mittels einem Benutzer Task abgebildet. Damit dieser ausgeführt werden kann wird eine weitere Schnittstelle in der Web-Applikation erstellt. Über diese kann das Resultat der Überprüfung für eine bestimmte Kreditanfrage übermittelt werden. Dieses wird sodann in der entsprechenden Anfrage und in einer Prozessvariable der Instanz gespeichert.

```
@PostMapping("/{requestId}/recheck/{creditworthy}")
@Operation(summary = "Check whether the provided request should really be rejected")
fun recheckLoanRequest(
    @PathVariable(name = "requestId") requestId: String,
    @PathVariable(name = "creditworthy") creditworthy: Boolean
): ResponseEntity<LoanRequestTO> {
    log.debug { "Rechecked the loan request '$requestId' - approved: $creditworthy" }
    val updatedRequest = creditFacade.recheckLoanRequest(requestId, creditworthy)
    return ResponseEntity.ok().body(LoanMapper.mapToTO(updatedRequest))
}
```



```
fun recheckLoanRequest(requestId: String, creditworthy: Boolean): LoanRequest {
    val matchingRequest = findLoanRequest(requestId)
    matchingRequest.updateResultOfCreditworthinessCheck(creditworthy)
    saveLoanRequest(matchingRequest)
    val processVars = mapOf(Pair("recheck_creditworthy", updatedRequest.getCreditworthiness()))
    processService.finishTask(taskId, processVars)
    return matchingRequest
}
```

Restliche Tasks und Ereignisse

Für die Ausführung der restlichen Prozessschritte ist keine komplexe Ausführungslogik hinterlegt. Die Tasks und Ereignisse sind so konfiguriert, dass sie lediglich Methoden aufrufen, welche eine

entsprechende Log-Nachricht in die Konsole der Web-Applikation schreiben. An und für sich wäre es jedoch auch bei diesen möglich eine tatsächliche Logik zu programmieren, durch welche z.B. ein Kreditangebot erstellt, oder eine Mail an den Kunden gesendet wird.

```
/** Wird aufgerufen vom Task "Kreditangebot erstellen" */
fun createCreditOffer(requestId: String) {
    log.info { "Credit for request '$requestId' can be granted - creating offer!" }
}

/** Wird aufgerufen vom Event "Angebot versendet" */
fun sendCreditOffer(requestId: String) {
    log.info { "Send credit offer for request '$requestId' to applicants e-mail" }
}

/** Wird aufgerufen vom Task "Über Ablehnung informieren" */
fun sendRejectionMessage(requestId: String) {
    log.info { "Send rejection message for request '$requestId' to applicants e-mail" }
}
```


Anhang 8: Transformation eines PMML-Modells in eine DMN-Entscheidungstabelle

Eine zweite Möglichkeit, um das ML-Modell aus Anhang 5 über einen Geschäftsregel Task in den Prozess zur Bearbeitung von Kreditanfragen einzubinden, ist dessen Umwandlung in eine DMN-Entscheidungstabelle. Die Grundidee zu einer solchen Umwandlung wird im Rahmen von Kapitel 6.3.3.2 thematisiert. In diesem Anhang wird eine Web-Applikation implementiert, durch welche diese tatsächlich durchgeführt werden kann. Die Applikation basiert dabei – wie die bisherigen Prototypen – auf der Programmiersprache Kotlin und dem Framework Spring Boot.¹

Vorbereitung - Schnittstelle zur Nutzung der Applikation

Um ein PMML-Modell in eine Entscheidungstabelle transformieren zu können, benötigt die Web-Applikation einen Zugriff auf dieses. Daher wird eine konkrete Schnittstelle entwickelt. Diese ist in der Lage ein PMML-Modell entgegenzunehmen. Zudem können der Schnittstelle einige weitere Parameter übergeben werden. Zu diesen zählen die IDs und die fachlichen Bezeichnungen für das DMN-Modell und die Entscheidungstabelle.

```
@PostMapping("/dmn")
fun buildDmnModel(
    response: HttpServletResponse,
    @RequestParam("pmml-file") rawPmmlTree: MultipartFile,
    @RequestParam(name = "model-id") modelId: String,
    @RequestParam(name = "model-name") modelName: String,
    @RequestParam(name = "decision-id") decisionId: String,
    @RequestParam(name = "decision-name") decisionName: String,
): ResponseEntity<ByteArrayResource> {
    response.contentType = "application/xml"
    val request = DmnModelRequest(modelId, modelName, decisionId, decisionName)
    return ResponseEntity.ok().body(dmnFacade.buildDmnModel(rawPmmlTree, request))
}
```

Schritt 1 – Extraktion der Pfade

Die Umwandlung eines Entscheidungsbaums in eine Entscheidungstabelle kann grob beschrieben in zwei Schritten erfolgen. Der erste Schritt umfasst dabei die Extraktion der Entscheidungsregeln aus dem Baum. Diese werden repräsentiert durch die einzelnen Pfade im Baum. Damit diese aus dem PMML-Modell ausgelesen werden können, muss dieses von einem generischen MultipartFile in die Form eines XML-Dokuments überführt werden, dass von Kotlin, bzw. der Web-Applikation interpretiert werden kann.

```
fun mapPmmlDocumentToXML(rawPmmlFile: MultipartFile): Document {
    return try {
        val factory = DocumentBuilderFactory.newInstance()
        val builder = factory.newDocumentBuilder()
        val document = builder.parse(rawPmmlFile.inputStream)
        document.documentElement.normalize()
        document
    } catch (ex: Exception) {
        throw RuntimeException("Could not map provided pmml file to xml: ${ex.message}")
    }
}
```

¹ Vollständige Implementierung verfügbar unter: <https://github.com/emaarco/pmmltodmn>

Aus diesem XML-Dokument müssen sodann alle Knoten des Baumes extrahiert werden. Folgend darauf kann der Wurzelknoten identifiziert werden. Ausgehend von eben diesem können sodann alle Regeln des Baumes identifiziert werden. Dazu wird ein rekursiver Algorithmus entwickelt.

Dieser sucht jegliche Unterknoten einer Wurzel. Für jeden von diesen wird eine weitere Funktion aufgerufen. Diese bildet die eigentliche Rekursion ab. Sie speichert den Knoten, der ihr übergeben wird, in einer Liste, und sucht für diesen nach weiteren Unterknoten. Für jeden gefundenen Knoten wird die Liste kopiert, und die Funktion erneut aufgerufen. Durch diesen Vorgang bilden sich die einzelnen Pfade. Er wird so lange wiederholt, bis ein Blatt gefunden wird. Ist dies der Fall, so ist ein Pfad abgeschlossen. Somit kann die Liste, die der Funktion übergeben wird, als ein einzelner Pfad interpretiert werden. Sie wird als ein eindeutiger Eintrag in einer Map gespeichert. Ist die Rekursion abgeschlossen, so sind in dieser Map alle Pfade des Baumes hinterlegt.

```
val treePaths: TreeMap<Int, List<Node>> = TreeMap()

fun buildDecisionTree(document: Document) {
    val allNodes = getAllNodes(document)
    if (allNodes.length > 0) {
        val rootNode = allNodes.item(0)
        val childNodes = getChildNodes(rootNode)
        childNodes.forEach { child: Node -> findAllChildrenRecursive(child, ArrayList()) }
    }
}

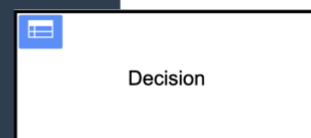
private fun findAllChildrenRecursive(node: Node, currentRow: MutableList<Node>) {
    currentRow.add(node)
    val children = getChildNodes(node)
    if (children.isEmpty()) {
        treePaths[treePaths.size + 1] = currentRow
    } else {
        children.forEach { child -> findAllChildrenRecursive(child, ArrayList(currentRow)) }
    }
}
```

Schritt 2 – Aufbau der DMN-Entscheidungstabelle

Sind durch den ersten Schritt alle Pfade des Entscheidungsbaumes bekannt, so kann im zweiten Schritt die eigentliche Umwandlung durchgeführt werden. Dazu wird eine spezifische Logik, bzw. eine Funktion entwickelt, welche zuallererst ein leeres XML-Dokument erzeugt.

In diesem wird sodann ein DMN-Modell initialisiert, welches eine einzelne Decision besitzt. Diese wiederum umfasst eine Entscheidungstabelle. Diese ist zu Beginn jedoch noch leer.

```
fun buildDmnTable(decisionTree: TreeInfo, request: DmnModelRequest): ByteArrayResource {
    val doc = initializeOutputDocument()
    val dmnModel = DmnModel(doc, request.dmnModelID, request.dmnModelName)
    val decision = Decision(doc, dmnModel, request.dmnDecisionID, request.dmnDecisionName)
    val decisionTable: Element = DecisionTable(doc, decision).decisionTable
    ...
}
```



Folgend darauf werden die Eingabefelder und die Ausgabefelder der DMN-Tabelle gesetzt. Diese sind abhängig von dem Data-Dictionary, welches ebenfalls im initial übergebenen PMML-Modell gespeichert ist (siehe Anhang 5). Außerdem wird die Hit-Policy der Entscheidungstabelle gesetzt. Die Grundlagen zu dieser werden in Kapitel 5.3.2.1 diskutiert. Sie wird standardmäßig mit dem

Wert „**FIRST**“ belegt. Dieser wird gewählt, da maximal eine Regel des Baumes zutreffen kann. Dies erhöht die Performance der Entscheidungstabelle, da nicht alle Regeln ausgewertet werden müssen – sondern die Auswertung beendet werden kann, sobald die erste Regel zutrifft.

```
// Provided data
val tree: TreeMap<Int, List<Node>> = decisionTree.allTreePaths
val dictionary: TreeDictionary = decisionTree.dictionary

// Set the input-fields (→ header)
val inputAttributes = InputAttributes(doc, dictionary.getNonTargetFields())
inputAttributes.appendTo(decisionTable)

// Set the output fields (→ header)
val targetAttribute: DataField = dictionary.getTargetAttribute()
val outputAttribute = OutputAttribute(doc, targetAttribute)
outputAttribute.appendTo(decisionTable)
...
```

Decision		Hit Policy: First	
When	And	Then	Annotations
INPUT 1	INPUT 2	OUTPUT 1	
string	string	string	
+	-	-	

Anschließend können die einzelnen Pfade des Entscheidungsbaumes in die Entscheidungstabelle überführt werden. Dazu wird die in Schritt 1 erstellte Map durchlaufen – und für jeden Pfad eine eigene Regel angelegt. Eine solche besteht aus einer Menge von Bedingungen, welche sich jeweils auf konkrete Spalten der Entscheidungstabelle beziehen. Jeder Knoten im Pfad repräsentiert eine solche Bedingung. Faktisch muss jede Bedingung in eine entsprechende Spalte eingefügt werden.

Insgesamt muss jedoch berücksichtigt werden, dass die Bedingungen in einer Syntax beschrieben werden müssen, die von der jeweiligen DMN-Engine, die im Produktivbetrieb zur Auswertung der Tabelle eingesetzt wird, unterstützt wird. Daher werden sie mittels der FEEL-Syntax beschrieben. Diese wird von allen gängigen DMN-Engines unterstützt. Die Grundlagen zu dieser Syntax werden in Kapitel 5.3.3.2 erörtert.

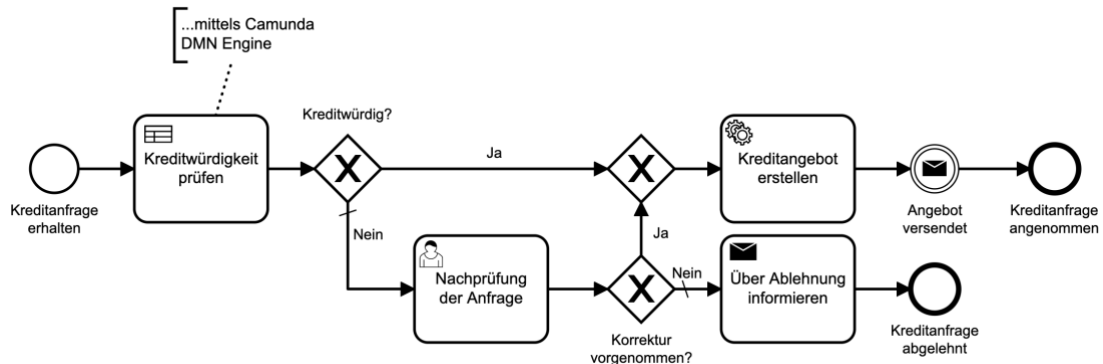
```
var ruleId = 0
// Print the rules to the xml
tree.forEach { (_, treeRoute: List<Node>) →
    ruleId = ruleId.plus(1)
    val rule = DecisionRule(doc, dictionary, treeRoute, ruleId)
    rule.appendTo(decisionTable)
}

// Print output-document
return getFinalDmnModel(doc)
}
```

Decision		Hit Policy: First	
When	And	Then	Annotations
INPUT 1	INPUT 2	OUTPUT 1	
string	string	string	
1	2	3	
+	-	-	

Anhang 9: Automatisierte Bearbeitung von Kreditanfragen mittels einer Entscheidungstabelle

Dieser Anhang baut auf den Anhängen 7 und 8 auf. Er beschreibt die technische Implementierung der zweiten Prozessvariante. In dieser wird die Prüfung der Kreditwürdigkeit eines Kunden durch eine Entscheidungstabelle automatisiert. Dabei wird, analog zur ersten Variante, die Camunda Process Engine eingesetzt. Diese ist, wie in allen anderen Prototypen, in eine Web-Applikation integriert, die auf der Programmiersprache Kotlin und dem Framework Spring Boot basiert.¹



Erstellung der Entscheidungstabelle

Um den Entscheidungsbaum, welcher in Form eines PMML-Modells vorliegt in eine DMN-basierte Entscheidungstabelle zu transformieren kann der Web-Service aus Anhang 8 aufgerufen werden. Diesem können das PMML-Modell, die IDs und die Bezeichnungen für die einzelnen Elemente übergeben werden. Der Response des Aufrufs ist eine Entscheidungstabelle. Diese kann als eine XML-, respektive eine DMN-Datei gespeichert und direkt in das obige Prozessmodell integriert werden. Ein Ausschnitt aus der Entscheidungstabelle ist Anhang 10 zu entnehmen.

Das Bild zeigt eine REST Client-Anfrage (POST) an `localhost:8085/api/dmn` mit folgenden Parametern:

KEY	VALUE	DESCRIPTION
pmml-file	credit.pmml	
model-id	approve-credit	
model-name	Kreditwürdigkeit	
decision-id	decision_check_creditworthiness	
decision-name	Prüfe Kreditwürdigkeit	

Die Response ist ein XML-Dokument, das eine DMN-Entscheidungstabelle enthält. Ein Ausschnitt des XML-Codes ist unten dargestellt:

```
1 <?xml version="1.0" encoding="UTF-8" standalone="no"?>
2 <definitions xmlns="https://www.omg.org/spec/DMN/20191111/MODEL/" xmlns:dc="http://www.omg.org/spec/DMN/20191111/DC/" xmlns:dmdid="https://www.omg.org/spec/DMN/20191111/DMNDI/" exporter="dmnandai"
3   id="approve-credit" name="Kreditwürdigkeit" namespace="http://camunda.org/schema/1.0/dmn">
4     <decision id="decision_check_creditworthiness" name="Prüfe Kreditwürdigkeit">
5       <decisionTable id="DecisionTable_18b24bc138b4431da61e4b71c6044a8d">
6         <input id="Input_785ffca9a15e4ae89bc221be7d307095" label="GENDER">
7           <inputExpression expressionLanguage="feel">
8             id="InputExpression_3d3a8dfea546442784a7b03557eee4ce" typeRef="string">
9               <text>gender</text>
10            </inputExpression>
11          </input>
12          <input id="Input_4b0025ed59e54c3db3adfb263b8c04a3" label="FLAG_OWN_CAR">
13            <inputExpression expressionLanguage="feel">
14              id="InputExpression_554432d71fe9434abb8a5935adfd3206" typeRef="string">
15                <text>flag_own_car</text>
16              </inputExpression>
17            </input>
18          </input>
19        </decisionTable>
20      </decision>
21    </definitions>
```

¹ Vollständige Implementierung verfügbar unter: <https://github.com/emaarco/bpmnandai>

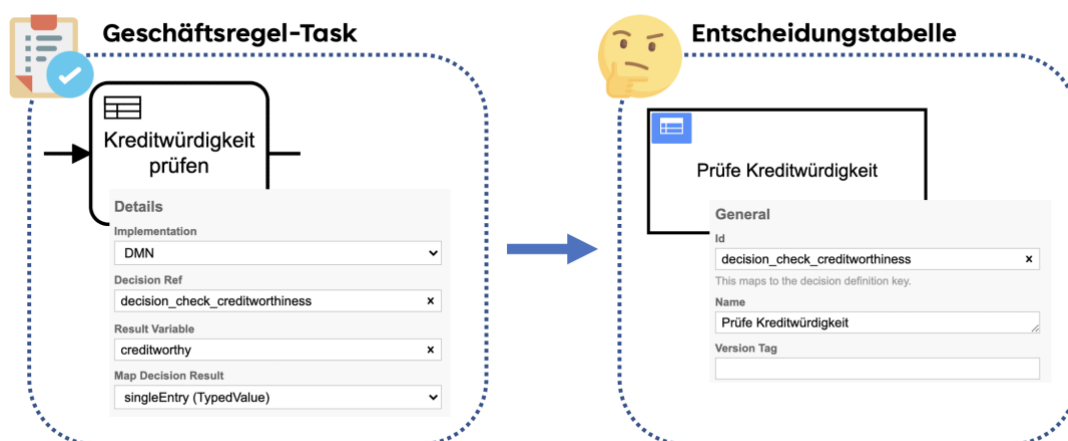
Startereignis – Kreditanfrage erhalten

Auch diese Variante kann durch den Aufruf einer konkreten Schnittstelle gestartet werden. Die Daten, welche dieser übergeben werden, sind identisch zu jener aus Anhang 7. Die Verarbeitung der Daten hingegen verläuft different. Denn während die Informationen zu einer Kreditanfrage in der ersten Variante lediglich in der Datenbank der Web-Applikation gespeichert werden, müssen sie in der zweiten Variante – zusätzlich dazu – auch in den Prozessvariablen gespeichert werden. Dies ist notwendig, damit die Camunda DMN-Engine, zur Auswertung der Entscheidungstabelle, auf die benötigten Daten zugreifen kann.

```
fun createLoanRequest(newRequest: NewLoanRequest): LoanRequest {  
    val requestId = UUID.randomUUID().toString()  
    val loanRequest = LoanRequest(requestId, newRequest)  
    creditService.saveLoanRequest(loanRequest)  
    val processVariables = getProcessVariables(loanRequest)  
    processService.startInstanceOfProcess(processKey, requestId, processVariables)  
    log.info { "Created a new loan request: $loanRequest" }  
    return loanRequest  
}
```

Aktivität – Kreditwürdigkeit prüfen

Auch in der zweiten Variante ist die Überprüfung der Kreditwürdigkeit über einen Geschäftsregel Task abgebildet. Um diesen auszuführen, wird die Camunda DMN-Engine eingesetzt. Diese ist in die Camunda Process Engine eingebettet. Somit kann sie direkt aus einem Prozessmodell heraus angesprochen werden. Dazu muss der Geschäftsregel Task so konfiguriert werden, dass er die Entscheidungstabelle, die in einem konkreten DMN-Modell abgebildet ist, aufrufen kann. Zudem wird festgelegt, in welcher Prozessvariable das Ergebnis gespeichert werden soll. Die Daten zur Auswertung der Entscheidungstabelle müssen nicht explizit übergeben werden. Sie werden von Camunda, auf Basis des DMN-Modells, automatisch aus den Prozessvariablen bezogen.



Restliche Tasks & Ereignisse

Die Implementierung der übrigen Prozessschritte ist identisch zu jenen aus Anhang 7. Für weitere Informationen zu diesen wird daher auf diesen Anhang verwiesen.

Anhang 10: Entscheidungstabelle zur Klassifikation der Kreditwürdigkeit von Kreditkunden

Camunda Modeler														
check-creditworthiness.dmn														
FlowRepo														
Edit DRD														
Open Overview														
Close Tab														
Rufte Kreditwürdigkeit Hit Policy: Unique														
When	And	And	And	And	And	And	And	And	And	And	And	And	Then	
GENDER	FLAG_OWN_C...	FLAG_OWN_R...	COUNT CHILD...	ANNUAL INCO...	EDUCATION T...	FAMILY STATUS	HOUSING TYPE	DAYS EMPLO...	JOB	AGE	CREDIT AMOU...	CREDITWORT...		
string	string	string	string	double	string	string	string	integer	string	integer	double	string		
1 "F"	-	-	-	-	-	"Civil marriage"	-	<= 1399.5	"Laborers"	-	<= 262710.0	"NO"		
2 "F"	-	-	-	-	-	"Married"	-	<= 1399.5	"Laborers"	-	<= 262710.0	"YES"		
3 "F"	-	-	-	-	-	"Widow"	-	<= 1399.5	"Laborers"	-	<= 262710.0	"NO"		
4 "F"	-	-	-	-	-	"Separated"	-	<= 1399.5	"Laborers"	-	<= 262710.0	"YES"		
5 "F"	-	-	-	-	-	"Single / not married"	-	<= 1399.5	"Laborers"	-	<= 262710.0	"YES"		
6 "M"	-	-	-	-	-	-	"House / apartment"	<= 260.5	"Laborers"	-	<= 164115.0	"NO"		
7 "M"	-	-	"No children"	-	-	-	"House / apartment"	[260.5, 1399.5]	"Laborers"	<= 40.0	<= 164115.0	"YES"		
8 "M"	-	-	"2+ children"	-	-	-	"House / apartment"	[260.5, 1399.5]	"Laborers"	<= 40.0	<= 164115.0	"YES"		
9 "M"	-	-	"1 children"	-	-	-	"House / apartment"	[260.5, 1399.5]	"Laborers"	<= 31.0	<= 164115.0	"NO"		
10 "M"	-	-	"1 children"	-	-	-	"House / apartment"	[260.5, 1399.5]	"Laborers"	[31.0, 40.0]	<= 164115.0	"YES"		
11 "M"	-	-	-	-	-	-	"House / apartment"	[260.5, 504.0]	"Laborers"	> 40.0	<= 164115.0	"YES"		
12 "M"	-	-	-	-	-	-	"House / apartment"	[504.0, 965.0]	"Laborers"	> 40.0	<= 164115.0	"NO"		
13 "M"	-	-	-	-	-	-	"House / apartment"	[965.5, 1189.0]	"Laborers"	> 40.0	<= 164115.0	"YES"		
14 "M"	-	-	-	-	-	-	"House / apartment"	[1189.0, 1290.5]	"Laborers"	> 40.0	<= 164115.0	"NO"		
15 "M"	-	-	-	-	-	-	"House / apartment"	[1290.5, 1399.5]	"Laborers"	> 40.0	<= 164115.0	"YES"		
16 "M"	-	-	-	-	-	"Co-op apartment"	<= 1399.5	<= 1399.5	"Laborers"	-	<= 164115.0	"NO"		
17 "M"	-	-	-	-	-	"Municipal apartment"	<= 1399.5	<= 1399.5	"Laborers"	-	<= 164115.0	"YES"		
18 "M"	-	-	-	-	-	"With parents"	<= 1399.5	<= 1399.5	"Laborers"	-	<= 164115.0	"NO"		
19 "M"	-	-	-	-	-	"Rented apartment"	<= 1399.5	<= 1399.5	"Laborers"	-	<= 164115.0	"NO"		
20 "M"	-	-	-	-	-	"House / apartment"	<= 977.0	[977.0, 1399.5]	"Laborers"	-	[164115.0, 262710.0]	"NO"		
21 "M"	"N"	-	-	-	-	"House / apartment"	<= 977.0	[977.0, 1399.5]	"Laborers"	-	[164115.0, 262710.0]	"NO"		
22 "M"	"Y"	-	-	-	-	"House / apartment"	<= 1399.5	[977.0, 1399.5]	"Laborers"	-	[164115.0, 262710.0]	"YES"		
23 "M"	-	-	-	-	-	"Co-op apartment"	<= 1399.5	<= 1399.5	"Laborers"	-	[164115.0, 262710.0]	"YES"		
24 "M"	-	-	-	-	-	"Municipal apartment"	<= 1399.5	<= 1399.5	"Laborers"	-	[164115.0, 262710.0]	"YES"		
25 "M"	-	-	-	-	-	"With parents"	<= 1399.5	<= 1399.5	"Laborers"	-	[164115.0, 262710.0]	"NO"		
26 -	-	-	-	<= 66150.0	"Secondary / secondary special"	-	-	> 1399.5	"Laborers"	<= 33.5	<= 262710.0	"YES"		
27 -	-	-	-	> 66150.0	"Secondary / secondary special"	-	-	> 1399.5	"Laborers"	<= 33.5	<= 144800.0	"NO"		
28 -	-	-	-	> 66150.0	"Secondary / secondary special"	-	-	> 1399.5	"Laborers"	<= 33.5	[144800.0, 262710.0]	"YES"		
29 -	-	"Y"	-	-	"Higher education"	-	-	> 1399.5	"Laborers"	<= 33.5	<= 262710.0	"YES"		
30 -	-	"N"	-	-	"Higher education"	-	-	> 1399.5	"Laborers"	<= 33.5	<= 262710.0	"NO"		
31 -	-	-	-	"Incomplete higher"	-	-	-	> 1399.5	"Laborers"	<= 33.5	<= 262710.0	"YES"		
32 -	-	-	-	"Lower secondary"	-	-	-	> 1399.5	"Laborers"	<= 33.5	<= 262710.0	"YES"		
33 -	-	-	"No children"	-	-	-	-	> 1399.5	"Laborers"	[33.5, 35.5]	<= 262710.0	"NO"		
34 -	-	-	"2+ children"	-	-	-	-	> 1399.5	"Laborers"	[33.5, 35.5]	<= 262710.0	"YES"		

XML Camunda Platform 4.11.1

Anhang 11: Korrektheit der Umwandlung des Baumes in eine Entscheidungstabelle

Um zu prüfen, ob die Umwandlung des Entscheidungsbaumes, von einem PMML-Modell zu einer Entscheidungstabelle korrekt verlaufen ist, werden die Testdaten aus Anhang 5 auf diese Tabelle angewendet. Die Klassifikationen, welche dadurch erzeugt werden, können mit dem initialen Test des ML-Modells aus Anhang 5 verglichen werden.

Im Ergebnis zeigt sich, dass die Anzahl der richtigen- und der falschen Vorhersagen in beiden der Tests identisch ist. Daher wird angenommen, dass die Regeln des Entscheidungsbaumes, und die der Tabelle identisch sind. Folglich liegt kein Unterschied in der Qualität der Ergebnisse vor. Die Transformation hat ausschließlich den Effekt, dass der Baum in einer anderen Form abgebildet wird und somit auf andere Weisen automatisiert, oder für dritte Zwecke eingesetzt werden kann.

